Studies in Systems, Decision and Control 458

Muhammad Atif
Jan Friso Groote

Understanding Behaviour of Distributed Systems Using mCRL2

MOREMEDIA

Springer

Studies in Systems, Decision and Control

Volume 458

Series Editor

Janusz Kacprzyk, Systems Research Institute, Polish Academy of Sciences, Warsaw, Poland

The series “Studies in Systems, Decision and Control” (SSDC) covers both new developments and advances, as well as the state of the art, in the various areas of broadly perceived systems, decision making and control–quickly, up to date and with a high quality. The intent is to cover the theory, applications, and perspectives on the state of the art and future developments relevant to systems, decision making, control, complex processes and related areas, as embedded in the fields of engineering, computer science, physics, economics, social and life sciences, as well as the paradigms and methodologies behind them. The series contains monographs, textbooks, lecture notes and edited volumes in systems, decision making and control spanning the areas of Cyber-Physical Systems, Autonomous Systems, Sensor Networks, Control Systems, Energy Systems, Automotive Systems, Biological Systems, Vehicular Networking and Connected Vehicles, Aerospace Systems, Automation, Manufacturing, Smart Grids, Nonlinear Systems, Power Systems, Robotics, Social Systems, Economic Systems and other. Of particular value to both the contributors and the readership are the short publication timeframe and the worldwide distribution and exposure which enable both a wide and rapid dissemination of research output.

Indexed by SCOPUS, DBLP, WTI Frankfurt eG, zbMATH, SCImago.

All books published in the series are submitted for consideration in Web of Science.

Muhammad Atif · Jan Friso Groote

Understanding Behaviour of Distributed Systems Using mCRL2

Springer

Muhammad Atif
Department of Computer Science
and Information Technology
The University of Lahore
Lahore, Pakistan

Jan Friso Groote
Department of Mathematics and Computer
Science
Eindhoven University of Technology
Eindhoven, Noord-Brabant
The Netherlands

ISSN 2198-4182 ISSN 2198-4190 (electronic)
Studies in Systems, Decision and Control
ISBN 978-3-031-23007-3 ISBN 978-3-031-23008-0 (eBook)
https://doi.org/10.1007/978-3-031-23008-0

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2023
This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

Dedicated to our families

Preface

Understanding a distributed system's behaviour means getting insight into whether the system meets its expectations. Giving such a surety is challenging, primarily when a system consists of multiple participants. These participants can interact with each other in far more ways than we can imagine.

In this book, we explain how to model the behaviour of communicating systems and generate, reduce and visualise it as a labelled transition system. We also show how to do model-checking, which allows checking properties on the behaviour, such as whether the behaviour is free of deadlocks. Nevertheless, as we use the modal mu-calculus with data, virtually any conceivable safety, liveness and fairness property can be formulated and checked.

Applying model-checking makes everyone's eyebrow high as the results are often surprising. Using counter-examples can reveal behaviour in distributed systems that no one, including the developers, deemed possible, and as such, it is an invaluable tool to obtain essential insight into the behaviour of modern programmed systems.

It is a general perception that model-checking has a steep learning curve. In this book, we try to mitigate this by introducing all concepts with small examples and thoroughly explaining system specifications line by line. We use mCRL2 (micro Common Representation Language 2) and the modal mu-calculus, which are specialised languages that specify a distributed system and formulate system properties. These languages come with an effective toolset, which is open source and can be freely used (www.mcrl2.org).

This book is introductory and comprises subjects from installing the toolset mCRL2 to analysing simple systems. We start with writing *hello world* type systems and gradually increase the complexity of examples and exercises. So, anyone having some knowledge of computer programming, logic and basic mathematics can study this book and will learn how to model-check communication protocols, distributed algorithms and interacting systems. We focus to explain every topic with sufficient

examples and exercises. A chapter on modelling and solving puzzles and another chapter on how to model heartbeat protocols illustrate the techniques.

Lahore, Pakistan — Muhammad Atif
Eindhoven, The Netherlands — Jan Friso Groote
February 2023

Acknowledgments

We acknowledge the contribution of many people who gave their feedback and support. Shazia Hussain (wife of Muhammad Atif) kept asking about the progress of the book and Paula Versteegen (wife of Jan Friso Groote) kept listening to an infinite stream of unsolicited explanations about system behaviour. We appreciate the guidance of Dr. Ali Khan, Associate Editor of Springer, who suggested publishing this book with *Springer Nature Book Services*. Mr. Prasanna Kumar, also from Springer, kindly pushed us to complete this book by sending emails from time to time.

We thank Ir. Olav Bunte (Ph.D. student at TU/e Netherlands) and Dr.ir. Jeroen Keiren (Assistant Professor at TU/e, Netherlands) for answering every question about the toolset mCRL2. Thanks also go to Dr.ir. Wieger Wesselink and Ir. Maurice Laveaux, as well as many others, for maintaining and developing the mCRL2 toolset. We are also thankful to Dr. Ijaz Ahmed and Mr. Imran Shafi, who gave their valuable feedback. Officials from the University of Lahore who always remained a motivation are Mr. Awais Raoof (chairman), Prof. Dr. Mujahid Kamran (former rector) and Prof. Dr. Muhammad Ashraf (rector). Lastly, we would like to acknowledge the constant support of Bakhtawar Atif, Fatima Atif, Muhammad Hassan Chattha and Muhammad Athar Chattha.

Lahore, Pakistan — Muhammad Atif
Eindhoven, The Netherlands — Jan Friso Groote
February 2023

Introduction

Welcome

Welcome to the first edition of the book. This book intends to create an interest to reason about the correctness of the behaviour of programmed systems. There are a lot of such systems around us and available in the literature that needs to be studied and corrected. For example, we can study the protocol of an ATM (automated teller machine), e-Tag system for road tax collection, e-Commerce systems or other systems having communicating processes. It can be surprising to know that many of the existing distributed algorithms used in such systems are not completely correct, which can effectively be found by formal verification. In model-checking, the complete behaviour of a system is modelled and then searched exhaustively.

Who Is This Book For?

This book is for those who want to use mCRL2 for automatically analysing system behaviour. The targeted systems for such an analysis are those systems having concurrent processes with inter-process communication. In this book, we study system modelling and model-checking techniques. Interestingly, model-checking provides beneficial results, but at the same time, it poses particular challenges as well. Formulating behaviour precisely, especially formalising properties that the behaviour must have, is not always easy. Not only it is hard to understand the behaviour and requirements, often given in ambiguous natural language, but correctly encoding them using minimal or maximal fixed point modalities requires some training and experience. Besides that, a modelled system may become so complex that the tools can no longer analyse it. This phenomenon is often referred to as the state space explosion problem.

In this book, we study these challenges and the methods to address those challenges in order to apply model-checking effectively.

What Is Covered in This Book?

Features of mCRL2 are covered so that a reader can start from elementary and small examples to learn how to make models and check them.

Your Feedback and Errata

We have tried our best to explain with examples and exercises, but we believe there is always some space for improvement. So, please let us know your comments, feedback or errata (if any) through email: matifch@gmail.com or by visiting `http://www.mcrl2.org`. The code given in this book is available over the web portal of Springer as "Extra Materials".

Contents

1 Introducing mCRL2 1
1.1 What is mCRL2 All About? 1
1.2 History of mCRL2 1
1.3 Why mCRL2? 2
1.4 Learning mCRL2 2
1.5 Other Educational Material 3
1.6 The mCRL2 Toolkit 4
1.7 Other Toolsets 6

2 Automata to Represent Behaviour 9
2.1 Why Make Models? 9
2.2 Developing Models in mCRL2 11
2.2.1 Hello-World Example 11
2.2.2 Using the Terminal 14
2.2.3 State-Space and Actions 16
2.3 Using the GUI 19

3 Communicating Processes 25
3.1 Communication Among Actions 25
3.1.1 A Coffee Vending Machine 27
3.2 Deadlock 30
3.3 Synchronous Versus Asynchronous Communication 32
3.4 Blocking and Renaming Actions 34
3.5 Hiding Actions 35

4 Behavioural Equivalences 37
4.1 Trace Equivalence 37
4.1.1 Weak Trace Equivalence 40
4.2 Strong Bisimulation 42
4.3 Branching Bisimulation 45
4.4 Divergence Preserving Branching Bisimulation 49

5 Data Types and Data-Dependent Behaviour 51
5.1 Sending/Receiving Data 51
5.1.1 Operators on Data 53
5.2 Conditions 59
5.3 Data Structures 60
5.3.1 Lists 60
5.3.2 Sets 66
5.3.3 Bags 68
5.4 User Defined Functions 70
5.4.1 Defining Constants 78
5.4.2 λ-Expressions 79
5.5 Constructors of a Data Type 81
5.6 Defining a *Struct* Data Type 82
5.7 Modelling the Alternating Bit Protocol 85
5.8 Modelling Milner's Cyclic Scheduler 89

6 Model-Checking 93
6.1 Hennessy-Milner Logic 94
6.2 Model-Checking with mCRL2 96
6.2.1 Parameterised Boolean Equation System 96
6.3 Action Formulas 100
6.4 Regular Formulas 102
6.5 Formulas with Data and Quantifiers 108
6.6 What to do if Model-Checking Becomes Time Consuming? 110
6.6.1 Reduce the Problem Size 111
6.6.2 Optimising and Transforming a PBES 111
6.6.3 Model-Check Explicit State Spaces 112
6.6.4 CADP 113
6.6.5 LTSmin and Pbessolvesymbolic 114

7 The Modal μ-Calculus 117
7.1 Minimal Fixed Points 118
7.1.1 Minimal Fixed Point Formulas with Data 124
7.2 Maximal Fixed Points 128
7.2.1 Maximal Fixed Points with Data 131
7.3 Combining Minimal and Maximal Fixed Points 133
7.4 Modelling a Movable Patient Support Unit and Its Requirements 137
7.4.1 Formal Specification 139
7.4.2 Requirements Analysis 143

8 Linear Processes and Parameterised BESs 149
8.1 Linear Processes 149
8.2 Optimising an LPS 152
8.2.1 Rewriting a Linear Process 153
8.2.2 Constant Elimination 154

8.2.3 Parameter Elimination 154
8.2.4 Sum Instantiation and Sum Elimination 155
8.2.5 Action Renaming 156
8.2.6 Unfolding Parameters 158
8.3 Parameterised Boolean Equation Systems 159
8.4 Optimising PBESs 160
8.4.1 Rewriting PBESs 160
8.4.2 Constant Elimination 162
8.4.3 Parameter Elimination 162

9 Applications: Puzzles and Games 165
9.1 Magic Square 165
9.2 Bridge Crossing 166
9.3 Weight Bars Crossing 170
9.4 The Wolf, the Goat and Cabbage 172
9.5 Jumping Frogs 173
9.6 Tic-Tac-Toe 178

10 Applications: Distributed Algorithms 183
10.1 Heartbeat Protocols 183
10.2 The Binary Heartbeat Protocol 184
10.3 The Static Heartbeat Protocol 193
10.4 The Expanding and the Dynamic Heartbeat Protocol 199

Appendix A: Specifications of Protocols in mCRL2 201

Appendix B: Solutions 207

References 229

Index 233

Chapter 1
Introducing mCRL2



This chapter provides basic information about mCRL2. Historical notes and comparisons with other similar tools are provided, too. Installations and configuration instructions are also there, along with a road map for learning mCRL2.

1.1 What is mCRL2 All About?

mCRL2 stands for "micro Common Representation Language 2". It is a modelling language used to specify distributed systems formally. Particularly, a system comprising concurrent and communicating processes because mCRL2 is based on *Algebra of Communicating Processes* (*ACP*) [5]. After formally modelling a system, we can analyse this model using model-checking. In model-checking, we investigate the correctness of a system concerning its requirements, i.e., formal verification. So, in short, we can analyse the behaviour of a distributed system by using the toolkit mCRL2. This technique has already been applied to analyse numerous systems. A few of them are [2, 14, 48, 50, 51].

1.2 History of mCRL2

Around 1980 process algebras were developed to describe and reason about the behaviour of programmed systems, which communicate with their environment and each other by sending and receiving messages. The major examples of such process algebras are *CCS*, (*Calculus of Communicating Systems)* [47], *ACP* (*Algebra of Communicating Processes*) [5] and *CSP* (*Communicating Sequential Processes*) [39].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0_1

These algebras were lacking data types, and their purpose was to study the essence of system behaviour, as well as the nature of process algebras themselves. Data plays a vital role in process specification. So, some specification languages were introduced using the process algebra extended with equational abstract data types. Examples are *LOTOS* (*Language of Temporal Ordering Specifications*) [11], *PSF* (*Process Specification Formalism*) [46], and μ*CRL* (*micro Common Representation Language*) [10, 30].

mCRL2 (micro Common Representation Language 2) is the successor of μCRL [27]. Besides equational data types, mCRL2 incorporates standard data types. For example, booleans, positive and natural numbers, integers and reals where all numbers have no bounds and properly represent the mathematical notion of numbers. Data structures like lists, sets, and bags are also built-in in mCRL2. mCRL2 also has (higher-order) functions and λ-expressions. Additionally, multi-actions, timed and probabilistic behaviour are also supported in the current version.

1.3 Why mCRL2?

mCRL2 is a very concise and, at the same time, extremely expressive language to specify and analyse the behaviour of distributed systems and protocols. It can easily express non-computable behavioural specifications and requirements. It goes without saying that for such descriptions, tool-supported analysis is not very fruitful. However, this implies that effectively using mCRL2 requires a good understanding of the language and formula writing in the μ-calculus. Properties in mCRL2 are expressed in the modal μ-calculus with data, which is far more expressive than other temporal logics such as LTL (Linear Temporal Logic), CTL (Computational Tree Logic), and CTL*. Compared to other behavioural specification languages, mCRL2 can express everything that most other behavioural modelling languages can do without any problem. A disadvantage of mCRL2 is that its conciseness requires reformulating certain commonly used programming constructs. For instance, it does not support while loops, assignments and returning values from functions.

1.4 Learning mCRL2

Before we start learning the modelling and analysis techniques, it may be helpful if we know our direction of learning and also figure out the route we must take to understand the toolset best. So, here is a brief roadmap.

1. The current chapter introduces mCRL2, its need for formal analysis and other similar approaches used in the formal specification and verification of distributed algorithms.

2. Chapter 2 describes the basics of modelling with elementary examples so a beginner can know about the fundamentals of modelling. This chapter explains how a process can be represented as an automaton and in the algebraic notation used in mCRL2.
3. In Chap. 3, we describe the details to specify a process using certain operators like plus, times, and the parallel operator, along with their relation to automata. We also describe recursive behaviour with examples and show how to use the toolset to simulate and generate state spaces.
4. Chapter 4 explains the use of behavioural equivalences, specification versus implementation, the τ-action, strong and branching bisimulation, and weak trace equivalence, and behavioural reduction. It also describes using the toolset to reduce automata and calculate equivalences.
5. Chapter 5 describes how to declare data types, the conditional and sum operators, and the use in actions, conditional operators and processes. It also describes how to communicate data among parallel processes with examples.
6. In Chap. 6, we describe Hennessy-Milner logic, action- and regular formulas including examples.
7. In Chap. 7, the μ-Calculus is described to express advanced properties formally.
8. In Chap. 8 we present linear processes and parameterised boolean equation systems. These are normal forms. A process is translated to a linear process which is easier to handle. A process and a modal formula are translated to a parameterised boolean equation system, which must subsequently be solved.
9. In Chap. 9, we show how to solve puzzles and games. Here we learn how to model such problems in mCRL2 and solve them by exhaustive search or verifying modal formulas.
10. In Chap. 10, we learn how mCRL2 is used to formally verify distributed algorithms. In particular, two heartbeat protocols are discussed, specified in mCRL2, and subsequently verified.
11. Appendix A provides complete models of the case studies given in Chap. 10.
12. Solutions of all exercises are provided in Appendix B.

1.5 Other Educational Material

On Coursera, there are over 60 short videos explaining the material in this book that are freely accessible with many associated exercises. Only obtaining a certificate that the course has been passed requires a fee. The videos are accessible at

https://www.coursera.org/learn/automata-system-validation
https://www.coursera.org/learn/system-validation-behavior
https://www.coursera.org/learn/system-validation-modal-formulas
https://www.coursera.org/learn/system-validation-software-protocols

The theoretical background of mCRL2 is described in [28]. An introduction in elementary process algebra is given in [19]. Good background material in process algebra is available in [6].

1.6 The mCRL2 Toolkit

The mCRL2 toolset can be downloaded from http://www.mcrl2.org for various platforms, like Windows, Mac OS X, Ubuntu, Fedora, etc. There are two versions for each platform, namely the released stable version and the "nightly build", which is the most recent version that is under active development.

Installing mCRL2. We can install the toolset as follows. Download the source or binaries according to the operating system and install it. Set the path to the *bin* directory/folder where the toolset is installed.

We have three options to start working, i.e., using the IDE (`mcrl2ide`), using the GUI (`mcrl2-gui`), or via the command terminal. For beginners, the IDE is recommended where we manipulate specifications and check formulas with a few clicks. Figure 1.1 shows the IDE.

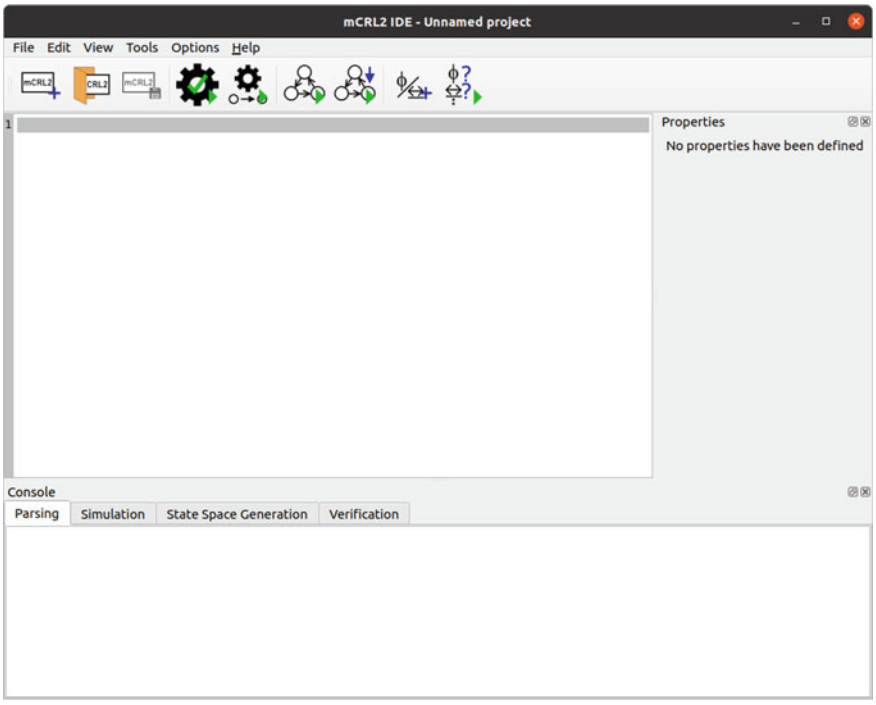


Fig. 1.1 Integrated Development Environment (IDE) of mCRL2

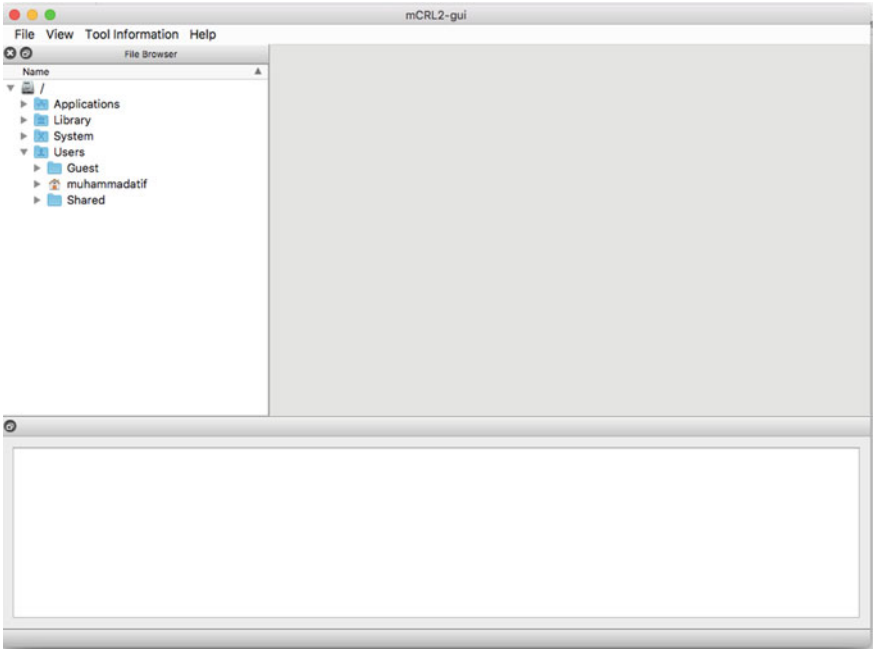


Fig. 1.2 mCRL2 GUI

We can create a new project and start writing mCRL2 code in the main window (see Sect. 2.2.1). Similarly, we can also test the installation by starting the GUI. While using the GUI, or the command line, we can use the tools with various options, which is impossible in the IDE. Especially for the advanced user, these options are necessary as they are essential to perform analyses on large and complex specifications. Figure 1.2 shows a sample screen of the GUI. We learn how to use GUI and IDE in the next chapter.

We can also work with the terminal window provided that the path to the *bin* directory is already set, as shown in Fig. 1.3 where a tool `mcrl22lps` is used as `mcrl22lps -version` to show the current version.

A video demonstrating the installation along with a "Hello World" example can be found on www.youtube.com with the title "Installing mCRL2 on MAC".



Fig. 1.3 Terminal window showing mCRL2 installation

1.7 Other Toolsets

There are several other toolsets, some of them with a generic expressivity to mCRL2, and others more targeted to particular application domains, such as time or probabilities.

The toolset CADP (Construction and Analysis Distributed Processes) [20] resembles mCRL2 to some extent. Both the toolsets use the modal μ-calculus for writing properties and are inspired by process algebra. However, mCRL2 is open source, while CADP requires a license for academics and commercial use. mCRL2 provides complete insight into linear processes and equation systems while these details are hidden in CADP from a user. Furthermore, visualising a system in 2D/3D and analysing a real-time system is possible in mCRL2, whereas both these features are much more limited in CADP. On the other hand, CADP supports the performance evaluation of a system while mCRL2 does not, and the specification language used in CADP, LNT (LOTOS New Technology), is more friendly to those used to programming.

Uppaal [9] is a toolset used for real-time systems. It uses directed graphs to specify a process which is very attractive for beginners, but is restricted in expressivity. It supports only LTL-based properties and is a closed source where its GUI is written in Java, and the verification engine is implemented in C++. It has a 64-bit version for Mac OS and Linux, but only a 32-bit version is available for Windows, so it can

only access limited memory. Uppaal allows for statistical model-checking using its SMC (Statistical Model-Checking) engine.

PRISM [44] is another probabilistic model checker which uses discrete and continuous-time Markov chains and decision processes. LTSmin is a language-independent model-checker [43] that can work with mCRL2 for state-space optimisation. This tool provides reachability analysers using symbolic, multi-core, and parallel algorithms.

Other model checkers have their own specification languages, and mCRL2 compares well to all of them. Examples of such model-checkers include Spin [42], PAT [52], nuSMV [18], FDR3 [21] and DiVinE [8].

Chapter 2
Automata to Represent Behaviour



In this chapter, we will learn to model a system in mCRL2. Notably, we want to model systems consisting of concurrent and communicating processes. We can also see how a process can be represented graphically in the form of an automaton. Parallelism among processes significantly increases the complexity of the behaviour of a system.

A system has processes, and a process has actions. By a process, we mean an entity (or a participant) of a system that interacts with other participants or even may interact with other systems.

2.1 Why Make Models?

If a system comprises concurrent processes, the behaviour of that system becomes complex to analyse. However, by making models of such behaviour, we can assess the correctness of a system. In a formal analysis, we try to figure out whether a system meets its functional requirements or not. For example, whether or not a deadlock occurs in a distributed system. We say a system is in a 'deadlock' state when no process can take action. To understand requirement analysis, see Chap. 6.

Consider the following three concurrent processes of a system where the requirement is that the value of i (a shared variable) never becomes negative.

Process 1

```
1 while true do
2 |  if i > 0 then
3 |  |  i := i − 1;
4 |  end
5 end
```

Process 2

```
1 while true do
2 |  if i ≈ 100 then
3 |  |  i := 0;
4 |  end
5 end
```

Process 3

```
1 while true do
2 |  if i < 100 then
3 |  |  i := i + 1;
4 |  end
5 end
```

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0_2

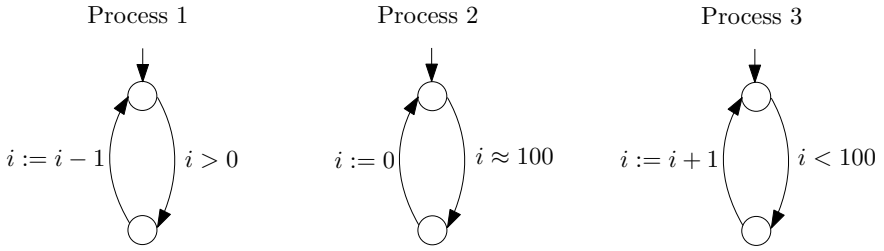


Fig. 2.1 Automata of processes

We can represent these processes in the form of automata, as shown in Fig. 2.1. Do you think the value of i can become negative? To answer this question, we must carefully observe each process's behaviour. The following sequence of actions shows a situation where the value of i becomes -1.

1. Suppose the process is in a state where $i = 99$ and process 3 makes $i = 100$.
2. Process 1 takes control and evaluates $i > 0$ but before executing $i := i - 1$, a clock interrupt occurs and process 1 loses control due to the context switching.
3. Process 2 takes control and determines that $i \approx 100$ is valid, and henceforth assigns 0 to i.
4. Then process 1 is resumed and decreases the value of i, thus i becomes -1.

The above sequence of actions (also called a *trace* of actions) is not always very easy to discover, whereas the functionality of each process seems simple. We can find such scenarios automatically by model-checking. This is why model-checking provides beneficial results, which are sometimes also surprising. Showcases of this type of analysis are available on http://www.mcrl2.org and a few of them are [2, 14, 48, 50, 51].

Exercise 2.1 Consider a printer spooler having three jobs to print, where two processes, say Process-A and Process-B, want to submit their printing jobs. Their behaviour is straightforward; namely, they check whether the next slot is free and if so, they place their print job in that slot. Slot 7 is available for the next job, as shown in Fig. 2.2. Do you think Process-A can overwrite the job of Process-B or vice versa, where the printer daemon periodically checks the next available slot?

Exercise 2.2 Assume there are two processes, P_0 and P_1, and they share a common boolean variable called *lock*. Is it possible for both processes to enter a critical section simultaneously if they are concurrent and work according to the following algorithm? Initially, the variable *lock* is false, and CS presents a critical section.

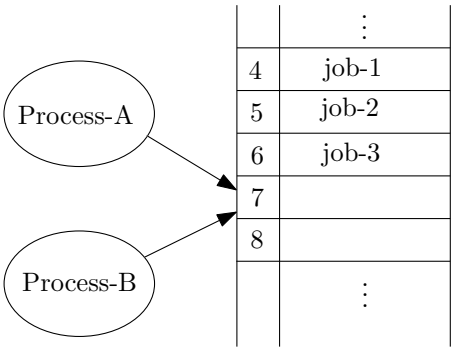


Fig. 2.2 Spooler directory

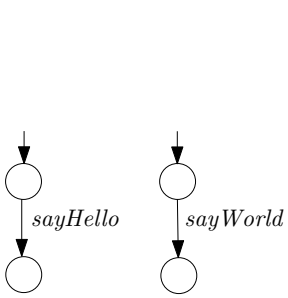
```
1 while true do
2 |   while lock do
3 |   |   /* wait */
4 |   end
5 |   lock := true;
6 |   CS();
7 |   lock := false;
8 |   NON-CS();
9 end
```

2.2 Developing Models in mCRL2

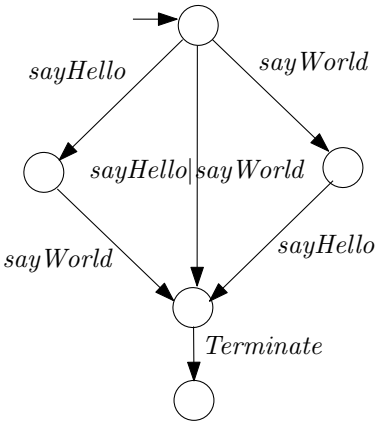
In this section, we will learn to develop models in mCRL2. By model-checking, we can automatically answer questions like the ones given in Exercises 2.1 and 2.2.

2.2.1 Hello-World Example

A "Hello World!" example is a computer program used to illustrate a language's basics by writing simple code, compiling it, and seeing its output. We do the same here, but, for the sake of concurrency, we consider a system having two parallel processes, say P_1 and P_2. Assume P_1 performs the action "sayHello" and P_2 performs "sayWorld". The behaviour of these processes is shown as automata in Fig. 2.3a, whereas the behaviour of a system as a result of putting them parallel is shown in Fig. 2.3b. The system behaviour exhibits all the following possibilities from its initial state where the dot separates the actions, and actions separated by a bar happen simultaneously.



(a) Two processes



(b) System behaviour due to two parallel processes

Fig. 2.3 Model of the "Hello World" example

- `sayHello.sayWorld` (first P_1 then P_2).
- `sayWorld.sayHello` (first P_2 then P_1).
- `sayHello|sayWorld` (both occur at the same time).

This system is specified in the syntax of mCRL2 as:

```
act sayHello, sayWorld;

proc P1 = sayHello;
     P2 = sayWorld;

init P1||P2;
```

Here `act`, `proc` and `init` are the keywords for declaring actions, processes, and initialisation, respectively. See Table 2.1 for the complete list of keywords (i.e., reserved words) in mCRL2. In the above example, `sayHello` and `sayWorld` are declared as actions while the processes P_1 and P_2 are defined in the `proc` section. The processes P_1 and P_2 are put in parallel by the || operator. The keyword `init` is followed by the process that represents the behaviour of this model. The processes P1 and P2 are put in parallel by the ‖ operator. Table 2.2 provides the list of the other reserved operators in mCRL2.

We can enter this model in the mCRL2 IDE (tool: `mcrl2ide`) using the exact syntax as indicated above as shown in Fig. 2.4. Use the icon pointed by the red arrow to see if the specification is well-formed or not.

We can simulate the system behaviour by using the next icon, i.e., right to the red arrow, which activates the tool `lpsxsim`. See Fig. 2.5. By clicking on the actions listed in the left upper window, the next action of the trace can be selected. Selecting

Table 2.1 Reserved words in mCRL2

Keyword	Meaning	Keyword	Meaning
Init	Initial process	Act	Declare actions
Proc	Process definition	Eqn	Declaring equations that define functions
Map	Declare a function	Sort	Defining a type
Var	Declare variables	Allow	Actions allowed in the state space
Comm	Synchronisation of actions	Hide	To hide actions, i.e., rename to τ
Nil	Empty regular formula	Sum	Summation of processes over a data type
Cons	Declare a constructor	Block	Action blocking for state space
In	Presence in a set or bag	Rename	Give an alternative name to actions
Bool ($\mathbb{B}$)	Boolean data type	Pos ($\mathbb{N}^+$)	Positive numbers 1, 2, 3, ...
Nat ($\mathbb{N}$)	Natural numbers	Int ($\mathbb{Z}$)	Integer numbers ..., $-2, -1, 0, 1, 2, \ldots$
Real ($\mathbb{R}$)	Real numbers	List	Data type for lists
True	Boolean value	Bag	Data type for multi sets
False	Boolean value	Set	Data type for sets
Whr	Where	End	End of a where
Lambda	Used for λ-expressions	Forall	For all values ($\forall$)
Exists	There exists ($\exists$)	Div	Divide operator
Mod	Remainder of a division	Struct	Declare structured data types
Delta (δ)	Deadlock	Glob	Declare a global constant
Tau (τ)	The hidden action	Val	Evaluating a boolean expression
Mu (μ)	Minimal fixed point	Nu (ν)	Maximal fixed point

Table 2.2 Reserved symbols in mCRL2

Symbol	Meaning	Symbol	Meaning
||	Parallel processes	->	Then part of a condition
<>	Else part of a condition	#	Parameter type separator
+	Choice operator	·	Sequential composition operator
|	Parallel actions	:	Is-of-type symbol

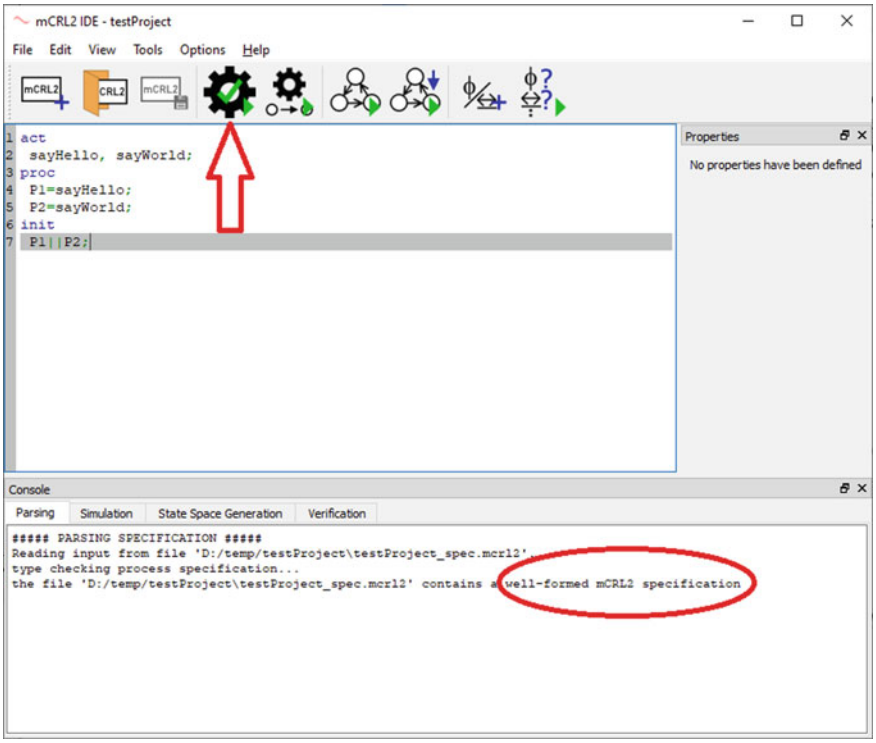


Fig. 2.4 mCRL2 IDE

a position in the trace listed in the right upper window allows the simulation to be restarted from an intermediate position in the trace. We can also see the state space using the icon right of the simulation button in the IDE as shown in Fig. 2.6.

2.2.2 Using the Terminal

Instead of using the IDE, it is also possible to start the tools for simulation, state-space generation, and many other forms of analysis by typing in the tool names using the command line in a terminal. This provides access to far more options of the tools than the IDE makes available. It is also helpful to automate verification tasks. Typically, people start modelling using the IDE and switch to the command line when the models become more complex and advanced.

When using the command line, we need a text editor to write the above code and save this file with the extension "`mcrl2`". To analyse and visualise this model through the toolset, we need to perform the following steps:

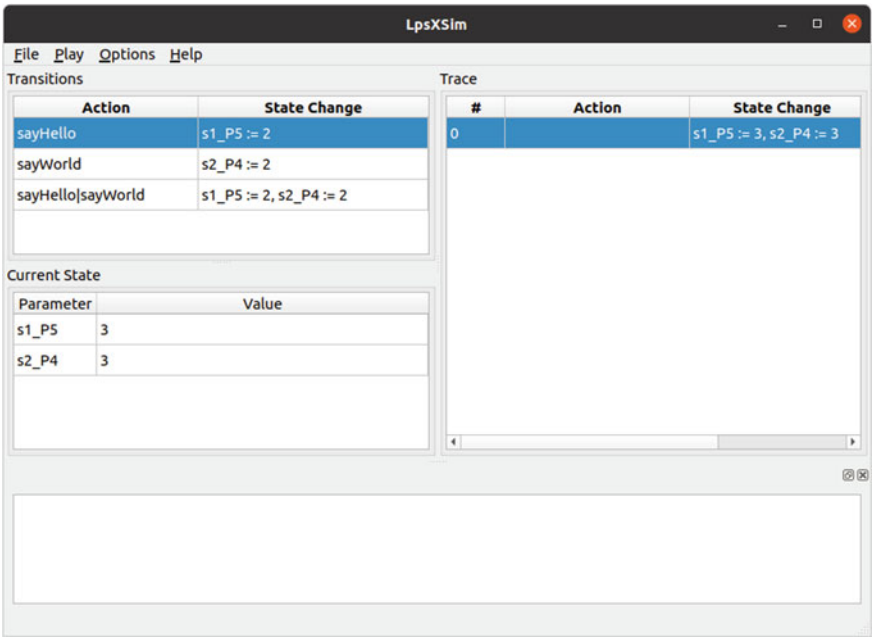


Fig. 2.5 HelloWorld example in a simulator

Linearising: Any process is first transformed to a normal form, namely a linear process, i.e., in `lps` format. Details of linearisation are given in Chap. 8. The tool for this purpose is `mcrl22lps`.

Simulating: A process in LPS format can be simulated using the `lpsxsim` tool as shown in Fig. 2.5.

State-space generation: A linear process can be transformed into a labelled transition system (LTS, see Definition 2.2.1) using the tool `lps2lts`. There are three main formats for labelled transition systems, namely `.lts`, `.aut` and `.fsm`. The `.lts` format is compact, not readable by humans, and contains all information about the process and state space. The two other formats are human-readable. The `.aut` format is used by many other toolsets, and is essentially a long list of transitions. The `.fsm` format is similar but contains information about the states.

Visualisation: There are different ways to visualise labelled transition systems. The major tool for this purpose is `ltsgraph`. Other tools are `ltsview` and `diagraphica`.

As shown below, we can perform the above-mentioned steps in a terminal/command window. It is assumed that the path of the binaries, i.e., *bin* folder/directory, is already set after the installation of mCRL2 (see details in Sect. 1.6).

```
mcrl22lps -v FirstModel.mcrl2 FirstModel.lps
lpsxsim FirstModel.lps
```

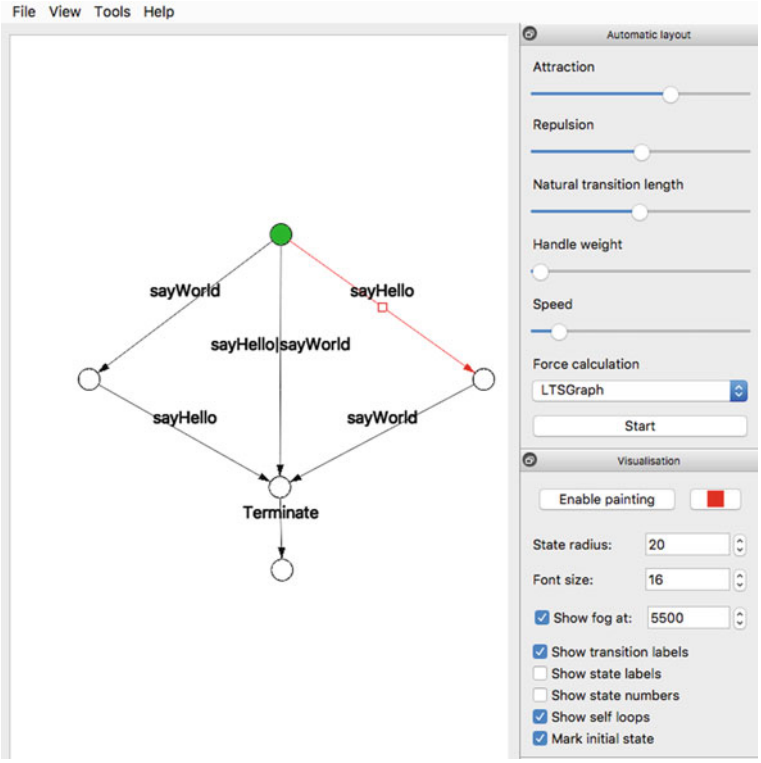


Fig. 2.6 Output of the tool `ltsgraph` for the "Hello World" example

```
lps2lts FirstModel.lps FirstModel.lts
ltsgraph FirstModel.lts
```

2.2.3 State-Space and Actions

A state-space or an LTS is a directed graph where action names are labelled on arcs (edges). See for example, Fig. 2.3b.

Definition 2.2.1 (Labelled transition system, LTS). An LTS is a 5-tuple $(S, Act, \rightarrow, s_0, T)$ where S is a set of states, Act is a set of actions, $\rightarrow \subseteq S \times Act \times S$ is a transition relation, (i.e., a transition can be from one state to another or the same state), s_0 is the initial state, and $T \subseteq S$ is the set of terminal states.

As the set of terminal states plays a minor role in transition systems representing behaviour, it is often omitted.

It is worth noticing that a state space often grows exponentially by increasing the number of parallel processes. If n parallel processes each have k states, the parallel

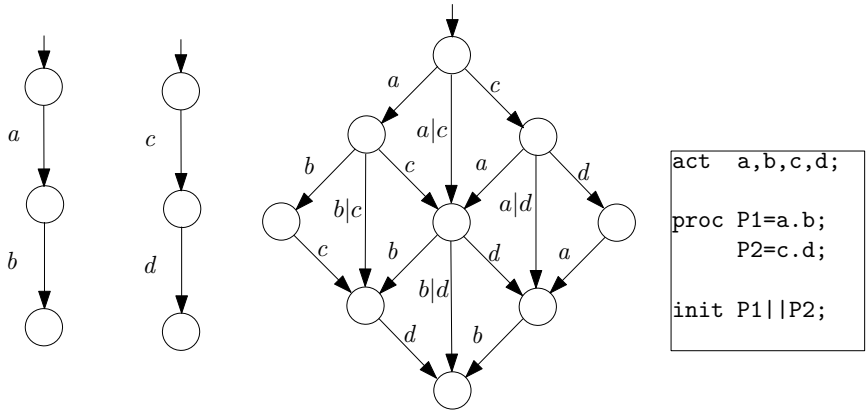


Fig. 2.7 Two parallel processes and their state space

behaviour has k^n states. For example, consider two parallel processes, P_1 and P_2, where P_1 performs an action a followed by another action b and likewise, P_2 performs an action c followed by another action d, as shown in Fig. 2.7 where the behaviour of $P_1 \parallel P_2$ and the mCRL2 code is also given. The two individual processes have 3 states. The parallel composition has $3^2 = 9$ states.

In mCRL2 '$\cdot$' is called the sequential composition operator, so `a.b` means the action a is followed by the action b. We use the '$+$' operator for a choice between two actions. For example, if the behaviour is doing either action a or action b but not both, then it is specified as `a+b`. It is the same as when we choose a cold drink or a hot drink but not both simultaneously. The sequential composition operator has precedence over the choice operator. A few combinations of the choice and sequential composition operators are shown in Fig. 2.8.

Exercise 2.3 Translate the automaton shown in Fig. 2.9 to an mCRL2 specification.

Exercise 2.4 Draw the automaton for process P specified as: `P=a.b+c.d.(e+f).`

Processes can also make recursive calls if a process reaches its initial state again after doing one or more actions. For example, `P=a.b.P` means that the sequence of actions a and b can happen an infinite number of times. The process `P=a.P+b.P` (i.e., equals to `P=(a+b).P`) means that both actions a and b can occur consecutively for any number of times. These examples are shown in Fig. 2.10a.

A process may contain sub-processes. For example, Fig. 2.10b shows that a process can do an action a, and after that it can do an infinite number of actions b, or it can return to the initial state by doing an action c. We can also define this in another way, i.e. by defining data-dependent behaviour, as discussed in Chap. 5.

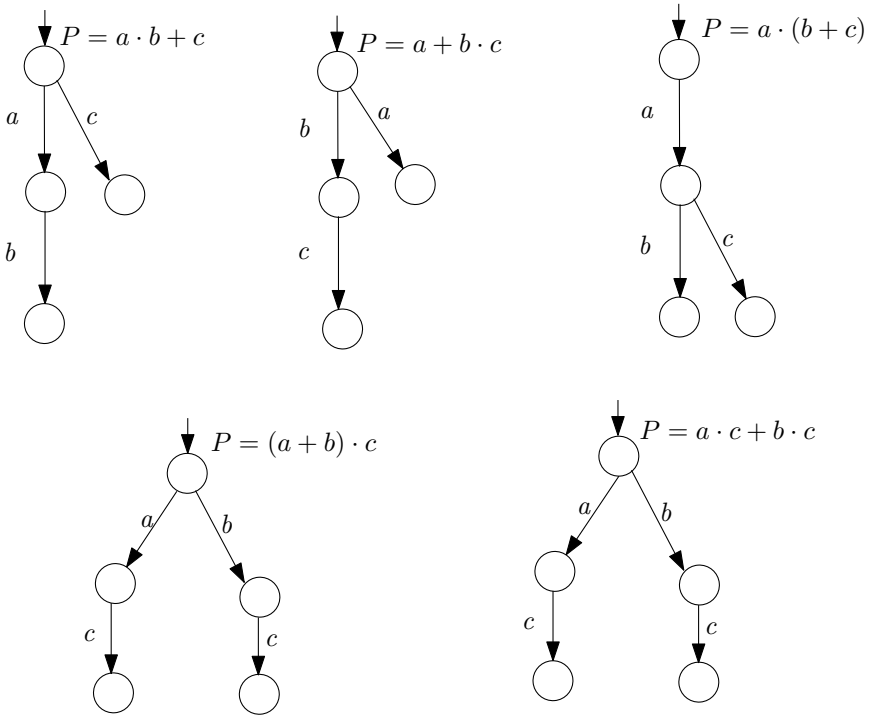


Fig. 2.8 Sequence and choice operators

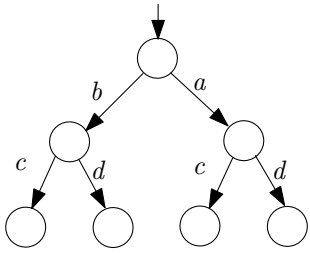


Fig. 2.9 Translating in mCRL2

A state is identified as a deadlock if no more actions can be performed from that state. In Figs. 2.7, 2.8 and 2.9, we can easily identify such states, and there are no deadlocks in Fig. 2.10a, b.

A state is designated as non-deterministic if there are multiple outgoing transitions labelled with the same state leading to different states. For example, in Fig. 2.11a, non-deterministic behaviour of an unreliable light switch is shown by the *switch_on* action; it may remain in an *off* state or it can move to an *on* state. Similarly, in Fig.

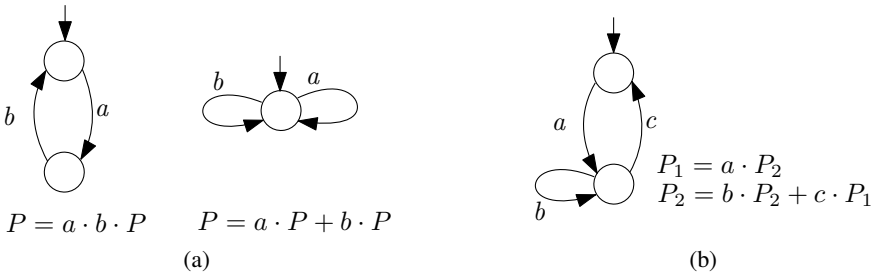


Fig. 2.10 Repeating actions

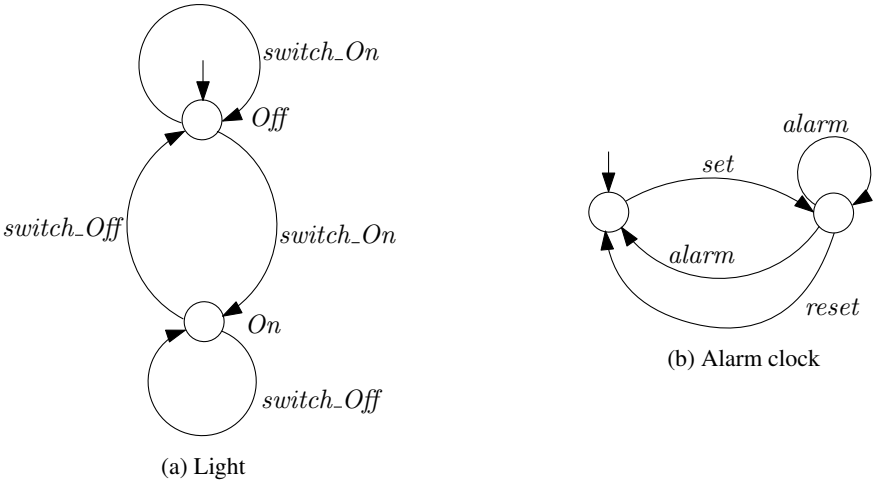


Fig. 2.11 Examples of non-deterministic behaviour [28]

2.11b, a nondeterministic alarm clock is shown which can keep alarming or can go to its initial state by the action, *alarm*.

Exercise 2.5 Express the Fig. 2.11a, b in mCRL2 specifications.

2.3 Using the GUI

This section discusses the GUI (Graphical User Interface) to simulate processes and generate a state space. The GUI is shown in Fig. 1.2. In "File Browser", we can create a directory/folder or mCRL2 files (having extension "mcrl2") by right-clicking. We can edit a file with the built-in editor `mcrl2xi` by a right-click on the mCRL2 file and in the "Editing" menu we select "`mcrl2xi`", i.e., a graphical editor for MCRL2 specifications.

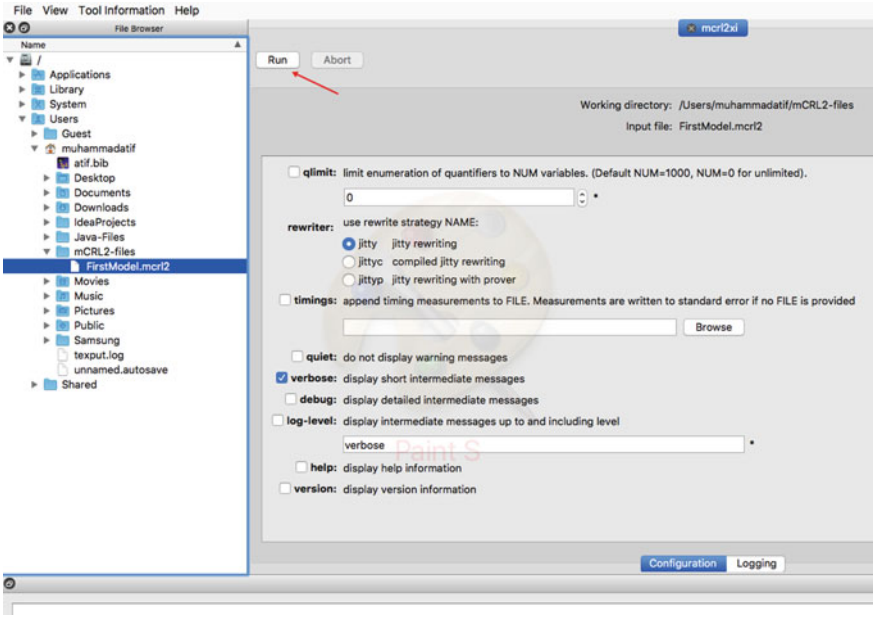


Fig. 2.12 Running the editor `mcrl2xi`

We can find options to run `mcrl2xi` as shown in Fig. 2.12. It is recommended to use default options at this stage and then click on the button "Run" to find the editor as shown in Fig. 2.13 where we can write specifications. To ensure mCRL2's syntax is correct, click on the "parse" icon. If the output window (right lower corner in Fig. 2.13) shows it is a valid specification, then we can close the editor window.

We can convert a valid mCRL2 specification to a linear process specification (LPS) by right-clicking on the mCRL2 file and then selecting "`mcrl22lps`" in the "Transformation" menu. We can give parameters to this tool and select the target filename as shown in Fig. 2.14. We can proceed further by clicking the button "Run", and if conversion to an LPS is successful, we get a green mark "[Ready]" and a message about the LPS file in the output window, as shown in Fig. 2.15.

We can see that a new file with the extension "lps" was created in the file browser. The "lps" file can be converted to a labelled transition system (LTS) by another tool, `lps2lts`. We can run that tool by right-clicking an lps file and selecting "`lps2lts`" in the "Transformation" menu. We can set options and target files for "lts" as shown in Fig. 2.16. At this stage, we recommend moving ahead with the default settings and clicking on the "Run" button to generate an "lts" file.

To view a labelled transition system, we have a tool called "`ltsgraph`" which can be initiated by right-clicking an "lts" file and then in the "Analysis" menu by selecting "`ltsgraph`". By clicking on the button "Run", we can see an LTS graphically, as shown in Fig. 2.6 (similar to Fig. 2.3b). Initially, the graph may have a different shape, but we can either use the "Start" button for its self-organisation

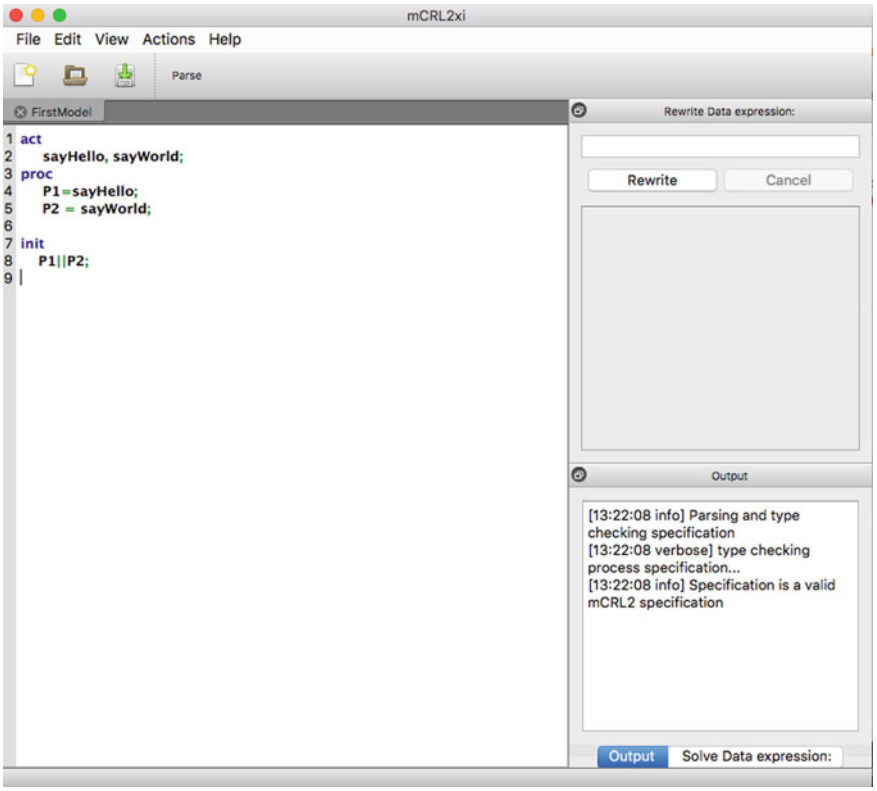


Fig. 2.13 Editing with mcrl2xi

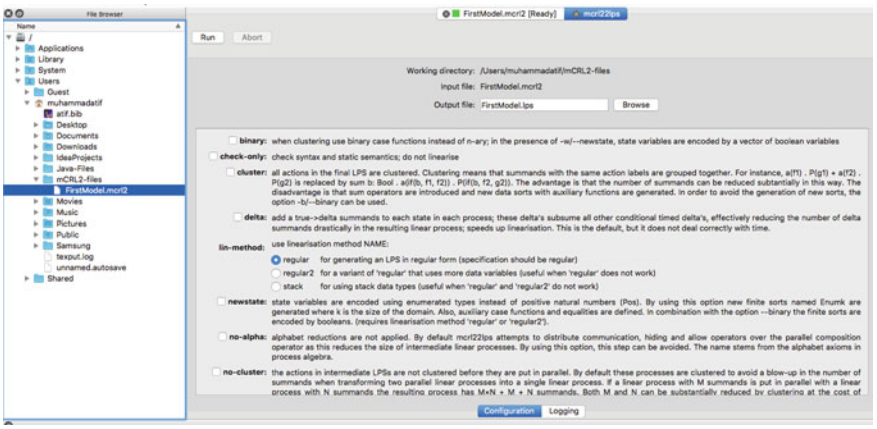


Fig. 2.14 Running the tool `mcrl22lps`

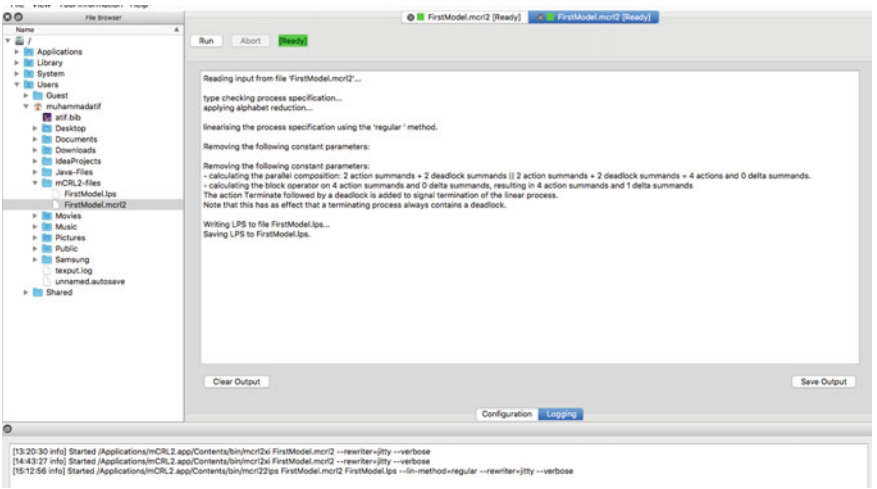


Fig. 2.15 Linearising a process using the tool `mcrl22lps`

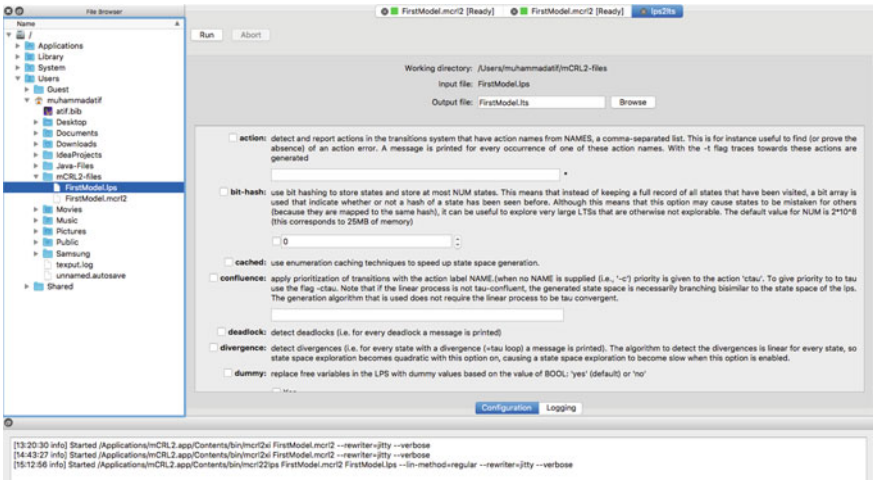


Fig. 2.16 Starting state space generation with `lps2lts`

(states, transitions, and their labels) or set them manually by dragging the states to the desired place with the mouse. Note that by right-clicking an object, such as a state, we can fix it to the canvas, and that state will not move even if the "Start" button is pressed.

The examples given up till now are elementary. But, viewing graphs this way is not recommended for extensive systems. The tool `ltsgraph` has an exploration

mode using which partial graphs can be depicted. Alternatively, we can use another tool `ltsview` where we can depict state spaces as graphical objects and simulate actions. Simulating an LPS can also be done directly using the graphical simulator `lpsxsim`, or the simple command line simulator `lpssim`.

Chapter 3
Communicating Processes



The previous chapter showed simple examples of processes performing actions that communicate with the outside world. This chapter shows how parallel processes can communicate with each other. It also shows that actions can be renamed to the hidden or internal action, which is an action that happens, but cannot be observed. Hiding actions this way helps reduce models' complexity, allowing them better understand their behaviour.

3.1 Communication Among Actions

Actions are events that happen atomically at some instant in time. When processes are put in parallel, actions can co-occur, and we call such a combined action a multi-action. A multi-action is denoted as a sequence of actions with a bar in between, $a_1|\cdots|a_n$. Typically, $a|b$, $a|b|b$ are multi-actions. The order of the actions in a multi-action does not matter. So, $a|b = b|a$ and $a|a|b = b|a|a$. The empty multi-action is denoted as τ, and a single action a is also a multi-action. For example, in Fig. 2.7, all actions of process P_1 (i.e., a, b) have the option to happen at the same time with each action of process P_2 (i.e., c, d) or vice versa. As a result of this, the multi-actions $a|c$, $a|d$, $b|c$, $b|d$ can be observed in Fig. 2.7.

Suppose there are three parallel processes $P1$, $P2$, and $P3$, and each of them has only one action a, b, and c, respectively. In mCRL2, we specify these processes as:

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-3-031-23008-0_3.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0_3

```
act a, b, c, d;

proc P1 = a;
     P2 = b;
     P3 = c;

init P1 || P2 || P3;
```

The set of multi-actions in this case is $\{a|b|c, a|b, a|c, b|c, a, b, c\}$. Here $a|b|c$ means that all three concurrent processes are doing an action at the same time. We can experience it in mCRL2 by generating the state space of this process using the tool `mcrl2ide`.

Let us consider another example where each process has two actions in a sequence, and they are specified as:

```
P1=a.b;
P2=c.d;
P3=e.f;
```

The set of multi-actions for the above processes is:

$$\begin{aligned}\{&a,\ b,\ c,\ d,\ e,\ f,\\ &a|c,\ a|d,\ a|e,\ a|f,\ b|c,\ b|d,\ b|e,\ b|f,\ c|e,\ c|f,\ d|e,\ d|f,\\ &a|c|e,\ a|c|f,\ a|d|e,\ a|d|f,\ b|c|e,\ b|c|f,\ b|d|e,\ b|d|f\}.\end{aligned}$$

Practically speaking, if we put processes in parallel, we are generally not interested in all multi-actions. So, to keep the labelled transition system concise, we restrict the appearance of multi-action, or even actions, using the *allow* operator. The allow operator is denoted in mathematical texts by $\nabla_V(p)$ where V is a set of multi-actions and p a process. In mCRL2 specifications, it is written as `allow` followed by a set of multi-actions and a process. It means only actions that occur in the allow set, an argument of the *allow*, will appear in a formal model. For example, see the following specification where multi-actions are not allowed and hence will not be part of the resulting state space.

```
act a, b, c, d;

proc P1 = a . b;
     P2 = c . d;

init allow( { a, b, c, d },
              P1||P2
          );
```

It is recommended to view the LTS using `ltsgraph`. The result is similar to Fig. 2.7 but without any multi-action consisting of more than one action. If we want to see any of such multi-actions, say $a|c$, in the LTS, we can allow them explicitly, as given below.

```
allow( { a, b, c, d, a|c }, P1||P2 );
```

If we want to give a particular name to a multi-action, we can apply the communication operator. In mathematical typesetting, we write it as Γ_C and in mCRL2 using the keyword `comm`. This operator renames a multi-action into a single action. We say that the actions in the multi-action communicate into this single action. The action b can be seen as sending some data in the process below. The action c can be viewed as receiving it. Both actions communicate to action m, which represents data handover. Note that the data is not explicitly present yet.

```
act a, b, c, d, m;

proc P1 = a . b;
     P2 = c . d;

init allow( { a, b, c, d, m },
       comm( { b | c -> m },
             P1||P2
      ));
```

Note that this is our general mechanism to model communication between processes. We let actions happen simultaneously in a multi-action, and declare that it is a particular communication by renaming such a multi-action to a single action. From the perspective of parallel processes this says that a communication can only take place if all participants, in general the sender and the receiver, are ready to communicate.

This is called synchronous communication. Many systems communicate asynchronously by sending mes- sages via some communication channel that arrive later. Using a separate process modelling the channel asynchronous communication can easily be modelled using synchronous communication. The reverse is much harder. This is why we prefer synchronous communication as the elementary primitive to model communication.

While using multiple renamings in a communication operator, the actions in the multi-actions on the left side of the arrow cannot overlap. It is not allowed to write `comm({ a|b->d, b|c->e},...)`.

Exercise 3.1 Use the `comm` operator to rename two multi-actions shown in Fig. 2.7 and observe the updated model. Why is it not possible to rename all multi-actions in one communication operator? Can you do it with two communication operators?

3.1.1 A Coffee Vending Machine

Let us talk about an example that is a bit more realistic. Consider a simple coffee vending machine that accepts a specific coin and then serves a cup. It interacts with a user who inserts a coin and gets coffee. We can say that the machine and the users are participants or processes of this system. We assume both processes never stop, i.e., they are in an infinite loop. Here is the formal specification of this system.

```
act insCoin, getCoffee, acceptCoin, serveCoffee, coin, coffee;

proc Customer = insCoin . getCoffee . Customer;
     Machine = acceptCoin . serveCoffee . Machine;

init allow( { coin, coffee },
      comm( { insCoin    | acceptCoin  -> coin,
              getCoffee  | serveCoffee -> coffee },
              Customer || Machine
          ));
```

According to this specification, the processes and their concurrent behaviour are shown in Fig. 3.1.

Now we want to add a feature that the machine can switch itself off at its initial state. It means that this machine either accepts a coin or goes to an *off* state by the *switchOff* action, as shown in Fig. 3.2. It is formally specified as:

```
act insCoin, getCoffee, acceptCoin, serveCoffee;
    coin, coffee, switchOff;

proc Customer = insCoin . getCoffee . Customer;
     Machine  = acceptCoin . serveCoffee . Machine
                +
                switchOff;

init allow( { coin, coffee, switchOff },
```

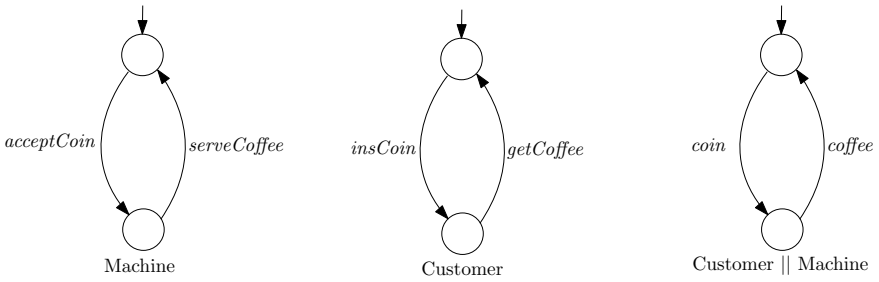


Fig. 3.1 Model of the coffee vending machine

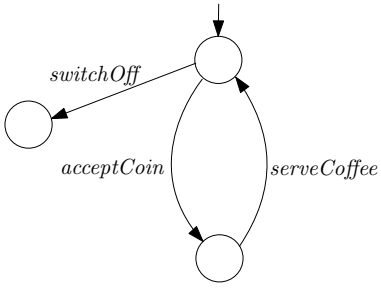


Fig. 3.2 Coffee machine with the *switchOff* option

```
      comm( { insCoin   | acceptCoin  -> coin,
              getCoffee | serveCoffee -> coffee },
               Customer || Machine
          ));
```

Exercise 3.2 Assume a modified version of the coffee machine above-mentioned that can also be switched off after it accepts a coin. Write its formal specification in mCRL2.

Let us add another process for the *Owner*, who can switch off the machine. We can do that by introducing a new process (here) called *Owner* to communicate with the machine. The new process is put in parallel to the above system. Let us say that the system with a customer and a machine specified above is denoted by "`Subsystem`" that works in parallel to the *Owner* process, as shown below.

```
act insCoin, getCoffee, acceptCoin, serveCoffee, coin, coffee,
    switchOff, setOff, getOff;

proc Customer  = insCoin . getCoffee . Customer;
     Machine   = acceptCoin . serveCoffee . Machine
                 +
                 setOff;


     SubSystem = allow(  { coin, coffee, setOff },
                 comm( { insCoin | acceptCoin -> coin,
                         getCoffee | serveCoffee -> coffee
                       },
                       Customer || Machine ));

     Owner     = setOff . Owner;

init allow( { coin, coffee, switchOff },
      comm( { setOff | setOff -> switchOff },
              SubSystem || Owner ));
```

Here the subsystem consists of two processes *Customer* and *Machine*. This subsystem allows a set of only three actions {*coin*, *coffee*, *setOff*} to appear in its model, and when we put this subsystem in parallel to the *Owner* process then we allow {*coin*, *coffee*, *switchOff*} for the whole system.

Note that the action name *setOff* is used for the two processes *Machine* and *Owner*. The renaming $setOff|setOff \rightarrow switchOff$ means that two simultaneous occurrences of the action *setOff* are replaced by the action *switchOff*. Giving the same name is not as such required but illustrates the possibility.

Exercise 3.3 Describe the effect if the Owner process in the above specification is changed from

```
Owner = setOff . Owner;
```

to

```
Owner = setOff;
```

3.2 Deadlock

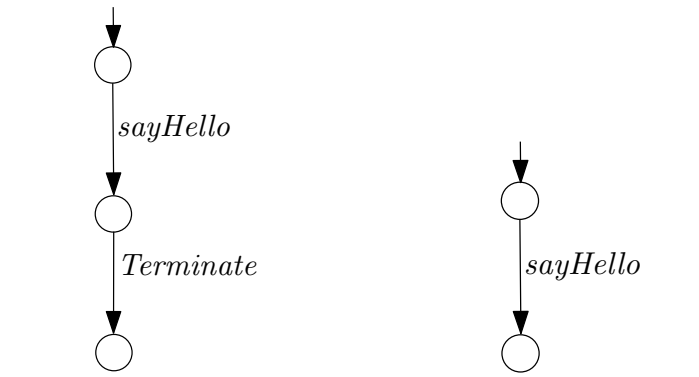
A state is a deadlock when no action can be taken in this state. It is denoted by δ in [28] and written by the reserved word *delta* in mCRL2. It is different from a normal termination, even though at the termination state, there is no further action to perform.

This difference can be observed in Fig. 3.3, where the process *P* has nothing to do after *sayHello* (Fig. 3.3a), so it is showing a normal termination by an extra transition *Terminate*. In Fig. 3.3b, there is no such transition because of the deadlock *delta* put after *sayHello*. It is important to know why we need to add a deadlock in a process definition. To understand this, let us consider the following two processes:

- ```
  Machine = acceptCoin.(serveCoffee+switchOff).Machine
                      +
                      switchOff;
  ```

- ```
  Machine = acceptCoin.(serveCoffee+switchOff.delta).Machine
                      +
                      switchOff;
  ```

In the first definition, *switchOff* can occur an infinite number of times whereas in the second definition it can occur at most once, i.e., due to the *delta*. In the first case, the action *switchOff* terminates after it happened, allowing next actions to happen. In the second case, the *delta* prevents termination and blocks the process after the action *switchOff* happens. When we do not desire a process to terminate, it is always wise to explicitly prevent it by putting a *delta* after the process or action.



(a) Normal termination (b) Deadlock

Fig. 3.3 Differentiating a normal termination and a deadlock

A deadlock can also occur in a system having inter-process communication where the individual processes have no deadlock. This type of deadlock usually appears as a surprise during model-checking where we check that our defined behaviour satisfies desired properties. For example, a deadlock happens in a situation where all processes are waiting for other processes to initiate communication. We discuss such scenarios in Chap. 7 along with the tools and techniques to figure out a deadlock automatically. One simple scenario for deadlock occurrence can be discussed here, where due to the communication of actions we face a deadlock.

Consider a phone which provides some information when, upon dialling, a user presses some specific key. The telephone disconnects a current after providing information. It is possible for a user to change their mind after dialling and disconnect without further communication. The behaviour of these two processes is shown in Fig. 3.4.

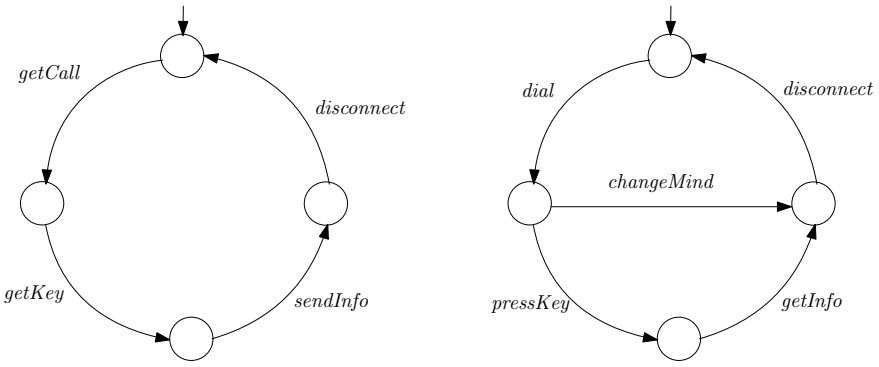
The following actions are supposed to communicate with each other.

- *dial* and *getCall*
- *pressKey* and *getKey*
- *sendInfo* and *getInfo*

We assume that *changeMind* and *disconnect* are atomic actions that are not part of any multi-action. We can see that there is no deadlock in Fig. 3.4a, b as there is some action to perform in every state.

Exercise 3.4 Write the formal specifications of the Telephone and the User processes according to Fig. 3.4 and put them parallel using *comm* and *allow* operators. Give a trace of actions leading to deadlock by observing the LTS.

This protocol works fine every time a user presses a key after dialling but there is a problem when a user changes his mind after dialling and disconnecting the call. In this situation, the user moves to its initial state again after disconnecting while the



(a) Telephone (b) User

Fig. 3.4 Example of two communicating processes

telephone is waiting for a key input. So, the result is a deadlock as the user cannot dial because the phone is waiting for some keypress. Hence, both processes are blocked.

3.3 Synchronous Versus Asynchronous Communication

We talk about synchronous versus asynchronous communication when modelling concurrent behaviour. In synchronous communication, all the synchronised processes progress simultaneously which means their communicating actions occur at the same time. Handshaking is a term that we use to reflect this synchronised behaviour. There can be two or more processes involved handshaking. Synchronous communication is said to be blocking because if a participant is not ready for handshaking but the other is waiting for the same, the process will be blocked. This blocking can be observed by putting the processes in Fig. 3.4 in parallel.

In asynchronous communication there is no direct connection of processes. Rather they use some intermediate processes, so-called channels. A channel establishes a direct connection with one or more processes and then communicates through another direct connection. In this way, a message is held by the intermediate process and sent to the target. This is shown in Fig. 3.5 where there is a process *Channel* communicating directly with a process *Sender* and afterward communicating with another process *Receiver*. During such interaction, we can transfer some message or data too, which we will learn about in Chap. 5. Here, *Sender* is communicating with *Channel* by the multi-action *sendHello*|*rcvMessage* and then the *Sender* can complete proceeding further without blocking. Similarly, the *Receiver* can receive by the multi-action *sendMessage*|*rcvHello* whenever it wants, provided that the *Channel* is ready for that. This is the technique for asynchronous communication between any two processes. Modelling of asynchronous communication is demonstrated with examples in Chap. 10.

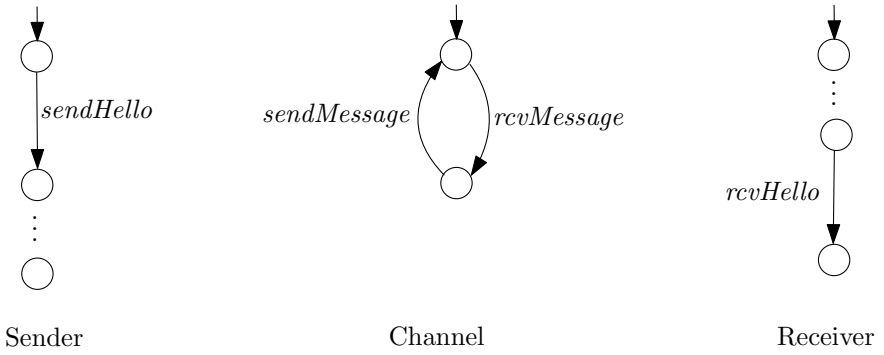


Fig. 3.5 Channel communication

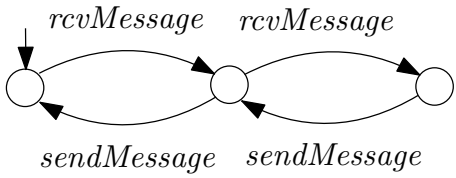


Fig. 3.6 Two-message buffer

Exercise 3.5 Give the complete formal specifications of the processes shown in Fig. 3.5, put them in parallel in such a way that they communicate properly and draw the LTS.

Note that the channel shown in Fig. 3.5 can contain one message at most. We say it has a buffer of one message only. The question is, how to increase this buffer size? If we model the channel as shown in Fig. 3.6 then it can contain one or two messages. Note that it is straightforward to make a buffer with capacity three or four. Later in this book, we discuss the techniques and data structures to compactly increase the buffer size, where it is even possible to specify an unbounded buffer. Note that much more complex channels exist, such as for instance a channel where messages can overtake each other.

Exercise 3.6 Change the model shown in Fig. 3.6, so that it must receive two messages before delivering both of them.

The following specification is inspired by the examples that come up with the mCRL2 installation. This example shows a deadlock due to communication of actions.

```
% Producer consumer example
%
% This is a simple example of a process containing a deadlock.

% Actions
act P_in_start, P_out_started, Q_out_started, Q_in_start;
    c1, c2;

% Processes.
proc P = P_in_start.P_out_started.P;

% Variant of Q that is ok.
%    Q = Q_out_started . Q_in_start . Q;
% Variant of Q causing a deadlock.
     Q = Q_in_start . Q_out_started . Q;

% And now make a network
init allow( { c1, c2  },
       comm( { P_in_start     | Q_out_started -> c1,
               P_out_started | Q_in_start    -> c2
             },
             P || Q ));
```

In the above example, the first action of P communicates with the second action of Q and vice versa. So, none of the processes progress and a deadlock occurs while both processes P and Q do not contain deadlocks themselves.

3.4 Blocking and Renaming Actions

The *allow* operator allows the actions or multi-actions occurring in its argument to appear in a system behaviour and it blocks all others. For example, if we look at the formal specifications of the coffee machine in Sect. 3.1.1, allowed actions are {*coin*, *coffee*, *switchOff*} and an action *setOff*, not in this set, cannot happen. It however plays an important role as a part of a communication.

Sometimes, we need a customised LTS where the effect of an action and all actions occurring after that action are not required. In such a case, we use the *block* operator which prevents particular actions from happening. Here is the example of the coffee machine in which the action *setOff* is blocked explicitly.

```
act insCoin, getCoffee, acceptCoin, serveCoffee, coin, coffee,
    switchOff, setOff, getOff;

proc Customer = insCoin . getCoffee . Customer;
     Machine = acceptCoin . serveCoffee . Machine
               +
               setOff;

     SubSystem = allow( { coin, coffee, setOff },
                    comm( { insCoin   | acceptCoin  -> coin,
                            getCoffee | serveCoffee -> coffee },
                     Customer || Machine ));

     Owner = setOff . Owner;


init allow( { coin, coffee, switchOff },
       comm( { setOff | setOff -> switchOff },
         block ( { setOff },
           SubSystem || Owner )));
```

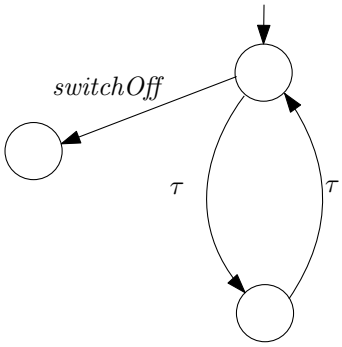
In renaming, we just give a different name to an action. For this purpose, there is a *rename* operator. It can be used to rename one action into another. For example, in the following code the actions a and b are allowed. Then the action b is blocked, only allowing a. Subsequently, the action a is renamed to c.

```
act a, b, c;

proc P  = a . b . P2;
     P2 = a + b . P;

init rename ( { a -> c },
        block( { b },
          allow ( { a , b }, P)));
```

Fig. 3.7 Hiding actions



3.5 Hiding Actions

Hiding an action means an action is renamed to τ. The action τ is an action that cannot by itself be observed. For instance in a coffee machine, we may order coffee, the coffee is brewed, and ten poured into a cup. We may see the action of ordering, and the action of pouring. But it may be that we do not see that the machine is brewing. For us this is a τ action. It happens, but we cannot observe it. The internal action is also called the hidden action. Recall that the τ is also the empty multi-action.

As an example, consider the following specification which is also discussed in Sect. 3.1.1 but here it is extended with the *hide* operator.

```
act insCoin, getCoffee, acceptCoin, serveCoffee;
    coin, coffee, switchOff;

proc Customer = insCoin . getCoffee . Customer;
     Machine  = acceptCoin . serveCoffee . Machine
                +
                switchOff;

init hide ( { coin, coffee },
       allow( { coin, coffee, switchOff },
         comm( { insCoin   | acceptCoin  -> coin,
                 getCoffee | serveCoffee -> coffee },
           Customer || Machine )));
```

So, with τ actions, Fig. 3.2 becomes Fig. 3.7. Hiding actions is helpful to simplify behaviour. We can concentrate on the remaining visible actions by hiding less relevant actions. For instance, when we are interested in finding a specific action, like *error*, we can hide all other actions by renaming them to τ. By applying a behavioural reduction, we can reduce a state-space without affecting the essence of its behaviour. Behavioural equivalence and behavioural reductions are discussed in the next chapter.

Chapter 4
Behavioural Equivalences



A labelled transition system can be converted to a more compact one while preserving the essence of its behaviour. This process is called state-space reduction. If two systems have the same behaviour as another, we say that both systems are behaviourally equivalent. For example, the process $a + a$ is equivalent to a as both can essentially only perform a single action a.

There are different types of behavioural equivalences because we can observe behaviour in different ways. Therefore, there are different techniques to determine an equivalence. In this chapter, we discuss these and learn how the mCRL2 toolset can help us to get a reduced state-space modulo different behavioural equivalences.

4.1 Trace Equivalence

A trace is a sequence of actions (also called an execution or a run) from a state. It shows the action that can be performed consecutively from that state. It is an execution of zero or more actions. If no action is taken, we call it an empty trace, denoted by ϵ. If no state is mentioned explicitly, the trace is generally supposed to start in the initial state.

Two systems are trace equivalent (also called strong trace equivalent) if we get the same set of traces from both of them. For example, all the LTSs in Fig. 4.1 are trace equivalent because they generate the same set of traces, i.e., $\{\epsilon, a, a \cdot b\}$.

This technique is very useful when we are interested in knowing the presence or absence of some particular action sequences. For example, before every *logout* action, a *login* action must exist. In Fig. 4.1, before every action b, there is an action a. Note that a process a is not trace equivalent to the process $a \cdot \delta$ because the set of traces for the first one is $\{\epsilon, a, a \cdot \mathit{Terminate}\}$ while it is $\{\epsilon, a\}$ for the other.

Definition 4.1.1 (Trace set) In an LTS $(S, A, \rightarrow, s, T)$ the set of traces *Traces(t)* for some state $t \in S$ is the minimal set that satisfies:

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0_4

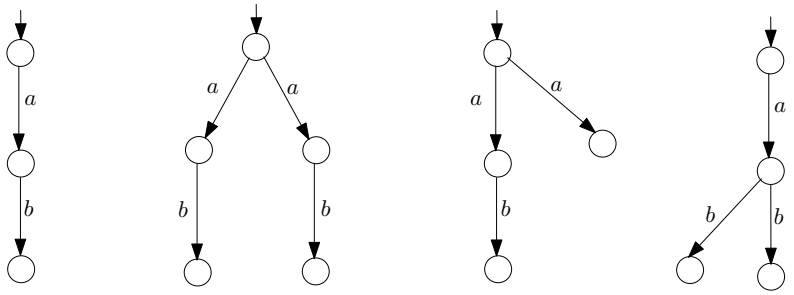
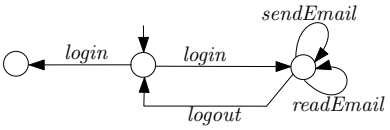


Fig. 4.1 Trace equivalent systems




```
act login,logout,sendEmail,readEmail;

proc Person=login.delta+login.Emailing;
     Emailing= logout.Person
               + sendEmail.Emailing
               + readEmail.Emailing;

init Person;
```

Fig. 4.2 Repetition of actions

1. $\epsilon \in Traces(t)$ where ϵ is an empty trace, i.e., a trace having no action,
2. $Terminate \in Traces(t)$ iff $t \xrightarrow{Terminate} t'$ where $t' \in T$, and
3. If $t \xrightarrow{a} t'$ for some $t' \in S$ and $\sigma \in Traces(t')$ then $a\sigma \in Traces(t)$.

If a process has a repetition of actions like the one shown in Fig. 4.2, the trace set is infinitely large. It is given by

$\{\ \epsilon,\ login,\ login \cdot sendEmail,\ login \cdot readEmail,\ login \cdot sendEmail,$
$login \cdot sendEmail \cdot sendEmail,\ login \cdot sendEmail \cdot readEmail,\ login \cdot sendEmail \cdot logout,$
$login \cdot readEmail \cdot sendEmail,\ login \cdot readEmail \cdot readEmail,\ login \cdot readEmail \cdot logout,$
$\ldots\ \}$

Exercise 4.1 Give the set of traces for Fig. 4.3. Also write an mCRL2 specifications for this behaviour.

We can automatically get a trace equivalent LTS that may have a smaller number of states and/or transitions. All the system behaviours shown in Fig. 4.1 are trace equivalent where the left-most has the smallest number of states and transitions. In mCRL2, `ltsconvert` is the tool used to get a trace equivalent LTS. If we apply this tool to the LTSs in Fig. 4.1, we get the left-most LTS as an output. With the option `-equivalence=trace` this tool preserves the property of trace equivalence. We discuss a few more equivalences in this chapter.

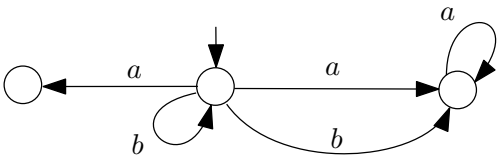


Fig. 4.3 Exercise trace equivalence

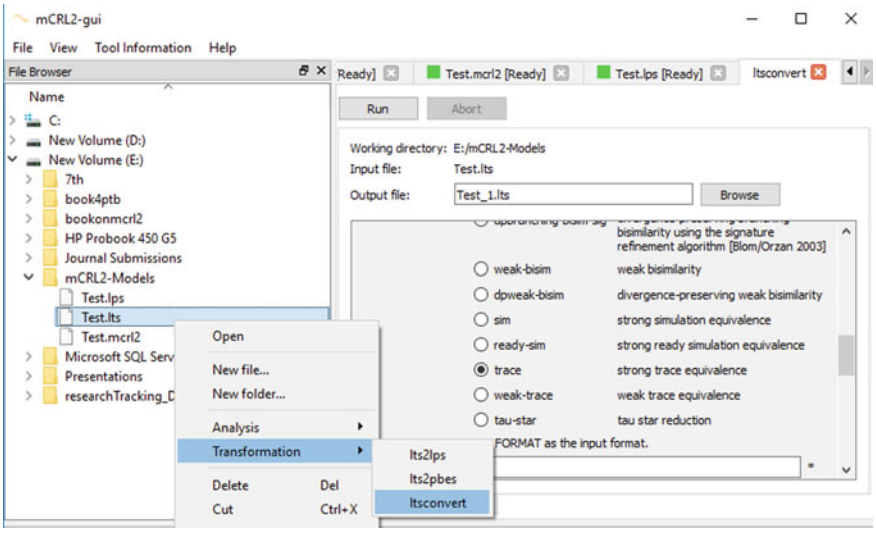


Fig. 4.4 Using *ltsconvert* in GUI

We can apply `ltsconvert` using the GUI. To do that, right-click on an LTS file and in the *Transformation* menu select `ltsconvert`. In the configuration, we can select *strong trace equivalence* and provide a name to the target LTS as shown in Fig. 4.4. We can see feedback from the tool in the output window. The reduced state-space can be viewed using `ltsgraph`. An automatic comparison can be performed using another tool `ltscompare`. To use `ltscompare`, right-click on any LTS file in GUI and select this tool from the menu *Analysis*. In the configuration, select another LTS for comparison and choose *strong trace equivalence* (as the equivalence type) to determine whether behaviourally both LTSs are trace equivalent or not.

We can use these tools on the terminal window as well. Here is a sample usage:

```
ltsconvert source.lts target.lts --equivalence=trace --verbose
ltsgraph target.lts
ltscompare source.lts target.lts --equivalence=trace --verbose
```

Exercise 4.2 Determine which of the labeled transition systems given in Fig. 4.5 are strong trace equivalent.

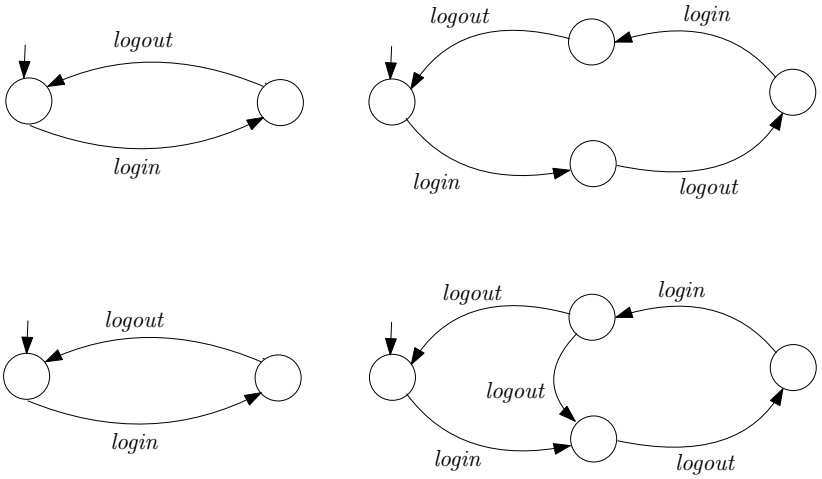


Fig. 4.5 Strong trace equivalent

Exercise 4.3 Trace equivalence does not preserve deadlock freedom. Give two trace equivalent transition systems, one with and one without a deadlock.

4.1.1 Weak Trace Equivalence

A trace is called a weak trace if all the hidden actions τ are ignored. For example, the trace $\tau \cdot a \cdot \tau \cdot b$ is converted to its equivalent weak trace $a \cdot b$. Two systems are behaviourally *weak trace equivalent* if they have the same set of weak traces.

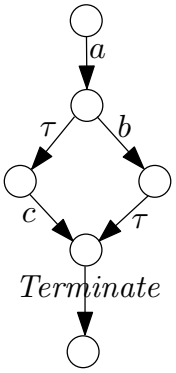
The tool `ltsconvert` converts a system to its weak-trace-equivalent system, automatically. In mCRL2 GUI, right-click on an LTS file and select `ltsconvert` in the menu *Analysis*. In the configuration, select *weak trace equivalence* and click on the "Run" button to obtain a weak trace equivalent LTS. The LTSs shown in Fig. 4.6a, c are weak trace equivalent.

We can observe this conversion on a command prompt as:

```
ltsconvert test.lts test_wtrc.lts --equivalence=weak-trace
--verbose
```

Sometimes applying a weak trace equivalence reduction leads to surprising results. Simply removing τ does not work if it changes system behaviour. For example, see the conversion from Fig. 4.7a, b where τ is replaced with an action from an infinite loop so that the actual behaviour is not changed.

Exercise 4.4 Consider the Fig. 4.7a and draw its weak trace equivalent LTS if the action c is not there.



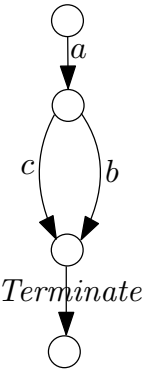
(a) A system behaviour

```
act  a,b,c,d;

proc  P=a.(d.c+b.d);

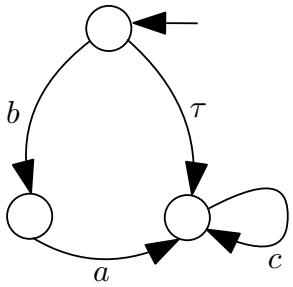
init allow({a,b,c,d},
      hide({d},
        P
           ));
```

(b) mCRL2 specifications

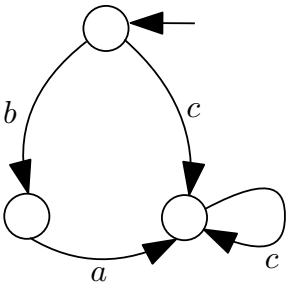


(c) Weak trace equivalent system

Fig. 4.6 Weak trace example



(a) A system with a τ action



(b) Weak trace equivalent system

Fig. 4.7 Weak trace equivalent LTSs

Getting and printing counterexamples

Comparing two LTSs through `ltscompare` gives a result as either *true* or *false*. To investigate the result, in particular when it is *false*, we generate and print a counterexample. Using the GUI, we can generate a counterexample w.r.t. some equivalences by selecting "counter-example" in the configuration of the `ltsconvert`. On the command prompt or in the terminal window we can do the same as:

```
ltscompare Test1.lts Test2.lts --counter-example -eweak-
trace -v
```

Here the flag `-eweak-trace` is a shorthand for `-equivalence=weak-trace`. If both LTSs are not equal then a file "Counterexample0.trc" is automatically generated which contains a trace of actions witnessing the result. We can view the trace using the tool `tracepp` to pretty print the trace.

```
tracepp Counterexample0.trc
```

In the GUI, we can use `tracepp` by right-clicking on a "trc" file.

4.2 Strong Bisimulation

Strong bisimulation is a behavioural equivalence that relates transition systems when their behaviour cannot be distinguished even under severe observation strategies. Two states in two transition systems are strongly bisimilar if every action that can be performed in one state can be simulated by some outgoing action in its counterpart.

Definition 4.2.1 (Strong bisimulation) Consider two LTSs $(S, A, \rightarrow, s_0, T)$ and $(S', A', \rightarrow', s_0', T')$. A relation $R \subseteq S \times S'$ is called a (*strong*) *bisimulation relation* iff for any two states $s \in S$, $s' \in S'$ and action a:

- if $s \xrightarrow{a} t$ and $s \; R \; s'$ then there exists a state $t' \in S'$ such that $s' \xrightarrow{a} t'$ and $t \; R \; t'$, and
- if $s' \xrightarrow{a} t'$ and $s \; R \; s'$ then there exists a state $t \in S$ such that $s \xrightarrow{a} t$ and $t \; R \; t'$.
- Both states s and s' or either terminating or not: $s \in T$ iff $s' \in T'$.

Two states $u \in S$ and $u' \in S'$ are (*strongly*) *bisimilar* iff there is a relation R such that uRu'. The two LTSs are (*strongly*) *bisimilar* iff their initial states s_0 and s_0' are strongly bisimilar.

As an example consider the processes a and $a + a$ as shown in Fig. 4.8. These are bisimilar where the dashed lines indicate the bisimulation relation R between states. The formal definition of strong bisimulation comes from Milner [47], but see also [28]. Let us discuss how bisimulation can be used in the toolset in mCRL2, in particular, how automatic conversion to a minimal but strongly bisimulation equivalent LTS can be performed, and how it can be determined whether two LTSs are strongly bisimilar.

For example, in Fig. 4.9 only the action a is possible from the initial state of each system regardless of the number of a actions. So, the initial states are bisimilar. After every action a, the states are also bisimulating because only action b is possible from

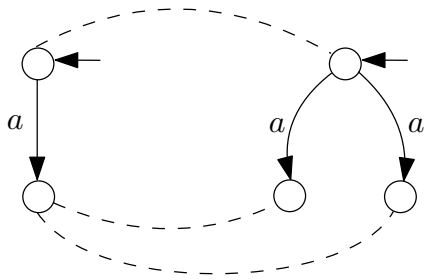


Fig. 4.8 Strongly bisimilar systems

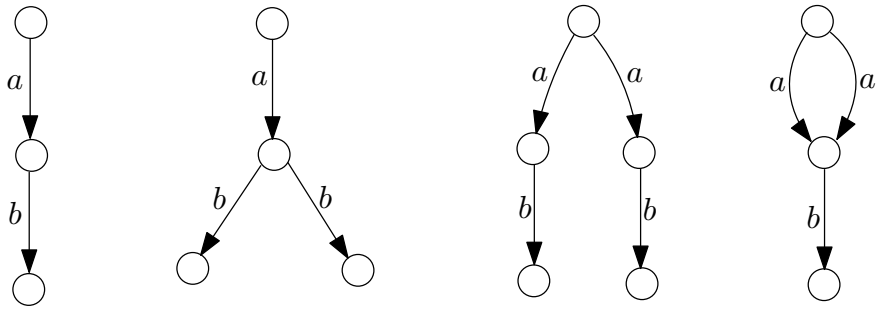


Fig. 4.9 All four systems are strongly bisimilar systems

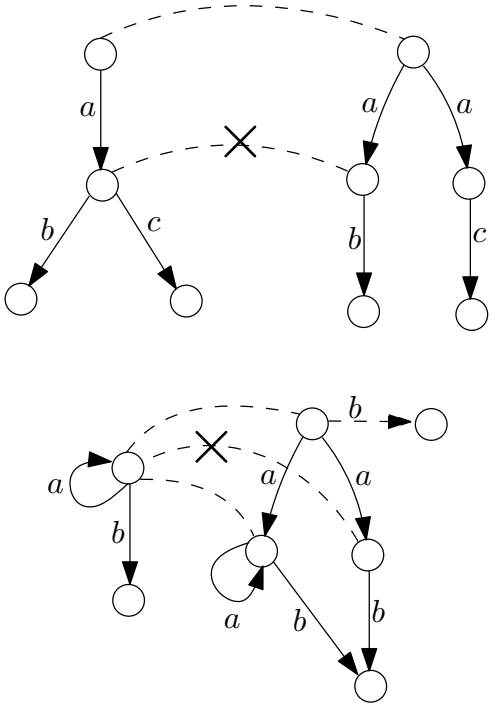


Fig. 4.10 Non-bisimilar systems

there. Every action b leads to a deadlock, so terminal states are also bisimulating. As all states are in a bisimulation relation in each system, we can conclude that these states are strongly bisimilar. As the initial states are related, the systems are bisimilar.

Examples of systems which are not bisimilar are shown in Fig. 4.10.

Exercise 4.5 Are the systems in Fig. 4.1 bisimilar? Solve the same question for Fig. 4.5.

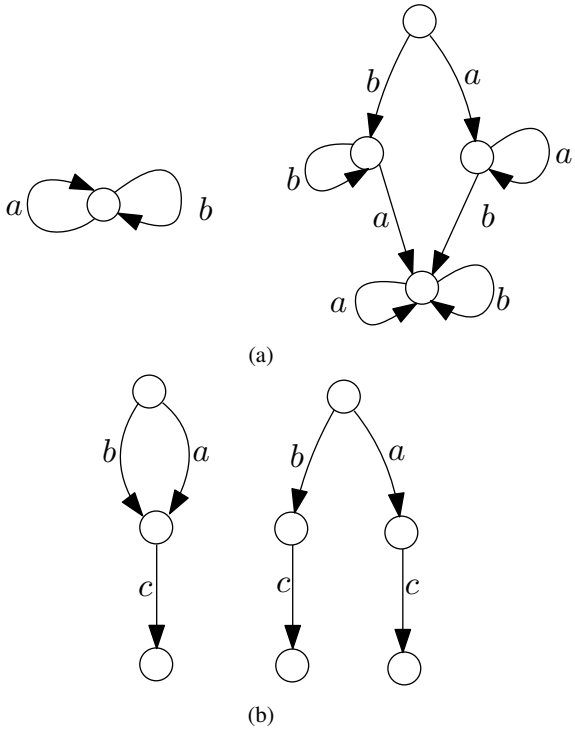


Fig. 4.11 Transition systems for Exercise 4.6

Exercise 4.6 Are the systems given in Fig. 4.11 strongly bisimilar?

The tools discussed in Sect. 4.1, i.e., `ltsconvert` and `ltscompare`, can also be used for converting a system to its minimal bisimilar LTS and for comparing two systems for bisimulation. To observe automatic conversion to a bisimilar LTS, let us try to answer Exercise 4.6 with the help of the toolset. A formal specification in mCRL2 of the large system in Fig. 4.11a is:

```
act
     a, b, c;
proc
     P =  b . P1 + a . P2;
     P1 = b . P1 + a . P4;
     P2 = a . P2 + b . P4;
     P4 = a . P4 + b . P4;
init
     allow( { a, b, c }, P);
```

The tool `mcrl22lps` by default attempts to simplify the specification (α reduction [5]), so the Fig. 4.12 is the system that is obtained by default. We can skip this

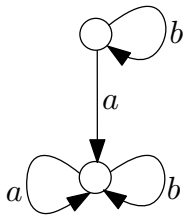


Fig. 4.12 With α reduction

reduction by selecting "no-alpha" in the configuration of `mcrl22lps` to obtain the LTS shown in Fig. 4.11a. On the terminal command window, we use `-no-alpha` at the command-line arguments. For example:

```
mcrl22lps test.mcrl2 test.lps --no-alpha --lin-method=regular
lps2lts test.lps test.lts -v
```

In both cases, whether α reduction is applied or not, we get an LTS which can be converted to its bisimilar equivalent using `ltsconvert`. Applying this tool is discussed in Sect. 4.1 but here we select "strong bisimilarity" in the configuration of this tool. In the terminal window, we can apply this tool as:

```
ltsconvert test.lts test_1.lts --equivalence=bisim --verbose
```

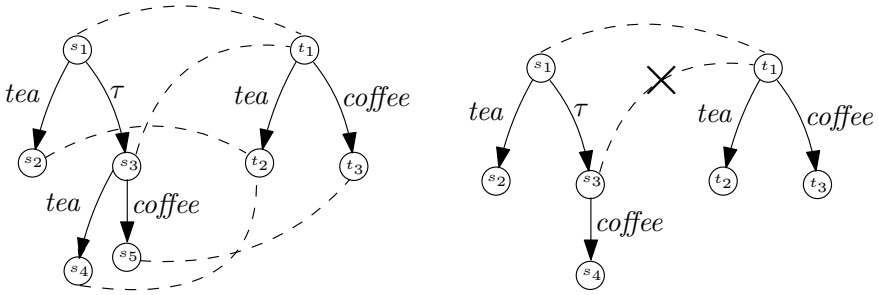
There are different algorithms for converting to a bisimilar LTS that have different time complexities, but all yield the minimal LTS modulo bisimulation. The resulting LTS can be viewed using `ltsgraph` which shows the small LTS shown in Fig. 4.11a. So, this result shows a significant reduction in state-space which is behaviourally the same. We can use `ltscompare` to determine whether two LTSs are bisimilar using the following command.

```
ltscompare test_1.lts test.lts --equivalence=bisim --verbose
```

4.3 Branching Bisimulation

Branching bisimulation is an equivalence that preserves the behaviour under a wide range of observations, like strong bisimulation, but in addition it takes into account that internal actions τ cannot be observed, and therefore it can make substantially larger reductions.

Two states are branching bisimilar, if the actions that can be done in one state can also be done in the other, possibly preceded by a number of (non-observable) τ-actions, and the resulting states should again be branching bisimilar. A τ-action can also be simulated by not doing an action at all, provided the state reached after



(a) Branching bisimilar behaviours (b) Non branching bisimilar behaviours

Fig. 4.13 Examples for branching bisimilarity

the tau is branching bisimilar to the other state. A technical requirement is that if an action a is simulated by a sequence $\tau \cdots \tau \cdot a$, then the state directly before the a must be branching bisimilar to the first initial state. Two systems are called branching bisimilar if we can relate their initial states. Fig. 4.13a is an example of branching bisimulation.

Most of the τ actions are removed when a branching bisimulation reduction is applied. For example, consider a process $a \cdot b$, i.e., an action a is followed by an action b. All the following processes are branching bisimilar to $a \cdot b$.

- $a \cdot \tau \cdot b$.
- $a \cdot b \cdot \tau$.
- $\tau \cdot a \cdot b$.
- $\tau \cdot \tau \cdot a \cdot \tau \cdot \tau \cdot b \cdot \tau \cdot \tau$.

Definition 4.3.1 (Branching bisimulation) Let $(S, A, \rightarrow, s_0, T)$ and $(S', A', \rightarrow', s'_0, T')$ be two LTSs. A relation $R \subseteq S \times S'$ is called a *branching bisimulation relation* iff for any two states $s \in S, t \in S'$ and action a such that sRt, it holds that:

1. If $s \xrightarrow{a} s'$, then
   - either $a = \tau$ and $s'Rt$ or
   - there is some sequence $t \xrightarrow{\tau} \cdots \xrightarrow{\tau} t'$ of (zero or more) τ-transitions such that sRt' and $t' \xrightarrow{a} t''$ with $s'Rt''$.
2. Symmetrically, if $t \xrightarrow{a} t'$, then
   - either $a = \tau$ and $t'Rs$ or
   - there is some sequence $s \xrightarrow{\tau} \cdots \xrightarrow{\tau} s'$ of (zero or more) τ-transitions such that $s'Rt$ and $s' \xrightarrow{a} s''$ with $s''Rt'$.
3. If s is a terminating state, then there is a sequence of (zero of more) τ-transitions $t \xrightarrow{\tau} \cdots \xrightarrow{\tau} t'$ such that sRt' where t' is also a terminating state.

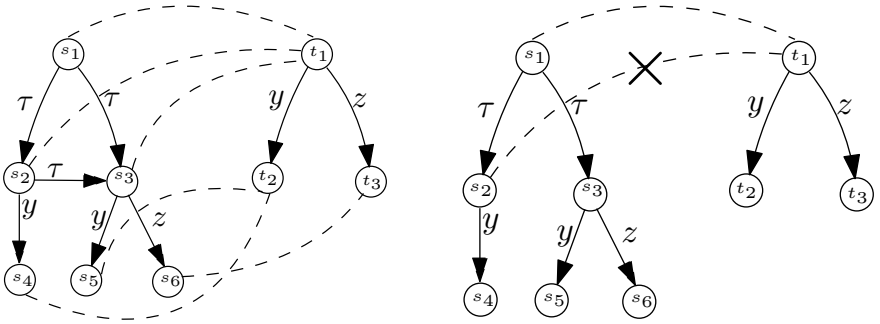


Fig. 4.14 Construction a branching bisimulation relation can be successful or fail

4. Again symmetrically, if t is a terminating state, then there is a sequence of (zero of more) τ-transitions $s \xrightarrow{\tau} \cdots \xrightarrow{\tau} s'$ such that $t\,R\,s'$ where s' is also a terminating state.

Two states are *branching bisimilar* iff they are related by a branching bisimulation relation R.

In Fig. 4.13a, the state s_1 relates to the state t_1 because at state s_1 the action τ can be mimicked by not doing a step in t_1. The action *tea* can directly be simulated in t_1. In state t_1 the action *tea* can directly be simulated in state s_1. The action *coffee* can be simulated in s_1 by the sequence $\tau \cdot \mathit{coffee}$. Note that the intermediate state s_3 is related to t_1 and the final state s_5 is related to t_3. All other actions in all other states also need to be carefully checked.

The states s_1 and t_1 are not branching bisimilar in Fig. 4.13b. Moving from state s_1 to state s_3 through an internal action means that state s_3 must be related to state t_1 as there is no action τ in t_1 that can simulate the τ in s_1. But this would mean that states s_3 and t_1 must be in the branching bisimulation relation. However, in state t_1, there is a transition labelled with *tea* that cannot be done in state s_3. Hence, s_3 and t_1 cannot be branching bisimilar. But this means that s_1 and t_1 cannot be branching bisimilar either.

Another example of branching bisimilarity/non-bisimilarity is shown in Fig. 4.14.

Exercise 4.7 Are the systems shown in Fig. 4.15 branching bisimilar?

The examples we discussed so far in this section have τ in one system but not in the other. According to Definition 4.3.1, we compare a state after the action τ with the initially related state of the other LTS in such a case. Let us discuss two LTSs where both sides have τ actions. For example, Fig. 4.16. We need to carefully observe the behaviour after a τ. Either we need that the action τ is mimicked by one or more actions τ, or the target state of the first system is related to the start state of the simulating one. An example of finding a bisimilar branch is shown in Fig. 4.17

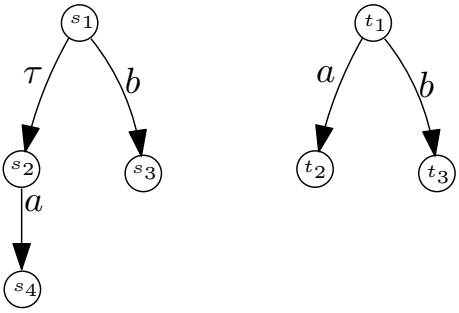


Fig. 4.15 Transition systems for Exercise 4.7

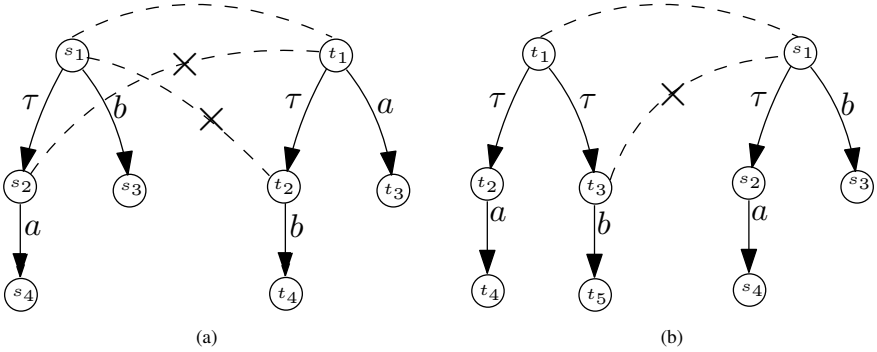


Fig. 4.16 Branching bisimilarity with non-deterministic τ's

where for instance the τ from state s_1 to state s_3 is mimicked by the τ from state t_1 to state t_3.

We can use the mCRL2 toolset to automatically get a minimal branching bisimilar LTS using the tool `ltsconvert`. For an experiment, let us specify the model shown on right of Fig. 4.17. In the syntax of mCRL2 it is specified as:

```
act a, b;

proc P = tau . tau . b + tau . a;

init P;
```

We can generate its LTS and apply `ltsconvert` in the GUI as discussed in Sect. 4.1 but with the configuration of *branching-bisim*. On the terminal windows we can use this tool as shown below where `Test.lts` is the source LTS and `Test_brBisimilar.lts` is the target LTS. The flag `-v` is a shorthand for verbose output.

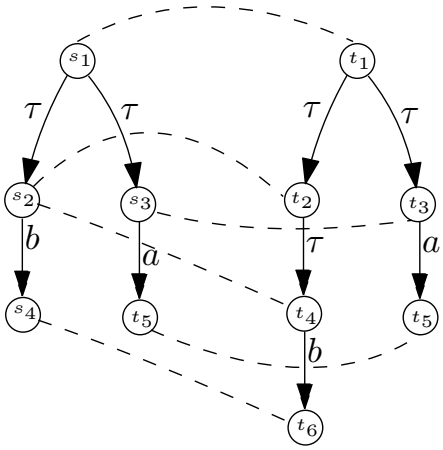


Fig. 4.17 Non-deterministic τ's

```
ltsconvert Test.lts Test_brBisimilar.lts -ebranching-bisim -v
```

We can view `Test_brBisimilar.lts` using `ltsgraph` in which we can notice that the branch $\tau \cdot \tau \cdot b$ is converted into $\tau \cdot b$. We can determine whether two LTSs are branching bisimilar by giving the single command:

```
ltscompare Test1.lts Test2.lts -ebranching-bisim -v
```

4.4 Divergence Preserving Branching Bisimulation

Branching bisimulation has the property that it does not preserve τ-loops. A process defined by

```
proc P = tau . P + a . delta;

init P;
```

has a τ-loop. Branching bisimulation reduction does remove this τ-loop as the process without a τ-loop is branching bisimilar. However, this is not always desired, for instance when one wants to be sure that the action a will surely be done. In the variant with the τ-loop this is not the case, as the τ can be chosen always, whereas in the reduced variant there is no alternative to doing the action a. The possibility to do an infinite sequence of τ's in a state is called a *divergence*.

Divergence preserving branching bisimulation is exactly the same as branching bisimulation, except that it guarantees that τ-loops will remain in place. It is applied to an LTS using the following command

```
ltsconvert Test.lts Test_brBisimilar.lts -edp-branching-
bisim -v
```

Divergence preserving branching bisimulation can also be used for comparisons.

Chapter 5
Data Types and Data-Dependent Behaviour



Data plays a vital role in the formal specification of processes. A process takes a decision according to the data it has. A process's data can be its ID, state and a record of sent/received messages etc. Usually, in the complex models, processes have data-dependent behaviour.

A process gets data through its parameters upon initialisation or by interacting with other parallel processes through multi-actions. In mCRL2, there is no way to declare global variables that are accessible to all processes, and this means that processes cannot communicate through shared variables. If somehow, the global variables are required, we can add a process to exchange data anytime with any process, and in this way that process behaves like a container of global variables. The built-in data types in mCRL2 are given in Table 5.1.

In mCRL2, the following data types are built-in.
We can define our own data types as well, like a data type for weekdays and a data type for the states of a process as discussed in Sect. 5.6.

5.1 Sending/Receiving Data

Let us revisit the "coffee machine" example discussed in Sect. 3.1.1 to understand sending/receiving of data. We change behaviour of both the user and the machine to accept coins of 1 and 2 cents. In the following specifications, the actions *insCoin*, *acceptCoin* and *coin* are declared to send/receive a value of type `Nat` ($\mathbb{N}$).

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-3-031-23008-0_5.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0_5

Table 5.1 Data types in mCRL2

Type name	Meaning	Symbol	Range
`Pos`	Positive integers	$\mathbb{N}^+$	$1, 2, 3, 4, 5, \ldots$
`Nat`	Natural numbers	$\mathbb{N}$	$0, 1, 2, 3, 4, 5, \ldots$
`Int`	Integer	$\mathbb{Z}$	$\ldots, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, \ldots$
`Bool`	Boolean	$\mathbb{B}$	*true, false*
`Real`	Real	$\mathbb{R}$	Rational Numbers approximation, i.e., $\frac{p}{q}$ where $p \in \mathbb{Z}, q \in \mathbb{N}^+$

```
act insCoin, acceptCoin, coin: Nat;
    getCoffee, serveCoffee, coffee;

proc Customer = insCoin(1) . insCoin(2) . getCoffee . Customer;
     Machine  = acceptCoin(1) . acceptCoin(2) . serveCoffee . Machine;

init allow( { coin, coffee },
       comm ( { insCoin   | acceptCoin  -> coin,
                getCoffee | serveCoffee -> coffee },
                  Customer || Machine
           ));
```

The actions *insCoin* and *acceptCoin* communicate with each other based on the same coin values. Two actions can communicate only if their arguments are exactly the same. So, *insCoin*(1) communicates with *acceptCoin*(1) and likewise, *insCoin*(2) communicates with *acceptCoin*(2). This is why the process *Machine* does not work if the user first inserts a coin of 2 and then of 1. However, practically a receiver does not know in advance about the value it is going to receive. So, a receiver in the above example is supposed to be ready for any coin. We can do that with the choice (+) operator. A somewhat better definition of the coffee machine is:

```
Machine = (acceptCoin(1) + acceptCoin(2)) .
          (acceptCoin(1) + acceptCoin(2)) .
          serveCoffee . Machine;
```

This definition still requires improvement, so that the machine not only can accept other coins as well but also ensures that it receives the required amount before serving coffee. We improve it later in this chapter but one quick improvement is to increase the options by adding more choice operators. If there are more '+' operators, we can write the same using the *sum operator* as well. For example, in the following code *acceptCoin* can accept a coin of any value, even other than 1 and 2 cents.

```
act insCoin, acceptCoin, coin: Nat;
    getCoffee, serveCoffee, coffee;

proc Customer = insCoin(2) . insCoin(1) . getCoffee . Customer;
    Machine  = sum x: Nat.acceptCoin(x) . sum y: Nat.acceptCoin(y) .
               serveCoffee . Machine;

init
    allow( { coin, coffee },
      comm( { insCoin   | acceptCoin  -> coin,
              getCoffee | serveCoffee -> coffee },

            Customer || Machine
          ));
```

Here `sum x: Nat` ($\sum_{x:\mathbb{N}} \ldots$) is a variable declaration for the next action and after that, the remaining actions can also use its value. We can say that it is accessible to all the actions before which it is declared.

The expression `sum x: Nat.acceptCoin(x)` is a representation of:

```
acceptCoin(0) + acceptCoin(1) + acceptCoin(2) + acceptCoin(3) + ...
```

According to currency denomination [37], there is no coin of worth 0, 3, 4, 6 etc. cents. We can remove *acceptCoin*(0) by changing the data type to `Pos` from `Nat`. But for removing 3, 4, 6 and other unavailable coins, we need to devise our own way which is discussed later in this chapter.

Exercise 5.1 Suppose there are two processes where one sends some value to the other and then receives twice that value. Specify these processes in mCRL2 and observe the system using `ltsgraph`. Try with value of the type `Int` as well as `Real`. For example, if a `Real` type value $\frac{1}{8}$ is given then $\frac{1}{4}$ is received.

Let us discuss another example where boolean data is communicated between two processes. We apply the *sum* operator with the data type `Bool`. See Fig. 5.1, where mCRL2 specifications are shown along with the resulting LTS. There are two concurrent processes P_1 and P_2. The process P_1 is keeping a value in variable v of type `Bool` and sending values *true* and *false*, alternatively. The other process P_2 is ready to receive any value of type `Bool` at anytime. Here, the '!' operator is used for negating a boolean value. See the other logical operators Table 5.4.

5.1.1 Operators on Data

Like a programming language, mCRL2 provides predefined operators for arithmetic, logical and relational operations.

- **Arithmetic Operators:** These operators are useful when processes need to perform certain calculations.

```
act send,receive,value:Bool;

proc P1(v:Bool)=send(v).P1(!v);
     P2=sum x:Bool.receive(x).P2;

init  allow({value},
       comm({send|receive->value},
            P1(true)||P2
           ));
```

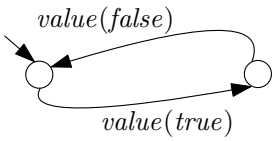


Fig. 5.1 Sending and receiving alternative values of type `Bool`

Let us make a model that shows the use of these operators in mCRL2 where a few processes are sending different types of values to a process and that process is applying a few operators given in Table 5.2. In the following mCRL2 specification, the processes P_1, P_2, P_3 and P_4 send some values of type `Int, Real, Pos`, and `Nat`, respectively to a process P and the process P applies some operators to these values. Working of the operators can be observed in the actions *testRound*, *testSucc*, *testRealDiv*, *testAbs*, and *testMinMax*. This example is self-explanatory w.r.t. the action names and its LTS is shown in Fig. 5.2. The action *testMinMax* is using two parameters, so it is declared as: `testMinMax: Int # Int;` where the # sign is used for the Cartesian product of types indicating that an action can have multiple parameters.

```
act sendInt, rcvInt, intValue: Int;
    sendReal, rcvReal, realValue: Real;
    sendPos, rcvPos, posValue: Pos;
    sendNat, rcvNat, natValue: Nat;
    testRound, testSucc, testDiv, testAbs: Int;
    testPlus, testRealDiv: Real;
    testMinMax: Int # Int;

proc % Four processes sending some values.
     P1        = sendInt(-4);
     P2(a:Int) = sendReal(4/a);
     P3        = sendPos(5);
     P4        = sendNat(10);
     % A process to receive values and store them locally.
     P = sum i: Int.rcvInt(i) .
         sum r: Real.rcvReal(r) .
         sum p: Pos.rcvPos(p).
         sum n: Nat.rcvNat(n) .
             ( testRound(round(r)) +
               testPlus(r+r) +
               testRealDiv(r/2) +
               testSucc(n-i) +
               testDiv(n div p) +
```

Table 5.2 Operators in mCRL2

Operator	Description	Usage	Results for: $a = 10$, $b = 3$, $c = -2$, $r = \frac{3}{4}$
`+`	Addition	`a+b`	13
`-`	Subtraction	`a-b`	7
`*`	Multiplication	`a*b`	30
`div`	Integer division	`a div b`	3
`mod`	Modulo	`a mod b`	1
`exp`	Exponentiation	`exp(a, b)`	1000
`/`	Real division	`a/b`	$\frac{10}{3}$
`abs`	Absolute value	`abs(c)`	2
`min`	Minimum	`min(a, b)`	3
`max`	Maximum	`max(a, b)`	10
`floor`	Previous integer	`floor(r)`	0
`ceil`	Next integer	`ceil(r)`	1
`round`	Round	`round(r)`	1
`sqrt`	Square root	`sqrt(a)`	3
		`sqrt(a*100)/10`	$\frac{31}{10}$
		`ceil(sqrt(a*100)/10)`	4
`pred`	Predecessor	`pred(a)`	9
`succ`	Successor	`succ(a)`	11
`Pos2Nat`	Casting from positive to natural numbers	`Pos2Nat(b)`	3
`Nat2Pos`	Casting from natural to positive numbers	`Nat2Pos(a)`	10
`Int2Nat`	Casting from integer to natural numbers	`Int2Nat(c-a)`	12
`Int2Pos`	Casting from integer to positive numbers	`Int2Pos(c)`	2

```
                  testAbs(abs(i)) +
                  testMinMax(min(i, p), max(n, -i))
                );

    init allow( { intValue, realValue, posValue, natValue,
                  testPlus, testRound, testSucc, testRealDiv,
                  testAbs, testMinMax },
           comm( { sendInt  | rcvInt  -> intValue,
                   sendReal | rcvReal -> realValue,
                   sendPos  | rcvPos  -> posValue,
                   sendNat  | rcvNat  -> natValue },
             P1||P2(7)||P3||P4||P));
```

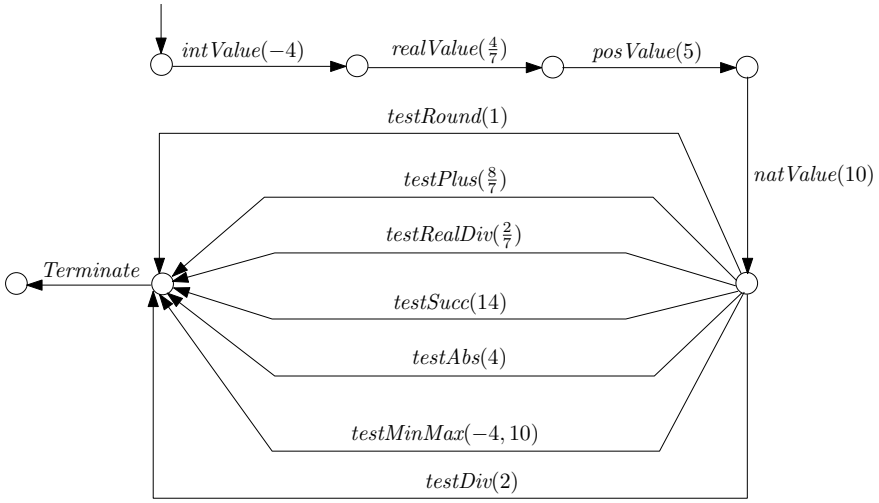


Fig. 5.2 Using arithmetic operators

Table 5.3 Relational operators in mCRL2

Operator	Description	Usage	Result $a = 10, b = 20$
<	Less than	$a < b$	*True*
<=	Less than or equal	$a <= b$	*True*
>	Greater than	$a > b$	*False*
>=	Greater than or equal	a>=b	*False*
==	Data equal ($\approx$)	a==b	*False*
!=	Not data equal ($\not\approx$)	a!=b	*True*

- **Relational Operators:**
  There are six binary relational operators in mCRL2 like C++/Java. These operators are $<, \leq, >, \not\approx$ and $\approx$, written in mCRL2 as <, <=, >, >=, !=, and ==, respectively. Suppose $a = 10$ and $b = 20$ then the use of these relational operators is described in Table 5.3.
- **Logical Operators:**
  There are four logical operators, $\neg$, $\wedge$, $\vee$ and $\Rightarrow$, written in mCRL2 as !, &&, ||, and =>, for *negation*, *and*, *or* and *implication*, respectively. In these operators, the negation operator has the highest precedence, then the 'and', subsequently the 'or', while the implication operator has at the lowest one. In the expression $a \wedge \neg b \Rightarrow b$, the negation is applied first and then the 'and' while the implication is applied last. To see the use of these operators, see Table 5.4 where we suppose a and b are of type *Bool* and are initialised as a = *true* and b = *false*.

Table 5.4 Logical operators in mCRL2

Operator	Description	Usage	Result
!	Not ($\neg$)	`!a`	*False*
&&	And ($\wedge$)	`a&&b`	*False*
		`a&&!b`	*True*
		`!(a&&b)`	*True*
		`!a&&b`	*False*
||	Or ($\vee$)	`a || b`	*True*
		`a||!b`	*True*
		`!(a||!b)`	*False*
		`!a||b`	*False*
=>	Implication ($\Rightarrow$)	`a=>b`	*False*
		`!a=>b`	*True*
		`b=>a`	*True*
		`a=>a`	*True*
		`a&&!b=>b`	*True*

Exercise 5.2 Figure out the change in the behaviour given in Fig. 5.1 when the process P_1 is replaced with any of the following definitions.

- $P_1(v : \mathbb{B}) = send(v).P_1(v \approx false)$.
- $P_1(v : \mathbb{B}) = send(v).P_1(v \approx true)$.
- $P_1(v : \mathbb{B}) = send(v).P_1(v \Rightarrow \neg v)$.

Exercise 5.3 Consider the following specifications and draw the resulting LTS manually.

```
act send1, send2, transfer1, transfer2: Nat;
    rcv1, rcv2: Nat;
    showValues: Nat#Nat;

proc P1 = send1(20) . send2(30);
     P2 = sum x: Nat.rcv1(x) . sum y: Nat.rcv2(y+x) . showValues(x, y);

init allow({ transfer1, transfer2, showValues },
       comm({ send1 | rcv1 -> transfer1,
              send2 | rcv2 -> transfer2 },
         P1||P2));
```

Let us discuss another example taken from the academic examples available in the mCRL2 installation. This example was written by Frank Stappers. Here we present a slightly modified version.

There are messengers, lazy- and hard workers that have a parameter n that indicates the identity of the person. Each person deals with tasks with the same number as

his identity. Workers and messengers receive instructions to start with a task using the action *start_r*. When a task is done the action *done_s* is done to indicate this. Messengers and lazy workers forward tasks to others using the action *start_s* and receive confirmation that a task has been done using the action *done_r*. Receiving and sending a done or start communicates to the actions *done* and *start*.

A hard worker gets a task, executes it and reports that it is done. Execution is not modelled explicitly, as we are interested in the communication pattern. A messenger with identity n receives a task and forwards it to person $n + 1$, waits until it is done and reports accordingly. A lazy worker n receives a task, and hands it over to persons $n + 1$ and $n + 2$, waits until both have done their work and reports back that the task is done.

There are three hard workers, 2, 3 and 6. There is one lazy worker with identity 1, and there are three messengers 0, 4 and 5. Together they must perform a task initially given to person 0. The interesting question is whether they can accomplish this task.

The model below shows the behaviour of the workers and messengers. The communication structure of this system is depicted in Fig. 5.3. Note that actions can only communicate when they have the same data parameters. In this case we allow workers to simultaneously start and stop their tasks and therefore we also allow multi-actions with multiple actions *start* and *done* to happen simultaneously.

```
act start_s, start_r, start, done_s, done_r, done: Nat;

proc Messenger(n: Nat)  = start_r(n) . start_s(n+1) .
                          done_r(n+1) . done_s(n) . Messenger(n);
     LazyWorker(n: Nat) = start_r(n) . start_s(n+1)|start_s(n+2) .
                          done_r(n+1)|done_r(n+2) . done_s(n) .
                            LazyWorker(n);
     HardWorker(n: Nat) = start_r(n) . done_s(n) . HardWorker(n);

init allow( { start,
              start | start,
              start | start | start,
              start | start | start | start,
              done,
              done | done,
              done | done | done,
              done | done | done | done
            },
       comm( { start_s | start_r -> start,
               done_s  | done_r  -> done },
             HardWorker(2) || HardWorker(3) || HardWorker(6) ||
             LazyWorker(1) ||
             Messenger(0) || Messenger(4) || Messenger(5) ||
             ( start_s(0) . done_r(0) )
           )
         );
```

Exercise 5.4 Can the constellation of workers in the example above accomplish task 0? Are all workers needed? When the hard worker 3 becomes lazy, can they still accomplish task 0?

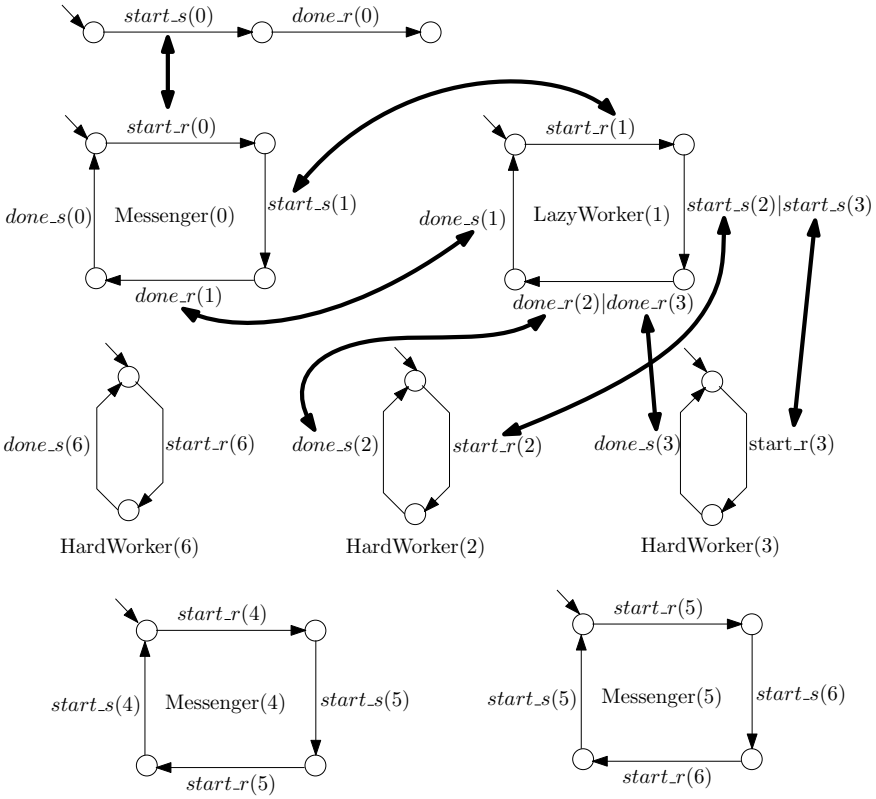


Fig. 5.3 Example of 3 messengers, 3 hard workers, and 1 lazy worker

5.2 Conditions

Processes take decisions based on data and adapt their behaviour accordingly. For example, upon receiving an odd value a process may take different actions than after receiving an even value. The conditional operator in mCRL2 has the following shape:

- **(condition)->process**. It is similar to the *if* statement used in programming languages like C++/Java. If the condition is true the process is executed. If the condition is false the whole process acts like a deadlock.
- **(condition)->process <> other process**. It is similar to *if-else* structure in C++/Java. Here <> represents the *else* part. The first process is executed if the condition is true, and the process behind the *else* is performed when the condition is false.

Consider the following mCRL2 specifications where a process performs a different action depending on its ID.

```
act evenAction, oddAction;

proc P(id: Nat) = ((id mod 2)==0) -> evenAction
                                  <> oddAction;
init P(10);
```

Along with the above conditional operator, mCRL2 also defines an operator called *if* to conditionally select a value. This operator is used as: *if*(*condition*, $value_1$, $value_2$) where the *condition* is a boolean expression. If the condition is true, $value_1$ is used. Otherwise, $value_2$ is the result of the expression. For example, see the following specification where a value in the recursive call is placed using the *if* operator.

```
act a;

proc P(n: Nat) = a . P(if(n mod 2 == 0, 0, 1));

init P(2);
```

The above specification can alternatively be written using the condition statement as given below:

```
act a;

proc P(n: Nat) = a . (n mod 2 == 0) -> P(0) <> P(1);

init P(2);
```

Exercise 5.5 Specify a system in mCRL2 where two processes P_1 and P_2 are sending odd and even values, respectively to a process P. The values are sent in an increasing order. The odd values are from 1 to 9 while the even values are from 0 to 8. The process P receives a value and responds to the respective process by sending a message *sendOdd* or *sendEven* in accordance to the received value. Both P_1 and P_2 can simultaneously send a value to, or receive a message from P.

5.3 Data Structures

mCRL2 provides a few built-in data structures and the relevant operators to manipulate that data. All the data structures store homogeneous data, i.e., the same type of data. Let us discuss lists, sets, and bags with examples in the following sections.

5.3.1 Lists

Lists are like arrays where each element has an index value. For the first element the index is 0 or in general, the nth element has the index $n - 1$. It means indices are

Table 5.5 Operations on lists

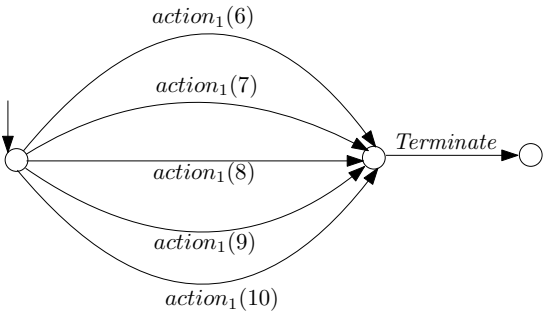
Operator	Description	Usage	Results for: $a = 10$, $list1 = [5, 10, 15]$, $list2 = [6, 7]$
`in`	Belongs to ($\in$)	`a in list1`	True
`#`	Length of a list	$\#list1$	3
`|>`	Adding an element to the front	`6 |> list1`	[6, 5, 10, 15]
`<|`	Appending an element to the back	`list1 <| 12`	[5, 10, 15, 12]
`++`	Concatenating two lists	`list2 ++ list1`	[6, 7, 5, 10, 15]
`.`	Accessing an element	`list1.2`	15
`head`	The first element	`head(list1)`	5
`tail`	A list without the first element	`tail(list1)`	[10, 15]
`rtail`	A list without the last element	`rtail(list1)`	[5, 10]
`rhead`	The last element	`rhead(list1)`	15
`==`	Equal	`list2==[6, 7]`	True
`!=`	Not equal	`list1!=list2`	True
`<`	Lexicographically less than	`list1<list2`	True
`>`	Lexicographically greater than	`list1>[10]`	False
`<=`	Lexicographically less than or equal	`list1<=[5]`	False
`>=`	Lexicographically greater than or equal	`list2>[7, 6]`	False

zero-based. We use the notation [. . .] to denote a list. As an example take [1, 4, 3, 3] as a list with four elements.

Elements are accessed with the '.' operator, which is also used for other very different purposes, such as sequential composition of processes. For example, *myList*.1 stands for the 2*nd* element in the list *myList*. Other operators and functions used with a list are discussed in Table 5.5. Lists are declared using the keyword `List` with its type. In mCRL2 syntax `List(Pos)` is the type of a list of positive integers having an arbitrary length.

The following specification shows a simple usage of a list where a list of 5 elements is given to a process and an action *action1* accesses each element of that list with the choice operator '+'.

Fig. 5.4 Using list elements




```
act action1: Nat;

proc P(arr: List(Nat)) = action1(arr.0) +
                         action1(arr.1) +
                         action1(arr.2) +
                         action1(arr.3) +
                         action1(arr.4);

init P([6, 7, 8, 9, 10]);
```

In the example above, we can convert all the '+' (choice) operators into a single sum operator as follows:

```
proc P(arr: List(Nat)) = sum x: Nat.(x<5) -> action1(arr.x);
```

The LTS for the specifications above is shown in Fig. 5.4. We can remove the action *Terminate* by putting the process *delta* after the action $action_1$.

Using lists, we can specify a coffee machine that is a bit more realistic. The machine now accepts common coins of denominations 1, 2, 5, 10, 20 and 50 cents. Suppose a cup of coffee costs 35 cents and it is served after getting at least the required money. The machine does not accept extra coins. This version of the coffee machine is given below.

```
act insCoin, acceptCoin, coin: Nat;
    getCoffee, serveCoffee, coffee;

proc Customer = ( insCoin(5) + insCoin(10) + insCoin(20) + insCoin(50)
                + getCoffee) . Customer;

     Machine(total: Nat, coinValues: List(Nat)) =
                (total<35)
                  -> (sum x: Nat.(x<6) -> acceptCoin(coinValues.x) .
                     Machine(total+coinValues.x, coinValues))
                 <> serveCoffee.Machine(0, coinValues);

init allow( { coin, coffee},
       comm( { insCoin   | acceptCoin  -> coin,
               getCoffee | serveCoffee -> coffee
```

```
        },
        Customer || Machine (0, [1, 2, 5, 10, 20, 50])));
```

The machine accepts a coin and then updates its variable *total* by adding the coin's value whereas a coin value is taken from the list *coinValues*. Accepting a coin is conditional, i.e., if $total < 35$. If enough money has been inserted, the only option is to serve coffee and reset the paid amount to zero.

The behaviour of the customer is to keep inserting coins until the machine stops taking coins and serves a cup of coffee. In this specification, the recursive call $Machine(0, coinValues)$ can also be written as $Machine(total = 0)$ explicitly stating that the parameter *total* must become 0 and all other parameters remain unchanged. This is helpful when only a few parameters require an update.

The expression `sum x: Nat.(x<6)->acceptCoin(coinValues.x)` represents the choice operator between the actions $acceptCoin(n)$ where n stand for the six coin values from the list *coinValues*.

Exercise 5.6 In the example above, modify the behaviour of the coffee machine, so that a customer may cancel an order and get money back. Moreover, if the given amount is more than 35, the customer may get the balance amount back.

Operations on Lists

There are number of operators and functions applicable to a list as shown in Table 5.5. Here we discuss an example which is a modified version of the file `list.mcrl2` that comes with the mCRL2 installation and is available in the directory `examples/language` or `share/mcrl2/examples/language`.

In this example, a few operators and functions are used to demonstrate their use. There are two processes P and P_1 where P is instantiated with a list to perform different actions and P_1 receives an arbitrary list and reports whether it equals the list [40, 20, 30].

```
act transferList, rcvList, sendList: List(Nat);
    r, nr;

proc P(ls: List(Nat)) =
              (40 in ls) -> tau . P( 10 |> ls ++ [#ls] <| 100 )
           +  (ls.1 == 30) -> tau . P( tail(ls) )
           +  (head(ls) == 20) -> tau . P( [rhead(ls)] ++ rtail(ls) )
           +  sendList(ls) . P();

% Process to receive an arbitrary list of natural numbers
     P1 = sum lst: List(Nat).rcvList(lst) .
          (lst==[40, 20, 30])-> r <> nr;

init allow( { transferList, r, nr },
       comm({ sendList | rcvList -> transferList },
         P([20, 30, 40]) || P1 ));
```

Assume the list *ls* in process P is $[20, 30, 40]$. The following is happening in the above code:

- Line 5 makes it [10, 20, 30, 40, 3, 100] because 10 is added at the left and [3, 100] is added to the right (3 is due to #*ls*).
- Line 6 makes it [30, 40] because 30 is there at the second position of *ls* and *tail(ls)* means the list *ls* without its first element.
- Line 7 makes it [40, 20, 30] because the condition there is *true* and *rhead*(*ls*) gives 40 while *rtail*(*ls*) produces [20, 30].
- Line 8 sends it to process P_1 and continues with P_1 with unchanged parameters.
- Line 11 receives the list and compares it with [40, 20, 30]. It performs the action *r* if the comparison is *true*. Otherwise the action *nr* is done.

We can view these lists using the tool`lpsxsim` as shown in Fig. 5.5. It allows to step through the actions of a process. It can be started from both the *GUI* and the *IDE* of mCRL2. On a terminal/command window, the tool is used as:

```
lpsxsim someFile.lps
```

Generating its LTS will result in an unlimited number of states/transitions and as a result the machine goes out of memory. This happens for the above specifications because the list *ls* may have continuous growth in recursive call at line 5. We can

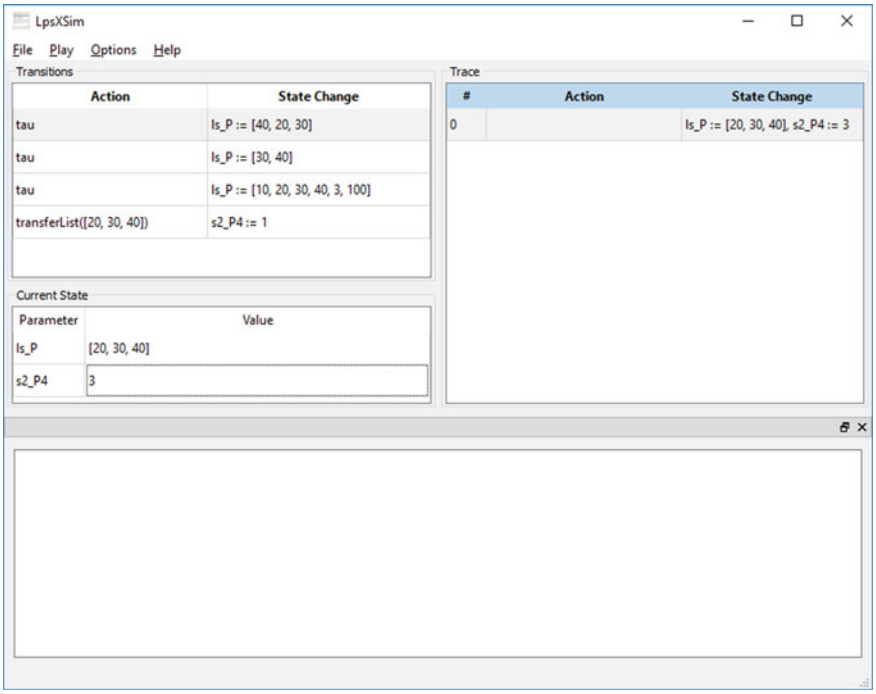


Fig. 5.5 Using the simulator `lpsxsim`

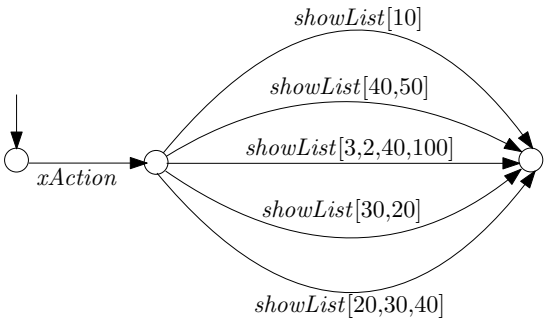


Fig. 5.6 Using lists in a list

observe this by using `lpsxsim` and repeatedly selecting the τ corresponding to line 5.

List of lists

A list can contain other lists just like an array of arrays in C++/Java. It is declared as *List*(*List*(*some_type*)). For example, a list containing lists of natural numbers is:

```
[[5, 6, 7], [10, 12, 15], [0, 1, 2, 3, 4], [5, 5]]
```

Suppose the list name is *lst* then #*lst* is its length 4 and `lst.1.2` stands for the third element in the second list, i.e., 15 in this example. Note that the size operator # has higher priority than the access operator '.', so #*ls*.1 is not a valid expression but #(*ls*.1) is fine for a list of lists. If $ls = [[]]$, then #*ls* is 1 and #(*ls*.1) is 0.

Let us discuss an example where we use lists of lists. Its LTS is shown in Fig. 5.6.

```
act xAction;
    showList: List(Nat);

proc P(ls: List(List(Nat)))  =
        ([40, 50] in ls)
        -> xAction . P2( [10]|> ls ++ [[#ls, #(ls.1), ls.0.2]<|100] );

     P2(ls: List(List(Nat))) =
        sum n: Nat.(n<#ls) -> showList(ls.n) . delta;

init allow({ showList, xAction },
       P([[20, 30, 40], [30, 20], [40, 50]]));
```

In this example, the process P receives a list of lists $ls = [[20, 30, 40], [30, 20], [40, 50]]$ and performs an action *xAction* after checking whether the list [40, 50] belongs to *ls* (which is *true* in this case). In the recursive call a list [10] is added to the front of *ls* and another list of lists [[3, 2, 40, 100]] is concatenated with *ls* because in the expression $[[\#ls, \#(ls.1), ls.0.2] \triangleleft 100]$, the subexpressions #*ls*, #(*ls*.1) and *ls*.0.2 have values 3, 2 and 40, respectively. So, *ls* becomes:

$$[[10], [20, 30, 40], [30, 20], [40, 50], [3, 2, 40, 100]].$$

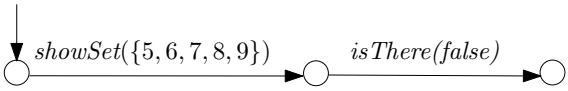


Fig. 5.7 A set used in an action in an LTS

Exercise 5.7 Specify three processes P_1, P_2, and P_3 which are continuously sending the lists [10], [20], and [30], respectively to a process P. The process P may receive the lists in any order and adds them to a list of lists. The list of lists can contain at most 3 elements and they are unique. Process P makes the list of lists empty after performing action τ when the list becomes full.

5.3.2 Sets

A set contains unordered and unique elements of the same type. In mCRL2, a set is either expressed by the explicit set notation {0, 4, 7} or by set comprehension $\{x{:}A \mid P(x)\}$, i.e., the set of all x elements having type A and satisfying some predicate $P(x)$. So, a set of all natural numbers between 4 and 10 is declared as either{5, 6, 7, 8, 9} or $\{x : \mathbb{N} \mid x < 10 \wedge x > 4\}$. The empty set is written as {}. The ordering and repetition of elements is ignored in a set declaration so, the set {1, 2} is same as {1, 1, 2, 1, 2}. Moreover, there is no index for an element in a set.

Let us discuss the usage of sets in mCRL2 with a simple example. The following specification shows a process P_1 that takes a set of natural numbers as a parameter and performs two actions over that set.

```
act showSet: Set(Nat);
    isThere: Bool;

proc P1(st: Set(Nat)) = showSet(st) . isThere(2 in st) . delta;

init P1({5, 5, 6, 6, 7, 8, 9, 9, 9});
```

The first action *showSet* simply displays the contents of the set in parameter while the other action *isThere* determines whether 2 is in that set or not, as shown in Fig. 5.7. Note that the repetition of elements in the set given as an argument in the *init* section is automatically ignored.

We can perform other operations on sets as well, e.g., union, intersection, difference, etc along with determining if a set is a subset, proper subset, superset, proper superset, and equal or not equal to some other set. Built-in operators of mCRL2 for sets are given in Table 5.6.

Table 5.6 Operations on sets

Operator	Description	Usage	Results for: $set1 = \{5, 10, 15\}$ $set2 = \{6, 7, 10\}$
`in`	Belongs to ($\in$)	`10 in set1`	True
`<=`	Subset ($\subseteq$)	`set1<=set2`	False
`<`	Proper subset ($\subset$)	`{5, 10, 15} < set1`	False
		`{10, 15} < set1`	True
		`{} < {}`	False
`+`	Union ($\cup$)	`set1+set2`	{5, 6, 7, 10, 15}
{_ : _ | *boolean expression*}	Set comprehension	`{x:Pos|x<6}`	{0, 1, 2, 3, 4, 5}
`-`	Difference	`set1-set2`	{5, 15}
		`set2-set1`	{6, 7}
`*`	Intersection ($\cap$)	`set1*set2`	{10}
`==`	Equality ($\approx$)	`set1=={10, 5, 15}`	True
		`{}=={{}}`	False
		`set1==set2`	False
`!_`	Complement	`!set1`	!{5, 10, 15}
		`!set1+set1`	$\{n \in \mathbb{N} \mid true\}$
		`2 in !set2`	True
`!=`	Inequality ($\not\approx$)	`set1!=set2`	True
`>=`	Superset ($\supseteq$)	`set1>={5, 15}`	True
		`set1>={10, 15, 5}`	True
		`set1>=set2`	False
`>`	Proper superset ($\supset$)	`set2>{7, 10}`	True

Exercise 5.8 Declare a set of odd numbers between -10 and 10 using set comprehension.

Let us discuss another example where two parallel processes can send/receive a set of values and perform some other operations. The actions specified on line 3 in the following code have type $Set(\mathbb{Z})$ which means they can communicate a set of integers while interacting. Their communication is specified on line 20. The other actions exhibit their meanings from their names and we can comprehend them by looking at Table 5.6 and Fig. 5.8.

```
1  act showSet, takeUnion, takeIntersection, takeDifference: Set(Int);
2      isEqual, isProperSubset, isSuperset, isProperSuperset: Bool;
3      sendSet, rcvSet, transferSet: Set(Int);
4
5  proc P1(st:Set(Int)) =
6           showSet(st) .
7           isEqual(st=={x:Int|x<=3 && x>-1}) .
8           isProperSubset(st<{x:Int|x<=3}) .
```

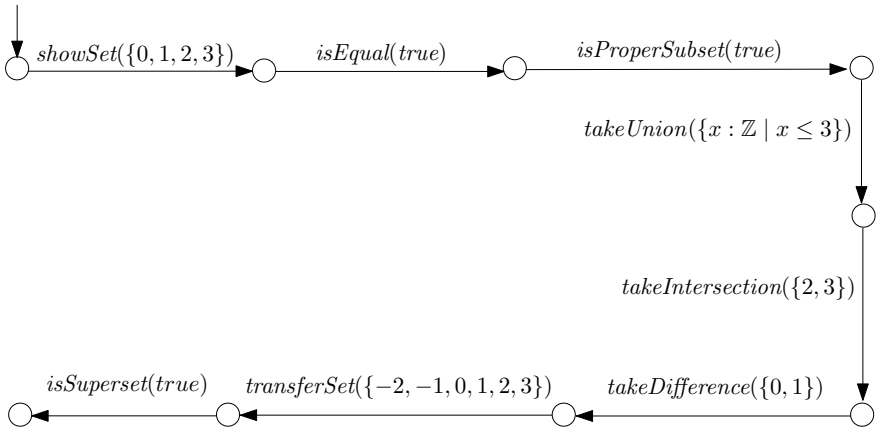


Fig. 5.8 Operations on sets

```
9           takeUnion(st+{x:Int|x<=3}) .
10          takeIntersection(st*{2, 3, 4, 5}) .
11          takeDifference(st-{2, 3, 4}) .
12          sendSet(st+{-1, -2}) . delta;
13
14    P2 = sum s:Set(Int) . rcvSet(s) . isSuperset(s>{1, -1, 0}) . delta;
15
16  init allow( { transferSet, showSet, isEqual,
17                isProperSuperset, isProperSubset,
18                takeUnion, isSuperset,
19                takeIntersection, takeDifference },
20        comm( { sendSet | rcvSet -> transferSet },
21          P1({0, 1, 2, 3}) || P2 ));
```

The data type *Set*(*D*) is very powerful, as it allows infinite sets and set comprehension. A disadvantage is that it is a complex structure which is difficult to calculate and it might slow down the generation of state spaces or carrying out model checking. For this reason a finite set *FSet*(*D*) is introduced. Insertion and deletion of elements in a finite set is much faster than in a 'real' set. A finite set can only contain a finite number of elements. It does not allow for set complement and set comprehension. But contrary to general sets, it is possible to obtain the number of elements in a finite set, by writing #*S* where *S* is a finite set.

Exercise 5.9 Why is it not really possible to define the size #*S* of an arbitrary set?

5.3.3 Bags

Bags are like sets except that elements in a bag can be repeated an arbitrary number of times. For example, {1:2, 2:1, 3:3} represents a bag having the elements 1, 2, and

3 which are occurring two, one, and three times, respectively. Bag comprehension is denoted as $\{x{:}D \mid f(x)\}$, which represents that element x of sort D occurs $f(x)$ number of times.

We can convert a set to a bag using the *Set2Bag* operator as shown on line 13 of the following specification. Similarly, a bag can be converted to a set by the *Bag2Set* operator. We can determine the number of occurrences of an element in a bag by the *count* operator as shown on line 4 in the following specifications where the number of occurrences of 2 is counted in a bag *bg*.

Most operators applicable to sets are also applicable to bags. An exception is "set complement" which does not exist on bags. Let us try to understand the usage of bags in mCRL2 with a simple example.

```
1  act sendInt, rcvInt, transferInt: Int;
2
3  proc P(bg: Bag(Int)) =
4        (count(2, bg)<2) -> sum i: Int.rcvInt(i) . P(Set2Bag({i})+bg);
5
6        Px(c: Nat) =
7          (c<2) -> sum x: Int.(x<3 && x>-2) -> sendInt(x) . Px(c+1);
8
9        Py(c: Nat) =
10         (c<2) -> sum x: Int.(x<3 && x>-2) -> sendInt(x) . Py(c+1);
11
12 init allow( { transferInt },
13        comm( { sendInt | rcvInt -> transferInt },
14          P(Set2Bag({0})) || Px(0) || Py(0) ));
```

In this example, a process P has as parameter *bg* representing a bag of integers. This process interacts with two processes P_x and P_y to receive values of type *Int*. The received values are added in the bag *bg* if the digit '2' occurs less than twice in the bag as shown in line 4. Both the processes P_x and P_y send at most two integers in the range from -1 to 2. The processes P_x and P_y have the same behaviour. We could have defined this process once and put it in the *init* section twice. This would have produced the same model.

Exercise 5.10 How can we write line 14 in the above specification without the Set2Bag operator?

Now we discuss another example in which a process receives some values from two processes and uses two bags, one for the positive values and other for the negative ones. In the two processes, one is fixed to send positive values while the other is for negative values from their pre-defined bags. This situation is depicted in Fig. 5.9.

```
1  act sendInt, rcvInt, transferInt: Int;
2
3  proc P(bg1: Bag(Pos), bg2: Bag(Int)) =
4           sum i: Int.rcvInt(i) .
5                       (i>0)
6                       -> P({Int2Pos(i):1}+bg1, bg2)
7                       <> P(bg1, {i:1}+bg2);
8
```

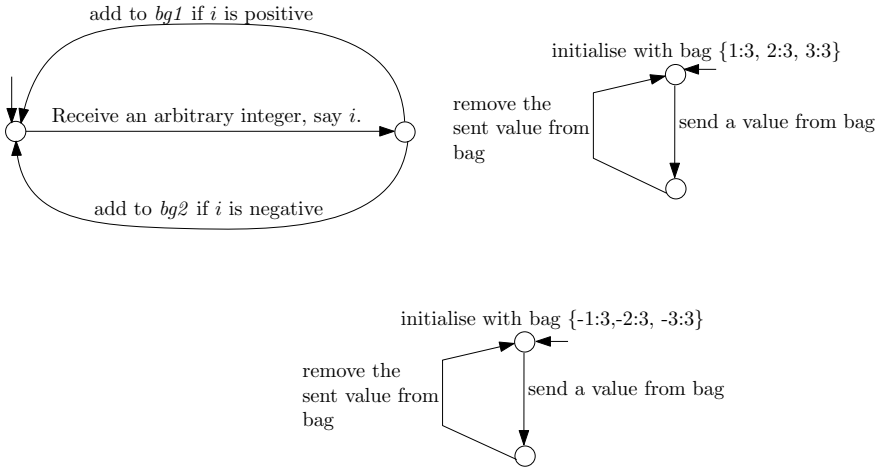


Fig. 5.9 Example of using bags

```
9         Px(b:Bag(Int)) = sum x:Int.(x in b) -> sendInt(x) . Px(b-{x:1});
10
11    init allow({transferInt},
12          comm({sendInt|rcvInt->transferInt},
13            P({1:0}, {1:0}) ||
14            Px({1:3, 2:3, 3:3}) ||
15            Px({-1:3, -2:3, -3:3}) ));
```

The process P in the above specification uses two bags bg_1 and bg_2 for positive and negative values, respectively as shown in line 3. The values are added to their respective bags by taking a union of bags as shown in lines 6 and 7. The process P_x is put in parallel twice; one with a bag with positive values and the other with a bag with negative values (see lines 14 and 15, respectively). The sent values are removed from their bags by taking a bag difference as shown in line 9.

Exercise 5.11 Specify a process that receives arbitrary integers and adds them to a bag provided that some divisor of the incoming value already exists in the bag.

5.4 User Defined Functions

We can define our own functions on data in mCRL2. These functions can be used by processes and other functions. A function is defined by a set of equations. The toolset interprets this function definition as a set of rewrite rules [4]. An equation is selected and executed by matching the function name and the arguments. It is similar to an overloaded function in C++/Java but there, a function is selected upon its name and the number/type of arguments whereas, in mCRL2, an equation is also selected upon the values of its arguments.

For example, we can define a function *XOR* representing the exclusive-or on booleans as follows:

```
map XOR: Bool#Bool -> Bool;

eqn XOR(true, true) = false;
    XOR(true, false) = true;
    XOR(false, true) = true;
    XOR(false, false) = false;
```

Using the keyword `map`, we declare a function, its parameters and return type. This is called the signature of a function. The types of multiple parameters for a function are separated by the Cartesian product operator # in the signature. The keyword `eqn` is used to define equations as shown in the above specification. Requiring equations for all possible values every time is not necessary. We can use variables for arbitrary values. The above example of the *XOR* function is again discussed with variables on page 73 to exhibit multiple ways to define a function.

For each group of equations, we can give the name and the type of the variables with the keyword `var`. For example, let us write a function *isEven* to determine whether a value is even or not. The function and its use by a process P are given below where the usage of keywords `map`, `var`, and `eqn` is shown on lines 1, 3, and 4, respectively.

```
1  map isEven: Int->Bool;
2
3  var n: Int;
4  eqn isEven(n) = (n mod 2==0);
5
6  act aAction, bAction;
7
8  proc P(a, b: Int) = ( isEven(a) && !isEven(b) )
9                      -> aAction
10                     <> bAction;
11
12 init P(10, 15);
```

In this specification, line 1 shows that the function name is *isEven*. It takes one parameter of type *Int* and returns a value of type *Bool*. So, this function is from *Int* to *Bool* which is expressed as $Int \rightarrow Bool$. Line 3 shows a variable declaration which is used later in line 4. The function returns $n \mod 2 \approx 0$, i.e., a truth value indicating whether n is divisible by 2 or not. The same function in the syntax of C++/Java is:

```
bool isEven(int n){ return n%2==0; }
```

We can also use the *if* operator for decision-making while defining a function. For example, the definition at line 4 in the above specification can be replaced with:

```
isEven(n)=if(n mod 2==0, true, false);
```

Inside *if*, there are three parts separated by two commas. In the first part, we write a boolean expression and if it is *true* then the second part is returned or otherwise the third part. In the second and third parts, the expressions have the same type, namely the type returned by the *if* expression. We can also make recursive calls in these parts as shown in the following examples.

Finding the maximum value in a list. To find the maximum value in a list, we can write separate equations for the empty list, one element list and more than one elements' list. The case of one element is simple as that element is supposed to be the maximum as shown in line 7 in the following specification. With two or more elements, we calculate the maximum using the built-in max operator (discussed in Table 5.2) as shown in line 8. We can exclude line 6, if this function is always used with non-empty lists.

```
1  map maxList: List(Nat)->Nat;
2
3  var n: Nat;
4      lst: List(Nat);
5
6  eqn maxList[]=0;                        %for the empty list
7      maxList([n])=n;                     %for one element
8      maxList(n|>lst)= max(n, maxList(lst));  %for more elements
```

Exercise 5.12 Write the above function without using the `max` operator. Hint: Use the *if* operator.

Exercise 5.13 Write a function to determine whether a given value is the maximum number in a list or not.

Calculating the power of two numbers. The function to calculate x^y is specified as:

```
1  map power: Int#Nat->Int;
2
3  var x: Int, y: Nat;
4  eqn power(x, 0) = 1;
5      (y>0) -> power(x, y) = x*power(x, y-1);
```

The function declaration in line 1 shows that there are two parameters, of type `Int` and `Nat`, respectively. The function returns a value of type integer. Line 4 shows the base case, i.e., the equation when the power is 0, i.e., $x^0 = 1$. If the power is larger than zero, we compute the result by recursion using: $x^y = x \times x^{y-1}$ which is shown in line 5 where "$(y > 0) \rightarrow$" is a condition for the respective equation. It means that the equation is applicable only if the given arguments makes the condition true.

The above mCRL2 specification in C++/Java is:

```
int power(int x, int y)
{
  if (y==0) return 1;
  return x*power(x, y-1);
}
```

Defining XOR (exclusive OR). We define this function in three ways to explain that sometimes a function can also be defined with fewer equations. The first version is already given above where the XOR was defined explicitly for all possible arguments.

The second version uses inequality on boolean values, and employs that inequality and XOR are equal.

```
map xor: Bool#Bool->Bool;

var a, b: Bool;
eqn xor(a, b) = a != b;
```

We can also apply a condition for selecting an equation and in this way the function definition is:

```
map xor: Bool#Bool->Bool;

var a, b: Bool;
eqn a == b -> xor(a, b) = false;
    a != b -> xor(a, b) = true;
```

Calculating the sum of the elements in a list. The following specifications show the addition of all elements in a list of natural numbers.

```
map sumList: List(Nat)->Nat;

var n: Nat;
    lst: List(Nat);
eqn sumList([]) = 0;
    sumList(n|>lst) = n + sumList(lst);
```

Exercise 5.14 Write a function to find the sum of the first n elements in a list of natural numbers.

Exercise 5.15 Write a function to find the maximum value from the first n numbers in a list of natural numbers.

Searching in a list. It is easy to find whether an element exists in a list with the help of the `in` operator, discussed in Table 5.5. Let us write a function to find the index of an element in a list and if it is not found, it returns -1.

```
map firstIndexOf: List(Nat)#Nat->Int;
    firstIndexOf: List(Nat)#Nat#Nat->Int;

var n, m, i: Nat;
    lst: List(Nat);
eqn firstIndexOf(lst, m) = firstIndexOf(lst, m, 0);
    firstIndexOf([], m, i) = -1;
    firstIndexOf(n|>lst, m, i) =
                  if(m==n, i, firstIndexOf(lst, m, i+1));
```

We provide two functions. The function *firstIndexOf*(lst, n) searches the index of n in the list lst. This is expressed using an auxiliary function *firstIndexOf* where the third argument is used to count the index. Initially, this third element is 0. It gets an increment of 1 upon every recursive call as shown in line 9.

Exercise 5.16 Write a function to find the last index of an element in a list of natural numbers. If the element is not there the function returns -1. Hint: You can use *rhead* and *rtail* functions discussed in Table 5.5.

Calculate the square of a sum. We define the square of a sum function in three different ways. Two of them are given below with their pros and cons while the third one is discussed in Sect. 5.4.2.

```
map squareOfSum: Int#Int->Int;

var x, y: Int;
eqn squareOfSum(x, y) = (x + y) * (x + y);
```

The above specification is easy to comprehend but the expression $x + y$ is executed twice by the rewriter. However, by using the whr clause, we can take care that the evaluation of $x + y$ happens exactly once as given below.

```
map squareOfSum: Int#Int->Int;

var x, y: Int;
eqn squareOfSum(x, y) = z * z whr z = x + y end;
```

The clause starting with `whr` ends with the keyword `end` and in this clause, we introduce a new variable that is not already present in the `var` section. This clause is similar to a let expression. More than one definition can be given, separated by commas, as shown in the following example.

Calculating a distance. The following example shows how we can calculate a distance using the formula $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

```
map distance: Int#Int#Int#Int->Int;
    squareOfMinus: Int#Int->Int;

var x1, x2, y1, y2, a, b: Int;
eqn distance(x1, x2, y1, y2) = sqrt(Int2Nat(x+y))
                                 whr x=squareOfMinus(x1, x2),
                                     y=squareOfMinus(y1, y2)
                                 end;
    squareOfMinus(a, b) = minus*minus whr minus=(a-b) end;
```

Determine whether a list is sorted or not. We want to know whether a given list is sorted. We use this example to show that existential and universal quantifiers can be used in data expressions. An existential quantifier $\exists d{:}D.\varphi(d)$ is true exactly if there is a d of sort D such that the formula $\varphi(d)$ holds. A universal quantifier $\forall d{:}D.\varphi(d)$

holds precisely if the formula $\varphi(d)$ is valid for all d of sort D. Examples of quantified expressions are $\forall x{:}\mathbb{N}.x > 0$, $\exists x{:}\mathbb{N}.x > 0$, $\forall x{:}\mathbb{N}.x + x = 2 * x$ and $\exists x{:}\mathbb{N}.x > y$. All these expressions are true, except the first one.

The function *isSorted* can now be defined as follows.

```
1  map isSorted: List(Int)->Bool;
2
3  var lst: List(Int);
4  eqn isSorted(lst) =
5            forall j: Nat.(j<#lst-1) => (lst.j <= lst.(j+1));
```

Line 5 shows that we call the list sorted if, for every index value j of type *Nat* that is less than the length of *lst*, every jth element is less or equal than the $(j+1)th$ element. So, in order to determine whether the whole list is sorted, we look at whether each consecutive pair of elements is sorted.

An alternative definition of the function using the function *if* is:

```
map isSorted: List(Int)->Bool;

var lst: List(Int);
eqn isSorted(lst)=if(#lst < 2, true,
                                head(lst)<=head(tail(lst)) &&
                                    isSorted(tail(lst)));
```

This definition poses that if there is only one element, i.e., the count elements is less than 2, then the list is already sorted. Otherwise, we compare the first two elements and define the function in terms of *isSorted* applied to a smaller list.

To improve the performance we can use a *whr* clause as given below.

```
map isSorted: List(Int)->Bool;

var lst: List(Int);
eqn isSorted(lst) = if(#lst<2, true,
                                head(lst) <= head(t) && isSorted(t))
                                      whr t = tail(lst) end;
```

Determine whether a list contains unique elements or not. We want to state that a list only contains unique elements. An empty list always contains unique elements. If the list contains more elements we check that the first element does not occur in the remainder, and we check that this remainder consists of unique elements only.

```
map are_all_unique:List(Nat)->Bool;

var n: Nat; lst: List(Nat);
eqn are_all_unique([]) = true;
    are_all_unique(n|>lst) = !(n in lst) && are_all_unique(lst);
```

Determine whether all elements in a list are the same. We define a function that determines whether a list consists of only the same elements. In a list that has at most

one element self evidently all elements are the same. Otherwise we compare the first two elements and check whether all elements in the tail of the list are also the same. The function *areAllSame* is specified as:

```
map areAllSame:List(Nat)->Bool;

var i,j: Nat;
    lst: List(Nat);
eqn areAllSame([])=true;
    areAllSame(i|>[]) =true;
    areAllSame(i|>j|>lst) = i==j && areAllSame(j|>lst);
```

Determine whether a number is prime. We consider a number p prime if all the numbers from 2 to square root of p do not divide it. In the syntax of mCRL2, the test for a prime number can be done as stated below. A function $sqrt : \mathbb{N} \to \mathbb{N}$ is available in mCRL2. It calculate the integer square root. The value of $sqrt(n)$ is the largest value m such that $m^2 \leq n$.

```
map is_prime: Nat->Bool;

var p: Nat;
eqn is_prime(p) = p>1 &&
                  forall q:Pos.(1<q && q<=sqrt(p)) => (p mod q)!=0;
```

Removing an element from a List. Suppose that a list of natural numbers is given and we have to remove a value from this list once if it is contained in the list. Otherwise, the given list remains unchanged. This can be defined by

```
1  map remove: List(Nat)#Nat->List(Nat);
2
3  var x, y: Nat;
4      lst: List(Nat);
5
6  eqn remove([], x) = [];
7      x == y -> remove(x |> lst, y) = lst;
8      x != y -> remove(x |> lst, y) = x |> remove(lst, y);
```

The base case is in line 6 where an empty list is returned as is. Line 7 shows that if the value to be removed, i.e., y matches with the first element x of the list then the list is returned without x. Line 8 shows the case where if y is different from x, the function removes y from the remaining list *lst* and then adds x to the updated list. This process continues through recursion until either the given value is found or the whole list is inspected.

Exercise 5.17 Modify the above definition such that if a value to be removed is not already in the list then it is appended to that list.

Insert an element at the *nth* position of a list. The following function inserts a value at the nth index of a list of natural numbers. If the index is beyond the list size, the element is appended to that list. The function is defined as:

```
1  map insertAt: List(Nat)#Nat#Nat->List(Nat);
2
3  var lst: List(Nat);
4      n, value, m: Nat;
5  eqn insertAt([], n, value) = [value];
6      insertAt(m|>lst, n, value) =
7               if(n==0,
8                  value|>m|>lst,
9                  m|>(insertAt(lst, Int2Nat(n-1), value)));
```

Line 5 shows that in an empty list the value is added without considering the index n. In lines 6–9, if the index n is 0 then the value is added to the front. Otherwise the value is put at position $n - 1$ in the list from which the first element is removed. That first element is put in front of the updated list. Note that for a natural number n, the expression $n - 1$ is an integer, and therefore, it must explicitly be converted to a natural number again.

Exercise 5.18 Specify a function to remove the element at the nth index in a list of natural numbers if it exists. Otherwise the list remains unchanged.

Exercise 5.19 Specify a function to insert a list, in a list of lists at a particular index.

Exercise 5.20 Consider a matrix represented by a list of lists of integers. Specify a function to replace an element at row i and column j with a new element.

Sorting. Suppose we want to sort the elements in a list in ascending order. There are different ways of doing this. Here, we define insertion sort using two functions *iSort* and *insert*. The function *insert* inserts its first argument at the right place in a sorted list, given as the second argument. The function *iSort* uses the function *insert* to insert all elements of a given list one by one in the sorted target list.

```
map iSort: List(Int)->List(Int);     % insertion sort
    insert: Int#List(Int)->List(Int);% insert the first argument
                                      % in a sorted list
var lst: List(Int);
    m, n: Int;
eqn iSort([]) = [];
    iSort(m|>lst) = insert(m, iSort(lst));
    insert(m, []) = [m];
    insert(m, n|>lst) = if(m<=n, m|>n|>lst, (n|>insert(m|>lst)));
```

Calculating the factorial. The following function computes the factorial of a given integer where line 5 is for the base case, i.e., for all values less than or equal to 1 and line 6 shows the computation of $(n + 1)!$ using $n!$ recursively.

```
1  map factorial: Int->Int;
2
3  var n, m: Int;
4  eqn (n<=1) -> factorial(n) = 1;
5      (n>1) -> factorial(n) = n*factorial(n-1);
```

Generating a one-dimensional list from the elements of a list of lists. By merging all elements of every list in a list of lists, we can convert a 2D list into a 1D one. The operator ++ concatenates two lists (see Table 5.5).

```
map twoDtoOneD: List(List(Nat))->List (Nat);

var l: List (Nat);
    L: List (List (Nat));

eqn twoDtoOneD([]) = [];
    twoDtoOneD(l|> L) = l ++ twoDtoOneD(L);
```

5.4.1 *Defining Constants*

We use constants in a specification to represent fixed values, such as a maximal buffer size or the number of messages or processes. Constants values in mCRL2 are defined as functions, i.e., we use the same keywords *map* and *eqn*. For example, in the following specifications, a constant value for N is specified and used.

```
act a;
map N: Pos;
eqn N = 10;

proc X=sum j:Pos.(j<=N)->a.delta;

init X;
```

The process X uses the constant $N = 10$ to allow ten non-deterministic a actions in its initial state.

Below is another example taken from the academic examples of the mCRL2 installation. This is a process with states $1, \ldots, N$, where each state i has transitions to all states 1 to $i + 1$. This means its transitions go maximally back to already detected states, and only one transition goes to a new state, i.e. $i + 1$. Such an example is particularly bad for random walks through the state space, as the state N has a very low probability of ever being reached.

The number of states is bounded by the constant $N = 1000$. In order to understand the structure of the example it is better to set N to 2 or 3 and visualise the state space using the tool `ltsgraph`.

```
act a;
map N: Pos;
eqn N = 1000;

proc X(i: Pos)=sum j: Pos.(j<=i+1 && j<=N) -> a . X(j) <> delta;

init X(1);
```

5.4.2 λ-Expressions

A function in mCRL2 can also be expressed as a lambda term (λ-term). The λ-calculus [7, 55] allows to write functions as a single term. A λ-term has two parts, namely a list of typed variables preceded by a lambda that represent the parameters of the function, and an expression that yields the result. For example, $\lambda z{:}\mathbb{N}.z^2$ represents a square function where $z : \mathbb{N}$ is the parameter and z^2 is the result of the function. A lambda term can be applied to arguments in an ordinary way. So, $(\lambda z{:}\mathbb{N}.z^2)(2)$ gives 4 and similarly, $(\lambda z{:}\mathbb{N}.z^2)((\lambda z{:}\mathbb{N}.z^2)(2))$ gives 16.

A variable declared in the parameter list is called a bound variable. We can use multiple bound variables to which arguments are assigned. For example, in $(\lambda p, q{:}\mathbb{N}.p^3 + q^2)(2, 3)$, arguments 2 and 3 are assigned to bound variables p and q, respectively, and therefore, the resulting expression becomes $2^3 + 3^2$. As another example, $(\lambda p{:}\mathbb{N}.p^3)((\lambda q, r{:}\mathbb{N}.p + q)(2, 3))$ represents $(2 + 3)^3$.

In mCRL2, we use a keyword `lambda` to write a λ-term. As an example, we write the *square_sum* function (already discussed in Sect. 5.4) using a λ-expression.

```
var x, y: Int;
eqn squareOfSum(x, y) = (lambda x: Int.z*z)(x + y);
```

This definition is equivalent to

```
var x,y: Int;
eqn squareOfSum(x, y) = (x + y) * (x + y);
```

where the expression $x + y$ is written twice, whereas in the λ-expression, it is stated only once.

It is not necessary to provide the arguments of a function occurring in the left hand side of an equation. The function can also be defined to be equal to another function, for instance formulated as a lambda term. As an illustration consider the definitions of functions f and g in the following example.

```
map f, g: Pos->Bool;

var y: Pos;
eqn f = lambda x: Pos.x > 0;
    g(y) = y > 0;

act a: Bool;

proc P = a(f(1)) . delta;

init P;
```

The functions f and g are semantically completely equivalent. Within the tools variants without lambdas often have a better performance, so, using g is more efficient in practice.

We can also use the *if* operator along with λ-expressions. For example, the following function divides two arbitrary natural numbers but always keeps the denominator lower than its nominator.

```
map divide: Nat#Nat->Real;

var a, b: Nat;
eqn divide(a, b) =
          (lambda x, y: Nat, z: Bool.if(z, a/b, b/a))(a, b, a>b);
```

We can reformulate the above specification by applying the function to its arguments, and replacing the explicit parameters a and b with a lambda with variables x and y. Furthermore, we show how the function can be used in a process.

```
map divide: Nat#Nat->Real;

eqn divide = lambda x, y: Nat.if(x>y, x/y, y/x);

act c: Real;

proc Pr(a: Nat, b: Nat) = c(divide(a, b)) . delta;

init Pr(2, 3);
```

Exercise 5.21 If two functions f and g are defined as: $f = \lambda x, y{:}\mathbb{N}.x^2 + y^2$ and $g = \lambda x{:}\mathbb{N}.2x$ then compute

(a) $f((g)(3), (g)(2))$, and
(b) $g(f(f(f(2, 2), f(1, 2)), f(3, 2)))$.

The above values can be computed through the following specification.

```
map f: Nat#Nat->Nat;
    g: Nat->Nat;

eqn f = lambda x, y: Nat.x*x+y*y;
    g = lambda x: Nat.2*x;

act a: Nat;

proc P = a(f((g)(3), (g)(2))) .
         a(g(f(f(f(2, 2), f(1, 2)), f(3, 2)))) .
         delta;

init P;
```

Given a function f it is sometimes useful to change the value for this function for particular arguments. This is possible using a function update which is applied as $f[x \to y]$. It shows a function f giving a value y when presented an argument with the value x. For all other arguments, the function f is unchanged.

For example, see the following function *square* which calculates the square of a number, but if it is applied to 0, it yields 1. Similarly, the value 1 is mapped to 2.

For the rest of the natural numbers, it computes the square properly. So, *square*(0), *square*(1), *square*(2) and *square*(3) give 1, 2, 4 and 9, respectively.

```
map square: Nat->Nat;

var n: Nat;
eqn square(n) = (lambda z: Nat.(z*z))[0->1][1->2](n);
```

A λ-expression taking constant arguments is defined as shown below.

```
map f: Nat;
eqn f = (lambda x: Nat.lambda y: Nat.lambda b: Bool.
                          if(b, y+1, x+1))(5)(8)(true);
```

The arguments given in parentheses are assigned to the bound variables from left to right. So, in the above example the variables x, y and b receive the constant values 5, 8 and *true*, respectively which makes f equals to 9.

Exercise 5.22 If $f(x) = 2x + 3$ and $g(x) = 3x + 2$ then write lambda expressions of each and compute $f(g(2))$ and $g(f(2))$ using the mCRL2 tools.

5.5 Constructors of a Data Type

Instead of declaring new functions using the keyword `map`, it is also possible to use the keyword `cons`. See the following example.

```
cons zero: PeanoNat;
     succc: PeanoNat->PeanoNat;
```

This is the standard way to declare Peano natural numbers, where *zero* represents the number 0 and *succ* is the successor function incrementing its argument by 1. For example *succ*(*zero*) represents 1 and *succ*(*succ*(*zero*)) stands for 2. Functions declared using the keyword `cons` are called *constructors* of that sort.

If a sort has constructors, all elements of that sort must be denotable using a constructor. Concretely, in the sort *PeanoNat* each element can be written as either *zero* or *succ*(n) where n is an element of *PeanoNat*.

Assume the specification above is extended with a constant.

```
map N: PeanoNat;
```

This declares that there is some constant N in the Peano natural numbers. We did not specify its concrete value, but as there are constructors in the Peano natural numbers, we know that N is equal to some term *succ*(...(*zero*)), which consists of zero or more applications of *succ* to the constant *zero*.

To define the datatypes of mCRL2, constructors are very important as they allow to precisely define the structure of the data by exactly specifying the shape terms can

have. In this book we do not use them directly, and therefore we do not elaborate on them further. When defining new data types, the *struct* data types, as explained in the next section, provide all necessary functionality.

5.6 Defining a *Struct* Data Type

We can define our own data type having values of our choice using struct or structured data types. The origin of these structured data types can be found in functional programming. A *struct* data type is like an enumerated type where all elements are listed explicitly.

For example, we can define a data type to represent weekdays. The keyword `sort` is used to specify a new data type and the keyword *struct* introduces the elements of that data type which are separated by the '|' operator as shown in the following example. Note that the operator '|' is also used in action communication with the keyword *comm* as discussed in Sect. 3.1, but that operator is completely unrelated. In the example below we show how the weekdays can be used in processes as any other data type.

```
1  sort weekDays = struct Monday | Tuesday | Wednesday | Thursday |
2                         Friday | Saturday | Sunday;
3
4  act sendDay, rcvDay, transferDay: weekDays;
5
6  proc Sender = sum d: weekDays.sendDay(d) . delta;
7       Receiver = sum d1: weekDays.rcvDay(d1) . delta;
8
9  init allow( { transferDay },
10        comm( { sendDay | rcvDay -> transferDay },
11                Sender || Receiver
12           ));
```

A new data type *weekDays* is specified in line 1 in the above specification. Actions using this data type are declared in line 4. The process *Sender* sends an arbitrary day of a week and the *Receiver* receives that day as shown in line 6 and line 7, respectively.

We can also define functions on our own data types. For example, in the following specification a function *nextDay* is introduced which gives the next day from the one given as an argument. In this specification, the process *Receiver* receives a day using the communication action *transferDay* and then performs the action *forwardDay* with the next day, as shown in Fig. 5.10.

```
sort weekDays = struct Monday | Tuesday | Wednesday | Thursday |
                       Friday | Saturday | Sunday;

map nextDay: weekDays->weekDays;

eqn nextDay(Monday) = Tuesday;
    nextDay(Tuesday) = Wednesday;
```

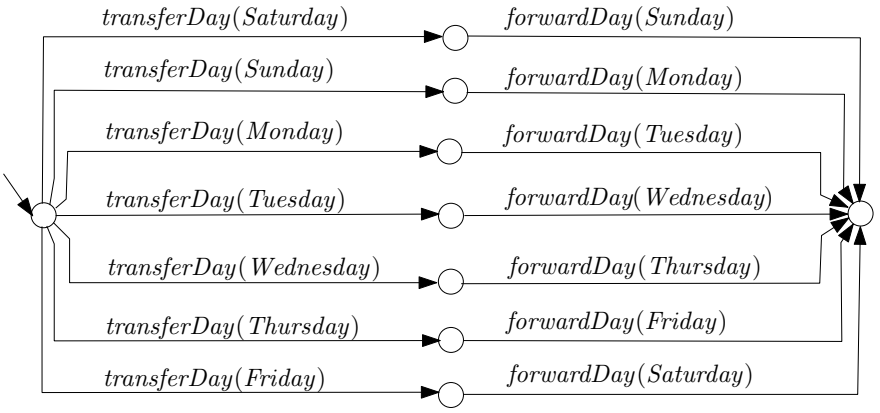


Fig. 5.10 Receiving a value from customised sort and showing its next one

```
    nextDay(Wednesday) = Thursday;
    nextDay(Thursday) = Friday;
    nextDay(Friday) = Saturday;
    nextDay(Saturday) = Sunday;
    nextDay(Sunday) = Monday;

act sendDay, rcvDay, transferDay: weekDays;
    forwardDay: weekDays;

proc Sender = sum d:weekDays . sendDay (d) . delta;
     Receiver = sum d1:weekDays. rcvDay(d1) .
                        forwardDay(nextDay(d1)) . delta;

init allow( { transferDay, forwardDay },
       comm( { sendDay | rcvDay -> transferDay },
               Sender || Receiver
           )
          );
```

Exercise 5.23 Write a function *nextNDay* to provide the day after n days from a given day. For example, *nextNDay*(*Saturday*, 2) gives *Monday* and the both *nextNDay*(*Saturday*, 0) and *nextNDay*(*Saturday*, 7) give *Saturday*.

In a struct data type we can define a recogniser for each element. The purpose of this function is to know whether its argument is the element to which it belongs. To define a recogniser function, we place the '?' sign after the element followed by the name of the recogniser. The signature of this function is fixed; it gets an element of the struct data type and returns a boolean value. So, both the parameters and the return data type are not explicitly mentioned when defining a recogniser. For example, we can define the elements of *weekDays* with recognisers as follows:

```
sort weekDays = struct Monday ? isMon | Tuesday ? isTue |
                       Wednesday ? isWed | Thursday ? isThu |
```

```
                        Friday ? isFri | Saturday ? isSat |
                        Sunday ? isSun;
```

We can redefine the function *nextDay* using recogniser functions as shown below.

```
map nextDay:weekDays->weekDays;

var d:weekDays;
eqn isMon(d) -> nextDay(d) = Tuesday;
    isTue(d) -> nextDay(d) = Wednesday;
    isWed(d) -> nextDay(d) = Thursday;
    isThu(d) -> nextDay(d) = Friday;
    isFri(d) -> nextDay(d) = Saturday;
    isSat(d) -> nextDay(d) = Sunday;
    isSun(d) -> nextDay(d) = Monday;
```

Each element in structure can contain other data members as well. For example, in the following specification a data type *MyCompounds* is defined where the elements have two, respectively three, arguments.

```
sort MyCompounds = struct composit2Nats(first: Nat, second: Nat) |
                          composit3Items(first: Nat, second: Int,
                                         third: List(Nat));
```

The element *composit2Nats* is a function applied to two natural numbers *first* and *second* while the element *composit3Items* has 3 data members of type *Nat*, *Int* and *List*(*Nat*), respectively. The functions *first*, *second* and *third* are projections. They are used to extract the corresponding argument from an elements in a struct. For example, *first*(*composit2Nats*(10, 20)) yields the first argument in the element *composit2Nats*, i.e., 10 in this case.

Here is an extension of the example above where projection and recogniser functions are used, and the data type is employed in a process.

```
sort MyCompounds = struct composit2Nats(first: Nat,
                                        second: Nat)?is2Nats |
                          composit3Items(first: Nat,
                                         second: Int,
                                         third: List(Nat))?is3Items |
                          otherData;

act sendData, rcvData, transferData: MyCompounds;
    natAction: Nat;
    intAction: Int;
    listAction: List(Nat);

proc Sender1 = sendData(composit2Nats(10, 20)) . delta;
     Sender2 = sendData(composit3Items(4, -1, [2, 3, 4])) . delta;
     Receiver =
        sum d1: MyCompounds.rcvData(d1) .
                (is2Nats(d1))
                   -> natAction(first(d1)) . natAction(second(d1))
                   <> (is3Items(d1)) -> natAction(first(d1)) .
```

```
                                        intAction(second(d1)).
                                        listAction(third(d1))
                                    <> delta;

init allow( { transferData, natAction, intAction, listAction },
       comm( { sendData | rcvData -> transferData },
                  Sender1||Sender2||Receiver ));
```

We provide another example in which the standard data type *Bit* with an auxiliary function *invert* is defined.

```
sort Bit = struct b0 | b1;

map invert: Bit->Bit;

eqn invert(b1) = b0;
    invert(b0) = b1;
```

5.7 Modelling the Alternating Bit Protocol

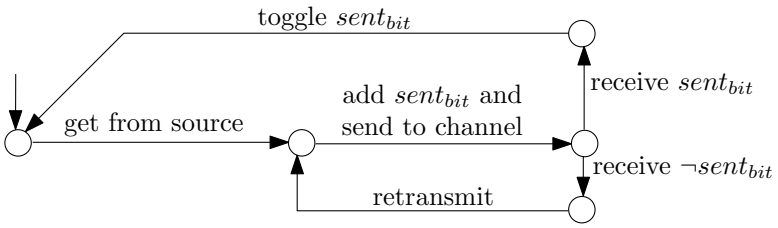
By studying this book till this point, it should be possible to model small distributed systems in mCRL2. Let us try and see how the alternating bit protocol is modelled, and do an exercise to formally specify another small system.

The alternating bit protocol [28, 32, 53] is a network protocol according to which a sender transfers data and a receiver is guaranteed to deliver this data even though packets sent between the sender and the receiver can be lost or become corrupted during transmission. There is only one proviso, namely that so now and then an undamaged packet is transferred from the sender to the receiver and vice versa.

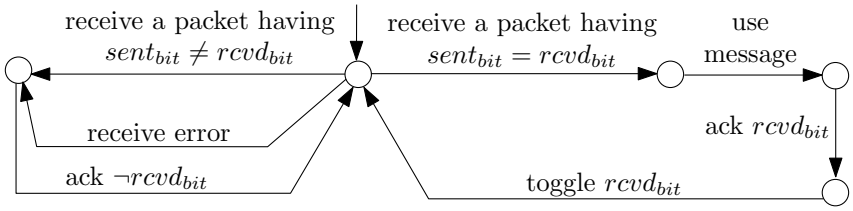
If a packet is lost or gets corrupted, it is reported by the receiver and it is then re-transmitted. Each packet comprises two parts, namely data and one-bit sequence number. The one-bit sequence number is toggled every time before sending the next new packet, i.e., it alternates for every two consecutive packets, except if these packets are retransmissions. The receiver receives a packet according to a predetermined bit. If the bit does not match the received packet is a retransmission and therefore discarded and negatively acknowledged to the sender. The sender retransmits each packet for which a negative acknowledgment is received.

Figure 5.11 shows the behaviour of both the sender and the receiver process. The sender process uses $sent_{bit}$ as a one-bit sequence to send a message and upon receiving the same value as acknowledgment, it generates the next packet. If the acknowledgment is the negation of the $sent_{bit}$, the last packet is retransmitted as shown in Fig. 5.11a.

The receiver process uses the $rcvd_{bit}$ to compare with the bit value of the incoming packet. If both are the same, the received packet is considered valid and it is delivered. Thereafter, its bit value is sent back as acknowledgment and the $rcvd_{bit}$ is toggled because the next packet is assumed to arrive with the alternating bit value.



(a) The sender process



(b) The receiver process

Fig. 5.11 The sender and the receiver processes of the alternating bit protocol

Figure 5.11b shows that it is also possible that the $rcvd_{bit}$ is different than expected or an error is received. In both cases a negative acknowledgment is sent. Initially, both the sender and the receiver have the same values (i.e., *true*) for the $sent_{bit}$ and the $rcvd_{bit}$ as shown on line 51, where the processes are initialised, in the specification listed in Fig. 5.12.

The channel between the sender and the receiver is also a process because of their asynchronous communication, i.e, the intermittent data transmission and no direct hand shake of the sender and the receiver. So, the channel receives a message from one process and hands over to the other. The channel is assumed lossy in this protocol, so a packet can be lost during transmission which is reported by the channel. The channel can process a maximum of one message at a time. A message is in the form of a packet if it travels from the sender to the receiver while it is just a message containing a single bit for the other way round.

The channel can report that it is loosing any type of message as shown in Fig. 5.13 where i is an internal action of the channel using which it is determined whether the data is lost. The action i is nondeterministic, in the sense that when i happens either the data can be lost, or be forwarded correctly. This models that there might be some mechanism, unknown to us, that determines what happens to the messages. By using this nondeterministic style, we do not have to know or model this mechanism. For the study of the alternating bit protocol the particulars of this mechanism are not really relevant.

```
sort D= struct d1 | d2;
     Channel_message= struct packet(data: D, bit: Bool) |
                              bit_msg(bit: Bool)?isBit_msg |
                              packet_lost |
                              bit_msg_lost;

act useMessage, getFromSource: D;
    rcvAtChannel, send2Channel, transfer2Channel,
    rcvFromChannel, send2Process,
                     transferFromChannel: Channel_message;
    i;

proc Sender(b: Bool) = sum d: D.getFromSource(d).Transmitter(d, b);
     Transmitter(d:D, b:Bool) = send2Channel(packet(d, b)) .
                   ( rcvFromChannel(bit_msg(b)) . Sender(!b)
                    +
                    (rcvFromChannel(bit_msg(!b))
                     +
                     rcvFromChannel(bit_msg_lost)
                    ) . Transmitter(d, b)
                   );

     Receiver(rcvd_bit: Bool) =
          sum d: D.rcvFromChannel(packet(d, rcvd_bit)) .
                     useMessage(d) .
                     send2Channel(bit_msg(rcvd_bit)) .
                     Receiver(!rcvd_bit)
                    +
                    (sum d:D.rcvFromChannel(packet(d, !rcvd_bit))
                             +
                             rcvFromChannel(packet_lost)
                    ) .
                    send2Channel(bit_msg(!rcvd_bit)) .
                    Receiver(rcvd_bit);

     Channel = sum m: Channel_message.
                  rcvAtChannel(m).
                  (i . send2Process(m)
                   +
                   i . (isBit_msg(m)
                        -> send2Process(bit_msg_lost)
                        <> send2Process(packet_lost)
                  )    ) . Channel;

init hide( { i },
      allow( { transfer2Channel, i, transferFromChannel,
                     useMessage, getFromSource },
        comm({ rcvAtChannel | send2Channel -> transfer2Channel,
               rcvFromChannel | send2Process ->
                                    transferFromChannel },
                Sender(true) || Channel || Receiver(true)
         ) ));
```

Fig. 5.12 The mCRL2 specification of the alternating bit protocol

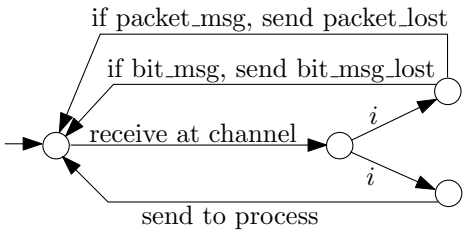


Fig. 5.13 Behaviour of a channel in ABP

But if we leave out the actions i, then, due to the communication mechanism, the receiving side of each channel can determine whether packets are lost or not. This is not desired, because this would allow us to make a transmission protocol that would refuse message loss by the channels, and always force the channels to deliver reliably.

In general it is necessary to know the following points to give formal specification of a protocol.

1. How many concurrent processes are there?
2. Is the communication synchronous or asynchronous?
3. Is the communication reliable or not?
4. Can messages be overtaken?
5. What are the buffer size of the involved channels?
6. What is the data (format and type) that is transferred from one place to another?
7. Is there any time triggered action/event?
8. What are the data structures and their initial values that a process uses?
9. Can a process fail?
10. Will the protocol terminate? If so, them upon which conditions and does it inform fellow components?
11. Which mechanisms in the protocol can be modelled with nondeterministic internal actions?

We need to address the above mentioned points from the informal description of a protocol to formally specify that protocol. Once you mastered how to specify data types and processes, you will find that the real difficulty in modelling lies in understanding the true nature of the protocol, which is often described quite imprecisely in natural language or semi-precise diagrams.

The description of the alternating bit protocol given in this section caters for all of the above-mentioned points. There are three concurrent processes, i.e., the sender, the receiver and their communication channels. We can use one communication channel because at any time only one direction of the communication channel is used. Either a packet travels or a packet's acknowledgment. If messages can travel in any direction at any time then we need to devise a separate channel between each pair of the sender and receiver because messages need to be transferred back and forth simultaneously.

Description of the formal specification of the alternating bit protocol. Here we describe the formal specification of the alternating bit protocol given Fig. 5.12. Line 1 shows the data that a process can send to the other with the combination of a bit. The bit is alternating for every two consecutive packets. Line 2 shows the type of messages. The sender process given in Fig. 5.11a in mCRL2's syntax is specified from line 13 to 21 and likewise, the receiver process given in Fig. 5.11b is described from line 23 to 34.

The channel process, which caters for both the sender and the receiver, is specified from line 36 to 43. As the same process is used for both sending and receiving messages from either side, it is possible that a sender may receive its own message. We solve this problem by separating message types for each side. The sender sends packets and the receiver either receives that packet or the packet_lost message whereas in its response the receiver sends bit_msg message and the sender either receives this message or the bit_msg_lost message.

There is another version of the specification for the alternating bit protocol which you can find in the directory `examples/academic/abp` in the mCRL2 installation. The specification of the alternating bit protocol in the distribution uses two separate channels to transfer messages between the sender and the receiver. In the alternating bit protocol this does not influence the behaviour as it happens that only one of these channels is used at any time. However, this is not generally the case. Minor changes in a specification can have a profound influence on the overall behaviour.

5.8 Modelling Milner's Cyclic Scheduler

Milner defined a cyclic scheduler to start and stop jobs [47]. Starting job i is represented by action $a(i)$ and stopping job i is represented by the action $b(i)$. In Milner's scheduler there are an arbitrary number N of processes, called cyclers, in a ring and cycler i controls starting and stopping job i. The scheduler guarantees two properties. All jobs must cyclically start in order, i.e., first job 0 must start, then job 1, etc. until job $N-1$, after which job 1 must be started. Furthermore, a started job must first be stopped, before it can be restarted.

Every cycler i receives a signal $signal(i)$ from the cycler before it to allow it to start its job. There is a special *Start* process to start the cycler 0. It can then perform action $a(i)$ to start job i. Subsequently, it either informs the next cycler that it can start, or it performs $b(i)$, indicating that job i has stopped. If both actions have been performed, cycler i starts over again waiting for a signal from the predecessor cycler.

Note that a cycler determines its next neighbour on the basis of its *id*, i.e., cycler i calculates the index of the next cycler as $(i+1) \mod N$. In this way, the first cycler is next to the last cycler and thus a ring is formed.

We specify the scheduler with only two cyclers. In the following specification, the sending/receiving of a signal to start a cycler is renamed to an internal action τ.

```
act a, b, signal, signalComm: Nat;

map N:Pos;

eqn N = 2;

proc Cycler(id: Nat) = signal(id) . a(id) .
                       (signal((id+1) mod N) . b(id)
                        +
                        b(id) . signal((id+1) mod N)
                       ) . Cycler(id);

     Start = signal(0) . delta;

     Scheduler2Procs =
          allow( { a, b, signalComm },
            comm( { signal | signal -> signalComm },
                 Start||Cycler(0)||Cycler(1)));

init hide( { signalComm }, Scheduler2Procs);
```

Exercise 5.24 Which of the following traces are possible in Milner's scheduler as given above?

1. $\tau \cdot a(0) \cdot \tau \cdot a(1) \cdot b(1) \cdot b(0) \cdot \tau$
2. $\tau \cdot a(0) \cdot a(1) \cdot b(0) \cdot b(1) \cdot \tau$
3. $\tau \cdot a(0) \cdot \tau \cdot a(1) \cdot b(0) \cdot \tau \cdot a(0) \cdot b(1)$
4. $\tau \cdot a(0) \cdot b(0) \cdot \tau \cdot a(1) \cdot \tau \cdot b(1)$

We can easily increase the number of cyclers in the above specification by changing the value of N and putting the required number of cyclers in parallel. As an exercise you can try to generate the state space for 3 or 4 processes. However, be warned, as the state space of Milner's scheduler grows exponentially in N, meaning that for large N the state space is too big to be generated in its entirety. This example also comes with the mCRL2 installation and it can be found in the directory `examples/academic/scheduler`.

Exercise 5.25 Give an mCRL2 specification of a vending machines that provides Coke, water and chocolate for 2, 1.80 and 2.10 euros, respectively. The vending machine accepts bank cards as well as coins of 10 cents, 20 cents, 50 cents, 1 euro and 2 euros.

A user can select a product and its quantity but the quantity must be less that 4 and greater than 0. The machine calculates the amount and gives the choice to pay either by cash or by card.

The user selects a payment choice and acts accordingly. If the card option is selected and the card is charged successfully, its receipt is printed and products are delivered. Otherwise, the choice of payment is again given by the machine with an error message. For the cash option, the machine continues receiving coins until the

required amount or more is received but invalid coins are rejected. Then the machine sends the product along with the spare change.

The machine shows a welcome screen if no user is using it and after its usage, it again shows the welcome screen.

Chapter 6
Model-Checking



In this chapter, we are going to learn about model-checking with mCRL2. Model-checking is the activity of checking properties of a given behaviour. In the context of software engineering such properties are also called requirements that express expected behaviours from the behavioural description. We use the terms 'property' and 'requirement' interchangeably.

A property describes some aspect of the global behaviour, such as "the system has no deadlock", but such properties can be far more involved. By formulating and checking such properties on behavioural specifications, we increase our trust that the behaviour is indeed what we believe that we had specified.

Generally, we first formulate a requirement in natural language, and subsequently translate it to a modal logic. Using tools we can determine whether the obtained formula is valid on the behaviour. This is called model-checking. In Fig. 6.1 it is shown how a specification and a requirement are formalised, and verified using model-checking, leading to the outcome *true* or *false*. In the case of *false*, we can get a counter example to explain why a certain property was not valid. Often a counter example is a simple sequence of actions, for instance showing a path to a deadlock, but counter examples can be much more complicated. In case of *true* a witness can be provided, which can help understanding why the requirement holds.

In this chapter we explain Hennessy-Milner logic [38], which is pretty basic, and extend it to the modal mu-calculus with data in the next chapter, leading to a very expressive property language. There are alternative modal logics such as LTL [49], CTL [13], and CTL*[16].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0_6

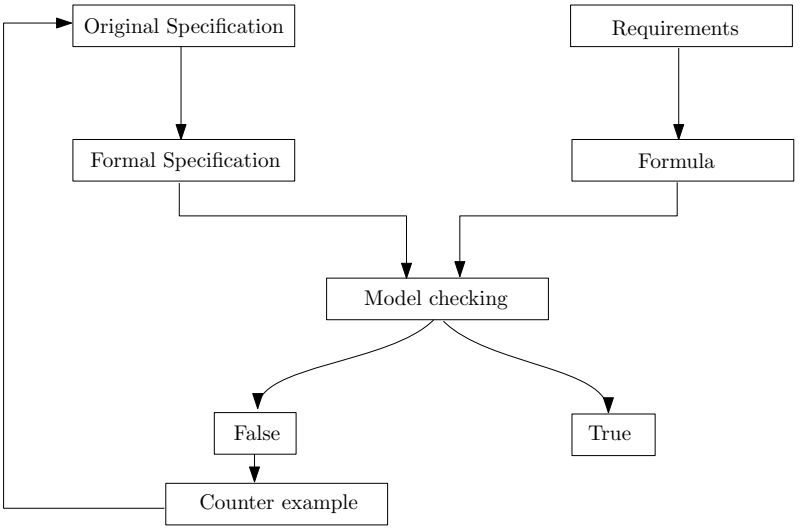


Fig. 6.1 Mechanism of model-checking

6.1 Hennessy-Milner Logic

Hennessy-Milner Logic (HML) is used to specify a property in terms of actions. A Hennessy-Milner formula is evaluated in a particular state and it is valid for a behaviour if and only if it holds in the initial state of this behaviour.

Definition 6.1.1 (Hennessy-Milner Logic) A formula in HML is written according to the following syntax where a is an action.

$$\varphi, \psi ::= \mathit{true} \mid \mathit{false} \mid \neg\varphi \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \Rightarrow \psi \mid \langle a \rangle \varphi \mid [a]\varphi.$$

In an LTS, the HML-formula *true* holds in every state and *false* holds nowhere, as shown in Fig. 6.2. In the syntax of Hennessy-Milner logic, negation ($\neg$), conjunction ($\wedge$), disjunction ($\vee$) and implication ($\Rightarrow$) have their usual logical meanings and they are expressed by the operators '`!`', '`&&`', '`||`' and '`=>`', respectively in mCRL2. The diamond modality $\langle a \rangle \varphi$ means that at least one action a can be done in the state such that φ holds in the state reached by doing the action a. It is false when either no action a is possible or φ is false after every a action. So, $\langle a \rangle \mathit{false}$ is always false whether an a action exists or not, because if an action a does not exist then it is clearly false and if action a exists, the formula *false* cannot hold in the state that is reached by doing an a.

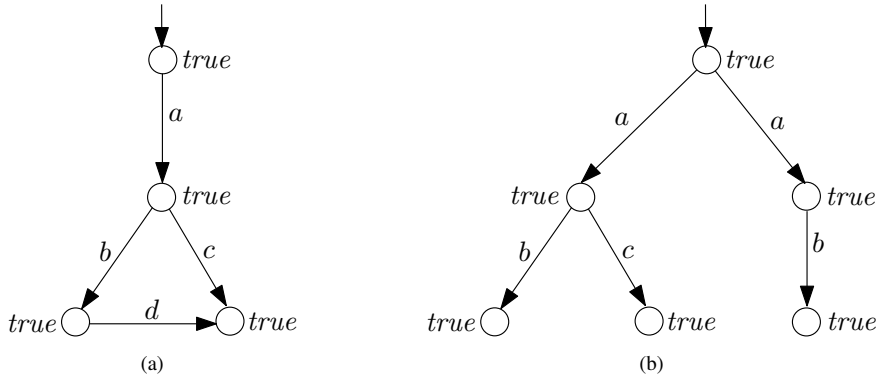


Fig. 6.2 LTSs showing that *true* holds at every state

The following formulas are true with regard to Fig. 6.2a.

$\langle a \rangle true$	There is at least one a action possible in the initial state.
$\langle a \rangle (\langle b \rangle true \vee \langle c \rangle true)$	After an action a, either an action b or an action c is possible.
$\langle a \rangle (\langle b \rangle true \wedge \langle c \rangle true)$	After an action a, both the actions b and c are possible.
$\langle a \rangle \langle b \rangle true$	There is an action a, after which action b occurs immediately.
$\langle a \rangle \langle b \rangle \langle d \rangle true$	In the initial state the trace $a \cdot b \cdot d$ is possible.

The box modality $[a]\varphi$ means that for all actions a that can be done in a state, the formula φ must hold in the state reached by doing this action a. Note that $[a]\varphi$ is true if no action a can be done in a state. Note also that $[a]true$ is a tautology (i.e., always true) whether the action a occurs in a state or not. If it occurs, thereafter the formula is true, because *true* always holds in the resulting state. If the action a does not occur, the formula is vacuously true.

The following formulas are true with regard to Fig. 6.2b.

$[a] \langle b \rangle true$	After every action a, at least one action b is possible.
$[a] (\langle b \rangle true \vee \langle c \rangle true)$	After every action a, at least an action b or an action c can be done.
$[b] false$	No action b is possible in the initial state.

An important property is that the box and diamond modalities are dual to each other, i.e., negating one yields the other. These identities and a few more are shown in Table 6.1.

Table 6.1 Identities in Hennessy-Milner Logic (HML)

$\neg \langle a \rangle \varphi = [a] \neg \varphi$	$\neg [a] \varphi = \langle a \rangle \neg \varphi$
$\langle a \rangle false = false$	$[a] true = true$
$\langle a \rangle (\varphi \vee \psi) = \langle a \rangle \varphi \vee \langle a \rangle \psi$	$[a] (\varphi \wedge \psi) = [a] \varphi \wedge [a] \psi$
$\langle a \rangle \varphi \wedge [a] \psi \Rightarrow \langle a \rangle (\varphi \wedge \psi)$	$[a] (\varphi \vee \psi) \Rightarrow \langle a \rangle \varphi \vee [a] \psi$

From the above equivalences, we can conclude that $[a]false \vee \langle a \rangle true$ is a tautology and $[a]false \wedge \langle a \rangle true$ is a contradiction.

Exercise 6.1 Which of the following formulas are true with regard to Fig. 6.2b.

1. $\langle a \rangle [a] false$.
2. $\langle a \rangle [c] false$.
3. $[b] false$.
4. $[a][b] false$.

Exercise 6.2 Write a formula in Hennessy-Milner logic which is true for Fig. 6.2a but false for Fig. 6.2b and vice versa.

6.2 Model-Checking with mCRL2

After specifying a system in mCRL2 and formulating a requirement, we can verify whether a requirement holds on the specification using model-checking tools. We can model-check a system with or without creating its state space explicitly. Both approaches have their pros and cons which we discuss later in this chapter.

6.2.1 Parameterised Boolean Equation System

We use parameterised boolean equation systems (PBES) to formally verify a formula without a state space [34]. In this technique, we linearise a specification written in mCRL2 and combine it with a formula as shown in Fig. 6.3. This formula can be written in Hennessy-Milner logic, the more advanced action- and regular formulas as presented in rest of this chapter, or in the full μ-calculus as given in Chap. 7.

We use the tool `lps2pbes` as a PBES generator and we use the tool `pbes2bool` to solve a PBES. If we get *true*, it means that the given formula holds in the initial state of the state space belonging to the mCRL2 specification. If the tool yields *false*, we can get a counter example that can help to understand why a formula does not hold.

Suppose `Test.mcrl2` and `myFormula.mcf` are the files for some mCRL2 specification and μ-calculus formula, respectively. We can apply model-checking as follows:

```
mcrl22lps -v Test.mcrl2 Test.lps
lps2pbes -v Test.lps -fmyFormula.mcf Test.pbes
pbes2bool Test.pbes -v
```

Here `-f` is the switch to indicate the file containing the modal formula and `Test.pbes` is the output file of the PBES generator.

The output of the PBES solver is the following when the option `-v`, which indicates that the tool runs in verbose mode, is used.

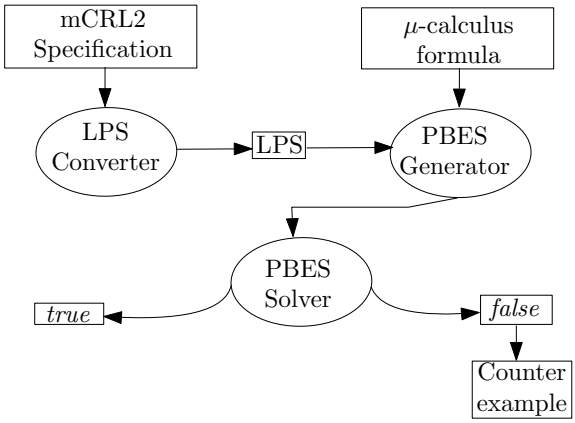


Fig. 6.3 PBES solving

```
Guessing input format: PBES in internal format
pbes2bool parameters:
input file:            Test.pbes
data rewriter:         jitty
substitution strategy: 0
search strategy:       breadth-first
solution strategy      lf
erase level:           none
Loading PBES in pbes format...
Processed 1 and generated 1 boolean variables.
Generated 1 BES equations in total, generating BES
Solving a BES with 1 equations using the local fixed point algorithm.
Solving equations of rank 0.
The solution for the initial variable of the pbes is true
true
```

There are many options to configure the PBES solver. See the help of `pbessolve` to explore the options. The most useful option among them is the `-s` switch that selects a "solve strategy" to solve a PBES optimistically.

We can also use the IDE of mCRL2 to add properties and verify them, as shown in Fig. 6.4. The IDE is far easier to use, but does not provide optimisation methods that may be needed to verify requirements on large and complex specifications. A property can be parsed to check syntax and then given to the PBES generator.

Properties can be verified one by one, or all at once. If a property is satisfied, it is shown in green and witnessing example is obtained by clicking on 'W'. An unsatisfied property is shown in red for which we can get a counter example clicking on 'C'.

Exercise 6.3 Explain the following Hennessy-Milner formulas and check them on all the LTSs shown in Fig. 6.5. Give a counter example for each if a formula does not hold.

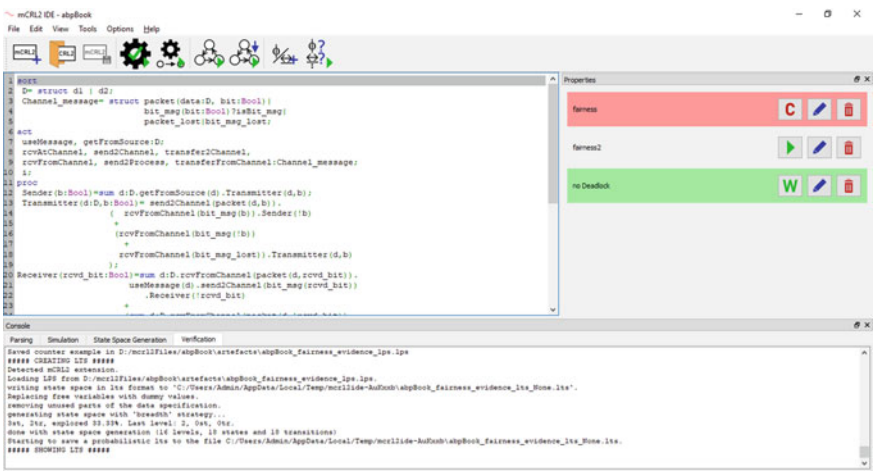


Fig. 6.4 Property verification using IDE

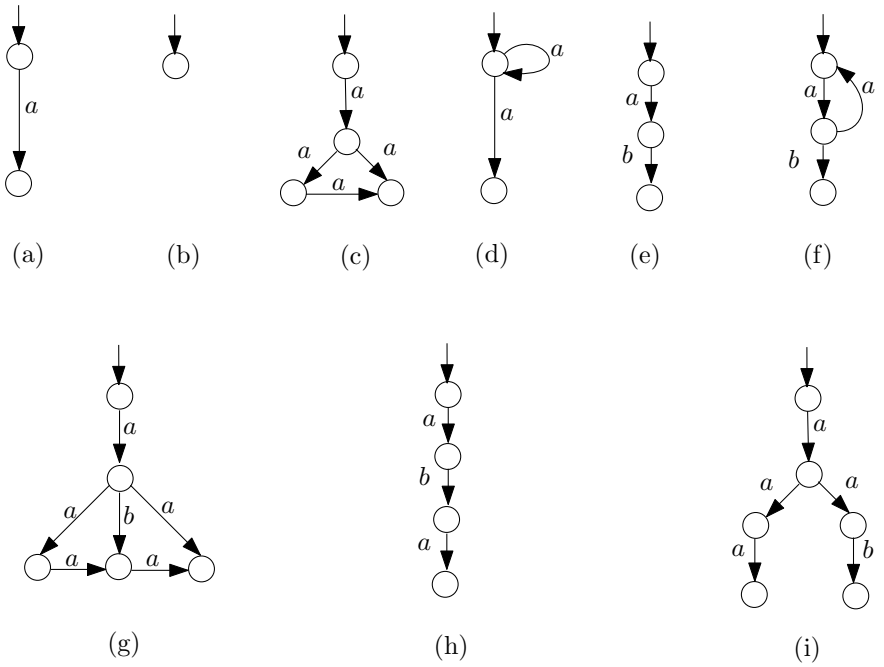


Fig. 6.5 LTSs for the Exercises 6.3 and 6.4

1. $\langle a \rangle [a] \langle a \rangle true$.
2. $[a] \langle a \rangle [a] false$.

Note: For learning the toolset, it is recommended to generate counter examples using the IDE and compare them with your examples.

A formula with an implication operator can be converted to a disjunction using the identity:

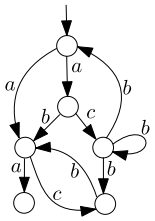
$$\varphi \Rightarrow \psi \approx \neg \varphi \vee \psi$$

So, $\langle a \rangle true \Rightarrow [b] false$ is the same as $[a] false \vee [b] false$, i.e., the formula is false only if both the actions a and b are possible at the initial state. This formula holds for all the LTSs given in Fig. 6.5 because $[b] false$ is true on the initial state of every LTS and it is in accordance with the domination law, i.e., $\varphi \vee true = true$ as shown in Table 6.2.

Exercise 6.4 See the answers for Exercise 6.3 and determine which of the following formulas hold for the LTSs given in Fig. 6.5.

1. $\langle a \rangle [a] \langle a \rangle true \Rightarrow [a] \langle a \rangle [a] false$.
2. $[a] \langle a \rangle [a] false \Rightarrow \langle a \rangle [a] \langle a \rangle true$.
3. $\neg false \vee true \Rightarrow \neg true$.

Exercise 6.5 Consider the following LTS and its mCRL2 specification.




```
act  a,b,c;

proc  P=(a.PL+a.(b.PL+c.PR));
      PL=a.delta+c.b.PL;
      PR=b.PR+b.b.PL+b.P;

init  P;
```

Which of the following formulas are true at the initial state? You can verify your answer using the IDE as shown in Fig. 6.4.

1. $[a][c](\langle b \rangle true \wedge [b] \langle b \rangle true)$.
2. $\langle a \rangle [c] \langle b \rangle \langle b \rangle \langle b \rangle \langle a \rangle true$.
3. $[a][c]([b] false \Rightarrow \langle a \rangle true)$.
4. $[a]([a] false \Rightarrow [c] false)$.

A formula can be simplified using the general equalities from propositional logic and Hennessy-Milner logic. In Table 6.2, some of these equalities are presented where φ and ψ are μ-calculus formulas.

Table 6.2 Equivalence between modal formulas

Propositional logic:	
$\varphi \wedge \psi = \psi \wedge \varphi$	$\varphi \vee \psi = \psi \vee \varphi$
$(\varphi \wedge \psi) \wedge \chi = \varphi \wedge (\psi \wedge \chi)$	$(\varphi \vee \psi) \vee \chi = \varphi \vee (\psi \vee \chi)$
$\varphi \wedge \varphi = \varphi$	$\varphi \vee \varphi = \varphi$
$\neg true = false$	$\neg false = true$
$\varphi \wedge true = \varphi$	$\varphi \vee true = true$
$\varphi \wedge false = false$	$\varphi \vee false = \varphi$
$\varphi \wedge (\psi \vee \chi) = (\varphi \wedge \psi) \vee (\varphi \wedge \chi)$	$\varphi \vee (\psi \wedge \chi) = (\varphi \vee \psi) \wedge (\varphi \vee \chi)$
$\neg(\varphi \wedge \psi) = \neg\varphi \vee \neg\psi$	$\neg(\varphi \vee \psi) = \neg\varphi \wedge \neg\psi$
$\neg\neg\varphi = \varphi$	$\varphi \rightarrow \psi = \neg\varphi \vee \psi$

Exercise 6.6 Draw two LTSs for each of the following formulas, so that each formula holds for one LTS and not for the other. Each answer is supposed to have at least one action a action at its initial state.

1. $[a](\langle b\rangle true \vee \langle a\rangle\langle b\rangle true)$.
2. $[a][b]([b]false \Rightarrow \langle c\rangle true)$.
3. $[a]([b]false \wedge \langle a\rangle\langle a\rangle true)$.

Exercise 6.7 Give a formula for each of the following requirements:

1. There should not be any action *switchOff* in the initial state.
2. Every initial action *switchOn* must directly be followed by an action *switchOff*.
3. There must be at least one d action in the initial state and each d action must directly be followed by another action e.

6.3 Action Formulas

Often requirements refer to multiple actions and in order to encode these into modal logic, this logic must be able to refer to more than just specific set of actions. For example, one may want to state that initially an arbitrary action must be done, but it does not matter which one that exactly is. Or alternatively, it may be desired to state that no action different from *switchOn* can be done, which requires to express all actions not equal to *switchOn*.

An action formula allows to express such sets of arbitrary actions. The syntax of an action formula is given below, where $a_1|\ldots|a_n$ represents a multi-action (described in Sect. 3.1).

$$\alpha ::= a_1|\ldots|a_n \mid true \mid false \mid \overline{\alpha} \mid \alpha \cap \alpha \mid \alpha \cup \alpha.$$

An action formula can be used in both box and diamond modalities, e.g., $[\alpha]\varphi$ and $\langle\alpha\rangle\varphi$. The formula $[\alpha]\varphi$ says that after any action in the set of actions represented by the action formula α the formula φ must hold. The formula $\langle\alpha\rangle\varphi$ expresses that there is an action in the set of actions represented by α after which φ holds in the resulting state.

The action formula *true* represents the set of all actions. Note that this can be confusing as at all other places *true* represents a boolean value. For example, $\langle true\rangle true$ says that an arbitrary action can be done in the initial state of a system. The first *true* is an action formula and the second one is a boolean value. Similarly, $[true]false$ expresses that no action is possible at the initial state.

In the figure given in Exercise 6.5, the formula $[true]\langle a\rangle true$ is false because after all first actions, the action a is not possible. But $\langle true\rangle\langle a\rangle true$ is true because there is an initial action, namely the action a at the left, followed by another action a.

The action formula *false* represents the empty set of actions, and as such it is hardly used. For instance $\langle false\rangle\varphi$ is always false, as it is impossible to select an action from an empty set. Dually, $[false]\varphi$ is always valid, because this formula says that an action from an empty set can be done, and there are no such actions, after which φ must hold.

The syntax $\overline{\alpha}$ represents an action other than α and in mCRL2, we express this with the '`!`' (not) operator. For example, $\langle\overline{a}\rangle true$ (in mCRL2: `<!a>true`) expresses that an action other than a is possible. This formula is false for the figure in Exercise 6.5. Note that, for action formulas the expected identity $\overline{true} = false$ and $\overline{false} = true$ hold as shown in Table 6.2.

It is possible to take intersections and unions in action formulas using the operators $\cup$ and $\cap$, expressed in mCRL2 syntax as `||` and `&&`, respectively. So, $\langle a \cup b\rangle true$ expresses that either an a or a b is possible in the initial state. And $[\overline{a \cup b}]false$ says that no action different from a and b can be done. The expected properties hold for union and intersection. So, $\overline{\alpha \cup \beta} = \overline{\alpha} \cap \overline{\beta}$ and $\overline{\alpha \cap \beta} = \overline{\alpha} \cup \overline{\beta}$. The use of intersection can be confusing as $a \cap b$ represents the empty set, namely the intersection of the sets of actions $\{a\}$ and $\{b\}$. Hence, as an example, $\langle a \cap b\rangle\varphi = false$.

Exercise 6.8 Consider the labelled transition system in Fig. 6.6 and determine whether the following formulas hold in its initial state.

1. $\langle true\rangle\langle true\rangle[true]false$.
2. $[true][true]false$.
3. $\langle true\rangle\langle switchOn\rangle[switchOff]false$.
4. $\langle true\rangle\langle switchOn\rangle[true]false$.

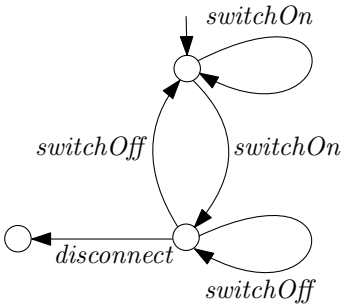
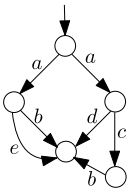


Fig. 6.6 A state space belonging to Exercise 6.8

Exercise 6.9 Consider the figure above and evaluate the following formulas on it.



1. $\langle a \rangle [true] \langle b \rangle true$.
2. $[true](\langle c \rangle true \wedge \langle e \rangle true)$.
3. $[a]\langle true \rangle [true] false$.
4. $[true](\langle d \rangle true \Rightarrow [b] false)$.
5. $[true][true]([true] false \vee [b] false)$.
6. $\langle \overline{a} \rangle true$.
7. $[\overline{a}] false$.
8. $\langle true \rangle (\langle \overline{b} \rangle \langle b \rangle true \wedge \langle \overline{d} \rangle true)$.
9. $\neg [\overline{a}] false$.

6.4 Regular Formulas

With action formulas, we can express sets of actions. Regular formulas extend this by allowing to express sets of traces of actions. As with action formulas, regular formulas are used inside the modal operators. A regular formula has the following syntax

$$R ::= \epsilon \mid \alpha \mid R \cdot R \mid R + R \mid R^* \mid R^+$$

where α is an action formula and ϵ represents the set containing the empty sequence of actions. The formula $\langle R \rangle \varphi$ means that in the current state, one of the traces from R can be done such that φ holds in the resulting state. The formula $[R]\varphi$ is valid in

Table 6.3 Equivalence between regular formulas. Operators μ and ν are discussed in Chap. 7

$\langle\epsilon\rangle\varphi = \varphi$	$[\epsilon]\varphi = \varphi$
$\langle false\rangle\varphi = false$	$[false]\varphi = true$
$\langle af_1 \cup af_2\rangle\varphi = \langle af_1\rangle\varphi \vee \langle af_2\rangle\varphi$	$[af_1 \cup af_2]\varphi = [af_1]\varphi \wedge [af_2]\varphi$
$\langle af_1 \cap af_2\rangle\varphi \Rightarrow \langle af_1\rangle\varphi \wedge \langle af_2\rangle\varphi$	$[af_1 \cup af_2]\varphi = [af_1]\varphi \wedge [af_2]\varphi$
$\langle \exists d : D.AF(d)\rangle\varphi = \exists d : D.\langle AF(d)\rangle\varphi$	$[\exists d : D.AF(d)]\varphi = \forall d : D.[AF(d)]\varphi$
$\langle \forall d : D.AF(d)\rangle\varphi \Rightarrow \forall d : D.\langle AF(d)\rangle\varphi$	$[\forall d : D.AF(d)]\varphi \Leftarrow \exists d : D.[AF(d)]\varphi$
$\langle R_1 + R_2\rangle\varphi = \langle R_1\rangle\varphi \vee \langle R_2\rangle\varphi$	$[R_1 + R_2]\varphi = [R_1]\varphi \wedge [R_2]\varphi$
$\langle R_1 \cdot R_2\rangle\varphi = \langle R_1\rangle\langle R_2\rangle\varphi$	$[R_1 \cdot R_2]\varphi = [R_1][R_2]\varphi$
$\langle R^*\rangle\varphi = \mu X.(\langle R\rangle X \vee \varphi)$	$[R^*]\varphi = \nu X.([R]X \wedge \varphi)$[1]
$\langle R^+\rangle\varphi = \langle R\rangle\langle R^*\rangle\varphi$	$[R^+]\varphi = [R][R^*]\varphi$
$\neg\langle R\rangle\varphi = [R]\neg\varphi$	$\neg[R]\varphi = \langle R\rangle\neg\varphi$
$\langle R\rangle false = false$	$[R]true = true$
$\langle R\rangle(\varphi \vee \psi) = \langle R\rangle\varphi \vee \langle R\rangle\psi$	$[R](\varphi \wedge \psi) = [R]\varphi \wedge [R]\psi$
$\langle R\rangle\varphi \wedge [R]\psi \Rightarrow \langle R\rangle(\varphi \wedge \psi)$	$[R](\varphi \vee \psi) \Rightarrow \langle R\rangle\varphi \vee [R]\psi$

a state s if for all traces from R that can be done in s, the formula φ is valid in the state that is reached by such a trace.

The regular formula α represents the singleton set containing the (multi-)action α. So, the formula $\langle a\rangle\varphi$ means exactly the same whether a is viewed as an action or whether a is a regular formula. The same applies to $[a]\varphi$.

The regular formula ϵ represents the empty trace. The formulas $\langle\epsilon\rangle\varphi$ and $[\epsilon]\varphi$ both say that φ must hold in the current state. Formally, this is characterised by the identities shown in Table 6.3. The regular formula ϵ per se is not very often used, but it is useful to understand it, because it is part of R^*, which is explained below.

The formula $R_1 \cdot R_2$ represents the concatenation of the sequences of actions in R_1 with those in R_2. Consider for example, $\langle a \cdot b\rangle true$. The regular formula $a \cdot b$ represents the set with the trace $a \cdot b$. The whole formula expresses that it is possible to do a trace in this set, which in this case means that it is possible to do an action a followed by an action b.

The meaning of $R_1 + R_2$ is the union of the traces in R_1 and R_2. So, $\langle a + b\rangle true$ means that either the action a or the action b can be done. The formula $[a \cdot b + c \cdot d]\varphi$ holds in a state s if for all traces $a{\cdot}b$ and $c{\cdot}d$ that can be done in s, the formula φ holds in all states that are reached by these traces.

The '$+$' and '$\cdot$' operators on regular formulas are in a sense just shorthands as they can be eliminated within modalities using the identities given in Table 6.3. The regular expression with a Kleene star R^* represents the set in which the traces of R occur zero or more times in sequence. So, $\langle a^*\rangle\varphi$ expresses that by doing zero or more actions a, a state can be reached where φ holds. The formula $[a^*]\varphi$ expresses that φ must hold in any state that is reachable by doing any number of a's. In particular, φ must hold in the current state, namely the state that is reached after doing zero a's. The meaning of R^+ is the same as that of R^*, except that at least one trace of R must

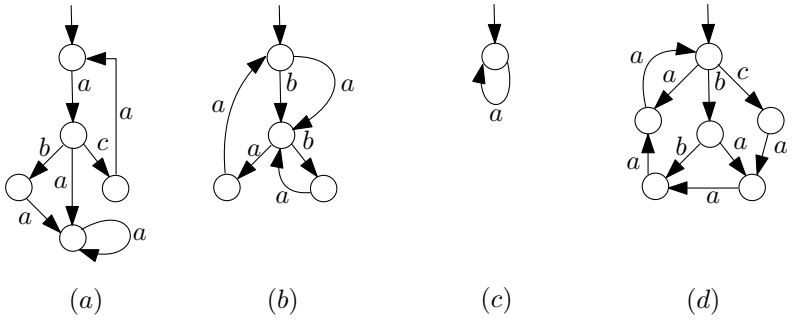


Fig. 6.7 Labelled transition systems on which $[true^*]\langle a\rangle true$ holds

be done. So, $[a^+]\varphi$ expresses that after doing one or more actions a the formula φ is valid.

In the examples up till now, we only used actions in regular formulas. By using action formulas we can express many interesting properties in a concise way. The formula $[true^*]\varphi$ is such an example. It expresses that the formula φ must hold in any state reachable from the initial state. The action formula *true* represent the set of all actions, and $true^*$ stands for the set of all sequences of actions. So, clearly $[true^*]\varphi$ expresses that in the states reachable by any trace, φ must hold. So, $[true^*]\langle a\rangle true$ expresses that in any reachable state an action a must be possible. This formula holds for all the LTSs in Fig. 6.7 and is invalid for all the LTSs in Fig. 6.8.

If we change our formula by adding $true^*$ before the action a we obtain $[true^*]\langle true^* \cdot a\rangle true$. This expresses that in any reachable state there must be a path to an action a. This change makes the formula for the LTS in Fig. 6.8c valid because a is not always immediately possible, but it can always be done after doing some other actions first. Note that this formula is still valid for all LTSs in Fig. 6.7 and not valid for any other LTS in Fig. 6.8.

The formula $[true^*]\langle true\rangle true$ means that there is no deadlock in any reachable state, or, in other words, after any sequence of actions, a next action is possible. We

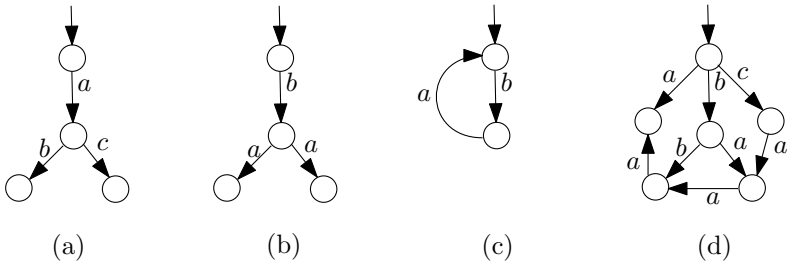


Fig. 6.8 Labelled transition systems on which $[true^*]\langle a\rangle true$ is invalid

can see that all LTSs given in Fig. 6.7 are deadlock-free, and those in Fig. 6.8 have at least one deadlock, except for Fig. 6.8c.

Instead of formulating that a property must hold for all reachable states, we can also say that we want to consider all states reachable through all but a specific action. The following formula for instance expresses that a behaviour is free of deadlocks unless an action *error* occurs:

$$[\overline{error}^*]\langle true\rangle true$$

Recall that the formula $[a]false$ expresses that it is not possible to do an action a. Hence, the formula $[true^* \cdot a]false$ expresses that an action a cannot occur in any reachable state.

Suppose we want to express that between an action a and an action b an action c must happen. This can be expressed by saying $[true^* \cdot a \cdot \overline{c}^* \cdot b]false$. If there is a sequence in which an action a is followed by a b without a c in between, then we end up in a state where *false* must hold and this is clearly impossible.

Now assume we want to express the property that between an action a and an action b, no action c is possible. In principle the following formula would be correct:

$$[true^* \cdot a \cdot true^* \cdot c \cdot true^* \cdot b]false.$$

But it may express too much. Consider the trace $a{\cdot}b{\cdot}c{\cdot}a{\cdot}b$. The formula is false as c can clearly happen between the first a and the second b. But we feel that the property holds for this trace as between each successive actions a and b no c can occur. We can rephrase this by saying that after an action a and a subsequent action c, an action b must occur. This is expressed by $[true^* \cdot a \cdot \overline{b}^* \cdot c]false$. This latter formula is generally the desired translation of the property.

Exercise 6.10 Which of the following formulas are true for the LTSs given in Figs. 6.7 and 6.8?

1. $[\overline{a}]\langle a\rangle true$.
2. $[\overline{a}]\langle \overline{a}\rangle true$.
3. $[\overline{a}^*]\langle a\rangle true$.

Exercise 6.11 Express the following properties as modal formulas

1. The action *error* cannot occur.
2. After the system has started (action *start*) it cannot be started again without being stopped (action *stop*) first.
3. It is possible to perform an action *hurrah*.
4. After a system has started it is always possible to stop it, as long as it did not stop.

Exercise 6.12 Write the following properties in a simpler way where R is an arbitrary regular formula.

1. $[R]\mathit{true}$.
2. $\langle R\rangle \mathit{false}$.
3. $[R^*]\mathit{false}$.
4. $\langle R^*\rangle \mathit{true}$.

An automatic teller machine. As an example we write down a number of requirements for an ATM (Automatic Teller Machine) and translate these requirements to modal formulas. In our ATM a user can insert a card, obtain money and take the card back. For this system, a few functional requirements and their respective formulas are given below. Note that behaviour, such as that of an ATM, can be modelled at various levels of abstraction and detail. Depending on the purpose more or less detail is required. In general, the more abstract the model and the requirements are, the more remote it is from the actual behaviour of the system. But if the behaviour is modelled in too much detail, it may be hard to get the modal formulas verified.

Inserting a card after another card is not possible. This means that it is not possible to insert the card twice, without removing it in between. It is formalised as:

$$[\mathit{true}^* \cdot \mathit{insert_card} \cdot \overline{\mathit{take_card}}^* \cdot \mathit{insert_card}]\mathit{false}.$$

An inserted card is eventually taken back. After a card insertion, as long as a card is not taken back, there must be a path towards a state where it is possible to take the card back. It is formulated as:

$$[\mathit{true}^* \cdot \mathit{insert_card} \cdot \overline{\mathit{take_back}}^*]\langle \mathit{true}^* \cdot \mathit{take_back}\rangle \mathit{true}.$$

This formula is again discussed in the next chapter where we provide a formula that expresses that *take_back* must inevitably take place.

It is always possible to cancel a transaction. All of the states are supposed to provide at least one possibility of the action *cancel*. It is formulated as:

$$[\mathit{true}^*]\langle \mathit{cancel}\rangle \mathit{true}.$$

Immediately switching off a machine is not possible after switching it on. This requirement is for instance useful if a graceful degradation (proper shutdown) is required after switching on a machine.

$$[\mathit{true}^* \cdot \mathit{switch_on} \cdot \mathit{switch_off}]\mathit{false}.$$

Entering a PIN code must be an immediate action after inserting an ATM card. This can be formalised by stating that one or more actions between *insert_card* and *pin_code* are not possible and that it is guaranteed that after the insertion of a card, the pin code is provided immediately.

$$[true^* \cdot insert_card \cdot true^+ \cdot pin_code]false \wedge [true^* \cdot insert_card]\langle pin_code\rangle true.$$

Only switching off puts an ATM into a deadlock state. It means taking another action after the action *switch_off* is not possible and all other actions will keep the system deadlock free.

$$[true^* \cdot switch_off \cdot true]false \wedge [\overline{switch_off}^*]\langle true\rangle true.$$

Cash cannot be withdrawn after transferring money or paying a bill or tax deduction. Suppose some processes can also affect your bank account. For example, *payBill*, *transferMoney* and *deductTax* and your bank has imposed some strange restriction that you cannot withdraw cash after these actions.

$$[true^* \cdot (transferMoney + payBill + deductTax) \cdot true^* \cdot withdrawCash]false.$$

The action *take_back* for a card is not possible at the initial state but it must occur exactly once in every sequence of actions.

$$[take_back]false \wedge [true^+]\langle take_back\rangle true.$$

Exercise 6.13 Is there any difference if we change the formula for "Only switching off puts an ATM into a deadlock state" as given above into the formula given below?

$$[\overline{switch_off}^*]\langle true\rangle true.$$

Exercise 6.14 How can we express a requirement that says that there is at least one *switch_off* action and only this action can put an ATM in a deadlock?

Exercise 6.15

1. Can the following pairs of formulas be differentiated by giving an LTS for which one formula is true and the other is false.
   (a) $\langle true^* \cdot a\rangle true$ and $\langle \overline{a}^* \cdot a\rangle true$.
   (b) $[true^*]\langle true\rangle true$ and $[true^+]\langle true\rangle true$.
2. Write a formula to show that only the actions *switch_off* and *shutDown* can put an ATM system into a deadlock state. In other words, a deadlock implies that any of the above actions has occurred.
3. What is the meaning of $[false^*]\langle true\rangle true$.

6.5 Formulas with Data and Quantifiers

In process specifications data plays an important role. In modal formulas data can also be used in various manners and especially for more involved properties it also plays a vital role. Let us start with actions.

Actions in modal formulas can carry data arguments, just as actions in processes. For instance, we can have an action $error(n)$ carrying a number n indicating the type of the error. Expressing that errors 17 and 25 can not occur, can be done with the following formula:

$$[true^* \cdot (error(17) \cup error(25)).]false.$$

Instead of the union between actions, $error(17) \cup error(25)$ we could have written the union of regular formulas $error(17) + error(25)$ with exactly the same effect.

We can also use variables, which must be declared using the universal quantifier *forall* ($\forall$) and the existential quantifier *exists* ($\exists$). A formula $\forall x{:}D.\varphi$ is true iff φ holds for all values of type D that the variable x can take. In $\exists x{:}D.\varphi$ only one value for x needs to make the formula φ true. In the tools the universal and existential quantifier are written as the following examples show:

- `forall i,j: Nat.[true* . a(i,j)]false`
- `exists b:Bool.<true* . a(b)>true`

An example of a modal property with data can be taken from the alternating bit protocol. If a message is read but not delivered, then the sender cannot read any other messages to send. The formula for the requirement is:

$$\forall d, e{:}D.[true^* \cdot getFromSource(d) \cdot \overline{useMessage(d)}^* \cdot getFromSource(e)]false.$$

In the formula above, we can reduce the complexity of the underlying parity game [36] by using quantifiers as close as possible near their use, as given below. This helps in efficient model-checking.

$$[true^*]\forall d{:}D.[getFromSource(d) \cdot \overline{useMessage(d)}^*]\forall e{:}D.[getFromSource(e)]false.$$

Quantifiers can even be written inside the modalities. The following identities can help to push quantifiers inside.

$$\exists d{:}D.\langle R(d)\rangle\varphi = \langle \exists d{:}D.R(d)\rangle\varphi,$$
$$\forall d{:}D.[R(d)]\varphi = [\forall d{:}D.R(d)]\varphi$$

provided the variable d does not occur in φ.

Let us discuss another example with regard to the LTS shown in Fig. 6.9 where the action $b(n)$ is followed by the sequence of actions $a(0)\cdots a(n-1)$, where n is a positive number smaller than 5. We want to express that the number m occurring in any action $a(m)$ is less than that used in a preceding action b.

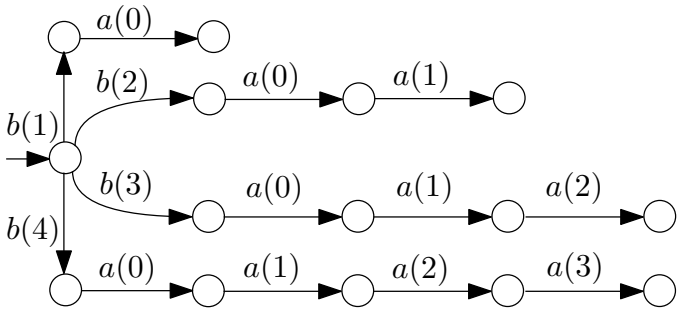


Fig. 6.9 Example for quantifiers

$$[true^*]\forall n{:}\mathbb{N}.[b(n) \cdot true^*](\forall m{:}\mathbb{N}.(n \leq m) \Rightarrow [a(m)]false).$$

Alternatively, the condition $n \leq m$ can also be written in negated form at the end.

$$[true^*]\forall n{:}\mathbb{N}.[b(n) \cdot true^*](\forall m{:}\mathbb{N}.[a(m)](n > m)).$$

The formula can now be read by saying that if an action $b(n)$ is followed by an action $a(m)$ then n must be larger than m.

There is yet another way to express that it is required that $n \leq m$ in the formulas above. We only want to allow actions $a(m)$ where $m < n$. The expression $m < n$ is a truth value. If we use a truth value inside a modality, and it is equal to *true*, it represents all actions, and if it is equal to false, it represents the empty set of actions. Using this, an equivalent form of the formulas above is given by:

$$[true^*]\forall n{:}\mathbb{N}.[b(n) \cdot true^*](\forall m{:}\mathbb{N}.[(m < n) \cap a(m)]false).$$

Quantifying over infinite data domains has the risk that the tools start enumerating all elements over such a domain, and they end up in an infinite loop. Therefore, it is wise to restrict quantified infinite domains explicitly. In our example we know that the values of n and m do not become larger than 4, and we can add that as extra conditions in the formula.

$$[true^*]\forall n{:}\mathbb{N}.(n \leq 4) \Rightarrow [b(n) \cdot true^*](\forall m{:}\mathbb{N}.(m \leq 4 \wedge n \leq m) \Rightarrow [a(m)]false).$$

The tools are in some cases able to find out that variables from unbounded domains can only occur in a bounded fashion. Especially, when moving quantified variables closer to their use, increases the likelihood that tools determine the bounds themselves.

When formulating modal formulas with data to be read by the tools, it is necessary to use the keyword `val` around all conditions. This is due to the fact that diamond modalities and equality operators like $<$ are otherwise confused. So, the formula

given above needs to be formulated as shown below when it is used as input for tools.

```
[true*]forall n:Nat.val(n<=4) => [b(n).true*]
          (forall m:Nat.val(m<=4 && n<=m) => [a(m)]false)
```

When writing such formulas, adding extra brackets to limit the scope of quantified variables is advised. Otherwise, the scope of quantifiers may be different than expected, for instance because the $\wedge$ binds stronger than quantifiers.

Quantification can be used on any data type. In Chap. 5, there is an example using the data of weekdays. See the LTS in Fig. 5.10 and its specification. We can express that every action *transferDay* is directly followed by an action *forwardDay* with its next weekday using the following formula.

$$\forall d{:}\mathit{weekDays}.\langle \mathit{transferDay}(d) \cdot \mathit{forwardDay}(\mathit{nextDay}(d))\rangle \mathit{true}.$$

Exercise 6.16 Consider the critical section problem where n processes use the actions $wish(i)$, $enter(i)$ and $leave(i)$ for expressing the wish to enter, entering and leaving the critical section where $i{:}\mathbb{N}$. Express the following requirements with data and quantifiers.

1. At any time all n processes have a way to express the wish to enter the critical section. This means that the $wish(i)$ action can be preceded by some other actions. This can for instance happen if the process is already in the critical section. It must then first leave before it can do the action *wish*.
2. Mutual exclusion is guaranteed. If some process enters the critical section using the action *enter* then no action *enter* can happen again without an occurrence of an action *leave* happening in between.

Exercise 6.17 Write a formula to express that if one person switches on a coffee machine no other person can ever switch the coffee machine on. Use the action *switch_on* to model that the coffee machine is being switched on.

Exercise 6.18 Express that only participants with an ID i smaller than n are allowed to do an action $start(i)$. Participants with an ID higher or equal than n are not permitted to execute the action start.

6.6 What to do if Model-Checking Becomes Time Consuming?

The verification of modal formulas can be time consuming, especially when we analyse complex systems. In such cases, it might be necessary to find the most appropriate way to carry out the verification. In this section we outline some approaches that can be taken.

6.6.1 Reduce the Problem Size

A good guideline is that model-checking a formula on a concrete model should never take longer than a few minutes. If this is not the case, the model or the formula should be adapted, such that model checking is faster. Especially, when still working on formulating the formulas and the model, model-checking should be swift, to obtain fast feedback.

A straightforward way to limit the run-time of model-checking is to limit the parameters of a problem. For instance, for a data-communication protocol, limit the number of communicated message to one, initially. If multiple processes communicate to obtain a certain task, try to verify the formulas on the smallest number of processes that still provide the basic functionality. If you model a controller of a stepper motor, with 1000 positions, first limit it to two, or maybe even one position, for the initial verification.

Only when the verification is successful in this sometimes extremely reduced setting, the parameters can be increased, and verification is allowed to take longer. Always try to predict the time verification will take when increasing these parameters, as the growth in time and memory is generally exponential in these parameters.

It is not always possible to verify a system in exactly the same circumstances as it could be used. For instance, the above mentioned controller of a stepper motor may only be verifiable up till 20 positions. In such a case this is still worthwhile, as most issues that occur with the parameter 1000, also exhibit themselves when the parameter is much lower. But insisting that a verification with parameter 1000 is required, which might be totally untenable, does not provide all the valuable insights that a verification with 20 or even less positions can already give.

Sometimes, reformulating the specification can reduce the state space substantially [26]. For instance, a traffic light controller can receive information from k car detection sensors. If the controller stores for each sensor whether it did detect cars, this contributes a factor 2^k to the size of the state space, quickly making it impossible to generate and explore the state space. However, when the traffic light controller only inquires a sensor, when it needs its information to make a decision, the controller typically has a state space linear in k. Modelling to keep a state space as small as possible, is as it stands the most contributing factor to make verifiable models.

6.6.2 Optimising and Transforming a PBES

As indicated in Sect. 6.2.1 a property and a linear process are transformed to a PBES (Parameterised Boolean Equation System). A PBES is a sequence of boolean fixed point equations that has a unique solution. If the initial variable of a PBES has solution true, the property is valid on the state space. In this way model-checking boils down to solving PBESs.

In Sect. 8.3, we explain PBESs in more detail. PBESs can be analysed and optimised on their own. In some cases applying optimisations to PBESs is pretty straightforward and can make a huge difference in the time required to solve such a PBES. Such optimisations are explained in Sect. 8.4.

6.6.3 Model-Check Explicit State Spaces

A different approach is to apply model-checking directly on a labelled transition system. This has as advantage that generating an LTS is often faster than checking a modal formula, and moreover the LTS can be reduced using bisimulation before commencing the model-check. A disadvantage can be that the whole LTS must be generated before model-checking can start whereas model-checking on a linear process may already lead to an answer by only inspecting a small part of the behaviour.

Assume we have already generated an LTS and a modal formula is given in a file `modal.mcf`. The instructions

```
lts2pbes -fmodal.mcf input.lts output.pbes
pbessolve output.pbes
```

are used to generate and solve the PBES. Generating a PBES out of a labelled transition system also works for LTSs in .aut and .fsm format. However, when using these, a separate file with a data type and action declarations must be provided as an argument to this tool.

It is generally very beneficial to reduce a labelled transition system using strong or divergence preserving branching bisimulation before generating and solving the PBES. This is done as follows:

```
ltsconvert -ebisim input.lts reduced.lts
```

Using trace equivalence or weak trace equivalence is not advised as these reductions do not preserve the validity of the modal formulas. We saw in Sect. 4.1 that trace equivalence does not preserve deadlocks. So, if the modal formula $[true^*]\langle true\rangle true$ can be shown to hold on a trace-reduced state space, this says nothing about the original which can contain deadlocks.

When used with care branching bisimulation and divergence preserving branching bisimulation can be used and can lead to much larger reductions of an LTS. Branching bisimulation is applicable to safety properties, but not to eventualities where actions can be reached via τ-loops as branching bisimulation removes such loops. Divergence preserving branching bisimulation keeps such loops, and it is generally safe to use, except if a formula counts the number of consecutive τ's in some way, as such sequences are compressed by divergence preserving branching bisimulation.

6.6.4 CADP

By explicitly generating a state space it is also possible to use other tools for verification purposes. A particularly powerful toolset is CADP (Construction and Analysis of Distributed Processes) comparable to mCRL2. A comparison of these tools is given in [15]. CADP is developed at INRIA Rhone-Alpes in Grenoble.

CADP can be connected to mCRL2 for model-checking as shown in Fig. 6.10. We get a labelled transition system in .aut format by using the tool `ltsconvert`. This LTS in .aut format is given to CADP where the tool *bcg_io* converts it into an LTS in binary coded graphs (bcg) format which it uses for model-checking.

In CADP, there is an on-the-fly model-checker, called evaluator [17] and it provides access to a number of different model-checking algorithms. The tool `bcg_open` involves an evaluator, a formula file (mcl) and a system in bcg format to perform model-checking. A formula in mcf has a slightly different syntax than μ-calculus formulas in mCRL2. After model-checking, the results of the tool `bcg_open` are either true or false along with a witnessing bcg. We can investigate the result by looking at the output of the tool *bcg_open* or by visualising the resulting bcg through the tools `bcg_draw`, or `bcg_edit`. We refer to the manuals for the above-mentioned tools to use them with some customisation. There is a GUI of

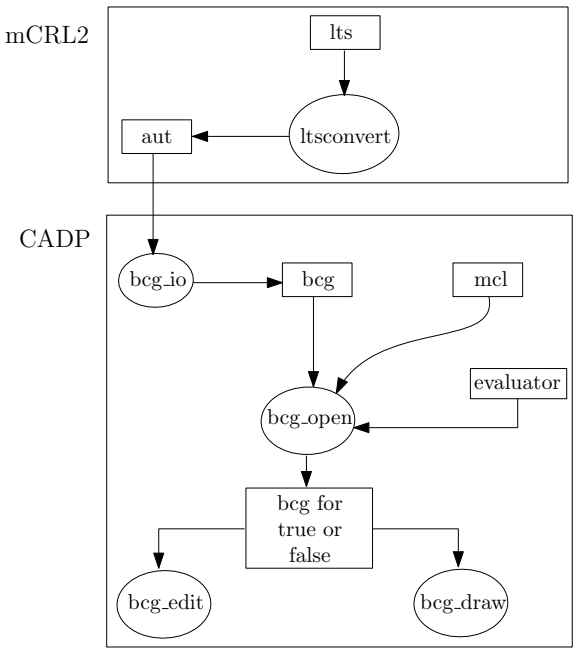


Fig. 6.10 Connecting mCRL2 with CADP

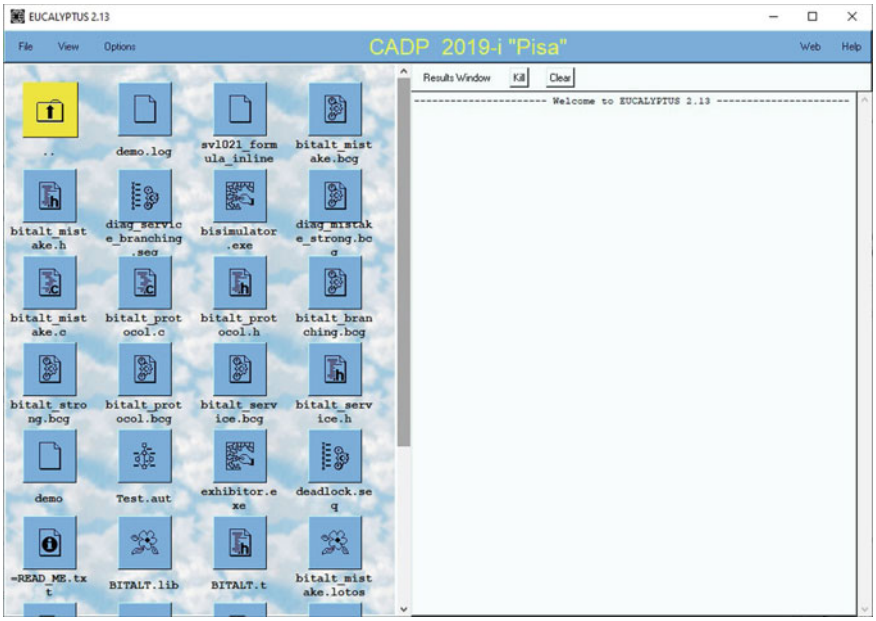


Fig. 6.11 xeuca for CADP

CADP called, `xeuca`, shown in Fig. 6.11 that is helpful in exploring all possible options provided by the tools in this toolset.

The tools can either be run through this graphical interface or run in a terminal. Below we show how to verify a property on a labelled transition system `test.lts` created in mCRL2, where the rest of the steps are done in CADP.

```
ltsconvert test.lts test.aut
bcg_io test.aut test.bcg
bcg_open test.bcg evaluator -verbose -bfs -diag results.bcg f1.mcl
bcg_edit results.bcg
```

6.6.5 LTSmin and Pbessolvesymbolic

Using so called symbolic methods, it is possible to verify properties on much larger state spaces. This is particularly effective when the behaviour has a regular structure, but leads to a very large state space. On inherently complex behaviours tools such as `pbessolve` that explores all states explicitly, can be faster. The symbolic tools represent the states of a labelled transition system symbolically, often using BDDs (Binary Decision Diagrams) or a similar structure.

The tool `pbessolvesymbolic` is used in the same way as `pbessolve`. A disadvantage of the symbolic solver is that it is not able to generate counter examples and witnesses, as it stands.

```
lps2pbes -fformula.mcf input.lps file.pbes
pbessolvesymbolic -v file.pbes
```

The toolset LTSmin is developed at Twente University and it is a generic toolset for analysing labelled transition systems [43]. It can be connected to various other tools, among which the mCRL2 toolset. To model-checks an LTS, this tool supports properties in LTL, CTL and the μ-calculus [56]. This tool is specialised in reachability checking while verifying safety properties. The tool is capable of performing reachability with symbolic state storage, full symbolic reachability using BDDs, distributed reachability by employing the MPI library, and multi-core reachability.

Chapter 7
The Modal μ-Calculus



We saw in the last chapter that a regular formula can express properties comprehensively but still there are some properties which are out of its scope. In particular, expressing that an action c must inevitably be done after an action a, is not possible with regular formulas. Note that this property does not hold in Fig. 7.1a. But if the b-loop would be removed, it would be valid.

Another property that one sometimes want to express, but which cannot be expressed with regular formulas is that an action d happens infinitely often in sequence. In Fig. 7.1b, this property is not valid, as at most two d's can be done in sequence. However, if the d-transition outgoing from state s_2 would be rerouted to end in s_2, this property would be true in both states s_1 and s_2.

The properties given above can be expressed in the modal μ-calculus [45]. The syntax of the modal μ-calculus is:

$$\varphi ::= \ true \mid false \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \Rightarrow \varphi \mid \langle a \rangle \varphi \mid [a]\varphi \mid \mu X.\varphi \mid \nu X.\varphi \mid X.$$

This calculus extends Hennessy-Milner logic by adding the minimal ($\mu X.\varphi$) and maximal ($\nu X.\varphi$) fixed point operators. The X is a variable representing the set of states where the formula is valid. Other variables like Y, Z can also be used. This X can be used inside φ. So, a typical formula is $\nu X.\langle a \rangle X$.

Exercise 7.1 The regular formula $\langle true^* \cdot c \rangle true$ expresses that there is a single path ending in a c, but it does not guarantee that on every path c must unavoidably be done. It holds in the LTS of Fig. 7.1a. Determine that none of the following formulas are suitable either to express that c must unavoidably be done. Also determine whether they are valid in Fig. 7.1a.

1. $[true^*]\langle c \rangle true$.
2. $\langle \overline{c}^* \cdot c \rangle true$.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-3-031-23008-0_7.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0_7

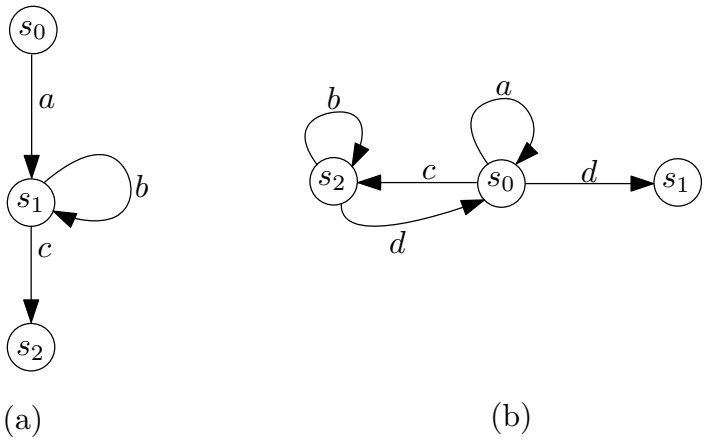


Fig. 7.1 Two transition systems

3. $[\overline{c}^*]\langle c\rangle true$.
4. $[true^* \cdot \overline{c}]false$.

7.1 Minimal Fixed Points

Liveness properties are expressed using a minimal fixed point. It means something will eventually happen after a finite number of steps. Consider the general shape of a minimal fixed point formula.

$$\mu X.\varphi(X).$$

We write $\varphi(X)$ to stress that X can occur in φ. The variable X represents a set of states where we assume this formula is valid. Given this set X, the formula $\varphi(X)$ also represents a set of states, namely those where $\varphi(X)$ is valid. If, in terms of sets of states, $X = \varphi(X)$, then X is called a solution. The formula $\mu X.\varphi(X)$ is valid in the minimal set of states that is a solution.

This minimal set of states is a subset of all solutions, and it is unique, provided in $\varphi(X)$, the variable X is preceded by an even number of negations. So, $\mu X.[a]X$, $\mu X.\neg[a]\neg X$ and $\mu X.X$ are ok, and $\mu X.\neg X$ is not. Formulas in which the bound variable X is preceded with an odd number of negations have no meaning and should not be used.

Consider as an example the formula

$$\mu X.[\overline{c}]X,$$



Fig. 7.2 The formula $\mu X.[\overline{c}]X$ is valid in all states

which we denote in tool syntax as `mu X.[!c]X`. It expresses that eventually a state is reached where no other actions than c can be done.

We want to evaluate this formula on the states of Fig. 7.2. If we take $X = \{s_0, s_1, s_2\}$ then we find that $\varphi(X)$ is also valid in all states $\{s_0, s_1, s_2\}$. So, this X is a solution. If we set X to any other set of states, we find that this set is not a solution, as in all those cases the set where $\varphi(X)$ is valid differs from that of X. So, $X = \{s_0, s_1, s_2\}$ is the only solution, and hence it is also the minimal solution.

An effective method to find the minimal set of states that is a solution of $X = \varphi(X)$ is to set X initially to $\varnothing$, calculate the set of states where $\varphi(X)$ is valid, set X to this set of states, and repeat this process until $X = \varphi(X)$. The set X found in this way is the minimal solution.

We apply this to the formula $\mu X.[\overline{c}]X$ and Fig. 7.2. If we take X to be the empty set of states, the set of states where $[\overline{c}]X$ is valid, is $\{s_1, s_2\}$. So, $X = \varnothing$ is *not* a solution. If we try $X = \{s_1, s_2\}$, then we find that $[\overline{c}]X$ is valid in $\{s_1, s_2, s_3\}$, and so $X = \{s_1, s_2\}$ is also not a solution. Calculation $[\overline{c}]X$ with $X = \{s_0, s_1, s_2\}$ yields again $X = \{s_0, s_1, s_2\}$, so, $X = \{s_0, s_1, s_2\}$ is indeed the minimal solution.

We now want to express the property that an action c must unavoidably happen. The formula $\mu X.[\overline{c}]X$ is not suitable as it says that ultimately a state is reached where only the action c is possible, but it is also valid in a deadlock state. In order to say that action c will unavoidably happen, we can say that ultimately we must end up in a state where only actions c can be done, and exclude the option for a deadlock explicitly. The formula

$$\mu X.[\overline{c}]X \wedge \langle true \rangle true$$

expresses this. This formula holds in states s_0 and s_1 of Fig. 7.2, but not in state s_2.

The formula

$$\mu X.\langle a \rangle X$$

is always false. This can be seen by taking the solution $X = \varnothing$. As it is not possible in any state to do an action a, and end up in no state, i.e., a state taken from the empty set of states, $\langle a \rangle X$ is also empty, and therefore, the empty set is a solution. As the

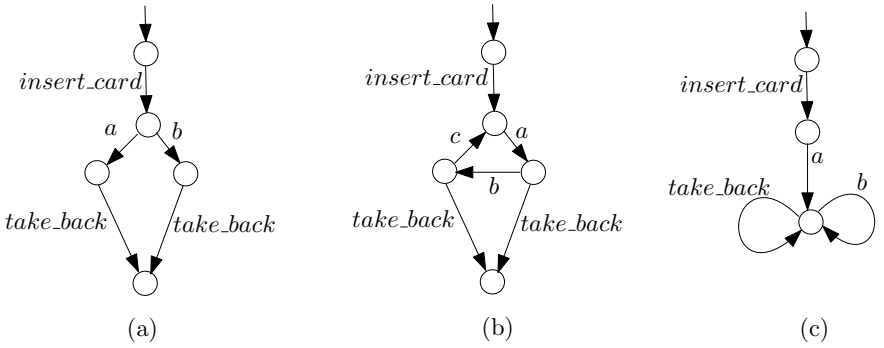


Fig. 7.3 Verifying a liveness requirement of an ATM

empty set is a subset of any other set, it is also the smallest solution. So, the formula $\mu X.\langle a\rangle X$ is valid in all states in the empty set, which is equivalent to saying that it is valid nowhere. This is exactly the same as the formula *false*, so, $\mu X.\langle a\rangle X = \mathit{false}$.

We now discuss a few more examples.

An inserted card is eventually taken back. When inserting a card in a cash dispenser, it must be taken back. In other words, after inserting a card, all the paths must have an inevitable action $take_back$. It is formulated as:

$$[\mathit{true}^* \cdot \mathit{insert_card}]\mu X.[\overline{\mathit{take_back}}]X \wedge \langle \mathit{true}\rangle \mathit{true}.$$

The formula is true for the initial state in Fig. 7.3a and false for the initial states of both the Fig. 7.3b, c. In both latter cases it is possible to do an infinite sequence of actions, without ever doing an action $take_back$. In Fig. 7.3c this is the sequence $insert_card{\cdot}a{\cdot}b{\cdot}b{\cdot}b\cdots$.

There is some action a after which an action c is inevitable. This is defined as:

$$\langle \mathit{true}^* \cdot a\rangle(\mu X.[\overline{c}]X \wedge \langle \mathit{true}\rangle \mathit{true}).$$

There is some state at which the a action is disabled. It means we can find a state in which no action a is possible. So, this formula is valid if either action a cannot be done in the current state, or there is a step towards a state where this is the cases ($\langle \mathit{true}\rangle X$). The minimal fixed point expresses that this state must be reached in a finite number of steps, i.e., it is not possible to infinitely often select $\langle \mathit{true}\rangle X$. At some point the part $[a]\mathit{false}$ must become valid.

$$\mu X.([a]\mathit{false} \vee \langle \mathit{true}\rangle X).$$

This formula is valid for all the LTSs given in Fig. 7.4 and would only be invalid if the action a were possible in every state.

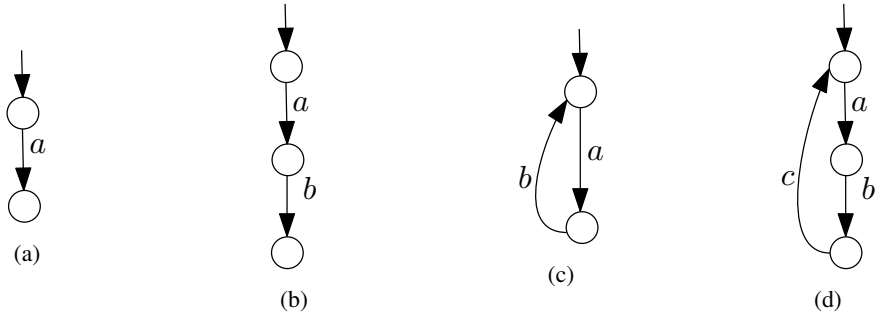


Fig. 7.4 Four transition systems

Exercise 7.2 For which of the LTSs given in Fig. 7.4 is the formula $\langle true^* \cdot a \rangle \mu X.[\overline{c}]X$ true?

Exercise 7.3 For which LTSs in Fig. 7.4 does the formula $\langle true^* \cdot a \rangle \mu X.[\overline{c}]X \wedge \langle true \rangle true$ hold?

Exercise 7.4 Can the following formulas be true for any state in an LTS? Provide an explanation.

1. $\mu Y.\langle a \cup b \rangle Y$.
2. $\mu Y.([true]Y \wedge \langle true \rangle true)$.

Let us discuss the following formula involving two actions in the box modality.

$$[true^*](\mu X.[\overline{a} \cap \overline{b}]X \wedge \langle true \rangle true). \tag{7.1}$$

This formula says that either an action a or an action b must be done ultimately.

Here, when $X = \varnothing$, we consider only the vacuously true cases for the expression $[\overline{a} \cap \overline{b}]X$. The set of actions $\overline{a} \cap \overline{b}$ is the set of all actions except a and b. Note that $\overline{a} \cup \overline{b}$ is equal to $\overline{a \cup b}$ which provides an alternative way to see this. So, in this case $[\overline{a} \cap \overline{b}]X$ says that no actions other than a or b can happen. Due to conjunction with $\langle true \rangle true$ the option of the deadlock is omitted. So, with $X = \varnothing$ the formula is only true in states in which only actions a and b are possible, and at least one of them is present. When replacing X by the set of actions where this is true, we find a set of actions where, either a or b must be done, or where in one step, not being a and b, a state is reached where only a or b is possible. If we repeat this, the formula clearly states that after any finite number of actions, not being a or b a state is reached in which it is enforced that an a or a b must be done. The $[true^*]$ prefix of the formula, expresses that this property must hold in any state.

We apply Formula 7.1 to a number of examples in Fig. 7.5 to illustrate how this approximation method works.

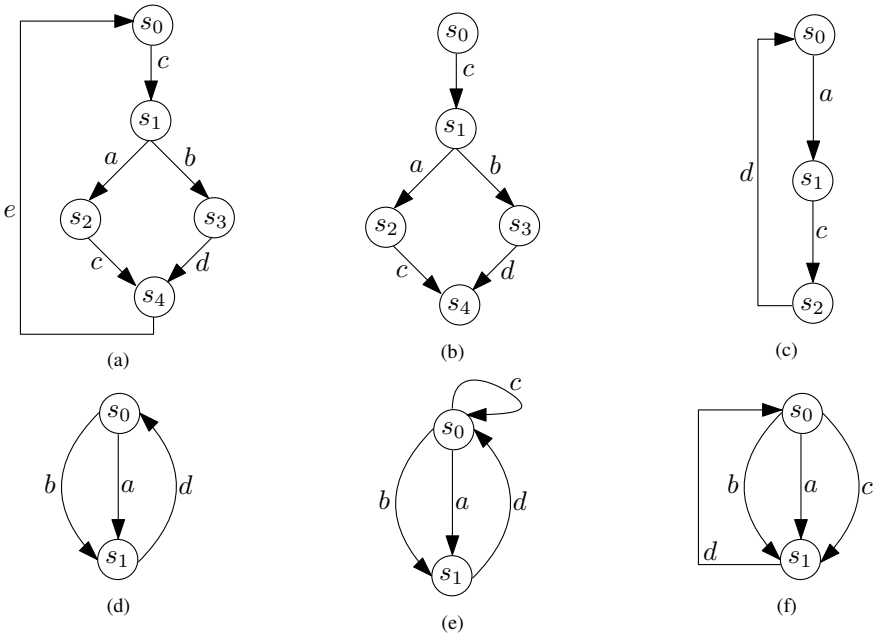


Fig. 7.5 Six transition systems to illustrate fixed point iteration

Figure 7.5**a.** We start with $X = \varnothing$ and in the first iteration,

$$X = \varnothing \cup \{s_1\}$$

because the expression $[\overline{a} \cap \overline{b}]X$ is true in state s_0 because an action a or b is enforced in this state. There is no other state with this property.
In the second iteration with $X = \{s_1\}$ we obtain

$$X = \{s_1\} \cup \{s_0\}$$

as from state s_0, we reach a state in $X = \{s_1\}$ by doing some action other than a and b, i.e., the action c. Note that there is no other state from which s_1 must be reached by all its outgoing not a and b actions.
In the third iteration with $X = \{s_0, s_1\}$ we get

$$X = \{s_0, s_1\} \cup \{s_4\}$$

because from state s_4, we do the action d and reach a state in $X = \{s_0, s_1\}$.
In the fourth iteration the set X becomes

$$X = \{s_0, s_1, s_4\} \cup \{s_2, s_3\}$$

as from both states s_2 and s_3, all actions go to a state in $X = \{s_0, s_1, s_4\}$.
So, the fixed point part of Formula (7.1) is true in $X = \{s_0, s_1, s_2, s_3, s_4\}$, which means in any reachable state of Fig. 7.5, Formula (7.1) is valid in the initial state. And indeed, in any state reachable from the initial state either an action a or an action b must be done within a finite number of steps.

Figure 7.5b. We start with $X = \varnothing$ and after the first iteration in the fixed point part it holds that

$$X = \{s_1\}.$$

Repeating this again we obtain

$$X = \{s_0, s_1\}$$

and this is also the fixed point. And indeed, in states s_2, s_3 and s_4 no action a or b must ultimately happen. Even stronger, there is no possibility that such actions will ever happen in these states. So, the fixed point is only valid in a part of the reachable states, and not in all. Hence, formula (7.1) is invalid in Fig. 7.5b.

Figure 7.5c. We start with $X = \varnothing$ and in the first iteration

$$X = \{s_0\}$$

because the expression $[\overline{a} \cap \overline{b}]X$ is true in state s_0 as action a is enforced in this state. In the second iteration with $X = \{s_0\}$ we get

$$X = \{s_0, s_2\}$$

as from state s_2, we must reach a state in $X = \{s_1\}$ by doing some action other than a or b, namely the action d in this figure. After the third iteration with $X = \{s_0, s_2\}$ we have

$$X = \{s_0, s_1, s_2\}.$$

From state s_1, we must do the action c and reach a state in $X = \{s_0, s_2\}$. So, the fixed point part holds for $X = \{s_0, s_1, s_2\}$. As these are all reachable state, Formula (7.1) is valid in the initial state of figure (c).

Figure 7.5d. We start with $X = \varnothing$ and in the first iteration, and obtain

$$X = \{s_0\}$$

because in state s_0 only actions a and b can be done, and it is not a deadlock. After the second iteration we obtain

$$X = \{s_0, s_1\}$$

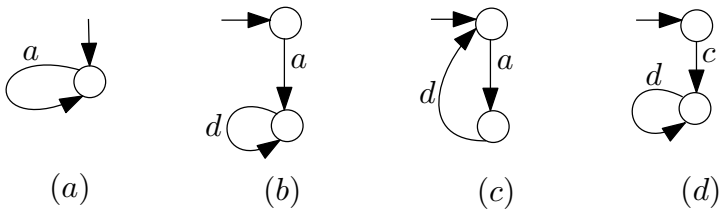


Fig. 7.6 Transition systems for Exercise 7.5

as from the state s_1, all actions other than an a and a b, which do not exist, lead to a state in $X = \{s_0\}$. As these are all actions, the fixed point part is true everywhere, and this means that Formula (7.1) holds for this labelled transition system.

Figure 7.5e, f. We start with $X = \varnothing$ and in the first iteration and get

$$X = \varnothing$$

because in no state we are enforced to do either an action a or b. In every state it is possible to do an action c or d instead. So, the fixed point part is invalid in any state. So, clearly the full formula $[true^*](\mu X.[\overline{a} \cap \overline{b}]X \wedge \langle true \rangle true)$ is invalid.

Exercise 7.5 1. Consider the LTSs in Fig. 7.6 and evaluate Formula (7.1). Would adapting the formula to

$$[\overline{a}^*]\mu X.[\overline{a} \cap \overline{b}]X \wedge \langle true \rangle true \tag{7.2}$$

change the results?

2. Evaluate Formula (7.2) on the LTSs in Fig. 7.5.

7.1.1 Minimal Fixed Point Formulas with Data

We can use data in fixed point formulas in the same way as we use it in regular formulas, by using data and variables in actions and quantifying them using the universal and existential quantifiers. But, it is also possible to use data as parameters in fixed point operators, which increases its expressivity enormously. We show a number of examples illustrating how data can be used in combination with minimal fixed point operators.

For the dining philosopher problem, no philosopher will ever starve. In the well-known dining philosopher problem, a group of philosophers alternatingly think and eat spaghetti, using two forks that they use as a shared resource. In this problem the question is whether the forks are shared in such a way such that each philosopher

can obtain the two forks necessary to eat. We formulate this as the property that each philosopher will always be able to do an action *eat* action, ultimately, as otherwise there is the possibility that a philosopher will starve.
We assume that there are three philosophers and defined by

```
sort Philosopher = struct p1 | p2 | p3;
```

The formula for no starvation is

$$[true^*](\forall p : Philosopher.\mu Y.([\overline{eat(p)}]Y \wedge \langle true\rangle true))$$

saying that at any moment, for each philosopher p, there will unavoidably be a moment that he may eat, i.e., that an action $eat(p)$ will take place within a finite number of steps.

For the alternating bit protocol, each message is either transferred or an error is reported. In the alternating bit protocol a sender process reads a message from a source using an action *getFromSource* and then the message is sent to the receiver or an error is reported using the action *transferFromChannel*. If the message is properly transferred, it is delivered using the action *transferFromChannel*. We may want to ask whether a message d will always be transferred in a finite number of steps, or the message is lost in the transmission from the sender to the receiver.
Recall we defined the following data types for the alternating bit protocol.

```
sort D = struct d1 | d2;
     Error = struct packet_lost;
```

The formula expressing the property mentioned above is:

$$\forall d : D.[true^* \cdot getFromSource(d)](\mu X.[\overline{useMessage(d) \cup transferFromChannel(packet_lost)}]X \wedge \langle true\rangle true).$$

Note that this formula is not true. The counter example shows that the delivery may be postponed infinitely when acknowledgements from the receive to the sender fail to arrive.

There is always a next day. See Fig. 5.10 where the requirement is that every weekday is immediately followed by the next one. For this figure, the action *transferDay* uses a weekday and another action *forwardDay* uses the next weekday provided by the function *nextDay*. Now assume that there are many more actions, so, we want to say that after some day, the next day will ultimately take place. This is formulated as follows:

$$\forall d : weekDays.[true^* \cdot transferDay(d)]\mu X.[\overline{forwardDay(nextDay(d))}]X \wedge \langle true\rangle true.$$

There is a weekday after which we get a holiday. In the example given above, we add an action *holiday* and a function *isHoliday* by

```
act holiday:Bool;

map isHoliday: WeekDays->Bool;

var d: Weekdays;
eqn isHoliday(d) = d==Saturday || d==Sunday;
```

Suppose there is an action *transferDay* using weekdays and this is followed by either an action *holiday(true)* or an action *holiday(false)* depending upon the weekday. We can state that there is a day in the week that is always followed by a holiday.

$$\exists s : weekDays.[true^* \cdot transferDay(s)](\mu X.[\overline{holiday(true)}]X \wedge \langle true\rangle true \wedge [holiday(false)]false).$$

The conjunct $[holiday(false)]false$ expresses that on any finite sequence from day s to the holiday, there cannot be indications that these days are not a holiday.

Exercise 7.6 What is the meaning of the following formula:

$$\forall s : weekDays.\langle true^* \cdot transferDay(s)\rangle(\mu X.[\overline{holiday(true)}]X \wedge \langle true\rangle true \wedge [holiday(false)]false).$$

It is also possible to use parameters in fixed points, quite comparable to the use of parameters in processes. These parameters are called *observation variables*. One can see the modal formula as an observer of the behaviour of the system. As the observations can be complex, the observer uses its own variables to store information about the observations. For instance, it can maintain a number to count the number of actions, but it can also use sets, lists and even variables ranging over functions.

We provide an example of a process that counts in a parameter n the number of *up* actions, and ultimately, the number must become equal to 5.

$$\mu X(n : \mathbb{N} = 0).([up]X(n+1) \vee n \approx 5).$$

The initial value for n is equal to 0, which must be indicated inside the fixed point operator. Whenever action *up* happens, the parameter n is increased by 1. After a finite time, the condition $x \approx 5$ must become valid, due to the minimal fixed point, and thus this formula expresses that after finite time the action *up* will have taken place five times. Note that in mCRL2 tool syntax this is written as

```
mu X(n: Nat=0).([up]X(n+1) || val(x == 5))
```

where the keyword `val` should not be forgotten.

If there is an action *up* then, most likely there are more actions in the behaviour, for instance *down*, *left* and *right*. We may want to express that a counter can go up and down, but ultimately the value five must be reached, whereas the actions *left* and *right* do not influence the counting. The required formula is

$$\mu X(n : \mathbb{Z} = 0).(([up]X(n+1) \wedge [down]X(n-1) \wedge [left \cup right]X(n)) \vee n \approx 5).$$

Exercise 7.7 Give an example that shows that the formula

$$\mu X(n : \mathbb{Z} = 0).(([up]X(n+1) \wedge [down]X(n-1)) \vee n \approx 5)$$

can be true while the action *up* does not take place five times.

There is a more compact way to take the actions into account that do not influence parameters. Consider the following example

$$\mu X(n : \mathbb{Z} = 0).(([up]X(n+1) \wedge [down]X(n-1) \wedge [\overline{up \cup down}]X(n)) \vee n \approx 5).$$

Instead of saying that the actions *up* and *down* do not influence the parameter n, we say that all actions other than *up* and *down* have no influence. If there are more actions, for instance *backward* and *forward*, then the state space reachable via these actions is also automatically taken into account.

Using data parameters in fixed points, it is also possible to analyse data that occurs in actions. Suppose that in a protocol two parties propose values, for instance using actions $prop_1(n)$ and $prop_2(m)$. Assume we want to formulate the property that after a finite number of steps the proposed values must be larger than zero and the same. A way to formulate this is by

$$\begin{aligned}
&\mu X(m : \mathbb{N} = 0, n : \mathbb{N} = 0).\\
&\quad (((\forall k : \mathbb{N}.[prop_1(k)]X(k,n)) \wedge\\
&\quad (\forall k : \mathbb{N}.[prop_2(k)]X(m,k)) \wedge\\
&\quad [\overline{\exists k : \mathbb{N}.prop_1(k) \cup prop_2(k)}]X(m,n)) \vee\\
&\quad (m > 0 \wedge m \approx n)).
\end{aligned}$$

Using the box modalities it is guaranteed that the values of the observation variables m and n are updated whenever actions $prop_1$ and $prop_2$ occur. If any other action happens, the values of m and n remain unchanged. Note that this is stated with the existential formula inside the box modality.

Because of the fixed point, after a finite number of action the right part of the or-operator must become true. So, after a finite number of actions, the values of m and n must have become larger than 0 and the same.

Exercise 7.8 Write a property that a unique number generator does not generate the same number twice. Using an action $gen(n)$ a number n is generated.

Exercise 7.9 Formulate the property that a safety device must sound k alarms within finite time. Sounding the alarm once is modelled by an action *alarm*. Sounding the alarm more often is acceptable. There can be other actions than sounding the alarm.

7.2 Maximal Fixed Points

A maximal fixed point is very similar to a minimal fixed point, and a typical example is

$$\nu X.\langle a\rangle X \qquad\qquad \texttt{nu X.<a>X},$$

where the tool syntax is written at the right. We discuss the meaning of this formula below. Typically, a maximal fixed point $\nu X.\varphi(X)$ is valid for the largest set of states X that satisfies $X = \varphi(X)$. Just as for the minimal fixed point, this maximal set is also uniquely defined, provided bound variables are preceded with an even number of negations.

Maximal fixed points represent safety properties that should always hold, such as for instance an invariant, whereas minimal fixed points express that a property should become valid within a finite number of steps. Consider as a typical example the following two formulas.

$$\mu X.(\langle a\rangle X \vee \langle b\rangle \mathit{true}) \qquad \text{and} \qquad \nu X.(\langle a\rangle X \vee \langle b\rangle \mathit{true}).$$

The minimal fixed point says that there is a finite trace with actions a followed by an action b. So, it is about a finite number of iterations through X. The maximal fixed point formula permits an infinite sequence through X. So, it says that there is either a finite trace of a's followed by a b, or there is an infinite sequence of a's. There are many interesting properties of fixed point formulas, some of which are given in Table 7.1. Typically, they can be used to transform a formula which helps to simplify formulas increasing their understandability.

The set of states where a maximal fixed point formula $\nu X.\varphi(X)$ is valid can be obtained by setting X initially to all states and iteratively calculate $X := \varphi(X)$ until a solution for X is reached. This solution is the set of states where $\nu X.\varphi(X)$ is valid.

Consider the following formula that says that an action b is possible in every reachable state.

$$\nu X.(\langle b\rangle \mathit{true} \wedge [\mathit{true}]X).$$

The formula is valid for Fig. 7.7a and b while it is invalid for Fig. 7.7c and d.

Table 7.1 Fixed point equations

$\mu X.\varphi(X) \Rightarrow \nu X.\varphi(X)$	
$\mu X.\varphi = \varphi$	$\nu X.\varphi = \varphi$
$\mu X.X = \mathit{false}$	$\nu X.X = \mathit{true}$
$\mu X.\langle R\rangle X = \mathit{false}$	$\nu X.[R]X = \mathit{true}$
$\neg\mu X.\varphi = \nu X.\neg\varphi[X := \neg X]$	$\neg\nu X.\varphi = \mu X.\neg\varphi[X := \neg X]$
$\mu X.\varphi = \varphi[X := \mu X.\varphi(X)]$	$\nu X.\varphi = \varphi[X := \nu X.\varphi(X)]$
if $\varphi[X := \psi] \Rightarrow \psi$ then $\mu X.\varphi \Rightarrow \psi$	if $\psi \Rightarrow \varphi[X := \psi]$ then $\psi \Rightarrow \nu X.\varphi$

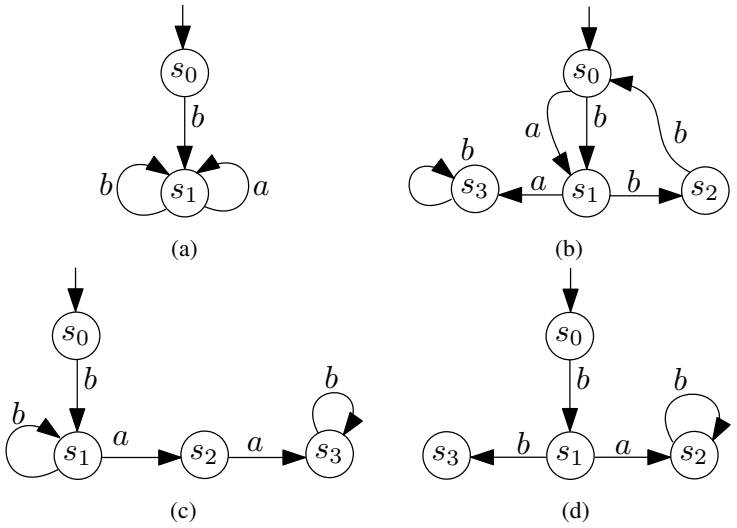


Fig. 7.7 Example LTSs used to illustrate maximal fixed point

Figure 7.7**a.** We start with $X = \{s_0, s_1\}$, namely the set of all states. In the first iteration, $[true]X$ holds in all states, as whenever an action can be done, a state in $\{s_0, s_1\}$ must be reached. The other part of the conjunct, $\langle b\rangle true$, is also satisfied because a b is possible in in all states. So, with $X = \{s_0, s_1\}$, the formula $\langle b\rangle true \wedge [true]X$ is valid in all states, and hence, $\{s_0, s_1\}$ is a solution. As we started iterating with all states, this is the unique maximal fixed point. Hence, the formula $\nu X.(\langle b\rangle true \wedge [true]X)$ is valid in all states.

Figure 7.7**b.** This case is exactly the same as the one above, but here the initial set of all states must be taken to be $X = \{s_0, s_1, s_2, s_3\}$.

Figure 7.7**c.** We start with $X = \{s_0, s_1, s_2, s_3\}$. In the first iteration, $\langle b\rangle true$ holds in states $\{s_0, s_1, s_3\}$ while $[true]X$ holds in all these states. So, after one iteration

$$X = \{s_0, s_1, s_3\} \cap \{s_0, s_1, s_2, s_3\}$$

In the second iteration, we start out with $X = \{s_0, s_1, s_3\}$. The conjunct $\langle b\rangle true$ holds in all these states, but $[true]X$ only holds in $\{s_0, s_3\}$ because only from these two states, all possible actions lead to X. So, after two iterations

$$X = \{s_0, s_1, s_3\} \cap \{s_0, s_3\}$$

In the third iteration, $X = \{s_0, s_3\}$ and again $\langle b\rangle true$ holds in $\{s_0, s_1, s_3\}$ but $[true]X$ now holds in $\{s_3\}$, only. So,

$$X = \{s_0, s_1, s_3\} \cap \{s_3\}$$

In the fourth iteration, $X = \{s_3\}$ and $[true]X$ holds in the same set. As a result, we reach the maximal fixed point $\{s_3\}$. So, the given formula holds only at the state s_3 and it is false in the initial state.

Exercise 7.10 Compute the maximal fixed point for $\nu X.(\langle b\rangle true \wedge [true]X)$ on the transition system in Fig. 7.7d. Does this formula hold in the initial state s_0?

Let us discuss the following example, where at least one b action is supposed to occur such that it leads to some state in the maximal fixed point.

$$\nu X.\langle b\rangle X.$$

This formula is satisfied in a state if there is an infinite sequence of actions b possible in such a state. Using the reasoning above, start out with X being all states. It is clear that after one iteration, only states with an outgoing action b remain. After two iterations, only states with an outgoing action b to a state with an action b remain in the maximal fixed point. After n iterations, only states with a trace $b{\cdot}b\cdots b{\cdot}b$ of length at least n remain in the approximation of the fixed point. The fixed point is only reached in states having an infinite sequence of outgoing actions b.

So, the above formula is valid in the initial states in the Fig. 7.7a–c and not in Fig. 7.7d. The respective maximal fixed points solutions for these figures are $\{s_0, s_1\}$, $\{s_0, s_1, s_2, s_3\}$, $\{s_0, s_1, s_3\}$ and $\{s_2\}$.

It is useful to compare the maximal fixed point formula $\nu X.\langle b\rangle X$ with its minimal counterpart $\mu X.\langle b\rangle X$. If we start iterating this for $X = \varnothing$, we find that $X = \varnothing$ is the minimal solution, and hence this formula is equal to false. Another way to understand this is by iterating through X, which can only happen finitely often with a minimal fixed point. For $\mu X.\langle b\rangle X$ to hold in a state, it must be possible to do an action b and end up in a state where the property represented by X is valid. This means that in this next state again an action b must be possible leading to a state in X. Clearly, this leads to an infinite sequence. However, the minimal fixed point says that such an iteration is finite. As a sequence of actions b can not both be finite and infinite, this leads to a contradiction and the formula is always false. For the maximal fixed point $\nu X.\langle b\rangle X$ it is allowed that iteration through X goes on infinitely. Hence, for the maximal fixed point the formula expresses that there is an infinite sequence of actions b possible.

We can combine fixed points and regular formulas to formulate more complex requirements. For instance, we may want to say that there is an action c in the reachable state space that is followed by an infinite sequence of actions b. This formula is valid in Fig. 7.1b, and not in (a). This property is expressed with a regular expression as:

$$\langle true^* \cdot c\rangle \nu X.\langle b\rangle X.$$

In the above formula, we can use multiple actions, as given below, to express the existence of an infinite sequence $a \cdot b \cdot c \cdot a \cdot b \cdot c \cdots$.

$$\langle true^* \rangle \nu X.\langle a \cdot b \cdot c \rangle X.$$

Note that this formula holds in Fig. 7.3b and Fig. 7.4d and in no other figure in this chapter.

Regular formulas can be expressed using fixed point formulas. The typical equivalences are

$$[R^*]\varphi = \nu X.([R]X \wedge \varphi) \text{ and}$$
$$\langle R^* \rangle \varphi = \mu X.(\langle R \rangle X \vee \varphi),$$

where R is an arbitrary regular formula. This means that any regular formula can be expressed using fixed points. However, the reverse is not true. Fixed point formulas are far more expressive than regular formulas. Yet, regular formulas are useful as they are generally more comprehensible.

Let us consider a practical setting where we can use a maximal fixed point formula, which cannot be formulated as a regular formula. We have a lottery where a price is drawn every week. Passing of a week is indicated by an action *week*. The action of allotting a price is indicated by the action *price*. We abstract from other actions such as buying a lottery ticket. The gambling authority has as requirement that at any time it is possible that there is a price every week, as otherwise the lottery would not be fair. A way of formulating this is

$$[true^*]\nu X.(\langle week \rangle X \wedge \langle price \rangle true).$$

This says in any state, there is an infinite sequence of actions *week* and in every week an action *price* is possible.

7.2.1 Maximal Fixed Points with Data

We can use data in maximal fixed point formulas in exactly the same way as in minimal fixed point formulas. So, we can use data in actions, we can use quantifiers and use parameters in the fixed points. We illustrate this by showing a number of examples of its use.

The first requirement comes from the alternating bit protocol. A requirement to avoid sending a message without reading it from the source, is expressed in [41] as:

$$\forall d : D.\nu X.([\overline{readFromSource(d)}]X \wedge [useMessage(d)]false).$$

It means new messages are not generated in the alternating bit protocol. We can also express the above formula as:

$$\forall d : D.[\overline{readFromSource(d)}^* \cdot useMessage(d)]false.$$

In [35], a requirement for the mutual exclusion problem is given as "two processes can never be in the critical section at the same time". This property is expressed there as:

$$\nu X.([true]X \wedge \forall b : \mathbb{B}.([enter(b)]\nu Y.([enter(\neg b)]false \wedge [\overline{leave(b)}]Y)))$$

Observe that this formula can also be stated as an equivalent regular formula.

$$\forall b : \mathbb{B}.[true^* \cdot enter(b) \cdot \overline{leave(b)}^* \cdot enter(\neg b)]false$$

Now we look at the use of parameters in maximal fixed points. Assume we have a system that can do actions *up* and *down*. We want to write down the safety formula that the number of occurrences of *down* can never exceed the number of occurrences of *up*. We maintain this number in a parameter n, and when an action *down* happens we require that $n > 0$.

$$\begin{aligned}\nu X(n : \mathbb{N} = 0).&[up]X(n+1) \wedge \\ &[down](n > 0 \wedge X(\max(0, n-1))) \wedge \\ &[\overline{down \cup up}]X(n).\end{aligned}$$

Note that the last line is added to say that whenever an action different from *up* or *down* is happening, the formula must also be checked, with the parameter n unchanged.

The data parameters in fixed points formulas can be as complex as needed. Suppose we have a protocol that reads, buffers and delivers data. Reading and delivering of data happens with actions $read(d)$ and $deliver(d)$. We formulate the property that all read data elements must be delivered in the same order. In other words, the protocol must deliver the data in a first-in first-out fashion.

We formulate this by a maximal fixed point formula using an observation variable which is a queue. Whenever the protocol reads data, this is put in front of the queue. Whenever the protocol delivers an element, we check that the delivered element matches the element at the end of the queue. It may feel odd to use a queue in the modal formula, whereas the protocol also must have data structures that mimic a queue. But this is exactly the point. We want to check whether these internal data structures are correct and therefore we compare the behaviour of the protocol using a clear cut queue which we certainly trust. We intentionally leave the domain D of data unspecified, such that this property can be used for any data domain.

$$\begin{aligned}\nu X(q : List(D) = []).((&\forall d : D.[read(d)]X(d \triangleright q)) \wedge \\ &(\forall d : D.[deliver(d)]((rhead(q) \approx d) \wedge X(rtail(q)))) \wedge \\ &[\overline{\exists d{:}D.read(d) \cup deliver(d)}]X(q))\end{aligned}$$

Exercise 7.11 A machine produces products with sequence numbers. The action $product(n)$ indicates that a new product with sequence number n is ready. Write

a requirement that it is only allowed to produce products with strictly increasing sequence numbers.

Exercise 7.12 Consider a wafer scanner, which imprints images onto silicon wafers, which is a crucial step in the construction of microchips. A wafer enters the machine using the action *enter*(w) and leaves the machine using the action *leave*(w) where w is the identity of a wafer. After the machine processed a lot containing a number of wafers the machine indicates that it finished using a *ready* action. Write the property that when the action *ready* happens, all wafers must have left machine. Write a second property that states that no two wafers with the same identity can be present at the same time in the machine, and a third property that when *ready* happens the wafers that left the machine have exactly the same identities as those that entered the machine.

7.3 Combining Minimal and Maximal Fixed Points

We can combine minimal and maximal fixed point operators where we can state properties such as whenever an event happens in some reachable state (maximal fixed point) then another action will be done (minimal fixed point). By nesting fixed points, *fairness* properties can be formulated. In such properties we can express that some activity will happen, provided other events will or will not happen sufficiently often. For instance, one can express that in a data communication protocol, data is guaranteed to be transferred, provided transmitted messages are not always lost and at least arrive sometimes.

We first focus on the simple combination of minimal and maximal fixed points. Actually, we have already seen some examples in the previous sections.

A typical example of a 'simple' minimal/maximal fixed point property is:

$$[true^* \cdot a]\langle true^* \cdot b\rangle true$$

saying that whenever an action a happens, there is a finite sequence of actions leading to the action b. To see that this is a maximal and a minimal fixed point, we rephrase this regular formula with explicit fixed points.

$$\nu X.([true]X \wedge [a]\mu Y.(\langle true\rangle Y \vee \langle b\rangle true)). \tag{7.3}$$

This is generally not a formula that we want to use, as it does not guarantee that b will take place. The formula above is for instance valid in the process (a) in Fig. 7.8, where after the action a the action b can be done, but it is also possible to choose c.

Alternatively, we can write down

$$[true^* \cdot a]\mu Y.[\overline{b}]Y \wedge \langle true\rangle true,$$

which we have already seen in the section on minimal fixed points. It expresses that after an action a the action b must happen on every path within a finite number of steps. The initial part of the formula $[true^*]\ldots$ is clearly a maximal fixed point and the rest uses a minimal fixed point operator. Formulas using this pattern are often used to formulate liveness properties. Note that this formula is valid in Fig. 7.8b but not in Fig. 7.8a.

The formula given above is also not valid for Fig. 7.8c because by always taking the action c, the action b can be prevented from happening. Such a situation occurs quite frequently in concrete systems. For instance, the action c may model the reception of an interrupt by pressing a button, not relevant for the actions a and b. As pressing the button infinitely fast, is physically impossible, we want to ignore this c-loop. One way of doing this is by weakening the property above by stating that after an action a, the action b remains possible, as long as b did not happen.

$$[true^* \cdot a \cdot \overline{b}^*]\langle true^* \cdot b\rangle true. \tag{7.4}$$

We recognise that this is a maximal fixed point formula followed by a minimal fixed point formula, and it is valid for the first three transition systems in Fig. 7.8.

But Formula (7.4) can be too strict, as it allows any action to occur unboundedly often before an action b. Maybe we want to state that action b can only be postponed indefinitely often by action c, but not by any other action. This can be expressed with the following formula, which is the first formula where we use nested alternating fixed points as iteration is possible through Y and Z, simultaneously. Note that this is not possible in formula (7.3), where once you start iterating in Y, you cannot return to X anymore.

$$[true^* \cdot a]\mu Y.\nu Z.([\overline{b \cup c}]Y \wedge [c]Z \wedge \langle true\rangle true). \tag{7.5}$$

This formula says that action b must happen within a finite number of steps, but the action c can happen infinitely often in between. Concretely, actions different from b and c can only happen finitely often, c can happen infinitely often in between, and no deadlock is allowed. So, either action b or a c-loop will ultimately happen. Formula (7.5) is valid in the first four transition systems of Fig. 7.8, but we may want to exclude the transition system Fig. 7.8d. This can be done by requiring explicitly that the action b is always possible, as follows:

$$[true^* \cdot a]\mu Y.\nu Z.([\overline{b \cup c}]Y \wedge [c]Z \wedge \langle true^* \cdot b\rangle true). \tag{7.6}$$

We can change the order of the fixed points $\mu Y.\nu Z.\cdots$ into $\nu Z.\mu Y.\cdots$. This leads to the following formula of which the meaning differs subtly from Formula (7.6).

$$[true^* \cdot a]\nu Z.\mu Y.([\overline{b \cup c}]Y \wedge [c]Z \wedge \langle true^* \cdot b\rangle true). \tag{7.7}$$

In this formula, when action c happens, we jump to Z outside of the minimal fixed point Y. This resets Y in the sense that counting the number of iterations in Y is

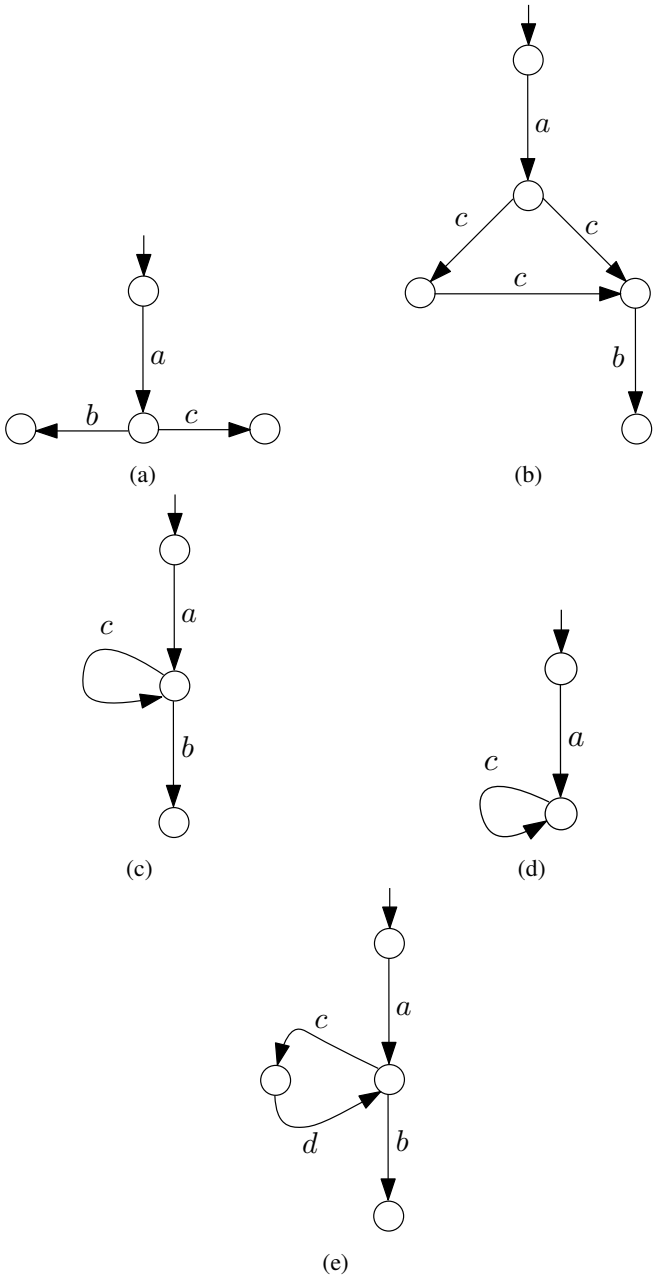


Fig. 7.8 Simple transition systems showing how action *b* can follow action *a*

reset to 0. In Formula (7.6) this counter is not reset, because Z is inside the scope of Y. Typically, Formula (7.6) is invalid for the transition system in Fig. 7.8e because action d can happen unboundedly often before action b. Formula (7.7) is valid as it says that after each action c only a bounded number of actions d can happen before b can be done. Which of the formulas is preferred depends on the particular practical situation.

Modal properties for the alternating bit protocol

In this section we provide a number of properties for the alternating bit protocol discussed in Sect. 5.7 using nested fixed point operators.

- A property of the ABP given in [12] says that if the action $getFromSource(d)$ is enabled infinitely often for any message d, then it is also taken infinitely often. It is formulated by

$$\forall d : D.([true^*]\nu X.\mu Y.\nu Z.([getFromSource(d)]X \wedge (\langle getFromSource(d)\rangle true \Rightarrow [\overline{getFromSource(d)}]Y) \wedge [\overline{getFromSource(d)}]Z)).$$

  Whenever $getFromSoure(d)$ is possible, and it is not taken, we cycle through Y, which means this is only finitely often possible. Otherwise, if $getFromSource(d)$ is not possible, actions other than $getFromSoure(d)$ can be taken infinitely often, cycling through Z. Note that the condition 'if $getFromSoure(d)$ is not possible' is redundant, and therefore not explicitly stated. If $getFromSoure(d)$ happens, iterating through the maximal fixed point variable X resets the minimal fixed point counter in Y, saying that again an arbitrary, but finite, number of actions different than $getFromSoure(d)$ can happen as long as $getFromSoure(d)$ is possible.
  This formula is false for the alternating bit protocol. The protocol with data d_1 and d_2 can regularly read both messages, but it may in some run of the system always decide to read datum d_1. This invalidates this property, which says that as d_2 can be read infinitely often, it must be read at some point.
- There is a path along which a message d (read through an action $getFromSource$) can be lost infinitely often via the action $transferFromChannel(packet_lost)$ without ever delivering d via $useMessage(d)$. It is given by The action $transferFromChannel(packet_lost)$ represents that an error is communicated, which indicates messages loss.

$$\langle true^*\rangle(\exists d : D.\langle getFromSource(d)\rangle (\nu X.\mu Y.(\langle transferFromChannel(packet_lost)\rangle X \vee \langle \overline{transferFromChannel(packet_lost)} \cup \overline{useMessage(d)}\rangle Y))).$$

  This property is true for the alternating bit protocol. It is indeed possible to always loose messages in transfer.
- For all messages d, it is possible to infinitely often start transferring datum d using the action $getFromSource(d)$. In a formula:

$$\forall d : D.\nu X.\mu Y.(\langle getFromSource(d)\rangle X \vee \langle \overline{getFromSource(d)}\rangle Y).$$

This property holds for the alternating bit protocol.

- The following formula expresses that if data is read by doing an action *getFromSource*(d), then it is eventually delivered using the action *useMessage*(d):

$$true^*t(\forall d : D.([getFromSource(d)](\nu X.\mu Y.([useMessage(d)]X \wedge [\overline{useMessage(d)}]Y)))).$$

This property does not hold for the alternating bit protocol, as it can lose the data in transfer continuously.

- Every data element d, once read through action *getFromSource*(d), is inevitably delivered through the action *useMessage*(d), if not permanently lost in transfer. This is formalised by the formula

$$[true^*]\forall d : D.[getFromSource(d)](\nu Y.\mu Z.([\overline{useMessage(d)} \cup \overline{transferFromChannel(bit_msg_lost)} \cup \overline{transferFromChannel(packet_lost)}]Z \wedge [transferFromChannel(bit_msg_lost) \cup transferFromChannel(packet_lost)]Y \wedge \langle true\rangle true)).$$

This property is valid. It formulates that if data transfer is fair, i.e., messages in transit are not always lost, all data that is received is properly transferred.

7.4 Modelling a Movable Patient Support Unit and Its Requirements

In this section, we consider a movable patient support unit which is used to move patients inside an MRI (Magnetic Resonance Imaging) scanner. The example is inspired by a software rejuvenation project at Philips Healthcare, which was carried out by formally modelling the new software, formulating and checking the requirements, and subsequently implementing the code according to the specification. We present here a simplified instance of a controller for the support unit.

The patient support unit is essentially a bed lying on a gurney and the bed can move left and right using a motor M as shown in Fig. 7.9. A docking station D is fixed with the gurney to sense if the unit is docked onto the scanner or not. The gurney is considered *docked* if it is attached and locked to an MRI scanner. The MRI scanner has a large size bore. A patient is pushed inside the bore for scanning. The scanning process starts when the bed is at the left-most position, i.e., entirely inside the bore. The motor can move the bed to the left and right if the unit is docked. If not docked, the motor applies the brake to fix the position of the bed.

The docking station has a mechanical lock which is automatically applied when the unit is docked. The lock opens automatically when the operator presses the undock button. Before undocking, the bed must be at the right-most position, i.e., completely out of the scanner. The bed signals the motor to stop when the bed is either completely

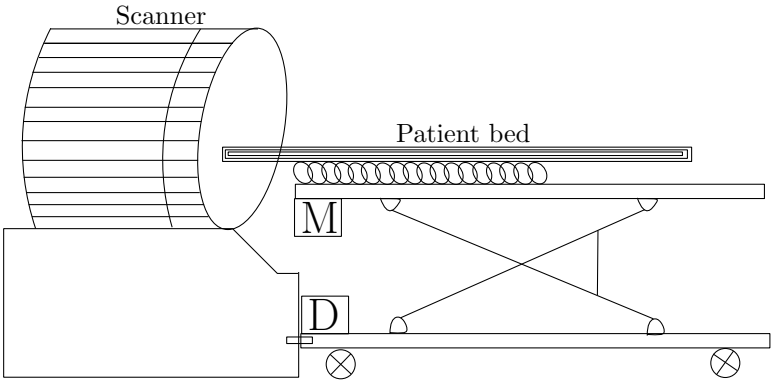


Fig. 7.9 A moveable patient support unit

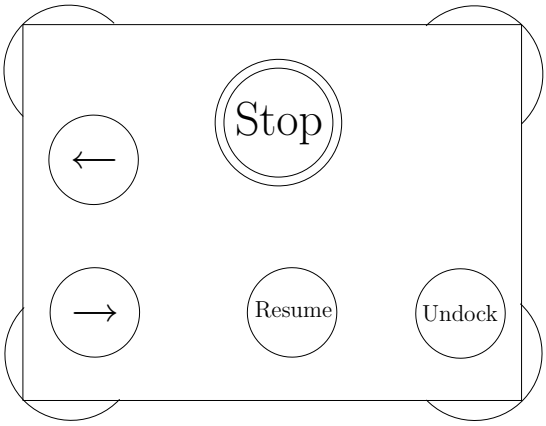


Fig. 7.10 The control panel for the movable patient support unit [28]

inside the bore (the left-most position) or completely over the gurney (the right-most position). The controller instructs the brake to apply when this happens.

An operator controls the bed using a fixed panel shown in Fig. 7.10. The panel contains the following buttons:

←	This button provides a signal to the motor M to move in the left direction until the bed reaches to the left-most position. This button works only if the bed is docked.
→	This button does exactly the same as the previous button, except for the right direction.
Stop	This button puts the patient support unit in emergency mode. In emergency mode the motors cannot be applied. In the undocked position, the brake of the motor M is applied to avoid left/right movement. In

	the docked position, the brake is released to get the patient out of the scanner.
Resume	This button finishes the emergency mode.
Undock	This button gives a signal to the docking station *D* to unlock the mechanical lock.

This example is also discussed in [28] where it is modelled using one process. Here, we make a model more in accordance with the real system. The model consists of two concurrent processes, namely the console controller *ConsoleC* and motor controller *GurneyC*.

The console controller is responsible for handling the buttons that are pressed. It also maintains whether the system is in emergency mode. The system must go in emergency mode when stop is pressed, and leaves emergency mode after pressing resume. In emergency mode, when the gurney is docked, the motor is stopped and the brake is released such that a patient can be moved in and out of the bore manually. The console controller only forwards button presses to the gurney controller if the command must be executed. So, a command to move the bed to left or right, is only forwarded if the system is not in emergency mode. Besides this, the gurney controller can request the console controller on whether the system is in emergency mode.

The gurney controller switches the motor on and off, applies the brake and gives instructions to undock the gurney. It takes care that in emergency mode the brakes are not applied when the gurney is docked.

7.4.1 Formal Specification

We present the formal specification in three parts, namely the console controller *ConsoleC*, the gurney controller *GurneyC* and the parallel composition of the two. The controllers use actions as listed in Table 7.2.

The console controller *ConsoleC* has one parameter *m* that indicates whether the patient platform is in normal or emergency mode. At line 10 it is shown that if the stop button is pressed, the stop command is forwarded to the gurney controller, and the console sets *m* to *emergency*. In the subsequent lines it is indicated what happens when other buttons are pressed. For instance at line 13 it is indicated what happens if the left button is pressed. Only if the console is in emergency mode, the command to rotate the motor to the left is forwarded.

```
1  sort Mode = struct normal | emergency;
2
3  act pressStop, stop_s;
4      pressResume;
5      pressUndock, undock_s;
6      pressLeft, left_s;
7      pressRight, right_s;
8
```

Table 7.2 The actions in the movable patient platform

Actions	Console controller
pressStop	Indication that the stop button is pressed
stop_s	Message to the gurney controller indicating that the stop button is pressed
pressResume	Indication that the resume button is pressed
pressUndock	Indication that that the undock button is pressed
undock_s	Message to the gurney that the undock button is pressed
pressLeft	Indication that the left button is pressed
left_s	Message to the gurney that the left button is pressed
pressRight	Indication that the right button is pressed
right_s	Message to the gurney that the right button is pressed
	Gurney controller
stop_r	Receive a stop command
undock_r	Receive an undock command
left_r	Receive a command to move the bed to the left
right_r	Receive a command to move the bed to the right
releaseBrake	Send a command to release the brake
motorOff	Instruct the motor to switch off
applyBrake	Instruct the brake to apply
unlockDock	Give the docking unit the command to undock
motorLeft	Switch the motor on to rotate in the left direction
dockIt	Receive the command that the gurney is docked
atLeftMost	Receive a message that the bed is in the leftmost position
atRightMost	Receive a message that the bed is in the rightmost position
motorRight	Instruct the motor to turn in the right direction
	Results of communication
stop	Result of a communication of stop_r and stop_s
undock	Result of a communication of undock_r and undock_s
left	Result of a communication of left_r and left_s
right	Result of a communication of right_r and right_s

```
9  proc ConsoleC(m: Mode) =
10         pressStop . stop_s . ConsoleC(emergency) +
11         pressResume . ConsoleC(normal) +
12         pressUndock . undock_s . ConsoleC(m)  +
13         pressLeft .
14           ( m==emergency )
15              -> ConsoleC(m)
16              <> left_s . ConsoleC(m) +
17         pressRight .
18           ( m==emergency )
```

```
19                -> ConsoleC(m)
20                <> right_s . ConsoleC(m);
```

The gurney controller *GurneyC* has four parameters governing its behaviour.

- The boolean *docked* indicates whether the gurney is docked to the MRI scanner.
- The boolean *rightmost* is true whenever according to the information that the gurney received the bed is in its right-most position. Note that the gurney has limited possibilities to measure the position of the bed. If the bed is in the rightmost position, and the patient platform goes into emergency mode, the bed can be moved freely, although the gurney controller still thinks it is rightmost position. If this is not desired, either extra sensors must be added, or the behaviour of the gurney controller must be changed.
- The boolean *leftmost* indicates whether the gurney is believed to be in the left-most position.
- The motor status *ms* indicates the current movement of the motor, which can be *movingLeft*, *movingRight* and *stopped*.

It is straightforward to see what the gurney controller does when receiving external inputs. For instance, at line 8 it receives a *stop* command from the console. It then switches the motor off, and releases or applies the brake depending on whether the gurney is docked.

Another example is given at line 29, where it receives a signal the bed is at the leftmost position. If the bed is stopped, i.e., the motor is not active, it records that the patient platform is in left-most position. If a motor is running, the motor is switched off, and the brake is applied, and the process parameters are set to reflect the current situation.

```
1  sort MotorStatus = struct movingLeft | movingRight | stopped;
2
3  act stop_r, undock_r, left_r, right_r;
4      releaseBrake, motorOff, applyBrake, unlockDock, motorLeft;
5      dockIt, atLeftMost, atRightMost, motorRight;
6
7  proc GurneyC(docked, rightmost, leftmost: Bool, ms: MotorStatus) =
8         stop_r .
9           motorOff .
10          ( docked -> releaseBrake
11                   <> applyBrake) . GurneyC() +
12        undock_r .
13          ( docked && rightmost )
14            -> applyBrake . unlockDock . GurneyC(docked=false)
15            <> GurneyC() +
16        left_r .
17            ( docked && ms!=movingLeft && !leftmost )
18              -> releaseBrake .
19                 motorLeft .
20                 GurneyC(rightmost=false, ms=movingLeft)
21              <> GurneyC() +
22        right_r .
```

```
23            ( docked && ms!=movingRight && !rightmost )
24              -> releaseBrake .
25                 motorRight .
26                 GurneyC(leftmost=false, ms=movingRight)
27              <> GurneyC() +
28            !docked -> dockIt.GurneyC(docked=true) +
29          atLeftMost .
30            (( ms==stopped )
31               -> GurneyC(docked, false, true, stopped)
32               <> motorOff .
33                  applyBrake .
34                  GurneyC(docked, false, true, stopped)
35            ) +
36          atRightMost .
37            (( ms==stopped )
38              -> GurneyC(docked, true, false, stopped)
39              <> motorOff .
40                 applyBrake .
41                 GurneyC(docked, true, false, stopped)
42            );
```

The console controller and the gurney controller communicate with the external world, and also with each other. The parallel composition below indicates how they are combined. The console controller can send stop, undock and movement commands to the gurney controller using actions ending on *_s*. Receiving these instructions is modelled with actions ending on *_r*. If a send and a receive action synchronises, an action results where the suffixes are dropped as indicated in lines 9 to 12.

Note that in the model instructions from the console are received by the gurney synchronously. Although this is generally an adequate way of modelling, especially when the message transfer time is negligible, messages are often buffered in real systems. In case this is important a message queue should be added between the console and the gurney.

```
1  act stop, undock, left, right;
2
3  init allow ( { pressStop, pressResume, pressUndock, releaseBrake,
4                 pressLeft, pressRight,
5                 motorOff, applyBrake, unlockDock, dockIt,
6                 motorLeft, motorRight, atLeftMost, atRightMost,
7                 stop, undock, left, right
8               },
9         comm ( { stop_s | stop_r -> stop,
10                undock_s | undock_r -> undock,
11                left_s | left_r -> left,
12                right_s | right_r -> right
13              },
14              ConsoleC(normal) ||
15              GurneyC(false, false, false, stopped)
16            ));
```

This model of a controller of a patient platform is simple, but in no way trivial. Formulating and verifying the properties in the next section led to many changes as the behaviour turned out to be flawed. Still, the model as it is may not reflect the desired behaviour of a patient platform, for instance because in emergency mode, the bed is not guaranteed to be in rightmost position when undocking, as doctors can freely move the bed back and forth. The purpose of the current description is to make such behaviour explicit, allowing such aspects to be discussed. In this concrete case one might argue that when in emergency mode, maximal freedom must be given to the medical personal to act, and the controllers is not responsible anymore, in which the current model is adequate. Or it can be concluded that this behaviour is unacceptable, and in emergency mode, the brakes must always apply, leading to a change in the model.

7.4.2 Requirements Analysis

Even a relatively simple behavioural model as that of a movable patient platform can easily contain mistakes, varying from simple mistypings and omissions, to a conceptually flawed solution. By formulating and verifying safety, liveness and fairness properties the quality of and trust in the model is considerably increased. We show that a number of essential properties hold. Note that formulating *all* desired properties is not easy, and requires substantial experience in the application domain. But even formulating a few already helps.

Undocking is safe. Undocking can only happen if the motor is off, the brakes are applied and the bed is detected to be in right-most position.

In terms of actions, we want to say that before an action *unlockDock* can take place, the action *applyBrake* must have taken place, in particular after a *releaseBrake*. Moreover, the bed must have been detected at the right-most position, i.e., an action *atRightMost* must have happened before the action *unlockDock*, in particular after an action *motorLeft*. Finally, after an action *motorLeft* or *motorRight* the motor must be switched off by a *motorOff* action, before an *unlockDock* can take place.

The formula below represents the property below and it is valid on the model.

$$
\begin{aligned}
&[\overline{applyBrake}^{*} \cdot unlockDock]\,false \wedge \\
&[true^{*} \cdot releaseBrake \cdot \overline{applyBrake}^{*} \cdot unlockDock]\,false \wedge \\
&[\overline{atRightMost}^{*} \cdot unlockDock]\,false \wedge \\
&[true^{*} \cdot motorLeft \cdot \overline{atRightMost}^{*} \cdot unlockDock]\,false \wedge \\
&[true^{*} \cdot (motorLeft \cup motorRight) \cdot \overline{motorOff}^{*} \cdot unlockDock]\,false.
\end{aligned}
$$

Exercise 7.13 How can we determine that the following two formulas are different?

1. $[\overline{applyBrake}^{*} \cdot unlockDock]\,false.$
2. $[true^{*} \cdot releaseBrake \cdot \overline{applyBrake}^{*} \cdot unlockDock]\,false.$

Pressing stop leads to stopping the motor and applying the brake. When pressing the stop button, the system goes in emergency mode. This means that the motor must be switched off and if the gurney is docked, the brake must be released.

In terms of actions this means that when the action *pressStop* happens, the action *motorOff* must take place, without having to do an action *pressStop* again. Furthermore, if *pressStop* takes place, and the system is docked, which can be determined by observing the actions *pressUndock* and *dockIt*, the action *releaseBrake* must happen, without doing *pressStop* again.

We formulate this property as follows. As it is possible to receive inputs arbitrarily often, before the brake and the motor are switched off, we use a relatively weak formalisation of this liveness property. The formula is valid on the model.

$$[true^* \cdot pressStop \cdot \overline{motorOff}^*]\langle \overline{pressStop}^* \cdot motorOff\rangle true \wedge$$
$$[(\overline{pressUndock}^* + true * \cdot dockIt \cdot \overline{pressUndock}) \cdot pressStop \cdot$$
$$\overline{releaseBrake}^*]\langle \overline{pressStop}^* \cdot releaseBrake\rangle true.$$

The motor stops when the bed is in an extreme position. When the patient bed interrupts the controller upon reaching to extreme left or extreme right position, the controller switches off the motor if it was on.

Formulated in terms of actions, this formula says that when one of the actions *atLeftMost* or *atRightMost* happens, when the motor is off, monitored by observing *motorLeft*, *motorRight* and *motorOff*, then the action *motorOff* must take place. However, the actions *pressResume*, *pressLeft*, *pressRight*, *atLeftMost* and *atRightMost* can infinitely often come before the motor is switched off. As these interface actions are relatively infrequent compared to the operational speed of the controller, this may not occur in practice, but the model allows it, and therefore we have to make it explicit in our modal formula.

We formulate the property very precisely using nested minimal and maximal fixed points. The formula holds.

$$[true^*.(motorLeft \cup motorRight) \cdot \overline{motorOff}^* \cdot (atLeftMost \cup atRightMost)]$$
$$\nu Y.\mu X.[\overline{motorOff \cup pressResume \cup pressLeft \cup pressRight \cup atLeftMost \cup atRightMost}]X \wedge$$
$$[pressResume \cup pressLeft \cup pressRight \cup atLeftMost \cup atRightMost]Y \wedge$$
$$\langle true\rangle true.$$

The bed undocks when the undock button is pressed. If the *Undock* button is pressed when the bed is at the right-most position, then unlocking of the docking station will happen.

Reformulated in actions, it is determined by looking at the actions *atRightMost* and *motorLeft* whether the bed is in the rightmost position. If in such a case the action *pressUndock* takes place, then the action *unlockDock* must inevitably happen. However, the actions *atLeftMost*, *atRightMost*, *pressUndock*, *pressLeft*, *pressRight*, *pressResume* and *pressStop* can arbitrarily often happen before the action *unlockDock* occurs.

The following modal formula represents this property and it holds on the model.

$$
\begin{aligned}
&[\mathit{true}^* \cdot \mathit{atRightMost} \cdot \overline{\mathit{motorLeft}}^* \cdot \mathit{pressUndock}] \\
&\qquad \nu Y.\mu X.[\overline{\mathit{unlockDock} \cup \mathit{atLeftMost} \cup \mathit{atRightMost} \cup \mathit{pressUndock} \cup \mathit{pressLeft} \cup \mathit{pressRight} \cup \mathit{pressResume} \cup \mathit{pressStop}}]X \wedge \\
&\qquad\qquad [\mathit{atLeftMost} \cup \mathit{atRightMost} \cup \mathit{pressUndock} \cup \mathit{pressLeft} \cup \mathit{pressRight} \cup \mathit{pressResume} \cup \mathit{pressStop}]Y \wedge \\
&\qquad\qquad \langle \mathit{true}^* \cdot \mathit{unlockDock} \rangle \mathit{true}
\end{aligned}
$$

The bed moves left when the ← button is pressed. If the button with the left arrow is pressed then the motor is switched on to move to the left. But this only happens when the gurney is docked, the platform is not in emergency mode, the bed is not in leftmost position and the motor is not already moving to the left.

To formulate this property in terms of actions, we use boolean observation variables b_1, b_2, b_3 and b_4, which are variables in the modal formula that record properties about actions that have been observed.

- The boolean b_1 determines whether the gurney is docked. Initially it is false. This variable is set when the action *dockIt* takes place, and it is set to false when *unlockDock* happens.
- The observation variable b_2 is also a boolean and it records whether the gurney is in normal, not emergency, mode. Initially true, it is set by the action *pressResume* and reset by *pressStop*.
- The boolean b_3 is true when the system is aware that the bed is in leftmost position. It is set when the action *atLeftMost* is happening, and it is reset when *motorRight* occurs.
- The last observation variable b_4 represents whether the bed is not moving to the left. It is set by the actions *motorOff* and *motorRight*, and it is reset by either the action *motorLeft*.

If all variables b_1, b_2, b_3 and b_4 are set, and if the action *pressLeft* happens, then the action *motorLeft* must happen, unless the action *atLeftmost* happens which can cancel switching on the motor. Both these actions can happen without the need for *pressLeft* to happen again. Moreover, there can be an unbounded number of button presses in between, i.e., the actions *pressStop*, *pressResume*, *atRightMost*, *pressLeft*, *pressRight* and *pressUndock* can happen arbitrarily often before the motor starts.

This is all formulated in the following modal formula. Note that formulas expressing complex properties can become sizeable.

$$
\begin{array}{l}
\nu X(b_1 : \mathbb{B} = \mathit{false}, b_2 : \mathbb{B} = \mathit{true}, b_3 : \mathbb{B} = \mathit{true}, b_4 : \mathbb{B} = \mathit{true}). \\
\qquad [\mathit{dockIt}]X(\mathit{true}, b_2, b_3, b_4) \wedge \\
\qquad [\mathit{unlockDock}]X(\mathit{false}, b_2, b_3, b_4) \wedge \\
\qquad [\mathit{pressResume}]X(b_1, \mathit{true}, b_3, b_4) \wedge \\
\qquad [\mathit{pressStop}]X(b_1, \mathit{false}, b_3, b_4) \wedge \\
\qquad [\mathit{atLeftMost}]X(b_1, b_2, \mathit{true}, b_4) \wedge \\
\qquad [\mathit{motorRight}]X(b_1, b_2, \mathit{false}, \mathit{true}) \wedge \\
\qquad [\mathit{motorOff}]X(b_1, b_2, b_3, \mathit{true}) \wedge \\
\qquad [\mathit{motorLeft}]X(b_1, b_2, b_3, \mathit{false}) \wedge \\
\qquad [\overline{\mathit{dockIt} \cup \mathit{unlockDock} \cup \mathit{pressResume} \cup \mathit{pressStop} \cup \mathit{atLeftMost} \cup} \\
\qquad\qquad\qquad\qquad \overline{\mathit{motorRight} \cup \mathit{motorOff} \cup \mathit{motorLeft}}]X(b_1, b_2, b_3, b_4) \wedge \\
\qquad ((b_1 \wedge b_2 \wedge b_3 \wedge b_4) \Rightarrow [\mathit{pressLeft}] \\
\qquad\qquad \nu Z.\mu Y.[\overline{\mathit{motorLeft} \cup \mathit{atLeftMost} \cup \mathit{pressStop} \cup \mathit{pressResume} \cup} \\
\qquad\qquad\qquad\qquad \overline{\mathit{atRightMost} \cup \mathit{pressLeft} \cup \mathit{pressRight} \cup \mathit{pressUndock}}]Y \wedge \\
\qquad\qquad\qquad [\mathit{pressStop} \cup \mathit{pressResume} \cup \mathit{atRightMost} \cup \mathit{pressLeft} \cup \\
\qquad\qquad\qquad\qquad \mathit{pressRight} \cup \mathit{pressUndock}]Z \wedge \\
\qquad\qquad\qquad \langle \overline{\mathit{pressLeft}}^* \cdot (\mathit{motorLeft} \cup \mathit{atLeftMost}) \rangle \mathit{true})
\end{array}
$$

The bed moves to the right when the → button is pressed. If the go-to-right button is pressed, then the motor to move the bed to the right is started provided the gurney is docked, the bed is not in emergency mode, the bed is not in rightmost position and it is not already moving to right.

We use again a number of observation variables to record the status of the bed, based on external observations.

- The variables b_1 and b_2 have the same meaning as in the previous formula.
- The observation variable b_3 records whether the controller thinks that the bed is in rightmost position. It is set when the action *atRightMost* happens, and it is reset when *motorLeft* occurs.
- The observation variable b_4 records whether the motor is not moving the bed to the right. It is set by the actions *motorOff* and *motorLeft*. It is made false when the action *motorRight* is observed.

Below we formalise this requirement. It has the same structure as the previous requirement and only differs in some details. Its essence is that if the action *pressRight* is happening, when all the observation variables b_1, b_2, b_3 and b_3 are set to true, then the action *motorRight* will happen, unless it is cancelled by an action *atRightMost*, and these actions can happen without having to perform an action *pressRight*. Moreover, the actions *pressStop*, *pressResume*, *atRightMost*, *pressLeft*, *pressRight* and *pressUndock* can happen arbitrarily often before the motor starts or the bed is detected to be at the right.

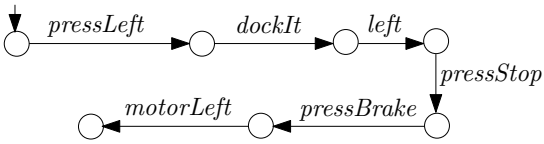


Fig. 7.11 A counter example for a naive formulation that motors cannot start in emergency mode

$$
\begin{aligned}
&\mu X(b_1 : \mathbb{B} = \mathit{false}, b_2 : \mathbb{B} = \mathit{true}, b_3 : \mathbb{B} = \mathit{true}, b_4 : \mathbb{B} = \mathit{true}).\\
&\quad [\mathit{dockIt}]X(\mathit{true}, b_2, b_3, b_4) \wedge\\
&\quad [\mathit{unlockDock}]X(\mathit{false}, b_2, b_3, b_4) \wedge\\
&\quad [\mathit{pressResume}]X(b_1, \mathit{true}, b_3, b_4) \wedge\\
&\quad [\mathit{pressStop}]X(b_1, \mathit{false}, b_3, b_4) \wedge\\
&\quad [\mathit{atLeftMost}]X(b_1, b_2, \mathit{true}, b_4) \wedge\\
&\quad [\mathit{motorRight}]X(b_1, b_2, b_3, \mathit{false}) \wedge\\
&\quad [\mathit{motorOff}]X(b_1, b_2, b_3, \mathit{true}) \wedge\\
&\quad [\mathit{motorLeft}]X(b_1, b_2, \mathit{false}, \mathit{true}) \wedge\\
&\quad ((b_1 \wedge b_2 \wedge b_3 \wedge b_4) \Rightarrow [\mathit{pressRight}]\\
&\qquad \nu Z.\mu Y.[\overline{\mathit{motorRight} \cup \mathit{atRightMost} \cup \mathit{pressStop} \cup \mathit{pressResume} \cup \mathit{atLeftMost} \cup}\\
&\qquad\qquad \overline{\mathit{pressLeft} \cup \mathit{pressRight} \cup \mathit{pressUndock}}]Y \wedge\\
&\qquad\quad [\mathit{pressStop} \cup \mathit{pressResume} \cup \mathit{atLeftMost} \cup\\
&\qquad\qquad \mathit{pressLeft} \cup \mathit{pressRight} \cup \mathit{pressUndock}]Z \wedge\\
&\qquad\quad \langle \overline{\mathit{pressRight}}^* \cdot (\mathit{motorRight} \cup \mathit{atRightMost})\rangle \mathit{true})
\end{aligned}
$$

In emergency mode motors cannot start. When the bed is in emergency mode, the motors cannot be switched on. On first sight this appears to be a straightforward safety formula, but when checking it, it turns out that there is a catch. Naively, we would assume that we can translate this property by saying that actions *motorRight* or *motorLeft* cannot happen between the actions *pressStop* and *pressResume*. The formula for this is

$$[\mathit{true}^* \cdot \mathit{pressStop} \cdot \overline{\mathit{pressResume}}^* \cdot (\mathit{motorRight} \cup \mathit{motorLeft})]\,\mathit{false}.$$

But this formula does not turn out to be true. The reason for this is that when *pressStop* is pressed, the gurney controller may already have decided to switch a motor on, and it is not informed in time of the emergency mode. This is shown in Fig. 7.11, where the command *pressLeft* is being executed when the action *pressStop* happens.

The actual situation is that when the action *pressStop* happens, it takes some time for the whole system to realise that it is in emergency mode. So, we need a formulation that is a little weaker, saying that if *pressStop* happens then, within a finite number of steps, a situation is reached where the motor cannot be switched on, until the action *pressResume* takes place, ending the emergency mode. Using the subformula $\langle\overline{\mathit{pressStop}}\rangle\mathit{true}$ it is expressed that it does not need to be necessary to press the stop button again to reach the situation where the whole system is in emergency mode. This formula is valid.

$$[\mathit{true}^* \cdot \mathit{pressStop}]\mu X.(([\mathit{true}]X \wedge \langle\overline{\mathit{pressStop}}\rangle\mathit{true}) \vee [\overline{\mathit{pressResume}}^* \cdot (\mathit{motorRight} \cup \mathit{motorLeft})]\,\mathit{false}).$$

When undocked the brake is always applied. We finish with a relatively simple safety formula, saying that when the gurney is undocked, the brake is always applied. Using two observation variables *docked* and $brake_applied$ we keep track of whether the gurney is docked and whether the brake is applied.

The last line of the formula below expresses the essence of this property, namely that either the gurney is docked, or the brake is applied. This formula is valid on our model.

$$
\begin{aligned}
&\nu X(docked : \mathbb{B} = false, brake_applied : \mathbb{B} = true).\\
&\quad [dockIt]X(true, brake_applied) \wedge\\
&\quad [unlockDock]X(false, brake_applied) \wedge\\
&\quad [applyBrake]X(docked, true) \wedge\\
&\quad [releaseBrake]X(docked, false) \wedge\\
&\quad \overline{[dockIt \cap unlockDock \cap applyBrake \cap releaseBrake]X(docked, brake_applied)} \wedge\\
&\quad (docked \vee brake_applied).
\end{aligned}
$$

Chapter 8
Linear Processes and Parameterised BESs



In this chapter, we introduce linear processes and parameterised boolean equation systems. These are normal forms. Any process can be transformed into a behaviourally equivalent linear process, also called a Linear Process Specification (LPS). Any modal formula and linear process can be transformed to a Parameterised Boolean Equation System (PBES) that has the solution true exactly if the formula holds on the process. The advantage of LPSs and PBESs is that they have a very simple format that allows for further processing and optimisation. For instance, generating a state space from an LPS is relatively straightforward.

8.1 Linear Processes

A linear process is a process that has the following abstractly formulated shape.

$$X(d : D) = \sum_{i \in I} \sum_{e_i : E_i} c_i(d, e_i) \to a_i(f_i(d, e_i)) \cdot X(g_i(d, e_i)).$$

It is a single process equation. The process variable X used at the right hand side must be the same as the one at the left. Other variables, such as Y or P, can be used, as long as they match at the left and right hand side.

In this description we use a single parameter d of sort D. It is allowed to use any number of parameters, including 0. If multiple variables are used, the expression $g_i(d, e_i)$ must also be multiplied to match the number of parameters. There are a number of summands, here represented using $\sum_{i \in I}$ where I is a finite index set. For each element i in the index set there is a summand

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-3-031-23008-0_8.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0_8

$$\sum_{e_i:E_i} c_i(d, e_i) \to a_i(f_i(d, e_i)) \cdot X(g_i(d_i, e_i)).$$

Each summand says that for some data element e_i of sort E_i if condition $c_i(d, e_i)$ holds, then multi-action $a_i(f_i(d, e_i))$ can take place where the next state is given by $X(g_i(d, e_i))$. It is essential that there is only one action before the variable X.

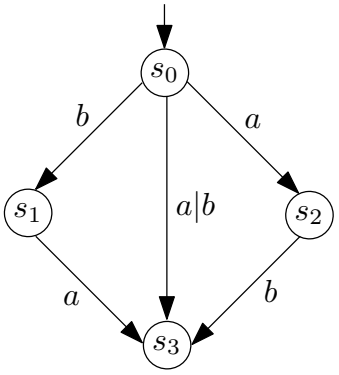
We consider a simple example `X=a.b.delta`. This is not linear process as not only no variable X occurs in the right hand side, but also two actions occur in sequence. The LPS of this process is:

```
proc P(s1_P: Pos) = (s1_P==3) -> b . P(s1_P = 2)
                    +
                    (s1_P==1) -> a . P(s1_P = 3);
```

The linear process has two summands. The sum operator $\sum_{e_i:E_i} \ldots$ has been omitted as no variable ranging over a sort is used. Note explicitly that each action occurs once and it is followed with the process variable used at the left hand side.

Linearisation eliminates most operators, such as the parallel, communication and allow operators, from the process. As the behaviour of the linearised process is still strongly bisimilar to the original process, the parallel nature of the original behaviour is still perfectly preserved in the linear process.

In Fig. 8.1 we provide two parallel processes P_1 and P_2 along with their mCRL2 specification. The linear process P given below, exhibits exactly the same behaviour as $P_1 \parallel P_2$. We also provide the initial process.




```
act a,b;

proc  P1=a.delta;
      P2=b.delta;

init  P1||P2;
```

Fig. 8.1 Two parallel processes and their LTS

```
act a, b;

proc P(s1_P1, s2_P2: Pos) =
               (s1_P1 == 1) -> a . P(s1_P1 = 2)
               +
               (s2_P2 == 1) -> b . P(s2_P2 = 2)
               +
               (s1_P1 == 1 && s2_P2 == 1) ->
                                    a|b . P(s1_P1 = 2, s2_P2 = 2);

init P(1, 1);
```

In this case there are three summands, and again the sum operator is omitted because it is not needed.

There are some further variations of linear processes. For instance when the condition is equal to true, it is sometimes also omitted as it does not change the behaviour. The process δ is sometimes added as an extra summand, but that does not change the behaviour either.

Exercise 8.1 Which of the following process equations is linear.

1. `P = true -> a . P.`
2. `P = a . P.`
3. `P = a . b . P.`
4. `P = a . Q; Q = b . P.`
5. `P = a . P + b . P;.`
6. `P = a . P + b . P + delta;.`

The tool `mcrl22lps` generates a linear process specification from most mCRL2 specifications. Although any process can be linearised [31], the tool `mcrl22lps` only linearises a subset. It cannot linearise processes where recursion is mixed with the parallel, communication and allow operator. So, typically, the following process fails to be linearised, as it mixes parallel and recursive behaviour.

```
act a, b;

proc X = a . delta || (b . X);
```

When not mixing recursion and the operators related to parallel behaviour, linearisation generally works as desired. All examples, with the exception of the one above, in this book can be linearised using the tool `mcrl22lps`.

There are various ways to linearise, which can be selected using the flag `--lin-method=...`. The default is regular, which works in most cases. However, when complex data manipulations are used in combination with terminating processes, it may be desired to use the option `--lin-method=regular2` or even `--lin-method=stack`, as otherwise linearisation may not finish. The resulting linear processes are less attractive, though.

For instance the following process can only be linearised using the `--lin-method=stack` option.

```
act a, b, c;

proc X = a . X . b + c;

init X;
```

In case linearisation using the default option does not work, it can help to disallow terminating processes explicitly. For instance, in the example above, the process does not terminate after the action c. Adding `delta` after the c allows default linearisation, but in this case it does change the behaviour of the process. In most practical process specifications, adding such a δ does not do harm, and it is often actually intended that processes cannot terminate after such a single action.

Linear processes can become large and are stored on disk in an internal, compressed format. A useful tool is `lpspp`, which stands for LPS Pretty Print, that prints the linearised process in a readable format. Reversely, a process file that contains a linear process can be transformed to internal format using the tool `txt2lps`.

Another tool `lpsinfo` provides information about an LPS, e.g., the number of multi-actions, summands, etc. Two tools `lpssim` and `lpsxsim` are used to simulate an LPS as shown in Sect. 2.2. The first one uses the command line, and the second one uses a graphical interface.

Using a linear process it is relatively easy to generate a state space. This is what the tool `Lps2lts` does. It starts with the initial state and puts it in the *todo*-buffer of states that need to be explored. For each state in the todo buffer, it goes through all summands of the linear process and evaluates whether the condition holds. If it holds a transition, or multiple transitions depending on E_i, are possible and these are added to the labelled transition system. The newly reached states that have not been explored yet are added to the *todo*-buffer. When the *todo*-buffer becomes empty the whole labelled transition system is generated. Note that by optimising the linear process, generating the state space can be sped up dramatically, and it is even possible to reduce the size of the state space while preserving the behaviour.

8.2 Optimising an LPS

We can optimise an LPS to obtain a reduced state space or to speed up its generation without changing the behaviour. This means that in general the reductions must preserve strong bisimulation. The following sections introduce the tools and techniques for such optimisations. Some of these optimisations are already applied by the lineariser automatically.

8.2.1 Rewriting a Linear Process

Using the rewriter `lpsrewr` on a linear process, it is possible to simplify a linear process by evaluating expressions. If the rewriter is applied to the linear process

```
act a, b;

proc P(m: Nat) =
        (1==2) -> a . P(0) +
        (forall n: Nat.(n==3 => n>0 )) -> b . P(m) +
        (forall n: Nat.(n==m => n*n==3*m)) -> b . P(m+1);

init P(1);
```

then the result is the following:

```
act  a, b;

proc P(m: Nat) =
       b . P(m) +
       (forall n: Nat. n==m => n*n==3*m) -> b . P(m + 1);

init P(1);
```

In the first summand the condition $1 == 2$ is rewritten to false, and the summand is deleted. The condition of the second summand can be determined to be equal to true, and therefore this condition is removed. The third summand is too complex for the simplifying rewriter and therefore it remains untouched.

There is however a quantifier-one-point rewriter, which searches for expressions of the form $\forall d : D.((d \approx e) \Rightarrow \phi(d))$ and $\exists d : D.(d \approx e \wedge \phi(d))$. Both expressions are equivalent to $\phi(e)$, i.e., the formula where d is replaced by e. The quantifier one-point rewriter searches for bound variables that can only have one value, and substitutes this value. The quantifier one-point rewriter does not apply simple rewriting, so this must be done separately. So, after applying `lpsrewr -pquantifier-one-point` and applying `lpsrewr` to the result we get the linear process

```
act a, b;
proc P(m: Nat) =
       b .P(m) +
       (m * m == 3 * m) -> b . P(m + 1);

init P(1);
```

8.2.2 Constant Elimination

Consider the following linear process.

```
act a: Nat;

proc P(n: Nat) = a(n) . P(n) + ( n == 2 ) -> a(0) . P(n+1);

init P(1);
```

This process infinitely often repeats the action $a(1)$. This is because the value of n remains 1 as the condition $n \approx 2$ can never be made true and the first summand does not change the value of n. We can optimise this linear process by substitution 1 for n. The resulting linear process becomes:

```
act a: Nat;

proc P = a(1) . P;

init P;
```

The tool `lpsconstelm` searches for parameters that are constant and replaces them by their constant values. This operation does not change the size of the reachable state space, but it can speed up its generation. Furthermore, the elimination of constant parameters can enable other optimisations of the linear process.

8.2.3 Parameter Elimination

Sometimes parameters do not influence the observable behaviour of a process and can therefore be removed. Consider the example

```
act up;

proc P(n: Nat) = up . P(n+1);

init P(0);
```

Here, the process variable n does not influence the behaviour, but it leads to an infinite state space. Using a simple algorithm it can be determined which variables do not influence the external behaviour, either directly, because these variables occur in actions or conditions, or indirectly, because the are used to determine the values of other variables that influence the behaviour. This is exactly what the tool `lpsparelm` does. It transforms the linear process above into the bisimulation

equivalent linear process below. Note that the infinite state space is transformed into a simple finite one.

```
act up;

proc P = up . P;

init P;
```

Sometimes variables cannot be removed directly but they are not relevant. Consider for instance the following process

```
act up: Nat;

proc P(n: Nat) = up(n) . P(n+1);

init P(0);
```

The tool `lpsparelm` is now unable to remove the parameter n as it is used in the action up. But it might be that the parameter n in $up(n)$ is not important, for instance because it needs to be investigated whether this process is deadlock free. By removing n from $up(n)$, lpsparelm can be applied. This can either be done by hiding the action up, effectively renaming it to τ, or by using the tool `lpsactionrename` explained below.

8.2.4 Sum Instantiation and Sum Elimination

If summands with sum operators in linear processes only have a finite number of elements that satisfy the condition, they can be replaced by explicit sums. This can be useful for two reasons. When generating the state space, the elements in the sum do not have to be enumerated all the time. This can save time, but it may also be more time consuming. The other reason is that after replacing a sum operator with summands, other optimisation tools may work better. For instance, rewriting is generally more successful on terms with concrete values.

The tool `lpssuminst` replaces summands in a linear process by instantiating them. It can be restricted to particular sorts, or to finite sorts. Note that if a sum to be eliminated ranges over an infinite domain the tool `lpssuminst` will not terminate.

As an example consider the following linear process:

```
act a: Nat;

proc P = sum n: Nat. (n+n==n*n && n<10) -> a(n) . P;

init P;
```

Applying `lpssumint` to this linear process yields the process

```
act  a: Nat;

proc P = a(0) . P + a(2) . P;

init P;
```

Sum instantiation works by enumerating all elements over the domains in the sum operator, until no elements are left. For infinite domains, this can be derived using the condition. In the example above the condition $n < 10$ puts a limit on the number of natural numbers that need to be investigated.

Sometimes, the value of the variable in a sum operator is determined using an equality. Consider for instance:

```
act  a: Real;

proc P = sum r: Real. (r == 22/7) -> a(r) . P;

init P;
```

It is not possible to enumerate the reals. But in this concrete case, using the condition $r \approx \frac{22}{7}$ it can be derived that r can only be $\frac{22}{7}$. This is what the tool `lpssumelm` is doing. It searches for equalities in the condition, and applies them to the summand, removing variables in the sum. It also removes variables in sums that are not used. Applied to the example above, it yields

```
act  a: Real;

proc P = a(22/7) . P;

init P;
```

Sum elimination finds its theoretical justification in the sum elimination theorem. The lineariser applies this theorem frequently when calculating the effects of communications.

8.2.5 Action Renaming

The renaming of actions in processes is discussed in Sect. 3.4. Using the rename operator an action label can be renamed into another action label. Using the hiding operator an action can be renamed into τ. But for linear processes it is sometimes useful to rename actions in a much more subtle way, for instance depending of the values

of the data values occurring in actions. This is what the tool `lpsactionrename` can do.

The tool requires two arguments. A linear process file and a rename file or regular expression that prescribes how renaming takes place. Using the flag `--regex=b/c` means that every letter `b` in an action label is replaced by `c`. By writing `--regex=^b$/c` each action 'b' is replaced by `c`. Using regular expressions only the action label in actions can be changed. It is not possible to take information about the data parameters into account.

Using a rename file it is possible to rename actions in a more refined way. Assume that it is called `rename.ren`, it is used by using the command

```
lpsactionrename -v --renamefile=rename.ren input.lps output.lps
```

The rename file typically looks as follows.

```
act error, error1;

var i: Int;

rename (i>1) -> a(i) => error;
       (i==1) -> a(i) => error1;
```

This file causes all $a(i)$ actions to be renamed to *error*, if $i > 1$. Each action $a(1)$ is renamed to $error_1$, and $a(0)$ remains untouched. The rename rules are applied in the sequence they occur. Writing the condition is optional and it is taken to be *true* if omitted. We can also rename an action to *tau* or *delta*. A file for renaming rules can contain *sort*, *cons*, *map*, *eqn* and *act* sections and these are added to the linear specification.

Using a rename file, it is also possible to remove and change and even add parameters to actions. A typical example is the following, where an action a with three arguments is renamed to an action a with two arguments if the first argument is larger than 0. Otherwise, if $i_1 = 0$, it is renamed into action b with $i_2 + 7$ as argument.

```
act a: Nat#Nat;
    b: Nat;

var i1,i2,i3: Nat;
rename (i1>0) -> a(i1,i2,i3) => a(i1, i2+i3);
       a(i1,i2,i3) => b(i2+7);
```

Exercise 8.2 Write a rename file that can be used to rename actions $a(i)$ to *error* in the following mCRL2 specification if i is even.

```
act a: Nat;

proc P(i: Nat) = (i>=0) -> a(i) . P(i+1);

init P(0);
```

8.2.6 Unfolding Parameters

The data parameters can have a complex structure. Consider the following example.

```
sort Pair = struct pair(first: Nat, second: Nat);

act a: Nat;

proc P(p: Pair) = a(first(p)) . P(pair(first(p), second(p)+1));

init P(pair(0,0));
```

The state space consists of states of the form $pair(0, i)$ for all $i \geq 0$, and hence is infinite. However, the behaviour only depends on the first argument of such pairs. The tool `lpsparelm` does not work as the parameter p plays a role in the external behaviour.

The parameter p is constructed using the constructor *pair* on two arguments. If we replace p by these arguments we can reconstruct p and also eliminate the second parameter using *lpsparelm*.

The tool `lpsparunfold` does exactly this. It takes process parameters of sorts with constructors and replaces them with the arguments.

We apply the following commands to the example above.

```
txt2lps file.txt file.lps
lpsparunfold --sort=Pair file.lps file_out.lps
lpsrewr file_out.lps file_rout.lps
```

The resulting linear process is the following, and contains a number of auxiliary data types.

```
sort Pair1;
     Pair = struct pair(first: Nat, second: Nat);

cons c_pair: Pair1;

map  pi_Pair1_,pi_Pair1_1: Pair -> Nat;
     Det_Pair1_: Pair -> Pair1;
```

```
      C_Pair1_: Pair1 # Pair -> Pair;

var   y2: Pair;
      y1: Pair1;
      d,d1: Nat;
eqn   C_Pair1_(c_pair, y2)  =  y2;
      C_Pair1_(y1, y2)  =  y2;
      Det_Pair1_(pair(d, d1))  =  c_pair;
      pi_Pair1_(pair(d, d1))  =  d;
      pi_Pair1_1(pair(d, d1))  =  d1;

act   a: Nat;

proc P(Pair_pp: Pair1, Pair_pp1,Pair_pp2: Nat) =
       a(Pair_pp1) .
         P(Pair_pp = c_pair, Pair_pp2 = Pair_pp2 + 1);

init P(c_pair, 0, 0);
```

Applying `lpsparelm` to the result yields

```
act   a: Nat;

proc P(Pair_pp1: Nat) = a(Pair_pp1) . P(Pair_pp1);

init P(0);
```

where we remove all generated data types that are not used in the behaviour anymore for readability. It is clear that by doing this, the state space is substantially reduced.

The tool `lpsparunfold` can be used on any sort that has constructors. For instance it can be applied to lists. Each list is replaced by its first element and by the tail. If it is known that lists only have a finite length, then by repeatedly applying `lpsparunfold` the lists can be completely replaced by the elements occurring in the list.

8.3 Parameterised Boolean Equation Systems

A modal formula and a linear process are transformed into a PBES (Parameterised Boolean Equation System). Each PBES has a unique solution. The modal formula is valid on the initial state of the process exactly if the initial variable has solution true. Calculating this initial solution can be a computationally demanding procedure. As a side note, we remark that many other problems can relatively straightforwardly be reformulated as a PBES that must be solved.

It goes too far go into depth here what a PBES is exactly, but for the moment it suffices to think of them as similar to process equations with parameters, see [28, 34] for exact descriptions. Just to give an impression, we give a rather arbitrary example of a PBES below:

$$\begin{array}{ll}\textbf{pbes} & \mu X_1(n:\mathbb{N}, l:\mathit{List}(\mathbb{N})) = ((n > 5) \wedge X_1(n, n\langle l\rangle)) \vee X_2(n, n+1, l);\\ & \nu X_2(n, m:\mathbb{N}, l:\mathit{List}(\mathbb{N})) = (\#l < m) \vee \forall i{:}\mathbb{N}.(i < m \Rightarrow X_1(n, \mathit{tail}(l)));\\ \textbf{init} & X_1(3, []);\end{array}$$

There are a number of boolean equations, preceded by a minimal (μ) or maximal (ν) fixed point symbol. There are PBES variables, h.l., X_1 and X_2, that can carry zero or more parameters. In the right hand side of each equation there is a boolean expression, in which the PBES variables can recursively occur, possibly with changed parameters. There is one initial instantiated PBES variable. Solving a PBES means to determine whether this initial instantiated variable is false or true in the context of the PBES equations.

The tool `lps2pbes` transforms a linear process and a modal formula into a PBES. Using the command `pbespp` a PBES can be printed in readable form and inspected.

8.4 Optimising PBESs

Note that the structure of a PBES resembles that of a linear process, although there can be multiple equations that must be preceded by a fixed point symbol, and at the right side boolean operators are used. But just as linear processes, PBESs are amenable to optimisations such as the removal of parameters.

8.4.1 Rewriting PBESs

Using the tool `pbesrewr` it is possible to simplify a PBES. In its simplest form, it applies the rewrite rules to a PBES. We describe PBESs here in teletype font, such that these examples can easily be redone using the toolset. Consider the following PBES.

```
pbes mu X(n: Nat) = val(n>=n) || X(n+1);

init X(0);
```

Using the tool `txt2pbes` this textual file can be transformed into a PBES file in internal format. By applying `pbesrewr` to it, it is reduced to

```
pbes mu X(n: Nat) = true;

init X(0);
```

This process can easily be solved, using for instance the tool `pbes2bool`, and the answer is true. A PBES in internal format can be printed using the pretty printer `pbespp` and the tool `pbesinfo` is used to obtain some basic information about a given .pbes file.

Besides the simplify rewriter, there are other rewriter algorithms. For instance the quantifier-all rewriter tries to eliminate all quantifiers by instantiating them with concrete values. Consider the following example.

```
pbes mu X(m: Nat) = forall n: Nat.(val(n<3) => X(n+1));

init X(0);
```

Parsing the input text, applying the quantifier-all rewriter and printing the result is done using the following sequence of instructions.

```
txt2pbes input.txt input.pbes
pbesrewr -pquantifier-all input.pbes output.pbes
pbespp output.pbes
```

The output that is obtained is the following

```
pbes mu X(n: Nat) = X(1) && X(2) && X(3);

init X(0);
```

Solving this PBES using the tool `pbessolve` yields false. This is done by instantiating the equations in a similar way as a state space is generated, and solving the resulting instantiated boolean equation system using standard algorithms.

There are variants of quantifier-all rewriter. The quantifier-finite rewriter replaces all quantifiers ranging over finite domains by conjunctions for $\forall$, and disjunctions for $\exists$. This can be useful because the elimination of quantifiers over infinite domains may not terminate.

The quantifier-one-point rewriter tries to eliminate quantifiers if it can determine that the bound variable in the quantifier must be equal to a concrete value based on the expression under consideration. Consider for instant the PBES:

```
pbes mu X(m: Nat) = exists n: Nat.(val(n==m+1) && X(n));

init X(0);
```

Using the quantifier-one-point rewriter this can be simplified to the following PBES, as it can determine that setting the existentially bound variable n to be $m + 1$ maintains the validity of the formula. The resulting PBES is given below:

```
pbes mu X(m: Nat) = X(m+1);

init X(0);
```

This PBES cannot be solved easily. We require to apply `pbesparelm`, explained below, to remove the parameter m from the equation for X in the PBES. Then it can easily be seen that this PBES is false.

8.4.2 Constant Elimination

Just as for linear processes, an optimisation technique is to replace parameters that have constant values by these constants. The tool `pbesconstelm` can do this. Consider for instance the following example.

```
pbes nu X(m: Nat) = val(m>0) && X(m+1) ||
                    val(m==0) && X(m);

init X(0);
```

The parameter m keeps the value 0, but that can only be seen by concluding that the condition $m > 0$ cannot be true. This is obtained by applying `pbesconstelm -c`. The flag `-c` is required to also investigate the expressions in a PBES. The result must explicitly be rewritten using `pbesrewr`, which is

```
pbes nu X = X;

init X;
```

As this is a maximal fixed point, the solution is equal to true.

Using the flag `-a` the tool `pbesconstelm` tries to push quantifiers as much as possible inside formulas, even distributing them over the PBES variables.

8.4.3 Parameter Elimination

Some parameters can be removed from a PBES without changing the solution of the PBES. This can be when a parameter does not occur in any expression, besides being passed on to other PBES variables. The effect of this can even be more substantial

than removing constants. PBESs that cannot be solved, can become solvable by applying parameter elimination. The following command shows how this is done.

```
pbesparelm -v input.pbes output.pbes
```

If it is applied to the following PBES:

```
pbes mu X(m: Nat) = X(m+1);

init X(0);
```

the result of applying `pbesparelm` is

```
pbes mu X = X;

init X;
```

which is easily shown to be false.

Chapter 9
Applications: Puzzles and Games



Many puzzles can very naturally be described in mCRL2 and solved using the available analysis techniques such as model-checking. In this chapter, six different puzzles are specified and solved.

9.1 Magic Square

A magic square is a square matrix filled with the numbers from 1 to n. The numbers are placed in such a way that the sum of the numbers in each row, column and diagonal is the same. For example, a 3×3 magic square is given below where 15 is the sum of every row, column and diagonal.

The question is, how to figure out all the possible solutions that form a magic square. Using the sum operator in mCRL2 the problem can concisely be formalised as shown below. By generating the state space, all solutions are found.

```
map all_unique:List(Pos)->Bool;

var n:Pos; L:List(Pos);
eqn all_unique([])=true;
    all_unique(n|>L)= !(n in L) && all_unique(L);
act a:Pos#Pos#Pos#Pos#Pos#Pos#Pos#Pos#Pos;

init sum n1,n2,n3,n4,n5,n6,n7,n8,n9:Pos.
             (n1+n2+n3==15 &&
              n4+n5+n6==15 &&
              n7+n8+n9==15 &&
              n1+n4+n7==15 &&
              n2+n5+n8==15 &&
              n3+n6+n9==15 &&
```

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-3-031-23008-0_9.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0_9

Table 9.1 3×3 Magic square

n_1	n_2	n_3
n_4	n_5	n_6
n_7	n_8	n_9

8	3	4
1	5	9
6	7	2

8	1	6
3	5	4
4	9	2

4	9	2
3	5	7
8	1	6

4	3	8
9	5	1
2	7	6

6	1	8
7	5	3
2	9	4

6	7	2
1	5	9
8	3	4

2	9	4
7	5	3
6	1	8

2	7	6
9	5	1
4	3	8

Fig. 9.1 The 8 solutions of the magic square

```
n1+n5+n9==15 &&
n7+n5+n3==15 &&
all_unique([n1,n2,n3,n4,n5,n6,n7,n8,n9])) ->
            a(n1,n2,n3,n4,n5,n6,n7,n8,n9).delta<>delta;
```

The magic square is filled with the numbers $n_1, \ldots, n_9$. The constraints that all diagonals, rows and columns must add up to 15 are explicitly stated. The predicate *all_unique*(L) gets a list of elements and yields true if all the elements in the list L are unique. Using this predicate it is guaranteed that $n_1, \ldots, n_9$ are all different. The action a, which takes 9 arguments, can happen exactly for every solution that exists for the magic square given in Table 9.1.

Generating the state space yields 8 symmetrical solutions. This can for instance be done using `mcrl2ide` or using the commands below. The solutions are listed in Fig. 9.1.

```
mcrl22lps -v -D square.mcrl2 square.lps
lps2lts -v square.lps square.lts
ltsgraph square.lts
```

Exercise 9.1 ** Sudoku puzzles can also be solved with mCRL2. You can find a puzzle of your choice and find the solution with model-checking.

9.2 Bridge Crossing

This is a puzzle where four people have to cross a bridge at night. See Fig. 9.2. Maximally two persons can cross the bridge simultaneously. One torch is available at the starting point. The torch is mandatory while crossing the bridge and it must be

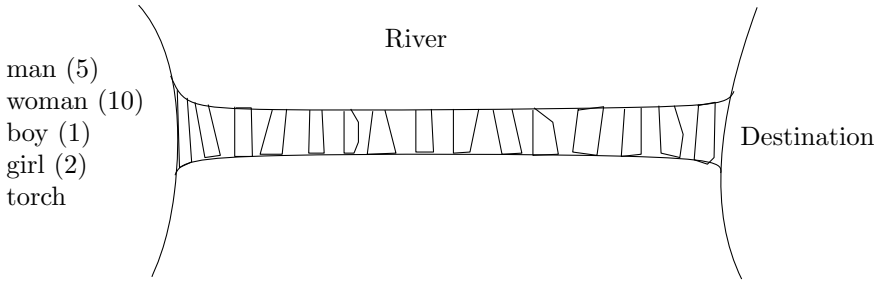


Fig. 9.2 Bridge crossing puzzle

brought back such that others can use it. The four people consist of a boy that can cross the bridge in 1 min, a girl that can do it in two minutes, and an elderly man and woman who require respectively 5 and 10 min to cross the bridge. The question is to find the minimal time in which all four people can cross the bridge.

Before proceeding, it is useful to try to solve the puzzle, either with classical means or by making a model in mCRL2 and analysing its behaviour. The puzzle is interesting because the solution is unexpected, but this may be hard to appreciate when seeing the solution without first thinking about it.

Below we model the process of crossing the bridge in mCRL2 describing all possible ways to cross the bridge. The datatype *PersonType* contains the persons and the datatype *Side* contains the two sides, namely *start* and *destination*. The process *Crossing* models how the persons can cross bridge.

```
sort PersonType = struct man | woman | boy | girl;
     Side = struct start | destination;

map Time2Cross: PersonType->Nat;

eqn Time2Cross(boy)=1;
    Time2Cross(girl)=2;
    Time2Cross(man)=5;
    Time2Cross(woman)=10;

act cross, return: Set(PersonType);
    allCrossed: Nat;

proc Crossing(S: Set(PersonType), t: Nat, torch: Side)=
       (S!={} && t<=19)
         -> (sum p1, p2: PersonType.
               (torch==start && p1 in S && p2 in S) ->
               -> cross({p1, p2}).
                  Crossing(S-{p1, p2},
                           t+max(Time2Cross(p1), Time2Cross(p2)),
                           destination) +
             sum p1, p2: PersonType.
               (torch==destination && !(p1 in S) && !(p2 in S))
               -> return({p1, p2}).
```

```
25                    Crossing(S+{p1, p2},
26                             t+max(Time2Cross(p1), Time2Cross(p2)),
27                             start)
28              ) +
29          (S=={})->allCrossed(t).delta;
30
31   init hide({cross, return},
32        Crossing({man, woman, boy, girl}, 0, start));
```

There are three actions. The action *cross*(P) with P a set of persons indicates which persons move to the destination side. The action *return*(P) indicates the set of persons P that return to the start side. The action *allCrossed*(t) takes place when all persons are at the destination side, where t indicates the total time it took to cross.

The process *Crossing* has three parameters, see line 14.

1. S: The set of persons that still must cross the bridge. Initially, $S = \{man, woman, boy, girl\}$.
2. t: The total time spent in each crossing the bridge up till now. At the beginning this time is set to 0.
3. *torch*: The side where the torch is. Initially, it is at the start.

In the process *Crossing* there are two main cases. If $S = \varnothing$, see line 29, all persons are at the other side and the *allCrossed*(t) action takes place.

If $S \neq \varnothing$, see line 15, then there are still persons that have to cross. Note that it is easy to see that the optimal time to cross cannot be larger than 19 min. When the boy accompanies the other persons and rushes back with the torch it requires $10 + 1 + 5 + 1 + 2 = 19$ min to cross the bridge. So, we limit the time to cross the bridge to 19 min. If we do not put a limit on t, the accumulated time can become unboundedly large. This is for instance possible when the boy rushes back and forth over the bridge alone. Although this is clearly pointless behaviour, it increases the time t to arbitrarily large values, creating an infinite state space.

In lines 16 to 21, the torch is at the start side. One or two persons can cross the bridge. If persons p_1 and p_2 are chosen to be the same then this models that only one person crosses the bridge. The persons p_1 and p_2 are removed from S as they are now going to the other side and the time t is increased with the maximum of the crossing time of both persons. Finally, the torch is registered to be at the destination side.

In lines 22 to 27 it is modelled how one or two persons move from the destination to the start side. It is completely symmetric, except that now the action *return* is used.

Using the tool `mcrl2ide` we can generate a state space, which turns out to have 429 states and 1426 transitions. By hiding the actions *cross* and *return*, and applying weak trace reduction we obtain the state space as shown in Fig. 9.3. This indicates that a crossing in 17 min is possible.

In order to find out how this is possible we write the formula that states that a crossing is possible in 17 min.

$$\langle true^* \cdot allCrossed(17) \rangle true$$

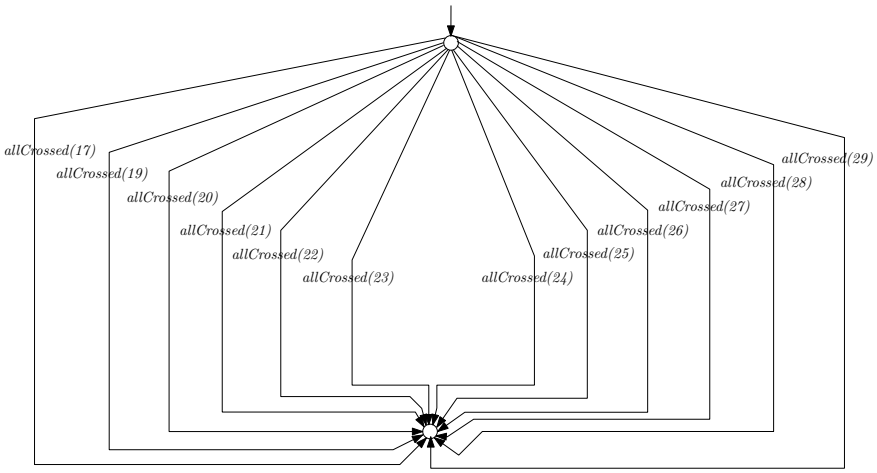


Fig. 9.3 Crossing times for the bridge obtained via weak trace reduction

As the formula is true, we are given a witness, which turns out to be the following.

1. Both the boy and girl cross the bridge together in 2 time units.
2. The girl comes back and the total time goes to 4.
3. The man and the woman cross the bridge together in 10 time units and then the total time becomes 14.
4. The boy comes back in 1 and the total time reaches 15.
5. In the end, both the boy and the girl cross together in 2 time units to make the total time 17.

We can also check our model to know whether the bridge crossing is possible in less than 17 min or not. For this purpose, the μ-calculus formula is:

$$[true^*]\forall i : \mathbb{N}.(i < 17) \Rightarrow [allCrossed(i)]false$$

The above formula is true. It means crossing of all persons is not possible in less than 17 time units.

When analysing such puzzles in mCRL2 the majority of the time goes into understanding and formalising the puzzle. But this is mentally less demanding than solving the puzzle outright. Provided the number of states is not too large, the tools are very helpful in obtaining the required solution. This problem is also modelled with mCRL2 in [24], along with some extra interesting examples.

Exercise 9.2 Why do we see actions $allCrossed(n)$ with values $n > 19$ in Fig. 9.3?

9.3 Weight Bars Crossing

The weight bars crossing puzzle is very similar to the bridge crossing puzzle of the previous section. One person wants to cross the bridge with four weight bars of 1kg, 2kg, 3kg and 4kg with the following constraints.

1. Maximally two weight bars can be transferred in one go.
2. The difference in total weights between every two successive transfers cannot be greater than 1.

We solve this problem using the model below. There are two processes P_1 and P_2. The process P_1 models the situation where the person is at the departing side. It has as arguments a set *here* with all weights that are at the departing side, and a value y that indicates the total carried weight of the previous crossing. The variable y equals 0 to indicate that this is the first crossing. When two equal weights are selected, they are considered to be the same.

The process P_2 is exactly the same as P_1, but models that the person goes back over the bridge. Its first parameter *here* also models the weights that reside at the departing side. The action *done* at line 22 is to show that all the weight bars have been transferred.

```
1  map all: Set(Pos);
2      weight: Pos#Pos->Pos;
3
4  var i,j: Pos;
5  eqn all = {1, 2, 3, 4};
6      weight(i,j) = if(i==j,i,i+j);
7
8  act moveAcross, moveBack:Set(Pos);
9      done;
10
11 proc P1(here: Set(Pos), y: Nat)=
12         sum i, j: Pos.(i in here && j in here &&
13           (y==0 || abs(weight(i,j)-y)<=1))->
14             moveAcross({i, j}).P2(here-{i, j},weight(i,j));
15
16       P2(here: Set(Pos), y: Nat)=
17         (here!={})
18         -> (sum i, j:Pos.(i in all-here && j in all-here &&
19               abs(weight(i,j)-y)<=1)->
20                 moveBack({i, j}).P1(here+{i, j}, weight(i,j))
21            )
22         <> done.delta;
23
24 init P1(all, 0);
```

The state space of this model consists of 90 states and 311 transitions. We want to know whether the action *done* is reachable. This is expressed by the following formula.

$$\langle true^* \cdot done \rangle true.$$

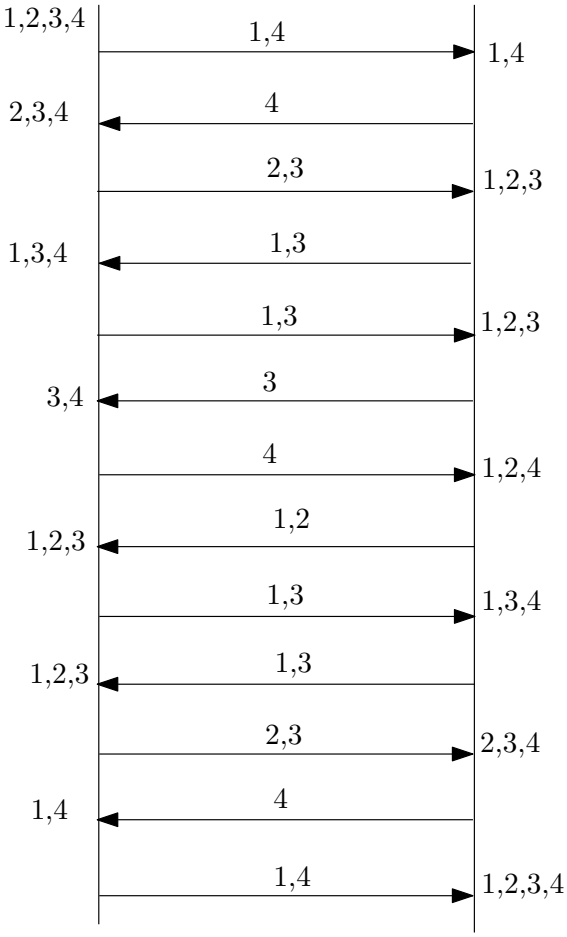


Fig. 9.4 Counter example

By model-checking we find that this formula is valid. The witness for this is depicted as a so-called message sequence chart in Fig. 9.4, where the departing side is on the left and each crossing is shown with arrows annotated with the transferred weight bars. This gives a shortest possible crossing sequence.

Exercise 9.3 The presented model does not include the possibility to cross the bridge empty handed. But if this is allowed, crossing could be more efficient. Extend the model to include this possibility and verify whether it is possible to bring all weights to the other side while crossing the bridge at least once empty handed, and figure out whether a shorter crossing is possible than the 13 crossings of Fig. 9.4.

9.4 The Wolf, the Goat and Cabbage

In approximately 780 AD, the scholar Alcuinus of York wrote a treatise with exercises to teach children how to think [1]. One of his exercises is about a river crossing problem. A farmer goes to a market and buys a cabbage, a wolf and a goat. While coming back home, the farmer has to cross a river with all these items. All the items remain safe in the presence of the farmer, but if left alone the wolf immediately eats the goat and, likewise, the goat will start eating the cabbage. A small boat is available to cross the river but it can carry only one item along with the farmer, and only the farmer can drive the boat. We need to provide a solution for a safe river crossing for all three items and the farmer. We model the problem in mCRL2 to find a solution. An alternative specification is given in [24].

```
1  sort Item=struct wolf | cabbage | goat;
2       Side = struct left | right;
3
4  map isSafe: Set(Item)->Bool;
5      all: Set(Item);
6      opposite: Side->Side;
7
8  var st:Set(Item);
9  eqn isSafe(st) = !({wolf,goat}<=st || {cabbage,goat}<=st);
10     all={wolf, cabbage, goat};
11     opposite(left) = right;
12     opposite(right) = left;
13
14 act move: Item#Side;
15     move: Side;
16     done;
17
18 proc P(here: Set(Item), s: Side)=
19        sum i:Item.
20          if(s==left, i in here && isSafe(here-{i}),
21                      i in all-here && isSafe(all-here-{i}))
22          -> move(i, s) .
23             P(if(s==left,here-{i}, here+{i}), opposite(s)) +
24        if(s==left, isSafe(here), isSafe(all-here))
25          -> move(s) . P(here, opposite(s)) +
26        (s==right && here=={})
27          -> done . delta;
28
29 init P(all, left);
```

In the specification given above, line 1 defines a data type *Items* for the item to be taken across and on line 2 both sides of the river are declared as *left* and *right*. Line 4 declares a function *isSafe* to determine whether items in a set are safe to be left unsupervised. This function returns false (see line 9) exactly if either the wolf and the goat, or the goat and the cabbage are in the set of items. Line 5 declares a function for the set *all* of all items, i.e., {*wolf*, *cabbage*, *goat*}. The mapping *opposite* gives the opposite shore.

The process P describes all the safe behaviour that the farmer can do. The process P has two arguments, namely a set of items *here* which are the items at the left side of the river. The parameter s indicates at which side the farmer is. In lines 19 to 23 it is described how one item i can be moved to the other side. Only moves that leave the items behind in a safe way are allowed.

In lines 24 and 25 it is described how the farmer can move alone to the other side. This is also only possible if he leaves the items safely behind. In the lines 26 and 27 it is described that if the farmer is at the right side together with all the items, then a *done* action can take place.

The state space of safe moves is remarkably small with only 11 states and 21 transitions. It can easily be visualised as can be seen in Fig. 9.5. From this picture it is immediately clear that there are essentially two ways to reach a state with the action *done*.

An alternative way to check whether there is a solution is by using the following modal formula saying that the action *done* can be reached.

$$\langle true^* \cdot done \rangle true$$

and get a witness such as given in Fig. 9.6 that explains how the farmer can safely cross the river.

Exercise 9.4 The formula below expresses that the action done will happen. Is it usable for our model? Is it valid?

$$\mu X.[\overline{done}]X \wedge \langle true \rangle true$$

9.5 Jumping Frogs

In this problem, there are six frogs sitting on stones in a pond in a row as shown in Fig. 9.7. The three frogs at the left-hand side want to go the right and vice versa. One stone in the middle is empty. A frog can jump to an empty stone if the stone is either adjacent or reachable by jumping over one frog. A frog can jump towards its desired direction only and at most one frog can sit on a stone. We need to devise a jumping strategy such that all frogs reach their destination.

In the required strategy, we need to know which frog should jump from which stone to which stone. So, to identify frogs and stones, we assign unique numbers to them. The numbers start from the left, as shown in Fig. 9.7 where the frogs 0, 1, 2, 3, 4 and 5 are sitting on the stones 0, 1, 2, 4, 5 and 6, respectively. The stone number 3 is empty and, as a first jump, any of the frogs 1, 2, 3 and 4 can jump to it.

There are exactly two solutions of this problem, obtainable by either generating the state space or by model-checking. The solutions are shown in Table 9.2, where the action $move(x, y, z, forward/backward)$ means frog x jumps from stone y to

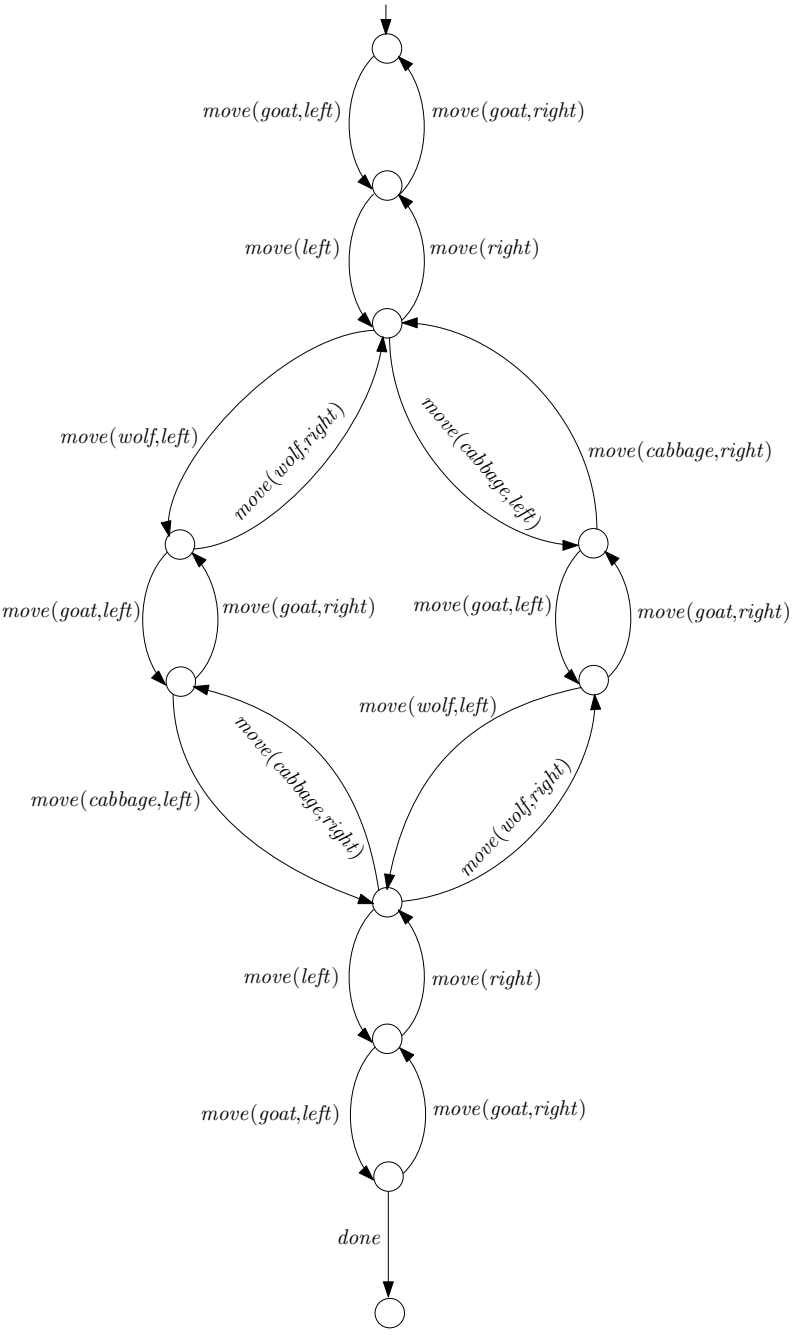


Fig. 9.5 LTS of a safe river crossing with a wolf, a goat and cabbage

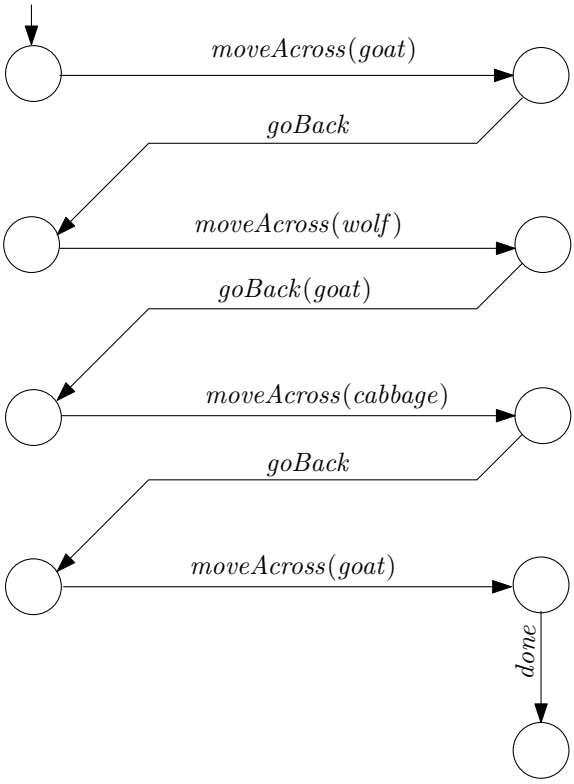


Fig. 9.6 Trace for the *done* action

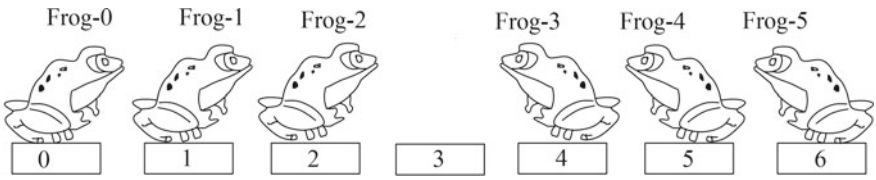


Fig. 9.7 Frog jumping problem

stone z in the *forward/backward* direction. The direction *forward* means moving to the right towards stone 6, while *backward* means moving left towards stone 0.

We model the frogs as concurrent processes. Each frog can jump one or two steps in a fixed direction. We devise another process called *Monitor* that communicates with the frogs and it monitors the jumping conditions.

The specification of this problem in the syntax of mCRL2 is given below. Line 1 defines a sort *Direction*. The process *Frog* is defined at line 14. Each frog has an ID, a position and a direction to move. Line 15 shows that if the direction is *forward*, a frog has the choice to jump to either $position + 1$ or $position + 2$ (see lines 15 and

Table 9.2 Jumping order obtained by model-checking

Strategy 1	Strategy 2
1. *move*(2, 2, 3, *forward*)	1. *move*(3, 4, 3, *backward*)
2. *move*(3, 4, 2, *backward*)	2. *move*(2, 2, 4, *forward*)
3. *move*(4, 5, 4, *backward*)	3. *move*(1, 1, 2, *forward*)
4. *move*(2, 3, 5, *forward*)	4. *move*(3, 3, 1, *backward*)
5. *move*(1, 1, 3, *forward*)	5. *move*(4, 5, 3, *backward*)
6. *move*(0, 0, 1, *forward*)	6. *move*(5, 6, 5, *backward*)
7. *move*(3, 2, 0, *backward*)	7. *move*(2, 4, 6, *forward*)
8. *move*(4, 4, 2, *backward*)	8. *move*(1, 2, 4, *forward*)
9. *move*(5, 6, 4, *backward*)	9. *move*(0, 0, 2, *forward*)
10. *move*(2, 5, 6, *forward*)	10. *move*(3, 1, 0, *backward*)
11. *move*(1, 3, 5, *forward*)	11. *move*(4, 3, 1, *backward*)
12. *move*(0, 1, 3, *forward*)	12. *move*(5, 5, 3, *backward*)
13. *move*(4, 2, 1, *backward*)	13. *move*(1, 4, 5, *forward*)
14. *move*(5, 4, 2, *backward*)	14. *move*(0, 2, 4, *forward*)
15. *move*(0, 3, 4, *forward*)	15. *move*(5, 3, 2, *backward*)

18, respectively). Similarly, for the *backward* direction, a frog can jump either to *position* − 1 or *position* − 2 (see lines 21 and 24). The change in position after each jump is administrated in every recursive call.

The process *Monitor* tracks the empty stone and maintains two sets, i.e., the set *right* to record which frogs moving in the forward direction are at their destination, and the set *left* doing the same for the frogs moving in the backward direction. The process *Monitor* is initiated as *Monitor*(3, {}, {}) to express that stone 3 is empty and no frog has reached its destination. Lines 29 and 30 show that the *Monitor* process stops when the sets *left* and *right* become {3, 4, 5} and {0, 1, 2}. Otherwise, the monitor notes each movement of a frog using the action *noteMove* and updates the sets *left* (line 35) or *right* (line 32), accordingly.

```
1  sort Direction = struct forward | backward;
2
3  map updateForward: Nat#Nat#Set(Nat)->Set(Nat);
4      updateBackward: Nat#Nat#Set(Nat)->Set(Nat);
5
6  var n, m: Nat; st: Set(Nat);
7
8  eqn updateForward(m, n, st)  = if(n<=3, st, {m}+st);
9      updateBackward(m, n, st) = if(n>=3, st, {m}+st);
10
11 act jump, noteMove, move:Int#Int#Int#Direction;
12     done;
13
14 proc Frog(id, position: Int, dr: Direction)=
15        (dr==forward)->(jump(id, position, position+1, dr).
```

```
                          Frog(position=position+1)
                        +
                          jump(id, position, position+2, dr).
                            Frog(position=position+2)
                       )
                    <>(jump(id, position, position-1, dr).
                          Frog(position=position-1)
                       +
                        jump(id, position, position-2, dr).
                          Frog(position=position-2)
                       );

      Monitor(empty: Nat, right, left: Set(Nat))=
        (left=={3, 4, 5} && right=={0, 1, 2})
        -> done . delta
        <> (sum k, prev: Nat.noteMove(k, prev, empty, forward) .
              Monitor(prev, updateForward(k, empty, right), left)
            +
            sum k, prev: Nat.noteMove(k, prev, empty, backward) .
              Monitor(prev, right, updateBackward(k, empty, left)));

init allow( { move, done },
      comm ( { jump | noteMove -> move },
              Frog(0, 0, forward) || Frog(1, 1, forward) ||
              Frog(2, 2, forward) || Frog(3, 4, backward)||
              Frog(4, 5, backward)|| Frog(5, 6, backward)||
              Monitor(3, {}, {})));
```

The state space of this problem has 81 states and 91 transitions and can neatly be visualised using the tool `ltsgraph`. By inspecting the state space, it is clear that there are only two paths to the action *done*.

We can also find the action *done* in the state space by using the following formula.

$$\langle true^* \cdot done \rangle true.$$

Strategy 2 shown in Table 9.2 is obtained as a witness of the above-mentioned formula. To find strategy 1, we change the above formula to:

$$\langle \overline{move(3, 4, 3, backward)}^* \cdot done \rangle true$$

This formula says that the action *done* must be reached on a trace without action $move(3, 4, 3, backward)$. Its witness gives us strategy 1 in Table 9.2.

Exercise 9.5 ** Change the above specification to two groups of 4 frogs and find a jumping strategy to let the frogs cross each other.

9.6 Tic-Tac-Toe

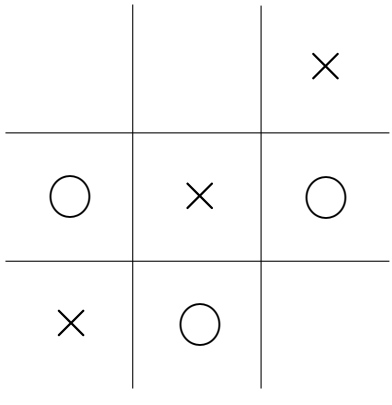
Tic-tac-toe is a two-player game played on a three-by-three grid. Marks '×' (cross) or a '◯' (naught) are placed on a grid alternatingly. A naught is the first mark to put on the board, and a player wins when completing either a row, a column, or a diagonal with the same mark first. For example, see Fig. 9.8 where the mark '×' is aligned in a diagonal showing a possible win. The game continues until a player wins, or no player wins and the grid is completely filled leading to a draw.

We can explore all the possible placements of both marks by modelling this game in mCRL2. Initially, the first player can place a naught in 9 places of the grid. Thereafter, the other player has 8 options as shown in Fig. 9.9. In this way, the possible options continue decreasing by 1 after every placement. Our model of the behaviour of this game contains 5479 states and 17109 transitions. This is too big to depict the LTS on a screen using the tool `ltsgraph`. However, using the *exploration mode* the tool `ltsgraph` can be used to traverse parts of the transition system by clicking any of the '⊕' signs as shown in Fig. 9.9.

A behavioural model. In this section we present an mCRL2 model with a process *Player* representing the game. This process uses a grid as a board on which the marks are placed. A data type *Mark* is defined at line 1 where '◯' and '×' are marks while *nil* means an empty field in the grid. The board as a grid is represented by a list of lists with marks as shown at line 2. Line 8 declares a function $isWinner(b, m)$ that determines whether the player with mark m won on the board b. All these functions are defined using equations from lines 12 to 23. The functions declared and defined from lines 25 to 38 are inspired by Exercise 5.20.

```
1  sort Mark = struct O | X | nil;
2       Board = List(List(Mark));
3
4  map Rows: Nat;
5      Columns: Nat;
6      initBoard: Board;
```

Fig. 9.8 Grid for tic-tac-toe



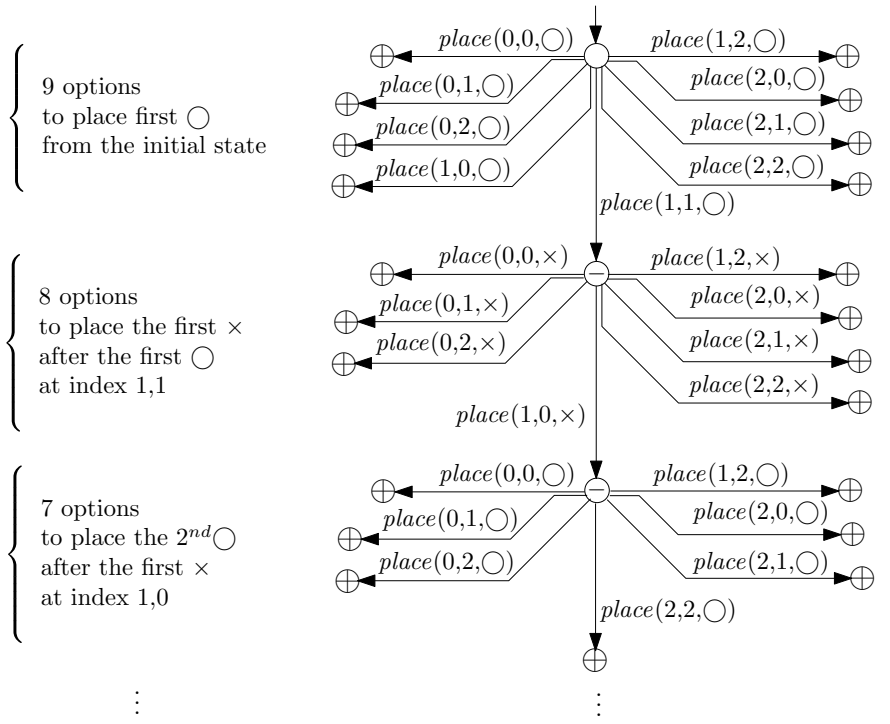


Fig. 9.9 LTS for tic-tac-toe

```
7       opponent: Mark->Mark;
8       isWinner: Board#Mark->Bool;
9
10  var m: Mark;
11      b: Board;
12  eqn Rows = 3;
13      Columns = 3;
14      initBoard = [[nil,nil,nil],[nil,nil,nil],[nil,nil,nil]];
15      opponent(X) = O;
16      opponent(O) = X;
17      isWinner(b,m) =
18          (exists row: Nat.(row<Rows && b.row.0==m &&
19                                  b.row.1==m && b.row.2==m)) ||
20          (exists col: Nat. (col<Columns && b.0.col==m &&
21                                  b.1.col==m && b.2.col==m)) ||
22          (b.0.0==m && b.1.1==m && b.2.2==m) ||
23          (b.0.2==m && b.1.1==m && b.2.0==m);
24
25  map replaceIn2D: Board#Nat#Nat#Mark->Board;
26      replaceIn1D: List(Mark)#Nat#Mark->List(Mark);
27
28  var llst: Board;
29      i, j: Nat;
```

```
30      e, n: Mark;
31      ls: List(Mark);
32
33  eqn replaceIn2D ([], i, j, n)=[];
34      replaceIn2D(ls|>llst,i,j,n)=if(i==0,replaceIn1D(ls,j,n)|>llst,
35                              ls|>replaceIn2D(llst, Int2Nat(i-1), j, n));
36      replaceIn1D([], j, n) = [];
37      replaceIn1D(e|>ls,j,n) = if(j==0,n|>ls,
38                                  e|>replaceIn1D(ls, Int2Nat(j-1), n));
39
40  act place: Nat#Nat#Mark;
41      win: Mark;
42
43  proc Player(b: Board, m: Mark) =
44        isWinner(b,opponent(m))
45        -> win(opponent(m)) . delta
46        <> (sum r,c: Nat.(r<Rows && c<Columns && b.r.c==nil)
47            -> place(r,c,m) . Player(replaceIn2D(b,r,c,m),opponent(m))
48           );
49
50  init Player(initBoard,O);
```

The game is initiated with the process *Player* which uses the initial empty board and allows the player with the mark '◯' to put the first mark. The definition of the process *Player* starts from line 43 and ends at line 48. It first checks whether the opponent won the game, and if not it allows that its mark is put on an empty field, after which the opponent can make a move.

Properties. We can write properties of this game in the modal μ-calculus.

1. Let us start with a safety property, which helps us to check whether we made mistakes in our model. A location in the grid cannot be used by both players, i.e., it is mutually exclusive. This property is expressed as:

$$\forall i, j : \mathbb{N}.(i < 3 \wedge j < 3) \Rightarrow \\ [true^* \cdot \exists m : Mark.place(i, j, m) \cdot true^* \cdot \exists m' : Mark.place(i, j, m')]false.$$

   The above property is satisfied.
2. A very interesting property is whether player ◯ has a winning strategy. This means that player ◯ can always put its mark such that he either wins, or for every counter move of player ×, player ◯ has a winning strategy. We use a minimal fixed point, but as the game is finite, a maximal fixed point would also do. This property is expressed as:

$$\mu Y.\ \exists i, j : \mathbb{N}.(\langle place(i, j, O)\rangle \\ (\langle win(O)\rangle true \vee (\langle true\rangle true \wedge \forall k, l : \mathbb{N}.[place(k, l, X)]Y)))$$

   This formula is false. If player × plays well, it can always force the game to become a draw.
3. The player starting the game with ◯ has a finite winning strategy if player × does a bad first move.

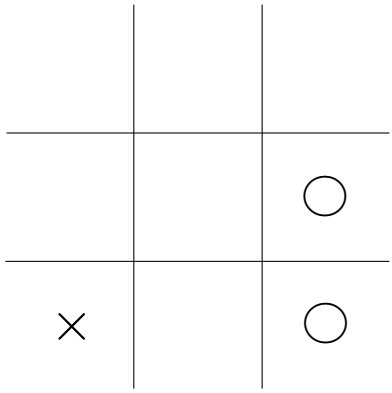


Fig. 9.10 A winning situation for player ◯

$$\langle \exists i, j : \mathbb{N}.place(i, j, O)\rangle$$
$$\langle \exists i, j : \mathbb{N}.place(i, j, X)\rangle$$
$$\mu Y.\exists i, j : \mathbb{N}.(\langle place(i, j, O)\rangle$$
$$(\langle win\rangle true \vee (\langle true\rangle true \wedge \forall k, l : \mathbb{N}.[place(k, l, X)]Y))).$$

This property holds. A witness reveals that the first three moves lead to the situation as depicted in Fig. 9.10.

Exercise 9.6 Is the following formula true w.r.t. the above tic-tac-toe model?

$$\forall i, j : \mathbb{N}.[(i < 3 \wedge j < 3) \wedge place(i, j, X)]false.$$

Exercise 9.7 Consider an initial board where there is a ◯ at coordinate $(0, 0)$ and a $\times$ at coordinate $(1, 0)$. Does player ◯ have a winning strategy in this case?

Chapter 10
Applications: Distributed Algorithms



This chapter discusses the application of mCRL2 to analyse distributed algorithms. Such algorithms are essential for the functioning of distributed systems. There are multiple participants in such a system and usually these participants communicate over a network. The participants strive towards achieving some common goal. Often these algorithms are described textually, supported by fragments specified in some semi-programming language and supplemented with some arguments why the algorithm is supposed to work. But it is not strange to find that the analysis is not precise, and that the algorithms can have behaviour that is quite unexpected.

As distributed algorithms are very tricky to get correct, modelling them precisely using for instance mCRL2, and model-checking that the common goal is achieved is an effective way to improve the quality of such algorithms. Experience shows that when applying such techniques to existing distributed algorithms, many fall short [3, 14, 25, 29, 48].

In this chapter we restrict ourselves to heartbeat protocols which are commonly used as parts of other distributed algorithms [23, 40, 54, 57, 58].

We study several versions of a heartbeat protocol, taken from [22], and show how to model-check that they are correct and safe. These protocols have been model-checked in [2]. The findings are quite remarkable, and we are quite sure that the original authors did not properly understand all the potential behaviour of their protocols.

10.1 Heartbeat Protocols

A heartbeat protocol is used to check whether all processes in a distributed system are still alive. If one of them dies, i.e., stops sending a heartbeat, the other processes must also die. Heartbeat protocols have multiple participants, and one of them is

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-3-031-23008-0_10.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0_10

designated as a central manager or a coordinator. They communicate over networks that may not be reliable and drop messages.

The central manager exchanges periodic heartbeat messages with other participants. The participants respond by sending their own heartbeat back. If participants observe no heartbeat for some time, they assume that a participant has died and they decide to stop functioning as well.

Participants can die voluntarily at any time, or they can die involuntarily as a consequence of not receiving heartbeats of others anymore. It might also be that due to message loss, heartbeats are not received, which can lead to a participant deciding to stop functioning, leading to the unintended termination of all participants. An important property of heartbeat protocols is resilience against message loss. The protocol that we study here is claimed to have good resilience [22].

Correctness of a heartbeat protocol consists of three aspects:

R1. If no participant inactivates itself voluntarily and the communication protocol does not lose messages, participants will stay alive and keep sending heartbeats.
R2. If one or more processes stops sending a heartbeat, all processes must be inactivated.
R3. If the communication protocol stops delivering messages, all processes must be inactivated.

We start out discussing a heartbeat protocol for two participants.

10.2 The Binary Heartbeat Protocol

The binary heartbeat protocol is intended for two processes. There are two processes $p[0]$ and $p[1]$ where the process $p[0]$ works as the coordinator. The original specification of $p[0]$ and $p[1]$ are described in Tables 10.1 and 10.2, respectively.

The binary heartbeat protocol is pretty straightforward. If active, the coordinator sends a heartbeat after a waiting time of every *tmax* time units (Table 10.1, lines 10-18). The process $p[1]$ responds by instantly sending a heartbeat back if it is alive (Table 10.2, lines 8-11). If the communication network does not lose the heartbeats in transmission, it guarantees that they are sent back and forth within a roundtrip time of *tmin* time units. If heartbeats get lost in transmission, and the coordinator is to send its next heartbeat, it halves the waiting time to the next heartbeat (Table 10.1, line 13, *rcvd* is false). If the waiting time gets smaller than the roundtrip time *min* (Table 10.1, line 15), it decides to stop the heartbeat protocol. In this case we say the coordinator quits involuntarily. If $p[1]$ does not receive a heartbeat within $3tmax - tmin$ time units it also quits involuntarily (Table 10.2, lines 12-15).

Both processes $p[0]$ and $p[1]$ can decide non deterministically to quit the protocol. This has been described in Table 10.1 at lines 7-9 and in Table 10.2 at lines 5-7.

The constants and variables used in both the coordinator and $p[1]$ are described here.

Table 10.1 The original specification of process $p[0]$ of the binary heartbeat protocol [22]

```
1  process p[0]
2  const tmin, tmax : integer                                  {0 < tmin ≤ tmax}
3  var active : boolean                                        {initially true}
4      rcvd : boolean                                          {initially true}
5      t : 0..tmax                                             {initially tmax}
6  begin
7    active→ if true→ skip
8              □ true→ active:= false
9              fi
10   □ timeout active ∧
11       {a time period of at least t units has passed without
     sending a beat message} →
12       if rcvd → t := tmax
13       □ ¬rcvd → t := t/2
14       fi;
15       if t < tmin → active := false
16       □ t ≥ tmin → send beat to p[1];
17                    rcvd:=false
18       fi
19   □ rcv beat from p[1] → if active → rcvd :=true
20                          □ ¬active → skip
21                          fi
22 end
```

***tmax*:** This constant represents the maximum waiting time for the next heartbeat message.

***tmin*:** This is also a constant that is an upper bound on the roundtrip time to send a heartbeat from $p[0]$ to $p[1]$ and back again. It is determined by the communication network.

active This boolean indicates whether the process is still actively taking part in the heartbeat protocol. Initially, it is true.

rcvd This boolean indicates that after sending out the last heartbeat, a response has been received.

***t*:** This variable is the waiting time to send out the next heartbeat. It is generally set to *tmax* but if no heartbeat is received within t time units, its value is halved.

Modelling the binary heartbeat protocol in mCRL2. It is clear that progression of time plays an important role in heartbeat protocols. mCRL2 fully supports modelling of time [33] but this is not explained in this book. In heartbeat protocols time can be modelled by a simple action *tick* representing the passing of a time unit. All processes must synchronise on the action *tick* as time progresses in all components with the same speed. The individual *tick* actions synchronise to the overall *ticking* action.

Given that we know how to model time, modelling the protocol in mCRL2 is a straightforward but tedious operation. We have four parallel processes, one for $p[0]$, one for $p[1]$ and two for the communication channels sending the heartbeats back

and forth. The whole specification is given in Appendix A.1. We explain the structure of all individual processes here.

The mCRL2 model of process *p*[0]. The mCRL2 model of process $p[0]$ corresponds directly to the pseudo specification in Table 10.1. It looks as follows.

```
1  proc P0(time: Nat, active, rcvd: Bool, t: Nat)=
2        active -> inactivate_p0(vol) . P0(0, false, false, TMAX) +
3        (t<=time && active)
4        -> (rcvd -> sendHB(p0) . P0(0, active, false, TMAX)
5                 <> ((t div 2 >= TMIN)
6                     -> sendHB(p0) . P0(0, active, false, t div 2)
7                     <> inactivate_p0(non_vol) .
8                        P0(0, false, false, TMAX)
9                    )
10          )
11       <> tick . P0(time=if(active,time+1,0)) +
12       from_p1 . P0(time, active, if(active,true,rcvd), TMAX);
```

In this specification, there is one extra parameter, namely *time*. It indicates the time, i.e., the number of *tick* actions since sending the last heartbeat. It is equal to 0 if the process is not active. The parameters *active*, *rcvd* and t are exactly as in the pseudo-code of the heartbeat protocol.

Line 2 shows that the process voluntarily quits the heartbeat protocol.
Line 3 contains a condition representing that the timeout takes place.
Line 4 represents that a heartbeat is sent ($sendHB(p_0)$) with a normal next timeout (*tmax*) when a heartbeat has been received in response to the previously sent heartbeat.
Lines 5-8 indicate what happens if no heartbeat is received as a result of the last heartbeat sent. If the timeout t, when halved, gets smaller than *tmin*, then $p[0]$ inactivates the heartbeat protocol non voluntarily (line 7 and 8). Otherwise, it sends a heartbeat with a halved timeout t (line 6).
Line 11 shows what happens if no timeout has to take place. In that case the clock can *tick*.
Line 12 indicates the receipt of a heartbeat using the action *from_p1*. Note that this message is ignored when the protocol is not active anymore.

The mCRL2 model of process *p*[1]. The mCRL2 specification for the process $p[1]$ is given below. It follows the description in Table 10.2.

```
1  proc P1(time: Nat, active: Bool)=
2         (active) -> inactivate_p1(vol).P1(0, false) +
3         from_p0 .
4           ((active)
5            -> sendHB(p1) . P1(0, active)
6            <> P1()) +
7         (time == 3*TMAX-TMIN && active)
8          -> inactivate_p1(non_vol) . P1(0, false)
9          <> tick . P1(if(active,time+1,time), active);
```

Table 10.2 The original specification of process $p[1]$ of the binary heartbeat protocol [22]

```
1  process p[1]
2  const tmin, tmax : integer                          {0 < tmin ≤ tmax}
3  var active : boolean                                {initially true}
4  begin
5     active→ if true→ skip
6              □ true→ active:= false
7              fi
8  □  rcv beat from p[0] →
9        if active → send beat to p[0]
10       □ ¬active → skip
11       fi
12 □ timeout active ∧
13    {a time period of at least 3tmax − tmin units has
14     passed without sending a beat message} →
15       active:=false
16 end
```

The parameter *time* indicates the number of actions *tick* that have passed since the last heartbeat that has been received. The parameter *active* indicates whether $p[1]$ is still active, and it is the same as the variable *active* in Table 10.2.

Line 2 describes that $p[1]$ inactivates itself voluntarily.
Lines 3-6 indicate what is happening if a heartbeat is received. If active, a heartbeat is sent back and *time* is set to 0. If not active anymore, the heartbeat is ignored.
Lines 7-8 indicate what happens if *time* reaches the timeout values $3 * tmax - tmin$. In that case $p[1]$ inactivates itself non-voluntarily.
Line 9 shows that if there is no time out, a *tick* action can happen modelling progression of time.

The mCRL2 model of the communication channels. There are two communication channels, i.e., from the process $p[0]$ to the process $p[1]$ and vice versa. Both channels can lose or delay a message but a round-trip delay cannot exceed *tmin* time for a heartbeat message [22]. Modelling the channels is not completely standard, because the roundtrip time must be handed over from the channel from $p[0]$ to $p[1]$ to the channel from $p[1]$ to $p[0]$. This is done using the actions *set_roundtriptime*(t) and *get_roundtriptime*(u). For each channel two non parallel mCRL2 processes have been used, one to represent that the channel is empty and the other to represent that a heartbeat is in the channel.

```
1  proc Channel_p0_to_p1_empty =
2         tick . Channel_p0_to_p1_empty +
3         rcvHB(p0) . Channel_p0_to_p1_full(0);
4
5       Channel_p0_to_p1_full(t: Nat) =
6         (t<TMIN) -> tick . Channel_p0_to_p1_full(t+1) +
7         (i . send_to_p1 . set_roundtriptime(t) + i . lose_message) .
8             Channel_p0_to_p1_empty;
9
10      Channel_p1_to_p0_empty(t: Nat) =
```

```
11          tick . Channel_p1_to_p0_empty(t) +
12          rcvHB(p1) . Channel_p1_to_p0_full(t) +
13          sum u: Nat.get_roundtriptime(u) . Channel_p1_to_p0_empty(u);
14
15        Channel_p1_to_p0_full(t: Nat) =
16          (t<TMIN) -> tick . Channel_p1_to_p0_full(t+1) +
17          (i . send_to_p0 + i . lose_message) .
18              Channel_p1_to_p0_empty(0) +
19          sum u: Nat.get_roundtriptime(u) . Channel_p1_to_p0_full(u);
```

The parameter t indicates the used roundtrip time of the current heartbeats that are sent back and forth.

Lines 2, 6, 11 and 16 indicate progression of time. In lines 6 and 16 if the roundtrip time is not below *tmin*, the channels must first deliver the heartbeats to guarantee that the time bound on a roundtrip is met.

Line 3 represents that the channel from $p[0]$ to $p[1]$ receives a heartbeat ($rcvHB(p_0)$).

Line 7 models that the channel from $p[0]$ to $p[1]$ forwards the heartbeat to $p[0]$ and informs the reverse channel of the roundtrip time used to forward the heartbeat via action *set_roundtriptime*(t). Or it indicates that the heartbeat message is lost using action *lose_message*. It continues at line 8 as the empty channel. The action i is used to indicate that delivering or losing the message is non-deterministically decided in the channel.

Line 12 indicates that the reverse channel from $p[1]$ to $p[0]$ receives a heartbeat from $p[1]$.

Lines 13 and 19 are used to obtain the roundtrip time from the forward channel.

Lines 17 and 18 model that the heartbeat is either delivered or lost. This is done non-deterministically using the action i.

The mCRL2 specification of the parallel composition. Below we present the parallel composition modelling the full heartbeat protocol. There are four parallel processes, namely one for $p[0]$, one for $p[1]$ and two channels, one from $p[0]$ to $p[1]$, and the other in the reverse direction.

```
init allow( { rcv_by_p1, rcv_by_p0, ticking,
              inactivate_p0, inactivate_p1,
              lose_message, i, heartbeat,
              pass_roundtriptime
            },
      comm( { send_to_p1 | from_p0 -> rcv_by_p1,
              send_to_p0 | from_p1 -> rcv_by_p0,
              sendHB | rcvHB -> heartbeat,
              tick|tick|tick|tick -> ticking,
              set_roundtriptime | get_roundtriptime ->
                                        pass_roundtriptime
            },
            P0(0, true, true, TMAX) || P1(0, true) ||
            Channel_p0_to_p1_empty || Channel_p1_to_p0_empty(0)
          ));
```

The communication and allow operators are standard, except for the progress of time. By letting four actions *tick* synchronise to the action *ticking* it is modelled that time must progress with the same pace in all four components.

The state space varies depending on the values of *tmin* and *tmax*, but it is pretty small. This allows us to easily verify the behavioural properties as shown in the next section.

Requirement Analysis. We formalise and discuss the three requirements as provided in Sect. 10.1.

R1. No participant inactivates itself non-voluntarily if the communication channels do not lose heartbeats and none of the participants deactivates itself voluntarily. In essence this says that under normal circumstances the heartbeat protocol will keep on functioning properly.
We formulate the requirement separately for process $p[0]$ and for $p[1]$.

$$[\overline{lose_message \cup inactivate_p1(vol)}^{*} \cdot inactivate_p0(non_vol)]false. \qquad (10.1)$$

This expresses that if no heartbeat is lost (*lose_message*) and process $p[1]$ did not activate itself voluntarily (*inactivate_p1*(*vol*)), then process $p[0]$ cannot terminate itself non-voluntarily for which it uses the action *inactivate_p0*(*non_vol*). When checking this formula we find that it holds in general, except when $tmin = tmax$. This is odd as [22] explicitly allows this case. In general, however, to minimise the probability of premature termination of the heartbeat protocol due to message loss, *tmax* should be chosen to be much larger than *tmin* and this may explain why the authors overlooked this situation.
The counter example to formula (10.1) is illustrated in Fig. 10.1. After time *tmax* the process $p[0]$ sends a heartbeat. As the roundtrip takes time *tmin*, begin equal to *tmax*, the process $p[0]$ can time out just before the heartbeat arrives. So,

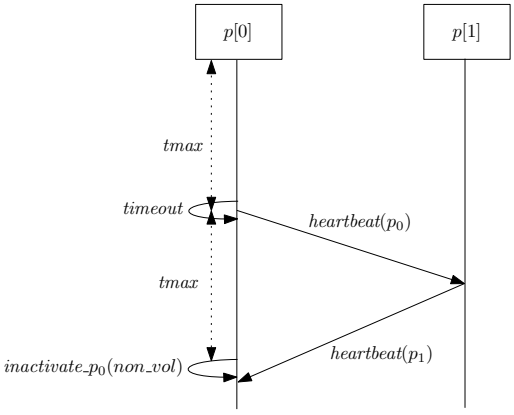


Fig. 10.1 Counter example for requirement R1 for $p[0]$ if $tmax = tmin$

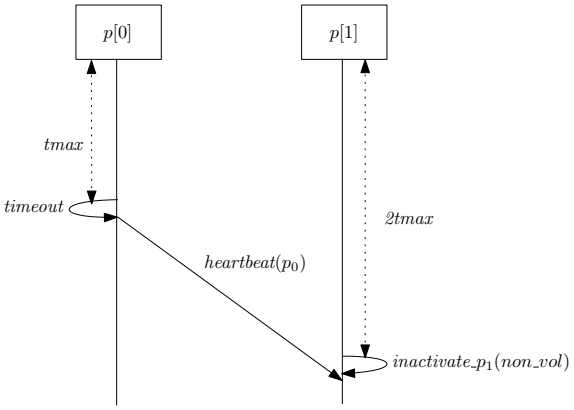


Fig. 10.2 Counter example for requirement R1 for $p[1]$ if $tmax = tmin$

the action *inactivate_p0*(*non_vol*) can happen under perfectly normal circumstances. The problem instantly disappears if $tmax > tmin$.
The formulation of this property for process $p[1]$ is completely symmetric.

$$[\overline{lose_message \cup inactivate_p0(vol)}^{*} \cdot inactivate_p1(non_vol)]false. \qquad (10.2)$$

Also this property only holds if $tmax > tmin$. In case $tmax = tmin$, we also find a counterexample, depicted in Fig. 10.2. The nature of this counter example is different from that of the previous one. Process $p[1]$ starts its timer. After $tmax$ time, process $p[0]$ sends a timeout. This arrives at the latest at time $tmax + tmin = 2 * tmax$. But process $p[1]$ must decide at the same time $2 * tmax = 3 * tmax - tmin$ that it must non-voluntarily inactivate itself, which it can do just before the heartbeat arrives. So, also in this case, although nothing is wrong, process $p[0]$ can go into non-voluntarily deactivation.

R2. The second property says that if one process deactivates itself, the other should follow. We formulate this property separately for $p[0]$ and $p[1]$, but both formulations are completely symmetric.
A first observation is that if a process is already deactivated, it should not deactivate itself again. Although not explicitly formulated above, we can understand that this is implicit in requirement R2. We record the activation status with an observation variable *active* that records whether process $p[0]$ is active. If $p[1]$ inactivates itself, observable through the action *inactivate_p1*(i) and $p[0]$ is active, then *inactivate_p0*(i) must unavoidably happen.

$$\nu X(active : \mathbb{B} = true).$$
$$[\exists i : inactivation_type.inactivate_p0(i)]X(false) \wedge$$
$$[\exists i : inactivation_type.inactivate_p0(i)]X(active) \wedge$$
$$[\exists i : inactivation_type.inactivate_p1(i)]$$
$$(active \Rightarrow (\mu Y.[\exists i : inactivation_type.inactivate_p0(i)]Y \wedge \langle true\rangle true)).$$

The symmetric formula for $p[1]$ is the following:

$$\nu X(active : \mathbb{B} = true).$$
$$[\exists i : inactivation_type.inactivate_p1(i)]X(false) \wedge$$
$$[\exists i : inactivation_type.inactivate_p1(i)]X(active) \wedge$$
$$[\exists i : inactivation_type.inactivate_p0(i)] \wedge$$
$$(active \Rightarrow (\mu Y.[\exists i : inactivation_type.inactivate_p1(i)]Y \wedge \langle true\rangle true)).$$

Both formulas are valid.

R3. The third property says that if the communication protocol stops delivering messages, all processes will be deactivated and stop sending heartbeats. Of course, this only needs to happen, if the processes are not already deactivated.
An alternative formulation of this property is that as long as a process is active either heartbeat messages are delivered, or, unavoidably, the process deactivates itself. This is what the following formula says.

$$([\exists i : inactivation_type.inactivate_p0(i)^*]$$
$$\mu X.[\exists i : inactivation_type.inactivate_p0(i) \cup rcv_by_p0 \cup rcv_by_p1]X \wedge \langle true\rangle true)$$
$$\wedge$$
$$([\exists i : inactivation_type.inactivate_p1(i)^*]$$
$$\mu X.[\exists i : inactivation_type.inactivate_p1(i) \cup rcv_by_p0 \cup rcv_by_p1]X \wedge \langle true\rangle true).$$

This formula holds for the protocol.
We may however contemplate another formulation of this property, which is possible as time keeps ticking in the model using actions *ticking*. We can formulate that if for some time no heartbeats are received then processes must have switched themselves off. Note that this is a maximal fixed point property as it is a property that must always hold.
Concretely, we can formulate this by counting how many actions *ticking* have passed without receiving heartbeats by actions *rcv_by_p0* and *rcv_by_p1* and store these in observation variables n_0 and n_1. We also observe whether processes $p[0]$ and $p[1]$ deactivated themselves in observation variables $active_0$ and $active_1$. In the last two lines of the following formula we state that if enough time has passed without receiving heartbeats, i.e., n_0 and n_1 have sufficiently large values, then the observation variables $active_0$ and $active_1$ should be false, indicating that processes $p[0]$ respectively $p[1]$ are deactivated.

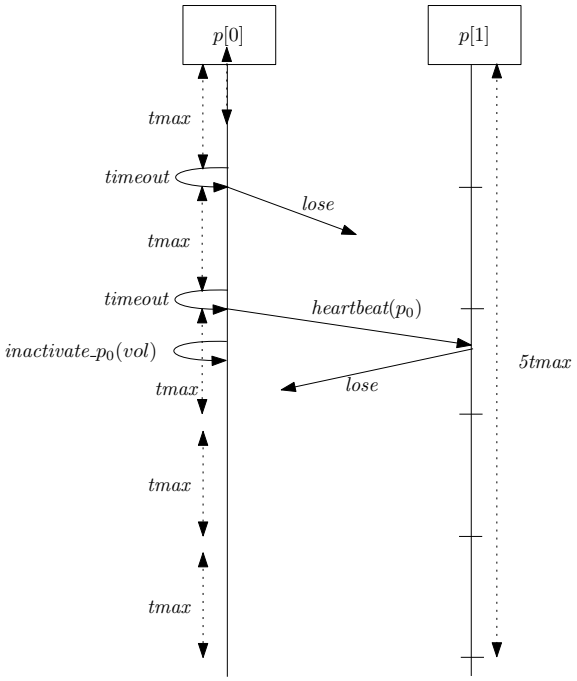


Fig. 10.3 Process $p[1]$ is still active while $p[0]$ is out of heartbeats for $5tmax$ time

$$
\begin{aligned}
&\nu X(n_0 : \mathbb{N} = 0, n_1 : \mathbb{N} = 0, active_0 : \mathbb{B} = true, active_1 : \mathbb{B} = true).\\
&\quad [ticking]X(\min(n_0 + 1, 6 * tmax), \min(n_1 + 1, 6 * tmax), active_0, active_1) \wedge\\
&\quad [rcv_by_p0]X(0, n_1, active_0, active_1) \wedge\\
&\quad [rcv_by_p1]X(n_0, 0, active_0, active_1) \wedge\\
&\quad [\exists i : inactivation_type.inactivate_p0(i)]X(n_0, n_1, false, active_1) \wedge\\
&\quad [\exists i : inactivation_type.inactivate_p1(i)]X(n_0, n_1, active_0, false) \wedge\\
&\quad [\exists i : inactivation_type.(inactivate_p0(i) \cup inactivate_p1(i)) \cup ticking \cup\\
&\qquad \overline{rcv_by_p0 \cup rcv_by_p1}]X(n_0, n_1, active_0, active_1) \wedge\\
&\quad ((n_0 \geq 3 * tmax \vee n_1 \geq 3 * tmax) \Rightarrow \neg active_0) \wedge\\
&\quad ((n_0 \geq 6 * tmax \vee n_1 \geq 3 * tmax) \Rightarrow \neg active_1)
\end{aligned}
\tag{10.3}
$$

This formula is valid, but only for the values for n_0 and n_1 mentioned in the last line. Process $p[0]$ may not receive a heartbeat for up to $3tmax$ time units and not have been switched off. The most extreme is that $p[0]$ can only be sure after $6tmax$ time units without receiving any heartbeat from $p[1]$ that $p[1]$ has deactivated itself. In Fig. 10.3 the situation is depicted where $p[1]$ is active while $p[0]$ did not receive any heartbeat from $p[1]$ for $5tmax$ time units. In the same vein, process $p[0]$ is only guaranteed to switch off after both $p[0]$ and $p[1]$ did not receive any heartbeats for $3tmax$ time units. Weakening these bounds immediately invalidates the formula.

Exercise 10.1 The following text appears in the article describing the heartbeat protocols presented in this chapter [22, p. 3].

> "... if $p[0]$ does not receive any beat message for a period of $2tmax$, then $p[0]$ becomes inactive."

This is clearly not in line with Formula (10.3). Adapt this formula and construct a counterexample showing that this statement is incorrect.

10.3 The Static Heartbeat Protocol

The static heartbeat protocol resembles the binary heartbeat protocol with as difference that there can be $n + 1$ participants $p[0]$ to $p[n]$ [22]. The process $p[0]$ is again the coordinator. The coordinator executes in essence a binary heartbeat protocol with each participant $p[i]$ $(1 \leq i \leq n)$ and again heartbeat messages can be lost in transit. So, the coordinator $p[0]$ exchanges periodic heartbeats with all other participants and it maintains a response time per participant which is halved when no responding heartbeat is received. The coordinator takes the minimum of all the response times. If it is less than *tmin*, which is the maximal roundtrip time for heartbeats to travel back and forth to a participant, then the coordinator becomes inactive non-voluntarily. Otherwise, it takes this minimum as the timeout for the next heartbeat.

Just as in the binary heartbeat protocol, each process can inactivate itself voluntarily at any moment in time. All other participants than the coordinator make themselves inactive non-voluntarily after not having received a heartbeat for $3tmax - tmin$ time units.

The description of the behaviour of the coordinator, as it appears in [22], is provided in Table 10.3. The behaviour of the participants is the same as that of process $p[1]$ in the binary heartbeat protocol, see Table 10.2. The main variables used in these processes are the following.

***tmax*:** This is a constant that represents the initial time for a the interval where heartbeats occur.

***tmin*:** This constant indicates the roundtrip time for a heartbeat to travel to a participant $p[i]$ and back again to the coordinator.

***active*:** This boolean indicates whether a process is still actively participating in the heartbeat protocol.

***rcvd*:** This is an array of booleans in which the coordinator maintains from whom it has received a heartbeat in the current round.

***tm*:** In this array the coordinator maintains the current heartbeat interval time for each participant. Initially, it is set to *tmax* and it is halved whenever no response is received during a round from that participant.

***t*:** This is the timeout period for the next heartbeat to be sent. It is set to the minimum value that occurs in the array *tm*.

***k*:** This is an auxiliary variable used locally in the coordinator to walk through the arrays *tm* and *rcvd*.

Table 10.3 The original specification of the coordinator $p[0]$ of the static heartbeat protocol

```
1  process p[0]
2  const tmin, tmax : integer                                  {0 < tmin ≤ tmax}
3  var active : boolean,                                       {initially true}
4      rcvd : array [1..n] of boolean,                         {initially true}
5      tm : array [1..n] of 0..tmax,                           {initially tmax}
6      t : 0..tmax,                                            {initially tmax}
7      k : 1..n + 1
8  par i : 1..n
9  begin
10   active→ if true→ skip
11           □ true→ active:= false
12   fi
13 □ timeout active ∧
14  {a time period of at least t units has passed
15   without sending a beat message} →
16      k := 1;
17      do k ≤ n → if rcvd[k] → tm[k] := tmax
18                 □ ¬rcvd[k] → tm[k] := tm[k]/2
19                 fi; k := k + 1
20      od; t :=MIN(tm);
21      if t < tmin → active := false
22      □ t ≥ tmin → BCAST
23      fi
24 □ rcv beat from p[i] → if active → rcvd[i] :=true
25                       □ ¬active → skip
26                       fi
27 end
```

The **BCAST** in line 22 in the above algorithm is function for sending $p[0]$'s heartbeat to all the participants. This function in [22] is defined as:

```
1 BCAST:: k := 1;
2         do k ≤ n → send beat to p[k];
3                    rcvd[k], k :=false, k + 1
4         od
```

The mCRL2 model of the static heartbeat protocol. The above algorithm is translated to mCRL2 and partly discussed below. We give the specification of the coordinator and the participants. We do not explain the channels or the parallel composition in this text as it is very much identical to those of the binary heartbeat protocol. The full specification is provided in Appendix A.2.

The full specification consists of a description of the coordinator $p[0]$ and a description of all other processes. We made a model with processes $p[1]$ and $p[2]$, but more can be added at the expense of larger state spaces. For each participant $i > 0$ two channels are added. One for heartbeats from $p[0]$ to $p[i]$ and one for sending

heartbeats back. Using a parameter t the roundtrip time is measured, and it is guaranteed that a message, if not lost, is always sent back and forth within *tmin* actions *tick*.

The coordinator $p[0]$ in mCRL2 The coordinator $p[0]$ is directly derived from the description in Table 10.3.

```
1  proc P0(time: Nat, active: Bool,
2                     rcvd: Set(proc_ID), tm: proc_ID->Nat, t: Nat) =
3         (active) -> inactivate(p0,vol) .
4                     P0(time=0, active=false, t=0) +
5         (active && time>=t)
6         -> ((minimum(rcvd, tm)>=TMIN)
7              -> broadcast(p1) .
8                 broadcast(p2) .
9                 P0(time=0,
10                   rcvd={},
11                   tm=tm[p1 -> if(p1 in rcvd, TMAX, tm(p1) div 2)]
12                        [p2 -> if(p2 in rcvd, TMAX, tm(p2) div 2)],
13                   t=minimum(rcvd, tm)
14                  )
15             <> inactivate(p0,non_vol) . P0(active=false))
16        <> tick . P0(time=if(active, time+1, 0)) +
17        sum id: proc_ID.rcv_by_p0(id) .
18                P0(rcvd=if(active, rcvd+{id}, {}));
```

We maintain the number of ticks after sending out the last heartbeats in the parameter *time*. The parameter *rcvd* is modelled as a set of process identifiers (instead of an array of booleans). A process identifier is an element of this set exactly if its entry in the array would contain true. The array *tm* is represented as a function from process identifiers to timeout values. All other parameters exactly match those in Table 10.3, except for k which we do not need.

In lines 3 and 4 the coordinator can deactivate itself voluntarily. In lines 5 to 15 it is modelled what happens when a timeout occurs. If no timeout occurs, in line 16, time is allowed to progress via a *tick* action.

If a timeout occurs and the adapted minimum of the values in *tm* are larger or equal than *tmin* heartbeats are broadcasted to all participants (lines 6-8). The definition of *minimum* is given in the full specification in Appendix A.2. It first halves the entries of *tm* if no reception of a heartbeat is recorded in the set *rcvd*, and then takes the minimum of these adapted values. The process then continues with *time* set to 0 as heartbeats have just been transmitted, *rcvd* set to empty as no heartbeats have been received in this round yet, *tm* is set to the expected timeouts per participant, and the timeout t is set to the minimum of these. Note that we use the function update notation $tm[p \rightarrow new_value]$ which represents the function *tm* that when applied to process identifier p delivers the new value.

In lines 17 and 18 it is specified what happens if a timeout is received, which is nothing more than adding the process identifier to the set *rcvd*.

The processes $p[i]$ in mCRL2. The participants $p[i]$ are specified in a pretty straightforward way in mCRL2.

```
1  proc P(id: proc_ID, time: Nat, active: Bool) =
2         (active) -> inactivate(id, vol) . P(id, 0, false) +
3         rcv_by_p(id) .
4           ((active)
5            -> respond(id) . P(id, 0, active)
6            <> P(id, 0, active)) +
7         (time == 3*TMAX-TMIN && active)
8         -> inactivate(id, non_vol) . P(id, 0, false)
9         <> tick.P(id, if(active,time+1,time), active);
```

The process has an explicit parameter *id* as the identifier of this process. In line 2 the process can voluntarily deactivate itself. In lines 3 to 6 it is described what happens if a heartbeat is received. If the process is active it instantly sends a heartbeat back and it sets the time *time* since the last heartbeat to 0. Otherwise, the heartbeat is ignored. In lines 7 and 8 it is shown how this participant involuntarily deactivates itself when there is no heartbeat for $3tmax - tmin$ time units. Otherwise, it allows time to pass by allowing the action *tick* to happen.

Requirement analysis. We check the primary requirements for heartbeat protocols. The modal formulas are very similar as for the binary heartbeat protocol, but here we try to formulate them more concisely.

R1. We start out with requirement R1. No process makes itself inactive non-voluntarily if no messages are lost or other process voluntarily inactivates itself. This property can be checked using the following formula.

$$[\overline{\exists q, q' : proc_ID.lose_message(q, q') \cup \exists p : proc_ID.inactivate(p, vol)}^{*} \cdot \exists p' : proc_ID.inactivate(p', non_vol)]false.$$

Note that we use an action *lose_message* with two parameters indicating the direction the heartbeat is sent when the message gets lost. Using the quantifiers we formulate this property for all processes at once. This is concise, but more time consuming to model-check. While model-checking it is always better to first check properties for concrete instances as in that case model-checking takes far less time and it is much easier to identify the nature of a problem if the formula is not true. When the model and the formulas are correct, the generic formulation is useful, as it allows for a more concise presentation.
This formula is only true if $tmax > tmin$ contrary to what is claimed in [22]. The problem is identical as that in the binary heartbeat protocol.

R2. Requirement R2 says that when one process inactivates itself, all processes must ultimately be inactivated. The following modal formula states this property for all process identifiers at once:

$$
\begin{array}{l}
\forall p : \mathit{proc_ID}. \\
\quad \nu X(\mathit{active} : \mathbb{B} = \mathit{true}). \\
\quad\quad [\exists i : \mathit{inactivation_type}.\overline{\mathit{inactivate}(p, i)}]X(\mathit{false}) \wedge \\
\quad\quad [\overline{\exists i : \mathit{inactivation_type}.\mathit{inactivate}(p, i)}]X(\mathit{active}) \wedge \\
\quad\quad [\exists i : \mathit{inactivation_type}, p' : \mathit{proc_ID}.((p \not\approx p') \cap \mathit{inactivate}(p', i))] \\
\quad\quad\quad (\mathit{active} \Rightarrow (\mu Y.[\overline{\exists i : \mathit{inactivation_type}.\mathit{inactivate}(p, i)}]Y \wedge \langle \mathit{true} \rangle \mathit{true})).
\end{array}
$$

The observation variable *active* represents that the process with process identifier p is active. The first and second box modalities in the maximal fixed point are used to keep track of the observation variable *active*.
In the third box modality the action formula $(p \not\approx p') \cap \mathit{inactivate}(p', i)$ is used. If $p \approx p'$ is true, $p \not\approx p'$ is false, which represents the empty set of actions. So, if $p \approx p'$, then the set of actions in the box modality is empty and nothing needs to be checked. Only if p and p' are different the minimal fixed point must be verified. This expresses that the action $\mathit{inactivate}(p', i)$ takes place and p' is different from p and if p is not already deactivated, then process p must be deactivated within a finite number of steps.
This formula is valid for the static heartbeat protocol.

R3. The third requirement says that if no messages are transferred anymore, then all processes must deactivate themselves. We formulate this as a safety formula, saying that if a sufficient number of actions *ticking* have passed, without heartbeat messages being delivered then all processes must have deactivated itself. These waiting times must be taken more liberally than suggested in [22].

$$
\begin{array}{l}
\nu X(\mathit{waiting} : \mathit{proc_ID} \to \mathbb{N} = \lambda \mathit{id} : \mathit{proc_ID}.0, \mathit{inactive} : \mathit{Set}(\mathit{proc_ID}) = \emptyset). \\
\quad [\mathit{ticking}]X(\mathit{waiting}[p_0 \to \min(\mathit{waiting}(p_0) + 1, 6\mathit{tmax})] \\
\quad\quad\quad\quad [p_1 \to \min(\mathit{waiting}(p_1) + 1, 6\mathit{tmax})] \\
\quad\quad\quad\quad [p_2 \to \min(\mathit{waiting}(p_2) + 1, 6\mathit{tmax})], \mathit{inactive}) \wedge \\
\quad (\forall \mathit{id} : \mathit{proc_ID}.[\mathit{rcv_heartbeat}(\mathit{id})]X(\mathit{waiting}[\mathit{id} \to 0], \mathit{inactive})) \wedge \\
\quad (\forall \mathit{id} : \mathit{proc_ID}.[\mathit{rcv_response}(\mathit{id})]X(\mathit{waiting}[p_0 \to 0], \mathit{inactive})) \wedge \\
\quad (\forall \mathit{id} : \mathit{proc_ID}.[\exists i : \mathit{inactivation_type}.\mathit{inactivate}(\mathit{id}, i)]X(\mathit{waiting}, \mathit{inactive} \cup \{\mathit{id}\})) \wedge \\
\quad [\overline{\exists \mathit{id} : \mathit{proc_ID}, i : \mathit{inactivation_type}.\mathit{inactivate}(\mathit{id}, i) \cup \mathit{ticking} \cup} \\
\quad\quad \overline{\mathit{rcv_heartbeat}(\mathit{id}) \cup \mathit{rcv_response}(\mathit{id})}]X(\mathit{waiting}, \mathit{inactive}) \wedge \\
\quad (\forall \mathit{id} : \mathit{proc_ID}.(\exists \mathit{id}' : \mathit{proc_ID}. \\
\quad\quad\quad \mathit{waiting}(\mathit{id}') \geq \mathit{if}(\mathit{id} \approx \mathit{id}', 3, \mathit{if}(\mathit{id} \not\approx p_0, 4, 6)) * \mathit{tmax}) \Rightarrow \mathit{id} \in \mathit{inactive}).
\end{array}
$$

In this formula we use a variable *waiting* that is a function from a process identifier p to the time process p waited without receiving a heartbeat. Initially, all values are set to 0, for which we use a lambda to denote the function that maps all process identifiers to 0. We use a set *inactive* of process identifiers to record which processes became inactive. When the action *ticking* happens, the values of all waiting processes is increased. The value is constraint to $6\mathit{tmax}$ to limit the number of BES variables being generated when calculating whether this formula holds.
When an action $\mathit{heartbeat}(\mathit{id})$ happens, one of the processes $p[1]$ or $p[2]$ receives a heartbeat, and the value in *waiting* is set to 0, accordingly. The same holds for the action $\mathit{response}(\mathit{id})$ which indicates that process $p[0]$ receives a heartbeat from process *id*.

The condition at the end says that if process id' did not receive a heartbeat for a sufficiently long time, then process id made itself inactive. The formula is somewhat complex and can better be understood with the next table.

	$p[0]$	$p[1]$	$p[2]$	id
$p[0]$	$3tmax$	$6tmax$	$6tmax$	
$p[1]$	$4tmax$	$3tmax$	$6tmax$	
$p[2]$	$4tmax$	$6tmax$	$3tmax$	
id'				

For example, only if $p[0]$ did not receive heartbeats for $6tmax$ time units, it is guaranteed that $p[1]$ and $p[2]$ deactivated themselves.

Especially noteworthy is that $p[2]$ (or symmetrically $p[1]$) needs to have no heartbeat for $4tmax$ time units for the coordinator $p[0]$ to be guaranteed to notice this. This is $3tmax$ for the binary heartbeat protocol. This suggests that the interplay between the two participants $p[1]$ and $p[2]$ adds new behaviour, although they do not directly communicate with each other. And indeed this is the case, as in the static protocol heartbeats sent in one round can be received in the next round, where it is interpreted as the responding heartbeat in the current round.

The concrete situation is depicted in Fig. 10.4. We set $tmax$ to 2 and $tmin$ to 1. Initially, process $p[0]$ waits for two time units to send out heartbeats to $p[1]$ and $p[2]$, and sets its timeout t to 2. One of the responses is lost. In Fig. 10.4 there is no response from $p[1]$.

Hence, after two more time units $p[0]$ sends out two heartbeats and sets the timeout t to 1 as $tm(p_1) = 1$ and $tm(p_2) = 2$. The response of $p[1]$ arrives in time, but the response from $p[2]$ arrives just after the timeout takes place. So, now $tm(p_1) = 2$ and $tm(p_2) = 1$. So, the timeout t is again set to 1. Note that such a scenario is not possible in the binary heartbeat protocol. The coordinator $p[0]$ sends out two heartbeats. Both processes $p[1]$ and $p[2]$ respond, but again the response from $p[2]$ is received by $p[0]$ after sending out the next round of heartbeats. At this moment $p[2]$ inactivates itself.

The coordinator is now tricked in believing that all is well. It properly receives the response of $p[1]$ and it takes the response of $p[2]$ from the previous round as the response of $p[2]$. So, it sets the timeout t to 2 and broadcasts two heartbeats. It receives a proper response from $p[1]$ and it again takes the response from $p[2]$ from the previous round as the response from $p[2]$, although $p[2]$ is inactive already for one time unit. So, the coordinator $p[0]$ again sets the timeout t to 2 and waits for responses. Now, as no response arrives from $p[2]$ it halves the timeout for the next heartbeat and waits 1 more time unit, before concluding that $p[2]$ is inactive.

At this point $p[2]$ is already inactive for $3tmax = 6$ time units, and $p[0]$ did not make itself inactive yet. This can be clearly seen in Fig. 10.4 where at the right $p[2]$ is not participating in the protocol for 6 time units, while $p[0]$ is still carrying out the protocol.

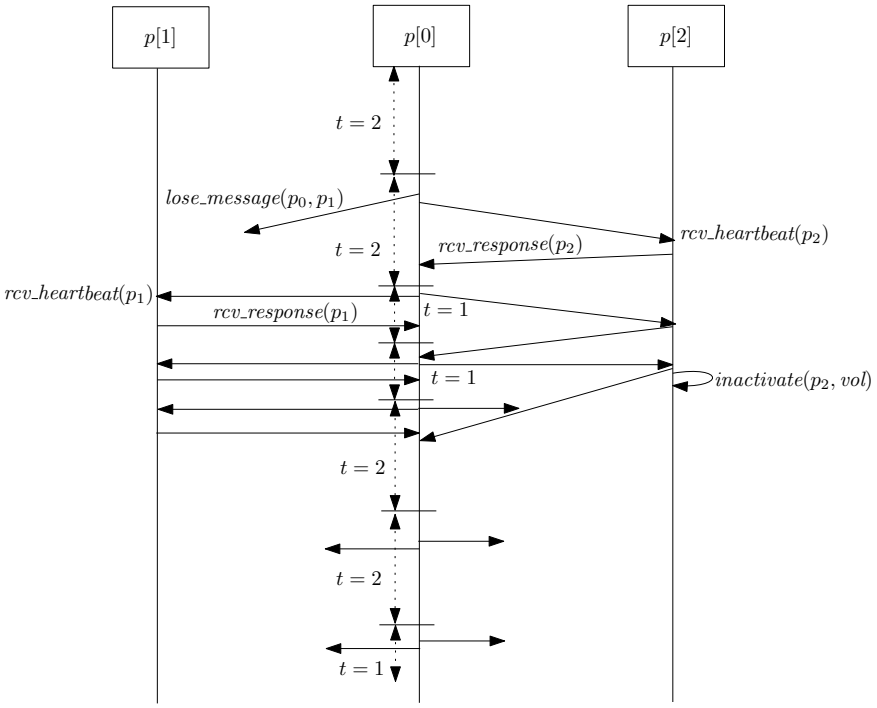


Fig. 10.4 Example showing that heartbeat messages can pass on to the next round

Exercise 10.2 The model given above is not realistic when time can stop. Concretely, this means that no actions *tick* would be possible at some time. Write a formula to check that the action *ticking* is always eventually possible.

10.4 The Expanding and the Dynamic Heartbeat Protocol

The heartbeat protocol can be extended by allowing participants to join. This is called the *expanding heartbeat protocol*. Participants can additionally be allowed to leave, in which case the protocol is called the *dynamic heartbeat protocol*. Both protocols are sketched in [22].

In the expanding heartbeat protocol, new participants can join the protocol anytime. The protocol can not be left. It is only possible for a process to inactivate itself and stop sending heartbeats, with the effect that the coordinator along with the other participants become inactive as well. The protocol starts with the process $p[0]$ only and later on any number of processes may join by sending their heartbeat messages.

If a process $p[i]$ wants to join with $1 \leq i \leq n$, it sends heartbeat messages every *tmin* time to the coordinator until a heartbeat from the coordinator is received. From

that point on the process continues with the static heartbeat protocol as explained in the previous section.

In the dynamic heartbeat protocol, processes can join as well as leave safely at anytime. Leaving safely means that the other participants will continue sending heartbeats.

Joining is as described above. For leaving, the heartbeats are extended with a boolean. If the heartbeat carries *true* it is a normal heartbeat. When a process $p[i]$ wants to leave it sends a heartbeat with a boolean *false*. It waits until the coordinator $p[0]$ responds with the boolean *false*. Then it can safely decide to abandon the heartbeat protocol.

Exercise ** Model and analyse both the expanding and dynamic heartbeat protocols as described in [22].

Appendix A
Specifications of Protocols in mCRL2

This appendix contains the full mCRL2 specifications of the binary and the static heartbeat protocol from Chap. 10.

A.1 Binary Heartbeat Protocol

```
sort proc_ID  = struct p0 | p1;
     inactivation_type = struct vol | non_vol;

map TMAX: Nat;
    TMIN: Nat;

eqn TMAX = 3;
    TMIN = 1;

act sendHB, rcvHB, heartbeat: proc_ID;
    send_to_p1, from_p0, rcv_by_p1;
    send_to_p0, from_p1, rcv_by_p0;
    set_roundtriptime, get_roundtriptime, pass_roundtriptime: Nat;
    inactivate_p0, inactivate_p1: inactivation_type;
    tick, ticking, lose_message, i;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
                  Processes for p[0]
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%


proc P0(time: Nat, active, rcvd: Bool, t: Nat)=
      active -> inactivate_p0(vol) . P0(0, false, false, TMAX) +
      (t<=time && active)
      -> (rcvd -> sendHB(p0) . P0(0, active, false, TMAX)
               <> ((t div 2 >= TMIN)
                   -> sendHB(p0) . P0(0, active, false, t div 2)
                   <> inactivate_p0(non_vol) .
```

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0

```
                        P0(0, false, false, TMAX)
                    )
          )
       <> tick . P0(time=if(active,time+1,0)) +
       from_p1 . P0(time, active, if(active,true,rcvd), TMAX);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%                 Processes for P[1]
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

proc P1(time: Nat, active:Bool)=
       (active) -> inactivate_p1(vol).P1(0, false) +
       from_p0 .
         ((active)
          -> sendHB(p1) . P1(0, active)
          <> P1()) +
       (time == 3*TMAX-TMIN)
        -> inactivate_p1(non_vol) . P1(0, false)
        <> tick . P1(if(active,time+1,time), active);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%                 Processes for Communication Channels
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

proc Channel_p0_to_p1_empty =
       tick . Channel_p0_to_p1_empty +
       rcvHB(p0) . Channel_p0_to_p1_full(0);

     Channel_p0_to_p1_full(t: Nat) =
       (t<TMIN) -> tick . Channel_p0_to_p1_full(t+1) +
       (i . send_to_p1 . set_roundtriptime(t) + i . lose_message) .
           Channel_p0_to_p1_empty;

     Channel_p1_to_p0_empty(t: Nat) =
       tick . Channel_p1_to_p0_empty(t) +
       rcvHB(p1) . Channel_p1_to_p0_full(t) +
       sum u: Nat.get_roundtriptime(u) . Channel_p1_to_p0_empty(u);

     Channel_p1_to_p0_full(t: Nat) =
       (t<TMIN) -> tick . Channel_p1_to_p0_full(t+1) +
       (i . send_to_p0 + i . lose_message) .
           Channel_p1_to_p0_empty(0) +
       sum u: Nat.get_roundtriptime(u) . Channel_p1_to_p0_full(u);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%                 Process for Heartbeat Protocol
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

init allow( { rcv_by_p1, rcv_by_p0, ticking,
              inactivate_p0, inactivate_p1,
              lose_message, i, heartbeat,
              pass_roundtriptime
            },
      comm( { send_to_p1 | from_p0 -> rcv_by_p1,
              send_to_p0 | from_p1 -> rcv_by_p0,
              sendHB | rcvHB -> heartbeat,
```

```
            tick|tick|tick|tick -> ticking,
            set_roundtriptime | get_roundtriptime ->
                                        pass_roundtriptime
          },
          P0(0, true, true, TMAX) || P1(0, true) ||
          Channel_p0_to_p1_empty || Channel_p1_to_p0_empty(0)
        ));
```

A.2 Static Heartbeat Protocol

```
sort proc_ID  = struct p0 | p1 | p2;   % Add as needed.
     inactivation_type = struct vol | non_vol;

map TMAX: Nat;
    TMIN: Nat;
    minimum: Set(proc_ID)#(proc_ID->Nat)-> Nat;
    function_tmax: proc_ID->Nat;

var n, m: Nat;
    id: proc_ID;
    rcvd: Set(proc_ID);
    tm: proc_ID->Nat;

eqn TMAX=3;
    TMIN=1;
    minimum(rcvd, tm)=min(if(p1 in rcvd, TMAX, tm(p1) div 2),
                          if(p2 in rcvd, TMAX, tm(p2) div 2));
    function_tmax(id) = TMAX;

act broadcast, rcv_from_p0, broadcasting,
    send_to_p, rcv_by_p, rcv_heartbeat,
    respond, rcv_from_p, responding,
    send_to_p0, rcv_by_p0, rcv_response: proc_ID;
    lose_message: proc_ID#proc_ID;
    set_roundtriptime, get_roundtriptime,
        pass_roundtriptime: proc_ID#Nat;
    inactivate: proc_ID#inactivation_type;
    i, tick, ticking;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%                  Process P[0]
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

proc P0(time: Nat, active: Bool,
                   rcvd: Set(proc_ID), tm: proc_ID->Nat, t: Nat) =
       (active) -> inactivate(p0,vol) .
                   P0(time=0, active=false, t=0) +
       (active && time>=t)
       -> ((minimum(rcvd, tm)>=TMIN)
            -> broadcast(p1) .
               broadcast(p2) .
```

```
                P0(time=0,
                   rcvd={},
                   tm=tm[p1 -> if(p1 in rcvd, TMAX, tm(p1) div 2)]
                        [p2 -> if(p2 in rcvd, TMAX, tm(p2) div 2)],
                   t=minimum(rcvd, tm)
                  )
             <> inactivate(p0,non_vol) . P0(active=false))
       <> tick . P0(time=if(active, time+1, 0)) +
       sum id: proc_ID.rcv_by_p0(id) .
               P0(rcvd=if(active, rcvd+{id}, {}));

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%                     Process for P[1..n]
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

proc P(id: proc_ID, time: Nat, active: Bool) =
       (active) -> inactivate(id, vol) . P(id, 0, false) +
       rcv_by_p(id) .
         ((active)
          -> respond(id) . P(id, 0, active)
          <> P(id, 0, active)) +
       (time == 3*TMAX-TMIN && active)
       -> inactivate(id, non_vol) . P(id, 0, false)
       <> tick.P(id, if(active,time+1,time), active);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Asynchronous communication channels with a round trip delay
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

proc Channel_p0_to_p_empty(id: proc_ID) =
       tick . Channel_p0_to_p_empty(id) +
       rcv_from_p0(id) . Channel_p0_to_p_full(id, 0);

     Channel_p0_to_p_full(id: proc_ID, t: Nat) =
       (t<TMIN) -> tick . Channel_p0_to_p_full(id, t+1) +
       (i . set_roundtriptime(id, t) . send_to_p(id) +
        i . lose_message(p0, id)
       ) . Channel_p0_to_p_empty(id);

     Channel_p_to_p0_empty(id: proc_ID, t: Nat) =
       tick . Channel_p_to_p0_empty(id, t) +
       rcv_from_p(id) . Channel_p_to_p0_full(id, t) +
       sum u: Nat.get_roundtriptime(id, u) .
                  Channel_p_to_p0_empty(id, u);

     Channel_p_to_p0_full(id: proc_ID, t: Nat) =
       (t<TMIN) -> tick . Channel_p_to_p0_full(id, t+1) +
       (i . send_to_p0(id) +
        i . lose_message(id, p0)) .
           Channel_p_to_p0_empty(id, 0) +
       sum u: Nat.get_roundtriptime(id, u) .
                  Channel_p_to_p0_full(id, u);
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% The parallel composition of all processes
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

init allow( { ticking, i, inactivate,
              broadcasting, rcv_heartbeat,
              responding, rcv_response,
              lose_message,
              pass_roundtriptime },
      comm( { tick|tick|tick|tick|tick|tick|tick -> ticking,
              broadcast|rcv_from_p0 -> broadcasting,
              send_to_p|rcv_by_p -> rcv_heartbeat,
              respond|rcv_from_p -> responding,
              send_to_p0|rcv_by_p0 -> rcv_response,
              set_roundtriptime | get_roundtriptime
                                        -> pass_roundtriptime },
        P0(0, true, {p1, p2}, function_tmax, TMAX) ||
        P(p1, 0, true) || P(p2, 0, true) ||
        Channel_p0_to_p_empty(p1) || Channel_p_to_p0_empty(p1, 0) ||
        Channel_p0_to_p_empty(p2) || Channel_p_to_p0_empty(p2, 0)));
```

Appendix B
Solutions

Chapter 2

2.1

Yes, Process-A can overwrite the job of Process-B in the following sequence of actions.

1. Process-B reads the spooler directory and finds the slot 7 free.
2. Process-A also reads the spooler directory and finds the slot 7 free.
3. Process-B adds its job name in the slot 7.
4. Process-A also adds its job in the same slot, i.e., it overwrites the job of Process-B.

2.2

Suppose P_0 reads the lock which is initially *false* and just before changing the lock to *true*, the process P_1 is scheduled, runs, and sets the lock to *true*. Process P_0 will also set the lock to *true* when the first process is resumed. In this way two both processes P_0 and P_1 will be in their critical sections, simultaneously. It is almost the same situation given in Exercise 2.1.

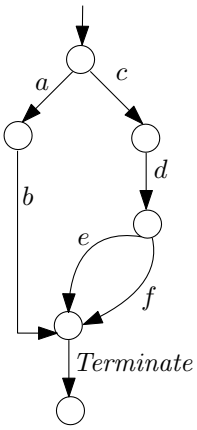
2.3

```
b.(c+d)+a.(c+d)
```

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0

2.4



2.5
For Fig. 2.11a

```
act switchOn, switchOff;

proc P1=switchOn.P1+switchOn.P2;
     P2=switchOff.P2+switchOff.P1;

init P1;
```

For Fig. 2.11b

```
act set, alarm, reset;

proc P1=set.P2;
     P2=alarm.P2+(alarm+reset).P1;

init P1;
```

Chapter 3

3.1
The *comm* operator cannot be applied to every multi-action in Fig. 2.7 as the actions overlap. However, we can rename both `a|c->m` and `b|d->p` in one communication operator in the specification given above Exercise 3.1. By adding a second communication operator, `a|d` and `b|c` can also be renamed.

3.2

The Machine process can be modified as:

```
Machine = acceptCoin.(switchOff+serveCoffee).Machine
          +
          switchOff;
```

3.3

The behaviour will remain unchanged, as the owner was already only able to switch the coffee machine off once.

3.4

```
act dial, getCall, callPlaced, pressKey, getKey, keyPressed, sendInfo;
    getInfo, infoExchange, disconnect, changeMind;

proc Telephone = getCall . getKey . sendInfo . disconnect . Telephone;
     User = dial . (pressKey . getInfo + changeMind) .
            disconnect . User;

init allow( { callPlaced, keyPressed, infoExchange,
              disconnect, changeMind },
   comm({ dial|getCall -> callPlaced,
          pressKey|getKey -> keyPressed,
          sendInfo|getInfo -> infoExchange
        },
         Telephone||User
       )
     );
```

Trace to deadlock: *callPlaced · changeMind · disconnect*

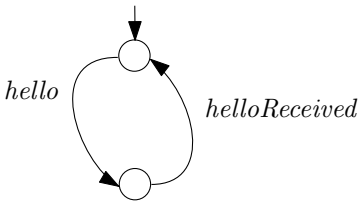
3.5

```
act sendHello, rcvMessage, hello;
    sendMessage, rcvHello, helloReceived;

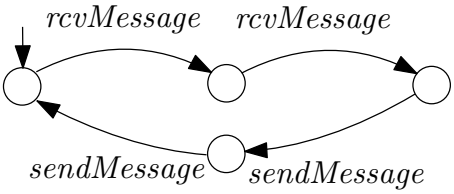
proc Sender   = sendHello . Sender;
     Channel  = rcvMessage . sendMessage . Channel;
     Receiver = rcvHello . Receiver;

init allow( { hello, helloReceived },
       comm( { sendHello | rcvMessage -> hello,
               sendMessage | rcvHello  -> helloReceived
             },
         Sender||Channel||Receiver
           )
          );
```

The LTS of the above specification is:



3.6



Chapter 4

4.1
The set of traces is $\{b^n a^m \mid m, n \geq 0\}$ where a^n stands for n consecutive actions a. The process is

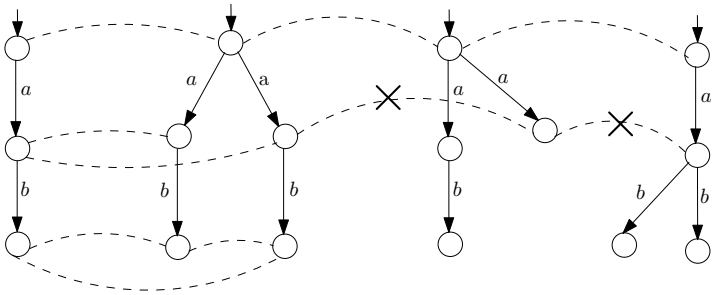
$$P = a \cdot \delta + b \cdot P + a \cdot P_1 + b \cdot P_1; \quad P_1 = a \cdot P_1.$$

4.2
The upper pair of figures is trace equivalent while the other is not due to occurring of two consecutive *logout* actions in the right figure whereas the left one does not contain such a trace.

4.3
The transition systems belonging to the processes $a \cdot Q + a \cdot \delta$ and $a \cdot Q$ with Q defined by $Q = b \cdot Q$ are trace equivalent. However, the first one has a deadlock, and the second has not.

4.4
It is an LTS for $b \cdot a \cdot \delta$.

4.5
For Fig. 4.1, all figures are strong bisimilar except the third one from right as shown in the following figure.



In Fig. 4.5, the upper pair of LTSs are strongly bisimilar. The lower one is not because of the two *logout* actions in a row in the right figure, which are not possible in its counterpart.

4.6
Yes. Both systems are strongly bisimilar.

4.7
LTSs are branching bisimilar.

Chapter 5

5.1

```
act send, receive, sent: Real;
    reply, rcv'reply, received: Real;

proc Sender = send(1/8) . sum x: Real.rcv'reply(x);
     Receiver = sum x: Real.receive(x) . reply(x*2) . Receiver;

init allow( { sent, received },
       comm( { send  | receive   -> sent,
               reply | rcv'reply -> received
             },
                 Sender||Receiver
          ));
```

5.2
The reader is advised to experiment with all the versions of the process P_1 and observe the respective LTSs. The first LTS is the same as in Fig. 5.1, the second has one state and the third has two states with a value(*false*) self-loop.

5.3
The resulting LTS has the following trace from the initial state.

```
transfer1(20) . transfer2(30) . showValues(20, 10) . Terminate.
```

Note that the state space generation tools may have difficulties with this example, as it may not be able to determine the value of variable y.

5.4

Yes, and workers 4, 5 and 6 are not even required to do so. If worker 3 becomes lazy the workers cannot accomplish the task. The last action that is performed is *done*(6) after which a deadlock occurs.

5.5

```
act  send, rcv,  receive: Nat;
     rcvOdd, rcvEven, sendOdd, sendEven, even, odd;

proc P = sum n: Nat.rcv(n).((n mod 2==0) -> sendEven
                                         <> sendOdd) . P;
     P1(i: Nat) = (i<10) -> send(i) . P1(i+2)
                  +
                  rcvOdd . P1();
     P2(i: Nat) = (i<10) -> send(i) . P2(i+2)
                  +
                  rcvEven . P2();

init  allow({ receive, even, odd },
       comm({ send     | rcv     -> receive,
              sendOdd  | rcvOdd  -> odd,
              sendEven | rcvEven -> even
            },
          P||P1(1)||P2(0)
          ));
```

5.6

```
act insCoin, acceptCoin, coin: Nat;
     getCoffee, serveCoffee, coffee;
     cancel, cancelOrder;
     returnMoney, getMoneyBack, moneyBack: Nat;

proc Customer = ( insCoin(5) + insCoin(10) + insCoin(20) + insCoin(50)
                + getCoffee.sum n: Nat.getMoneyBack(n)
                ) . Customer
                +
                cancel . sum n: Nat.getMoneyBack(n) . Customer;

     Machine(total: Nat, coinValues: List(Nat)) =
                (total<35)
                -> (sum x: Nat.(x<6) -> acceptCoin(coinValues.x) .
                           Machine(total+coinValues.x, coinValues))
                <>
                   serveCoffee . returnMoney(Int2Nat(total-35)) .
                           Machine(0, coinValues)
                +
                (total>0)-> cancel . returnMoney(total) .
                Machine(total=0);

init allow({ coin, coffee, cancelOrder, moneyBack },
       comm({ insCoin     | acceptCoin   -> coin,
              getCoffee   | serveCoffee  -> coffee,
              cancel      | cancel       -> cancelOrder,
```

```
            returnMoney | getMoneyBack -> moneyBack },
          Customer || Machine (0, [1, 2, 5, 10, 20, 50])));
```

5.7

```
act sendList, rcvList, transferList: List(Nat);

proc P1 = sendList([10]) . P1;
     P2 = sendList([20]) . P2;
     P3 = sendList([30]) . P3;

     P(ls: List(List(Nat))) =
        sum lst: List(Nat).!(lst in ls) -> rcvList(lst) . P(lst|>ls) +
        (#ls==3) -> tau . P([]);

init allow({ transferList },
       comm({ sendList | rcvList -> transferList },
              P([]) || P1 || P2 || P3 ));
```

5.8

```
{x:Int|x>-10 && x<10 && x mod 2==1}
```

5.9

An arbitrary set can have an infinite size, for instance for the set with all natural numbers $\{n : \mathbb{N} \mid \mathit{true}\}$. But we do not have a data type containing all natural numbers and infinity. So, we do not know the type of $\#S$ with S an arbitrary set. Using cardinality theory we may probably be able to overcome this, but that makes the language mCRL2 unnecessarily complex.

5.10

```
P({0:1}) || Px(0) || Py(0);
```

5.11

```
proc P(bg2:Bag(Pos)) =
        sum i: Pos, j:Pos. (j in bg2 && i mod j==0 )
        -> rcvInt (i) . P({i:1}+bg2);
```

5.12

```
map maxList: List(Nat)->Nat;

var n, m: Nat;
    lst: List(Nat);
eqn maxList([]) = 0
    maxList([n]) = n;    %for one element
    maxList(n|>m|>lst) = if(n>maxList(m|>lst), n, maxList(m|>lst));
```

Note that as *maxList* is calculated twice in the right hand side, this is not a very efficient definition.

5.13

```
map isItMax: List(Nat)#Nat->Bool;
    maxList: List(Nat)->Nat;

var n: Nat;
    lst: List(Nat);
eqn maxList([]) = 0;
    maxList(n|>lst) = if(n>maxList(lst), n, maxList(lst));
    isItMax(lst, n) = maxList(lst)==n;
```

Note that the definition of *maxList* is not very efficient in this way. If efficiency is important, for instance when a large LTS must be generated, the double occurrence of *maxList* in the right hand side must be avoided. But if performance is not an issue, clarity of the specification is preferable.

5.14

```
map sumListUptoN: List(Nat)#Nat->Nat;

var m, n: Nat;
    lst: List(Nat);
eqn sumListUptoN([], n) = 0;
    sumListUptoN(m|>lst, n) = if(n==0,
                                 0,
                                 m + sumListUptoN(lst, Int2Nat(n-1))
                               );
```

5.15

```
map maxListUptoN: List(Nat)#Nat->Nat;

var m, m', n: Nat;
    lst: List(Nat);
eqn maxListUptoN([], n) = 0;
    maxListUptoN([m], n) = if(n==0, 0, m);
    maxListUptoN(m|>m'|>lst, n) =
                  if(n==0,
                     0,
                     if(n==1,
                        m,
                        max(m, maxListUptoN(m'|>lst, Int2Nat(n-1)))
                     )  );
```

5.16

```
map lastIndexOf: List(Nat)#Nat->Int;
    lastIndexOf: List(Nat)#Nat#Nat->Int;

var n, m, i: Nat;
```

```
    lst: List(Nat);

eqn lastIndexOf(lst, m) = lastIndexOf(lst, m, #lst);
    lastIndexOf([], m, i) = -1;
    lastIndexOf(n|>lst, m, i) =
             if(m==rhead(n|>lst),
                i-1,
                lastIndexOf(rtail(n|>lst), m, max(0, i-1)));
```

5.17

```
map remove: List(Nat) # Nat -> List(Nat);

var x, y: Nat;
    lst: List(Nat);
eqn remove([], x) = x|>[];
     x == y -> remove(x |> lst, y) = lst;
     x != y -> remove(x |> lst, y) = x |> remove(lst, y);
```

5.18

```
map removeNthElement: List(Nat)#Nat->List(Nat);

var lst: List(Nat);
    n, m: Nat;

eqn removeNthElement([], n) = [];
    removeNthElement(m|>lst, n) =
             if(n==0, lst, m|>(removeNthElement(lst, Int2Nat(n-1))));
```

5.19

```
map insertList: List(Nat)#List(List(Nat))#Nat->List(List(Nat));

var lst, ls: List(Nat);
    llst: List(List(Nat));
    n: Nat;
eqn insertList(lst, [],n) = [lst];
    insertList(lst, ls|>llst, n) =
                  if(n==0, lst|>ls|>llst,
                           ls|>insertList(lst, llst, Int2Nat(n-1)));
```

5.20

In the following specification, `replaceIn2D` is the required function and it uses an auxiliary function `replaceIn1D` to replace element j in a one-dimensional list.

```
map replaceIn2D: List(List(Nat))#Nat#Nat#Nat->List(List(Nat));
    replaceIn1D: List(Nat)#Nat#Nat->List(Nat);

var llst: List(List(Nat));
    i, j, n, e: Nat;
    ls: List(Nat);
```

```
eqn replaceIn2D ([], i, j, n) = [];
    replaceIn2D(ls|>llst, i, j, n) =
             if(i==0, replaceIn1D(ls, j, n)|>llst,
                      ls|>replaceIn2D(llst, Int2Nat(i-1), j, n));
    replaceIn1D([], j, n) = [];
    replaceIn1D(e|>ls, j, n) =
             if(j==0, n|>ls,
                      e|>replaceIn1D(ls, Int2Nat(j-1), n));
```

5.21

(a) 52
(b) 16180

5.22

```
map f: Nat->Nat;
    g: Nat->Nat;

eqn f = lambda x: Nat.2*x+3;
    g = lambda x: Nat.3*x+2;

act a: Nat;

proc P = a(f(g(2))) . a(g(f(2))) . delta;

init P;
```

$f(g(2)) = 19$ and $g(f(2)) = 23$.

5.23

```
sort weekDays = struct Monday | Tuesday | Wednesday | Thursday |
                       Friday | Saturday | Sunday;
map nextDay: weekDays->weekDays;
    nextNDay: weekDays#Nat->weekDays;
var d: weekDays;
    n: Nat;

eqn nextDay(Monday) = Tuesday;
    nextDay(Tuesday) = Wednesday;
    nextDay(Wednesday) = Thursday;
    nextDay(Thursday) = Friday;
    nextDay(Friday) = Saturday;
    nextDay(Saturday) = Sunday;
    nextDay(Sunday) = Monday;
    nextNDay(d, 0) = d;
    nextNDay(d, 1) = nextDay(d);
    (n>1) -> nextNDay(d,n) = nextNDay(nextDay(d), Int2Nat(n-1));
```

5.24

Except trace number 2, all are possible.

5.25

```
%%%%%%%%%% Actions by a Machine %%%%%%%%%%%%%%%
act receiveRequest: Nat#Nat;
    showPrice: Nat;
    showCashOption; % Machine to give option to pay by cash
    showCardOption; % Machine to give option to pay by card
    receiveMoney: Nat;
    receiveInvalid;
    receiveInvalidCoin;
    cardInserted;
    chargeSuccess;
    chargeFailed; % Machine fails to charge the card
    printReceipt; %After charging money this action will be taken
                  %by the Machine
    sendProduct: Nat#Nat; % Machine send the selected product with
                          %selected quantity
    sendChange: Nat;% Machine return back the change
    welcome; % The machine shows welcome message when there is no user.


%%%%%%%%%% Actions by a USER %%%%%%%%%%%%%%%
    sendRequest: Nat#Nat; %Selected product with required quantity is
                          %given by this action
    selectCash; %When a user wants to pay in cash
    selectCard; %When a user wants to pay through card
    insertMoney: Nat; % User can insert valid coins in machine
    insertInvalid; % User inserts invalid coin
    insertInvalidCoin;
    insertCard; % To insert a bank card in machine
    receiveProduct: Nat#Nat; % User receives the selected product with
                             % selected quantity
    getReceipt; % Receive Receipt from Machine
    receiveChange: Nat;

% results of communications are here
    requestSent: Nat#Nat; % Request communication
    cashSelected;
    cardSelected;
    invalidCardTaken; % Invalid card realisation
    moneyAccepted: Nat; % Handing over and taking over coins
    productTaken: Nat#Nat; % After cash or card product is received
    validCard;
    invalidCard;
    invalidCoin;
    returnChange: Nat;
    receiveReceipt;

map calPrice: Nat#Nat->Nat; % product code and quantity are given and
                            % the price is calculated in cents.
                            % Assume the following code scheme:
                            % 0 for coke
                            % 1 for water
                            % 2 for chocolate

var code, qty: Nat; % Product code and quantity.
```

```
eqn % coke for 200 cents, i.e., 2 euros
    (code==0)->calPrice(code, qty)=200*qty;
    % water for 180 cents, i.e., 1.80 euros
    (code==1)->calPrice(code, qty)=180*qty;
    % chocolate for 210 cents, i.e., 2.1 euros
    (code==2)->calPrice(code, qty)=210*qty;

%%%%% Machine Process %%%%%%%%%%%%%%%%%%%
proc Machine =
        welcome . Machine
        +
        sum item, quantity: Nat.receiveRequest(item, quantity) .
        showPrice(calPrice(item, quantity)) .
                        TakeCashOrCard(item, quantity);

     TakeCashOrCard(item: Nat, quantity: Nat) =
        (showCardOption . TakeCard(item, quantity)
         +
         showCashOption.TakeCashFor(item, quantity, 0)
        );

     TakeCashFor(item: Nat, qty: Nat, money: Nat) =
        sum anyCoin: Nat.
           (anyCoin<5 && calPrice(item, qty)>money)
           -> receiveMoney([10, 20, 50, 100, 200].anyCoin) .
              TakeCashFor(money=money+[10, 20, 50, 100, 200].anyCoin)
        +
        (calPrice(item, qty)<=money)
         -> sendProduct(item, qty) .
            sendChange(Int2Nat(money-calPrice(item, qty))) .
            Machine
        +
        receiveInvalidCoin .
        TakeCashFor();

     TakeCard(item, quantity: Nat) =
        chargeSuccess .
        sendProduct(item, quantity) .
        printReceipt .
        Machine
        +
        chargeFailed .
        TakeCashOrCard(item, quantity);

%%%%%% User Process %%%%%%%%%%%%%%5
     User =
        sum code, quantity: Nat.
        (code<=2 && quantity>=1 && quantity<=3)
        -> sendRequest(code, quantity) .
           MoneyOptions;

     MoneyOptions =
        selectCash .
        ProvideCash
        +
```

```
          selectCard .
          ProvideCard;

       ProvideCash =
          sum coin: Nat.insertMoney(coin) . ProvideCash
          +
          insertInvalidCoin . ProvideCash
          +
          sum product, qty: Nat.receiveProduct(product, qty) .
               sum return: Nat.receiveChange(return) .
               User;

       ProvideCard =
          insertInvalid . MoneyOptions
          +
          insertCard .
          sum product, qty: Nat.receiveProduct(product, qty) .
          getReceipt .
          User;

  init allow( { welcome, showPrice,  requestSent, cashSelected,
                cardSelected, moneyAccepted, productTaken,
                invalidCard, returnChange, validCard,
                receiveReceipt, invalidCoin },
          comm( { receiveRequest | sendRequest -> requestSent,
                  selectCash | showCashOption -> cashSelected,
                  selectCard | showCardOption -> cardSelected,
                  receiveMoney | insertMoney -> moneyAccepted,
                  receiveProduct | sendProduct -> productTaken,
                  insertInvalid | chargeFailed -> invalidCard,
                  sendChange | receiveChange -> returnChange,
                  insertCard | chargeSuccess -> validCard,
                  printReceipt | getReceipt -> receiveReceipt,
                  insertInvalidCoin | receiveInvalidCoin -> invalidCoin },
            Machine||User
            ) );
```

Chapter 6

6.1

1. *true*. 2. *true*. 3. *true*. 4. *false*.

6.2

The formula $\langle a\rangle\langle b\rangle\langle d\rangle true$ holds for Fig. 6.2a and is invalid for Fig. 6.2b. The formula $[a][b][d]false$ is valid for Fig. 6.2b and is false for Fig. 6.2a. Note that there are many correct answers.

6.3

1. The formula $\langle a\rangle[a]\langle a\rangle true$ says that there is at least one action a at the initial state such that all subsequent actions a are followed by at least one other action a.

Algorithmically, it can be described by:

```
1  if an action a exists in the initial state then
2  |  if after this first action a there are more actions a then
3  |  |  foreach action a found in line 2 do
4  |  |  |  if another subsequent action a is found then
5  |  |  |  |  true
6  |  |  |  else
7  |  |  |  |  false
8  |  |  |  end
9  |  |  end
10 |  else
11 |  |  true
12 |  end
13 else
14 |  false
15 end
```

The respective answers are given below, followed by the line number of the above algorithm providing this answer:

(a) *true*, line 11.
(b) *false*, line 14.
(c) *false*, line 7.
(d) *true*, line 11 (There is an action a after which no action a exists).
(e) *true*, line 11.
(f) *true*, line 5.
(g) *false*, line 7.
(h) *true*, line 11
(i) *false*, line 7.

2. The formula $[a]\langle a\rangle[a]$*false* says that for any action a in the initial state there is a subsequent action a after which no further action a occurs. Algorithmically, it can be described as:

```
1  if for all actions a in the initial state then
2  |  if if a subsequent action a exists then
3  |  |  if this last action a is not followed by another action a then
4  |  |  |  true
5  |  |  else
6  |  |  |  false
7  |  |  end
8  |  else
9  |  |  false
10 |  end
11 else
12 |  true
13 end
```

The answers are below, followed by the line number where the above algorithm provides an answer:

(a) *false*, line 6.
(b) *true*, line 12.
(c) *true*, line 4.
(d) *false*, line 9.
(e) *false*, line 9.
(f) *false*, line 6.
(g) *true*, line 4.
(h) *false*, line 9.
(i) *true*, line 4.

6.4

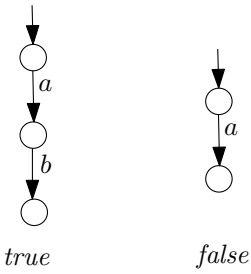
1. (a) *false*. (b) *true*. (c) *true*. (d) *false*. (e) *false*. (f) *false*. (g) *true*. (h) *false*. (i) *true*.
2. (a) *true*. (b) *false*. (c) *false*. (d) *true*. (e) *true*. (f) *true*. (g) *false*. (h) *true*. (i) *false*.
3. (a) *false*. (b) *false*. (c) *false*. (d) *false*. (e) *false*. (f) *false*. (g) *false*. (h) *false*. (i) *false*.

6.5

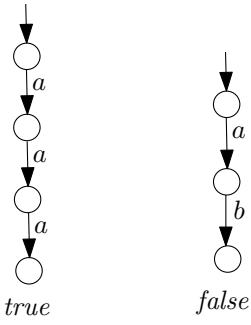
1. *false*. 2. *true*. 3. *true*. 4. *false*.

6.6

1.



2. The above figure answers this question if we swap *true* and *false*.
3.



6.7

1. $[switchOff]false$. 2. $[switchOn]\langle switchOff\rangle true$. 3. $\langle d\rangle true \wedge [d]\langle e\rangle true$.

6.8

1. $true$. 2. $false$. 3. $true$. 4. $false$.

6.9

1. $false$. 2. $false$. 3. $true$. 4. $true$. 5. $false$. 6. $false$. 7. $true$. 8. $true$. 9. $false$.

6.10

1. Valid for all transition systems. 2. Valid except for Fig. 6.7d and Fig. 6.8b–d. 3. Valid for all transition system, except Fig. 6.8b, c.

6.11

1. $[true^* \cdot error]false$. 2. $[true^* \cdot start \cdot \overline{stop}^* \cdot start]false$. 3. $\langle true^* \cdot hurrah$. 4. $[true^* \cdot start \cdot \overline{stop}^*]\langle true^* \cdot stop\rangle true$.

6.12

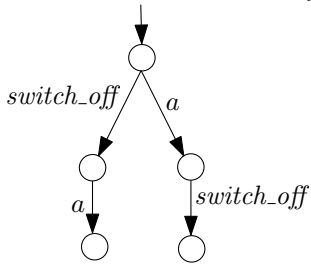
1. $true$ 2. $false$. 3. $false$. 4. $true$.

6.13

The formula $[\overline{switch_off}^*]\langle true\rangle true$ is valid for the following LTS whereas the formula

$$[true^* \cdot switch_off \cdot true]false \wedge [\overline{switch_off}^*]\langle true\rangle true$$

is invalid for this transition system.



6.14

$\langle true^* \cdot switch_off\rangle true \wedge [\overline{switch_off}^*]\langle true\rangle true$.

6.15

1.

(a) We cannot differentiate them. A way to see this is to assume that if the left modal formula is valid, then also the right formula should be valid, and vice versa. If $\langle true^* \cdot a\rangle true$ holds, then there is a trace with an action a. So, there is also a first action a on that trace. This shorter trace makes the formula $\langle \overline{a}^* \cdot a\rangle true$ valid. Reversely, if $\langle \overline{a}^* \cdot a\rangle true$ is valid, there is a trace with only an a at the end. But this trace makes $\langle true^* \cdot a\rangle true$ valid.

(b) These formulas can be distinguished by a single deadlock state. For this the first formula is false. The second formula is true, as it requires that each state reachable in one or more steps should have an outgoing transition. But the initial state is not required to have an outgoing transition.

2. $[\overline{(switch_off \cup shutDown)}^*]\langle true\rangle true$.
3. It means after doing arbitrary trace of actions, taken from the empty set of actions, one more action is possible. But the only sensible such trace is the empty trace. So, $[false^*]\langle true\rangle true = [\epsilon]\langle true\rangle true = \langle true\rangle true$, or in words, it expresses that an action is possible in the initial state.

6.16

1. $[true^*]\forall i{:}\mathbb{N}.(val(i \leq n) \Rightarrow \langle true^* \cdot wish(i)\rangle true)$.
2. $[true^* \cdot (\exists id_1{:}\mathbb{N}.enter(id_1)) \cdot \overline{(\exists id_2{:}\mathbb{N}.leave(id_2))}^* \cdot (\exists id_3{:}\mathbb{N}.enter(id_3))]false$.

6.17

$[true^*]\forall i{:}\mathbb{N}.[switch_on(i).\overline{(\exists j{:}\mathbb{N}.switch_on(j))}^*]\forall k{:}\mathbb{N}.[switch_on(k)]val(i \approx k)$.

6.18

$[true^* \cdot \exists i{:}\mathbb{N}.val(i \geq n) \wedge start(i)]false$.

Chapter 7

7.1

1. $[true^*]\langle c\rangle true$ means that an action c can be done in each reachable state. This is invalid in Fig. 7.1a.
2. $\langle \overline{c}^* \cdot c\rangle true$ means that there is a trace of actions without c's ending in the action c. It is equivalent to $\langle true^* \cdot c\rangle true$ and valid in Fig. 7.1a.
3. $[\overline{c}^*]\langle c\rangle true$ represents that whenever no action c is done, an action c is immediately possible. This formula is clearly not valid in Fig. 7.1a for instance because the initial state does not have an outgoing c-transition.
4. $[true^* \cdot \overline{c}]false$ stands for no action other than c can be done. Clearly invalid in Fig. 7.1a.

7.2

(a) *true*. (b) *true*. (c) *false*. (d) *true*.

7.3

(a) *false*. (b) *false*. (c) *false*. (d) *true*.

7.4

1. If we take Y to be empty, we see that it is not possible to do an action a or b and reach to a state in an empty set. So, the empty set is a solution and hence, the given formula is always false.

2. Again try the empty set as a solution. Then the formula $[true]Y$ means if we do an action we reach to state in an empty set. This can be only be true for a deadlock state. But the other part $langletrue\rangle true$ means that an action must be done. As a deadlock and an arbitrary action cannot happen at the same time, the given formula is always false.

7.5

1. Formula (7.1) holds for the Fig. 7.6a, c. The modified version is true for all LTSs except Fig. 7.6d.
2. The modified version holds for Fig. 7.5a, c.

7.6
This says that for every weekday there is an occurrence that is followed by a holiday without any holidays in between. This is obviously not true for a natural model of a normal week.

7.7
Consider the transition system that can only do an action *left*. Then both conjuncts $[up]X(n+1)$ and $[down]X(n-1)$ are true, as *left* is not *up* or *down*. This makes the formula true without inspecting the state space after the action *left*. Note that this exercise shows that all actions must be mentioned in box modalities, as otherwise parts of the state space may not be inspected.

7.8
$\forall n : \mathbb{N}.[true^* \cdot gen(n) \cdot true^* gen(n)]false.$

7.9
$\mu X(n : \mathbb{N} = 0).(([alarm]X(n+1) \wedge [\overline{alarm}]X(n)) \vee (n \approx 5)).$

7.10
We start out with $X = \{s_0, s_1, s_2, s_3\}$. We see that $\langle b\rangle true$ holds at $\{s_0, s_1, s_2\}$ while $[true]X$ holds in all states. So, after one iteration

$$X = \{s_0, s_1, s_2\} \cap \{s_0, s_1, s_2, s_3\}$$

In the second iteration, $X = \{s_0, s_1, s_2\}$ and $[true]X$ holds in $\{s_0, s_2\}$ because only from these states all actions lead to X. So,

$$X = \{s_0, s_1, s_2\} \cap \{s_0, s_2\}$$

In the third iteration, $X = \{s_0, s_2\}$ and for this set $[true]X$ holds at $\{s_2\}$. So,

$$X = \{s_0, s_1, s_2\} \cap \{s_2\}$$

In the fourth iteration $X = \{s_2\}$ remains the same. So, we conclude that this is the maximal fixed point. As s_0 is not in this set, the formula does not hold in initial state.

7.11

$$\nu X(m:\mathbb{N}=0).(\forall n:\mathbb{N}.[product(n)](n>m \wedge X(n))) \wedge [\overline{\exists n:\mathbb{N}.product(n)}]X(m).$$

7.12

In the first formula we simply count the number of wafers in the machine. When the action *ready* happens this number must be 0. Note that we ignore the identities of the wafer, which makes this formula much easier to check automatically.

$$\begin{aligned}\nu X(n:\mathbb{N}=0).([\exists w:\mathbb{N}.enter(w)]X(n+1) \wedge \\ [\exists w:\mathbb{N}.leave(w)]X(\max(0,n-1)) \wedge \\ [\overline{\exists w:\mathbb{N}.enter(w) \cup leave(w)}]X(n) \wedge \\ [ready](n \approx 0)).\end{aligned}$$

In the second formula we maintain a set S with identities of wafers. When an identity is inserted it may not already be in the set. Note that there are very many different potential sets. Hence, verifying formulas with such sets can only be done if the number of considered sets does not become very large.

$$\begin{aligned}\nu X(S:Set(\mathbb{N})=\{\}).((\forall w:\mathbb{N}.[enter](w \notin S \wedge X(S \cup \{w\}))) \wedge \\ (\forall w:\mathbb{N}.[leave(w)]X(S \setminus \{w\})) \wedge \\ [\overline{\exists w:\mathbb{N}.enter(w) \cup leave(w)}]X(S)).\end{aligned}$$

In the third formula we check whether when we enter a wafer no wafer with that identity is in the machine, when a wafer leaves the machine, a wafer with that identity is in the machine, and when the machine finishes a lot, the set of wafer identities that occur in the machine is empty.

$$\begin{aligned}\nu X(S:Set(\mathbb{N})=\{\}).((\forall w:\mathbb{N}.[enter](w \notin S \wedge X(S \cup \{w\}))) \wedge \\ (\forall w:\mathbb{N}.[leave(w)](w \in S \wedge X(S \setminus \{w\}))) \wedge \\ [\overline{\exists w:\mathbb{N}.enter(w) \cup leave(w)}]X(S) \wedge \\ [ready](S \approx \varnothing)).\end{aligned}$$

7.13

Consider the processes: `P=applyBrake.releaseBrake.unlockDock` and `Q=unlockDock`. For process `P` the first formula is true while the second one is false. For process `Q` the first formula is false, while the second is true.

Chapter 8

8.1

1. Linear. 2. Linear, although the condition is not explicit. 3. Not linear. There are two actions before P. 4. Not linear. The process variable at the right side does not

match the variable at the left. 5. Linear. 6. Not strictly linear, but as delta does not really change the behaviour we accept is as linear.

8.2

```
act error;
var i: Nat;
rename (i mod 2==0)-> a(i) => error;
```

We can also write the above by defining a function as given below.

```
act error;
map isEven: Nat->Bool;
var i: Int;
eqn isEven(0) = true;
    (i>0) -> isEven(i) = !isEven(max(0,i-1));
var i: Nat;
rename isEven(i) -> a(i) => error;
```

Chapter 9

9.2

At line 19 in the model we restrict t to be smaller than 19 min. For these values we investigate one extra crossing. As the longest crossing time is 10 min the times in Fig. 9.3 go up to 29 min. Suppose that the state space would be very large, and we would be in need of reducing it, we could have restricted the formalisation, such that no crossing requiring more time than 19 would be investigated.

9.3

An adapted model is the following. The weight of a previous crossing y is now an integer, where the value -1 represents that there was no previous crossing.

```
map all: Set(Pos);
    weight: Pos#Pos->Pos;

var i,j: Pos;
eqn all = {1, 2, 3, 4};
    weight(i,j) = if(i==j,i,i+j);

act moveAcross, moveBack:Set(Pos);
    done;

proc P1(here: Set(Pos), y:Int)=
       sum i, j: Pos.(i in here && j in here &&
         (y==-1 || abs(weight(i,j)-y)<=1))->
           moveAcross({i, j}).P2(here-{i, j},weight(i,j))
```

```
        + (y<=1) -> moveAcross({}).P2(here,0);

      P2(here: Set(Pos), y:Int)=
        (here!={})
        -> (sum i, j:Pos.(i in all-here && j in all-here &&
              abs(weight(i,j)-y)<=1)->
                moveBack({i, j}).P1(here+{i, j}, weight(i,j))
             + (y<=1) -> moveBack({}).P1(here, 0)
           )
        <> done.delta;

init P1(all, -1);
```

The formula that expresses that the action *done* can be reached including crossing the bridge at least once empty handed is the following.

$$\langle true^* \cdot moveAcross(\{\}) \cdot true^* \cdot done \rangle true.$$

This formula is valid.

The formula that expresses that a crossing of less than 13 steps is possible is

$$\begin{aligned}&\mu X(n : \mathbb{N} = 0).\\ &\quad ((n \leq 13) \wedge \langle \exists s : Set(\mathbb{N}^+).(moveAcross(s) \cup moveBack(s)) \rangle X(n+1)) \vee\\ &\quad \langle done \rangle (n < 13).\end{aligned}$$

In this formula the observation parameter n records the number of crossings. It is limited to 13 to avoid that an infinite number of situations must be evaluated. The formula is false. Note that in the tool syntax, the conditions $n \leq 13$ and $n < 13$ must be surrounded by the `val` keyword.

9.4

No. The given formula is *false* because there are many paths from the initial state which do not contain the action *done*. For example, the farmer can move back and forth with the goat. In this way the action *done* will be postponed indefinitely. We are not interested in whether the action *done* will take place, but we want to know that a path to *done* exists.

9.6

True, because at the initial state, it is not possible to put $\times$ on the board.

9.7

By adapting the initial board in the mCRL2 specification above, and checking the second property, it follows that player $\bigcirc$ can indeed always win the game.

Chapter 10

10.1
Set *tmin* to 1 and *tmax* to 2. In the one but last line of Formula (10.3) we write $n_0 \geq 2 * tmax + 1$ We find the following counter example.

$$ticking \cdot ticking \cdot heartbeat(p_0) \cdot ticking \cdot i \cdot lose_message \cdot ticking \cdot heartbeat(p_0) \cdot ticking.$$

Clearly, process $p[0]$ does not receive a heartbeat for 5 (which is more than $2 * tmax$) ticks, but it did not inactivate itself. This problem was observed using model-checking in [2].

10.2
A reasonable candidate for this formula is $[true^*]\langle true^* \cdot ticking\rangle true$. It is valid in the model.

10.2**
There are many ways to model the expanding and dynamic heartbeat protocols. The article [2] reports about one particular attempt to model these protocols.

References

1. Flaccus Albinus Alcuinus. Propositiones ad acuendos juvenes. Manuscript, 780
2. Atif, M., Mousavi, M.R.: Formal specification and analysis of accelerated heartbeat protocols. In: Proceedings of the 2010 Summer Computer Simulation Conference, SCSC '10, pp. 403–412, San Diego, CA, USA (2010). Society for Computer Simulation International
3. Atif, M., Mousavi, M.R., Osaiweran, A.: Formal verification of unreliable failure detectors in partially synchronous systems. In: Proceedings of the 27th Annual ACM Symposium on Applied Computing, SAC '12, pp. 478-485, New York, NY, USA (2012). Association for Computing Machinery
4. Baader, F., Nipkow, T.: Term Rewriting and All That. Cambridge University Press (1998)
5. Baeten, J.C.M., Basten, T., Reniers, M.A.: Process Algebra: Equational Theories of Communicating Processes, 1st edn. Cambridge University Press, New York, NY, USA (2009)
6. Baeten, J.C.M., Weijland, W.P.: Process Algebra, Cambridge Tracts in Theoretical Computer Science, vol. 18. Cambridge University Press (1990)
7. Barendregt, H.P.: The Lambda Calculus - Its Syntax and Semantics, Studies in Logic and the Foundations of Mathematics, vol. 103. North-Holland (1985)
8. Barnat, J., Brim, L., Rockai, P.: Divine multi-core - a parallel LTL model-checker. In: Cha, S.D., Choi, J.-Y., Kim, M., Lee, I., Viswanathan, M. (eds.), Automated Technology for Verification and Analysis, 6th International Symposium, ATVA 2008, Seoul, Korea, Oct. 20–23, 2008. Proceedings, Lecture Notes in Computer Science, vol. 5311, pp. 234–239. Springer (2008)
9. Behrmann, G., David, A., Larsen, K.G.: A tutorial on uppaal. In: Bernardo, M., Corradini, F. (eds.) Formal Methods for the Design of Real-Time Systems, International School on Formal Methods for the Design of Computer, Communication and Software Systems, SFM-RT 2004, Bertinoro, Italy, Sept. 13–18, 2004, Revised Lectures, Lecture Notes in Computer Science, vol. 3185, pp. 200–236. Springer (2004)
10. Blom, S.S.C., Fokkink, W.J., Groote, J.F., van Langevelde, I.A., Lisser, B., van de Pol, J.C.: μCRL: a toolset for analysing algebraic specifications. In: Comon, G.B.H., Finkel, A. (eds.) Computer Aided Verification, 13th International Conference, CAV 2001, Paris, France, July 18–22, 2001, Proceedings, Lecture Notes in Computer Science, vol. 2102, pp. 250–254. Springer (2001)
11. Bolognesi, T., Brinksma, E.: Introduction to the ISO specification language LOTOS. Comput. Netw. **14**, 25–59 (1987)
12. Bradfield, J.C.: Introduction to modal and temporal mu-calculi. In: Brim, L., Křetínský, M., Kučera, A., Jančar, P. (eds.) CONCUR 2002 – Concurrency Theory, pp. 98–98. Springer, Berlin, Heidelberg (2002)
13. Clarke, E.M., Emerson, E.A.: Design and synthesis of synchronization skeletons using branching time temporal logic. In: Kozen, D. (ed.) Logics of Programs, pp. 52–71. Springer, Berlin, Heidelberg (1982)

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0

14. Cranen, S.: Model checking the flexray startup phase. In: Stoelinga, M., Pinger, R. (eds.) Formal Methods for Industrial Critical Systems, pp. 131–145. Springer, Berlin, Heidelberg (2012)
15. Cranen, S., Groote, J.F., Keiren, J.J.A., Stappers, F.P.M., de Vink, E.P., Wesselink, W., Willemse, T.A.C.: An overview of the mCRL2 toolset and its recent advances. In: Piterman, N., Smolka, S.A. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, pp. 199–213. Springer, Berlin, Heidelberg (2013)
16. Emerson, E.A., Halpern, J.Y.: Sometimes and not never revisited: on branching versus linear time temporal logic. J. ACM **33**(1), 151–178 (1986)
17. Evaluator in CADP. https://cadp.inria.fr/man/evaluator.html. Accessed: 2019-11-06
18. Ferrante, O., Benvenuti, L., Mangeruca, L., Sofronis, C., Ferrari, A.: Parallel nusmv: A nusmv extension for the verification of complex embedded systems. In: Ortmeier, F., Daniel, P. (eds.) Computer Safety, Reliability, and Security - SAFECOMP 2012 Workshops: Sassur, ASCoMS, DESEC4LCCI, ERCIM/EWICS, IWDE, Magdeburg, Germany, Sept. 25–28, 2012. Proceedings, Lecture Notes in Computer Science, vol. 7613, pp. 409–416. Springer (2012)
19. Fokkink, W.J.: Introduction to Process Algebra. Texts in Theoretical Computer Science. An EATCS Series. Springer (2000)
20. Garavel, H., Mateescu, R., Lang, F., Serwe, W.: Cadp 2006: a toolbox for the construction and analysis of distributed processes. In: Proceedings of the 19th International Conference on Computer Aided Verification, CAV'07, pp. 158–163. Springer, Berlin, Heidelberg (2007)
21. Gibson-Robinson, T., Armstrong, P.J., Boulgakov, A., Roscoe, A.W.: FDR3: a parallel refinement checker for CSP. Int. J. Softw. Tools Technol. Transf. **18**(2), 149–167 (2016)
22. Gouda, M.G., McGuire, T.M.: Accelerated heartbeat protocols. In: Proceedings. 18th International Conference on Distributed Computing Systems (Cat. No.98CB36183), pp. 202–209 (1998)
23. Gouda, M.G., McGuire, T.M.: Alert communication primitives above TCPi. J. High Speed Netw. **9**(2), 139–150 (2000)
24. Groote, J.F., de Vink, E.P.: Problem solving using process algebra considered insightful. In: Katoen, J.-P., Langerak, R., Rensink, A. (eds.) ModelEd, TestEd, TrustEd - Essays Dedicated to Ed Brinksma on the Occasion of His 60th Birthday, Lecture Notes in Computer Science, vol. 10500, pp. 48–63. Springer (2017)
25. Groote, J.F., Keiren, J.J.A., Luttik, B., de Vink, E.P., Willemse, T.A.C.: Modelling and analysing software in mCRL2. In: Arbab, F., Jongmans, S.-S. (eds.) Formal Aspects of Component Software, pp. 25–48. Springer, Cham (2020)
26. Groote, J.F., Kouters, T.W.D.M., Osaiweran, A.: Specification guidelines to avoid the state space explosion problem. Softw. Test. Verification Reliab. **25**(1), 4–33 (2015)
27. Groote, J.F., Mathijssen, A., van Weerdenburg, M., Usenko, Y.S.: From μCRL to mCRL2: motivation and outline. Electron. Notes Theor. Comput. Sci. **162**, 191–196 (2006)
28. Groote, J.F., Mousavi, M.R.: Modeling and Analysis of Communicating Systems. The MIT Press (2014)
29. Groote, J.F., Pang, J., Wouters, A.G.: Analysis of a distributed system for lifting trucks. J. Log. Algebraic Methods Program. **55**(1–2), 21–56 (2003)
30. Groote, J.F., Ponse, A.: Proof theory for μCRL: a language for processes with data. In: Andrews, D.J., Groote, J.F., Middelburg, C.A. (eds.) Semantics of Specification Languages (SoSL), Proceedings of the International Workshop on Semantics of Specification Languages, Utrecht, The Netherlands, 25–27 Oct. 1993, Workshops in Computing, pp. 232–251. Springer (1993)
31. Groote, J.F., Ponse, A., Usenko, Y.S.: Linearization in parallel pCRL. J. Log. Algebraic Methods Program. **48**(1–2), 39–70 (2001)
32. Groote, J.F., Sellink, M.P.A.: Confluence for process verification. Theor. Comput. Sci. **170**(1–2), 47–81 (1996)
33. Groote, J.F., Usenko, Y.S., Reniers, M.A.: Verification of networks of timed automata using mCRL2. In: 2008 IEEE International Parallel & Distributed Processing Symposium, pp. 1–8, Los Alamitos, CA, USA, Apr. 2008. IEEE Computer Society
34. Groote, J.F., Willemse, T.A.C.: Parameterised boolean equation systems. In: Gardner, P., Yoshida, N. (eds.) CONCUR 2004 - Concurrency Theory, pp. 308–324. Springer, Berlin, Heidelberg (2004)

35. Groote, J.F., Willemse, T.A.C.: Model-checking processes with data. Sci. Comput. Program. **56**(3), 251–273 (2005)
36. Hansen, H.H., Kupke, C., Marti, J., Venema, Y.: Parity games and automata for game logic (extended version). CoRR arXiv:1709.00777 (2017)
37. He, J.-B., Zhang, H.-M., Liang, J., Jin, O., Li, X.: Paper currency denomination recognition based on GA and SVM. In: Tan, T., Ruan, Q., Wang, S., Ma, H., Di, K. (eds.) Advances in Image and Graphics Technologies - 10th Chinese Conference, IGTA 2015, Beijing, China, June 19–20, 2015, Proceedings, Communications in Computer and Information Science, vol. 525, pp. 366–374. Springer (2015)
38. Hennessy, M., Milner, R.: Algebraic laws for nondeterminism and concurrency. J. ACM **32**(1), 137–161 (1985)
39. Hoare, C.A.R.: Communicating sequential processes. In: Jones, C.B., Misra, J. (eds.) Theories of Programming: The Life and Works of Tony Hoare, pp. 157–186. ACM/Morgan & Claypool (2021)
40. Holbrook, H.W., Singhal, S.K., Cheriton, D.R.: Log-based receiver-reliable multicast for distributed interactive simulation. SIGCOMM Comput. Commun. Rev. **25**(4), 328–341 (1995)
41. Janssen, S.: Tools for parameterized boolean equation systems. Master's thesis, Eindhoven University of Technology, Netherlands (2008)
42. de Jonge, M., Ruys, T.C.: The SpinJai model checker. In: van de Pol, J.C., Weber, M. (eds.) Model Checking Software - 17th International SPIN Workshop, Enschede, The Netherlands, Sept. 27–29, 2010. Proceedings, Lecture Notes in Computer Science, vol. 6349, pp. 124–128. Springer (2010)
43. Kant, G., Laarman, A., Meijer, J., van de Pol, J.C., Blom, S.C.C., van Dijk, T.: Ltsmin: high-performance language-independent model checking. In: Baier, C., Tinelli, C. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 21st International Conference, TACAS 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11–18, 2015. Proceedings, Lecture Notes in Computer Science, vol. 9035, pp. 692–707. Springer (2015)
44. Klein, J., Baier, C., Chrszon, P., Daum, M., Dubslaff, C., Klüppelholz, S., Märcker, S., Müller, D.: Advances in symbolic probabilistic model checking with PRISM. In: Chechik, M., Raskin, J.-F. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 22nd International Conference, TACAS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2–8, 2016, Proceedings, Lecture Notes in Computer Science, vol. 9636, pp. 349–366. Springer (2016)
45. Kozen, D.: Results on the propositional μ-calculus. In: Nielsen, M., Schmidt, E.M. (eds.) Automata, Languages and Programming, pp. 348–359. Springer, Berlin, Heidelberg (1982)
46. Mauw, S., Veltink, G.J.: A process specification formalism. Fundam. Inf. **13**(2), 85–139 (1990)
47. Milner, R.: A Calculus of Communicating Systems, Lecture Notes in Computer Science, vol. 92. Springer (1980)
48. Ploeger, B., Somers, L.J.: Analysis and verification of an automatic document feeder. In: Cho, Y., Wainwright, R.L., Haddad, H., Shin, S.Y., Koo, Y.W. (eds.) Proceedings of the 2007 ACM Symposium on Applied Computing (SAC), Seoul, Korea, March 11–15, 2007, pp. 1499–1505. ACM (2007)
49. Pnueli, A.: The temporal logic of programs. In: 18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 Oct.–1 Nov. 1977, pp. 46–57. IEEE Computer Society (1977)
50. Remenska, D., Willemse, T.A.C., Verstoep, K., Templon, J., Bal, H.: Using model checking to analyze the system behavior of the LHC production grid. Future Gener. Comput. Syst. **29**(8), 2239–2251 (2013)
51. Stappers, F.P.M., Weber, S., Reniers, M.A., Andova, S., Nagy, I.: Formalizing a domain specific language using SOS: an industrial case study. In: Sloane, A.M., Aßmann, U. (eds.) Software Language Engineering - 4th International Conference, SLE 2011, Braga, Portugal, July 3–4, 2011, Revised Selected Papers, Lecture Notes in Computer Science, vol. 6940, pp. 223–242. Springer (2011)

52. Sun, J., Liu, Y., Dong, J.S., Pang, J.: PAT: towards flexible verification under fairness. In: Bouajjani, A., Maler, O. (eds.) Computer Aided Verification, 21st International Conference, CAV 2009, Grenoble, France, June 26–July 2, 2009. Proceedings, Lecture Notes in Computer Science, vol. 5643, pp. 709–714. Springer (2009)
53. Tel, G.: Introduction to Distributed Algorithms. Cambridge University Press, New York, NY, USA (1994)
54. Ting, Y., Shan, F.M., Lu, W.B., Chen, C.H.: Implementation and evaluation of failsafe computer-controlled systems. Comput. Ind. Eng. **42**(2), 401–415 (2002)
55. Turing, A.M.: Computability and λ-definability. J. Symb. Logic **2**(4), 153–163 (1937)
56. van de Pol, J.C., Meijer, J.: Synchronous or alternating? - LTL black-box checking of mealy machines by combining the learnlib and ltsmin. In: Margaria, T., Graf, S., Larsen, K.G. (eds.) Models, Mindsets, Meta: The What, the How, and the Why Not? - Essays Dedicated to Bernhard Steffen on the Occasion of His 60th Birthday, Lecture Notes in Computer Science, vol. 11200, pp. 417–430. Springer (2018)
57. Vogels, W.: World wide failures. In: Proceedings of the 7th Workshop on ACM SIGOPS European Workshop: Systems Support for Worldwide Applications, EW 7, pp. 115–120, New York, NY, USA (1996). Association for Computing Machinery
58. Wang, G., Gao, Z., Zhang, L., Cao, J.: Prediction-based multicast mobility management in mobile internet. In: Pan, Y., Chen, D., Guo, M., Cao, J., Dongarra, J. (eds.) Parallel and Distributed Processing and Applications, pp. 1024–1035. Springer, Berlin, Heidelberg (2005)

Index

A
Action formula, 100
Allow operator, 26, 34

B
Bag, 68
Bcg_draw, 113
Bcg_edit, 113
Bcg_open, 113
Behavioural equivalences, 37
Bisimulation, 42
Block operator, 34
Boolean equation system, 149
Bound variable, 79
Box modality [ ], 95
Branching bisimulation, 45, 47
  divergence preserving, 50

C
Choice operator, 17
Comm operator, 27
Communicating actions, 51
Communication operator, 27
Conditional actions, 59
Constants, 78
Construction and Analysis Distributed Processes (CADP), 6, 113
Constructor, 81
Counter example, 41, 93

D
Data structures, 60
Deadlock, 30
Diagraphica, 15
Diamond modality ⟨ ⟩, 94
Divergence, 49
Divergence preserving branching bisimulation, 50

E
Existential quantifier, 74
Exploration mode, 178

F
Fairness, 133
Finite set, 68
Fixed point, 117
Functions
  function update, 80
  projection functions, 84
  user defined, 70

G
Graphical User Interface (GUI), 19
Guarded actions, 59

H
Handshaking, 32
Heartbeat protocol
  dynamic, 199
  expanding, 199
Hello-world example, 11
Hennessy-Milner Logic (HML), 94
  formula, 94
Hidden action, 35
Hiding operator, 35

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Atif and J. F. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*, Studies in Systems, Decision and Control 458,
https://doi.org/10.1007/978-3-031-23008-0

I
Integrated Development Environment (IDE), 12
Internal action, 35

L
Labelled transition system, 16
Lambda calculus, 79
Lambda (λ) expression, 79
Lambda term, 79
Linear process, 149
Linear Process Specification (LPS), 149
List, 60
List of lists, 65
List operations, 63
Lps2lts, 15, 152
Lps2pbes, 96
Lpsactionrename, 157
Lpsconstelm, 154
Lpsinfo, 152
Lpsparelm, 154, 163
Lpspp, 152
Lpsrewr, 153
Lpssim, 12, 23, 152
Lpssumelm, 156
Lpssuminst, 155
Lpsxsim, 15, 23, 64, 152
Ltsconvert, 38, 41
Ltsgraph, 20
Ltsview, 15, 23

M
Maximal fixed point, 117, 128
Mcrl22lps, 15, 151
Mcrl2gui, 4
Mcrl2ide, 4
Mcrl2xi, 19
Minimal fixed point, 117, 118
Modal μ-calculus, 117
Model-checking, 93, 96
μ-calculus, 117
Multi-action, 25
Multi-action in a formula, 100

O
Observation variable, 126, 145
Operator
  allow, 26
  block, 34
  comm, 27
  hide, 35
  rename, 34
Operators on data, 53

P
Parameterised Boolean Equation Systems (PBES), 96, 149
Patient support unit, 137
PBES solver, 96
Pbes2bool, 96, 161
Pbesconstelm, 162
Pbesinfo, 161
Pbespp, 160, 161
Pbesrewr, 160
Projection function, 84
Property, 93

Q
Quantifier, 108
  existential, 74
  exists, 108
  forall, 108
  universal, 74
Quantifier one-point, 153

R
Recogniser, 83
Regular formula, 102
Rename operator, 34
Requirements, 93
Reserved symbols, 13
Reserved words, 13

S
Sending/receiving data, 51
Sequential composition operator, 17
Set, 66
  finite, 68
Sorting, 77
State space, 16
State space explosion, 64
Strong bisimulation, 42
Struct data type, 82

T
Tau, 35
Tools
  diagraphica, 15
  lps2lts, 15, 152
  lps2pbes, 96

lpsactionrename, 157
lpsconstelm, 154
lpsinfo, 152
lpsparelm, 154
lpspp, 152
lpsrewr, 153
lpssim, 23, 152
lpssumelm, 156
lpssuminst, 155
lpsxsim, 12, 15, 23, 64, 152
ltsconvert, 38, 41
ltsgraph, 15, 20
ltsview, 15, 23
mcrl22lps, 15, 151
mcrl2gui, 4
mcrl2ide, 4
mcrl2xi, 19
pbes2bool, 96, 161
pbesconstelm, 162
pbesinfo, 161
pbespp, 160, 161
pbesrewr, 160
tracepp, 41
txt2lps, 152
txt2pbes, 160
Trace equivalence, 37
Tracepp, 41
Txt2lps, 152
Txt2pbes, 160

U
Universal quantifier, 74
Uppaal, 6

W
Witness, 93